Journal of Computational Physics 477 (2023) 111902

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

Uncertainty quantification in scientific machine learning: Methods, metrics, and comparisons

Apostolos F. Psaros [a,1], Xuhui Meng [b,1], Zongren Zou [a], Ling Guo [c], George Em Karniadakis [a,d,*]

[a] Division of Applied Mathematics, Brown University, Providence, RI 02906, USA
[b] Institute of Interdisciplinary Research for Mathematics and Applied Science, School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan 430074, China
[c] Department of Mathematics, Shanghai Normal University, Shanghai, China
[d] School of Engineering, Brown University, Providence, RI 02906, USA

ARTICLE INFO

Article history:
Received 29 January 2022
Received in revised form 10 December 2022
Accepted 29 December 2022
Available online 12 January 2023

Keywords:
Scientific machine learning
Stochastic partial differential equations
Uncertainty quantification
Physics-informed neural networks
Neural operator learning
Bayesian framework

ABSTRACT

Neural networks (NNs) are currently changing the computational paradigm on how to combine data with mathematical laws in physics and engineering in a profound way, tackling challenging inverse and ill-posed problems not solvable with traditional methods. However, quantifying errors and uncertainties in NN-based inference is more complicated than in traditional methods. This is because in addition to aleatoric uncertainty associated with noisy data, there is also uncertainty due to limited data, but also due to NN hyperparameters, overparametrization, optimization and sampling errors as well as model misspecification. Although there are some recent works on uncertainty quantification (UQ) in NNs, there is no systematic investigation of suitable methods towards quantifying the *total uncertainty* effectively and efficiently even for function approximation, and there is even less work on solving partial differential equations and learning operator mappings between infinite-dimensional function spaces using NNs. In this work, we present a comprehensive framework that includes uncertainty modeling, new and existing solution methods, as well as evaluation metrics and post-hoc improvement approaches. To demonstrate the applicability and reliability of our framework, we present an extensive comparative study in which various methods are tested on prototype problems, including problems with mixed input-output data, and stochastic problems in high dimensions. In the Appendix, we include a comprehensive description of all the UQ methods employed. Further, to help facilitate the deployment of UQ in Scientific Machine Learning research and practice, we present and develop in [1] an open-source Python library (*github.com/Crunch-UQ4MI/neuraluq*), termed NeuralUQ, that is accompanied by an educational tutorial and additional computational experiments.

© 2022 Elsevier Inc. All rights reserved.

* Corresponding author.
E-mail address: george_karniadakis@brown.edu (G.E. Karniadakis).
[1] The first two authors contributed equally to this work.

https://doi.org/10.1016/j.jcp.2022.111902
0021-9991/© 2022 Elsevier Inc. All rights reserved.

Contents

1. Introduction . . . 3
   1.1. Motivation and scope of the paper . . . 3
   1.2. Problem formulation . . . 4
   1.3. Novel contributions of this work . . . 5
   1.4. Organization of the paper . . . 5
2. Neural PDEs and neural operators . . . 6
   2.1. Solving forward and mixed PDE problems: overview of PINN method . . . 7
   2.2. Learning operator mappings: overview of DeepONet method . . . 7
3. Modeling total uncertainty . . . 8
   3.1. Uncertainty in function approximation . . . 8
   3.2. Uncertainty in PINNs . . . 11
   3.3. Uncertainty in DeepONets . . . 11
4. Methods for uncertainty quantification . . . 11
   4.1. Bayesian methods . . . 12
   4.2. Ensembles . . . 12
   4.3. Functional priors (FPs) . . . 14
   4.4. Solving stochastic PDEs (SPDEs) . . . 15
   4.5. Towards a unified view of the presented methods . . . 16
5. Accuracy and uncertainty quality evaluation and improvement . . . 18
   5.1. Evaluation: accuracy and uncertainty quality evaluation . . . 18
   5.2. Post-training improvement: calibration . . . 19
6. Comparative study . . . 20
   6.1. Discontinuous function approximation . . . 21
   6.2. Mixed deterministic PDE: nonlinear time-dependent diffusion-reaction equation . . . 29
   6.3. Mixed stochastic elliptic equation . . . 34
   6.4. Forward deterministic PDE: nonlinear static diffusion-reaction equation . . . 35
   6.5. Operator learning and out-of-distribution data detection: 2D flow in heterogeneous porous media . . . 37
   6.6. Performance versus cost . . . 44
7. Summary . . . 46
   7.1. Novel contributions . . . 46
   7.2. Findings . . . 47
   7.3. Outlook and future research directions . . . 47

Code availability . . . 48
CRediT authorship contribution statement . . . 48
Declaration of competing interest . . . 48
Data availability . . . 48
Acknowledgements . . . 48
Appendix A. Supplementary material to Section 3: additional topics in total uncertainty modeling . . . 48
   A.1. Statistics of the predictive distribution . . . 48
   A.2. Point estimates (standard neural network training) . . . 49
   A.3. Hyperparameters and hyperpriors . . . 50
   A.4. Posterior tempering for model misspecification . . . 50
Appendix B. Supplementary material to Section 4: additional details of employed UQ methods . . . 51
   B.1. Gaussian processes (GPs) . . . 51
   B.2. Bayesian methods . . . 51
   B.3. Ensembles . . . 57
   B.4. Functional priors (FPs) . . . 59
   B.5. Solving stochastic PDEs (SPDEs) with U-NNPC(+) . . . 60
Appendix C. Supplementary material to Section 5: additional metrics and calibration details . . . 60
   C.1. Additional evaluation metrics . . . 60
   C.2. Post-training calibration . . . 61
Appendix D. Supplementary material to Section 6: details and additional computational results . . . 62
   D.1. Details regarding Fig. 19 for the function approximation problem of Section 6.1 . . . 62
   D.2. Additional results for the function approximation problem of Section 6.1 . . . 62
   D.3. Additional results for the mixed PDE problem of Section 6.2 . . . 66
   D.4. Additional results for the mixed stochastic problem of Section 6.3 . . . 67
   D.5. Additional results for the forward PDE problem of Section 6.4 . . . 68
   D.6. Details regarding the DeepONet input for the operator learning problem of Section 6.5 . . . 68
   D.7. Additional results for the operator learning problem of Section 6.5 . . . 73
   D.8. Hyperparameters used in the comparative study . . . 75
   D.9. Neural network architectures used in the comparative study . . . 78
References . . . 78

1. Introduction

1.1. Motivation and scope of the paper

Accurate modeling and prediction of the dynamic response of physical systems remains an open scientific problem, despite many successful research efforts in the past decades. Persistent challenges pertain, indicatively, to solving ill-posed problems, e.g., with unknown parameters and boundary conditions, to assimilating noisy and/or gappy data, and to reducing computational cost, especially for high-dimensional systems. In this regard, physics-informed and more generally scientific machine learning (SciML) are emerging interdisciplinary areas of research, which can offer effective new tools for addressing some of the aforementioned challenges; see [2] for a comprehensive review as well as [3,4].

In general, machine learning, and especially deep learning which uses neural networks (NNs), can deal with massive amounts of data and make accurate predictions fast; see, e.g., [5,6]. SciML includes a variety of techniques aiming at seamlessly combining observational data with available physical models. They can be divided into three main categories: (1) the ones that seek to emulate physical systems by utilizing large data amounts, such as the deep operator network (DeepONet) developed by Lu et al. [7] (see also [8,9] for different versions of DeepONet and [10] for theoretical results, as well as [11] for a different approach termed Fourier neural operator); (2) the ones that encode physics into the NN architecture, such as the architectures developed by Darbon and Meng [12]; and (3) the ones that add physical soft constraints in the NN optimization process, such as the physics-informed neural network (PINN) developed by Raissi et al. [13] (see also [14–19] for different versions of PINN and [20,21] for theoretical results). These techniques flourish, in principle, where the applicability of conventional solvers diminishes. Specifically, they can address mixed problems when only partial observational data and information regarding the underlying physical system are available (e.g., [13]); it is straightforward to incorporate noisy and multi-fidelity data (e.g., [22,23]); they can provide physically consistent predictions even for extrapolatory tasks (e.g., [24]); they do not require computationally expensive mesh generation (e.g., [13]); and open-source software is available and continuously extended (e.g., [25,26]).

Quantifying prediction uncertainty associated with, for example, noisy and limited data as well as NN overparametrization, is paramount for deep learning to be reliably used in critical applications involving physical and biological systems. The most successful family of UQ methods so far in deep learning has been based on the Bayesian framework [27–42]. Alternative methods are based, indicatively, on ensembles of NN optimization iterates or independently trained NNs [43–56], on the evidential framework [57–65], on combining NNs with Gaussian Processes [66–72], as well as on the conformal prediction framework [73,74]. Although Bayesian methods and ensembles are thoroughly discussed in this paper, the interested reader is also directed to the recent review studies in [75–87] for more information. Despite the extensive research efforts in the field of UQ for deep learning, there are still several open challenges, as summarized below:

1. It is not straightforward to address problems involving large measurement noise levels, associated typically with unknown and heteroscedastic noise distributions, as well as limited numbers of measurements and rare events (see, e.g., [88,89]). In such cases, not only predictions with wider uncertainty bands are obtained, but results can often be erroneous and misleading.
2. Further, it is not easy nor commonly computationally efficient to select and tune hyperparameters within the context of UQ. Such hyperparameters can, broadly speaking, include prior distributions, regularization constants, and NN architectures (see, e.g., [90]).
3. Furthermore, overparametrization of NNs, which leads to the local minima problem in standard NN training, leads to overly multi-modal posterior distributions in UQ. Developing efficient methods for obtaining representative samples from such distributions is currently an open problem (see, e.g., [39,91]).
4. Evaluating the performance of UQ methods is also not as straightforward as in standard NN training, where validation or test measurements are utilized. The reason is that the true data generating distribution is not known and depends on the number and quality of measurements as well as the selected hyperparameters (see, e.g., [92]).
5. Moreover, uncertainty predictions in most studies pertain to single inputs. Such models provide only marginal distribution predictions, which disregard correlations between, for example, points in the domain that are close to each other (see, e.g., [93,94]).
6. Finally, assessing the robustness of a UQ model in cases of distribution shift, i.e., when model inputs follow a distribution that is different from the measurement distribution, is still challenging (see, e.g., [95–97]). Developing UQ models that can detect such cases and output less confident predictions, i.e., admit "ignorance" regarding these inputs, is critical for safely deploying UQ in applications.

In this paper, we address or at least explore the extent to which most of these challenges affect the problem solutions and leave as future work the study of rare events, active learning, and joint predictions, among other topics.

Clearly, in the context of SciML, which may involve differential equations with unknown or uncertain terms and parameters, UQ becomes an even more demanding task; see Fig. 1 for a sketch of problems of interest and Fig. 2 for a summary of considered uncertainty sources. Despite some recent progress towards combining UQ with either data-driven or physics-informed optimization objectives [23,24,98–122], UQ methods are still scarcely utilized within SciML. One potential reason is that UQ methods are under-utilized even within the broader deep learning community; it is thus a developing field that

is not universally trusted and understood yet. In addition, physical considerations, although of utmost importance in many problems, make the SciML techniques more complicated and computationally expensive than their standard deep learning counterparts, and further reinforce the hesitation of the machine learning community towards UQ.

Motivated by the aforementioned open issues, in this paper we present a unified and comprehensive framework of UQ in SciML with emphasis placed on techniques founded on the PINN and DeepONet underlying frameworks and their extensions. The framework can be combined with approaches other than PINN and DeepONet, such as the Fourier neural operator developed in [11]. Specifically, (1) we consider various classes of problems and summarize pertinent solution methods; (2) provide a detailed deep learning-based uncertainty modeling procedure; (3) review and integrate into SciML selected and relevant UQ methods; (4) review and integrate uncertainty evaluation metrics as well as post-training improvement approaches; and (5) present an extensive comparative study, in which various uncertainty modeling and quantification methods are tested on problems of interest in computational science and engineering. Overall, we review, compare, and extend UQ methods for NNs to address function approximation, solving differential equations and learning operator mappings between infinite-dimensional function spaces. To this end, we inherit from the significant contributions in the field of UQ for scientific computing (e.g., [123–125]) and the related software (e.g., [126–128]), as well as from deep learning (e.g., [5]) and the related UQ methods (e.g., [83]). With the present paper, we intend to lay the groundwork for UQ to be routinely utilized in SciML, with the expectation that more focused studies will be undertaken in the future by other researchers.

1.2. Problem formulation

Consider a nonlinear partial differential equation (PDE) describing the dynamics of a physical system as follows:

$$\mathcal{F}_{\lambda,\xi}[u(x;\xi)] = f(x;\xi), x \in \Omega, \xi \in \Xi, \tag{1a}$$

$$\mathcal{B}_{\lambda,\xi}[u(x;\xi)] = b(x;\xi), x \in \Gamma, \tag{1b}$$

where x is the D_x-dimensional space-time coordinate, Ω is a bounded domain with boundary Γ, ξ is a random event in a probability space Ξ, while $f(x;\xi)$ and $u(x;\xi)$ represent the D_u-dimensional source term and sought solution evaluated at (x,ξ), respectively. Further, $\mathcal{F}_{\lambda,\xi}$ is a general differential operator; $\mathcal{B}_{\lambda,\xi}$ is a boundary/initial condition operator acting on the domain boundary Γ; and b is a boundary/initial condition term. Furthermore, $\lambda(x;\xi)$, with $x \in \Omega_\lambda \subset \Omega$, denotes the problem parameters.

In this regard, we consider four problem scenarios, as summarized in Table 1 and Fig. 1, and delineated below. We refer to problems where the operators $\mathcal{F}_{\lambda,\xi}$ and $\mathcal{B}_{\lambda,\xi}$ are known, i.e., the differential operators that define them are specified, as *neural PDEs*, because we use neural networks (NNs) throughout the present work to solve them. We refer to problems involving unknown operators $\mathcal{F}_{\lambda,\xi}$ and $\mathcal{B}_{\lambda,\xi}$ as *neural operators*, because we use NNs to learn them.

The first scenario pertains to a "forward" deterministic PDE problem, where $\mathcal{F}_{\lambda,\xi}$ and $\mathcal{B}_{\lambda,\xi}$ are deterministic and known. That is, ξ is fixed and dropped for simplicity. The problem parameters $\lambda(x)$ are also known and $u(x)$ is unknown. Given noisy data of the functions f and b sampled at random finite locations in Ω and Γ, respectively, the objective of this problem is to obtain the solution $u(x)$ at every $x \in \Omega$. The dataset in this case is expressed as $\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b\}$, where $\mathcal{D}_f = \{x_i, f_i\}_{i=1}^{N_f}$, $\mathcal{D}_b = \{x_i, b_i\}_{i=1}^{N_b}$, $f_i = f(x_i)$, and $b_i = b(x_i)$. Clearly, although we use the same symbol x_i in $\mathcal{D}_f$ and $\mathcal{D}_b$, the data locations in the space domains of f and b are in general different, and this holds for all the datasets in this paper.

The second scenario pertains to a "mixed" deterministic problem, where $\mathcal{F}_{\lambda,\xi}$, $\mathcal{B}_{\lambda,\xi}$ are deterministic and known, while $\lambda(x), u(x)$ are partially unknown. Given noisy data of u and λ sampled at random finite locations in Ω and Ω_λ, respectively, as well as noisy data of f and b, the objective of this problem is to obtain the solution $u(x)$ at every $x \in \Omega$ and the problem parameters $\lambda(x)$ at every $x \in \Omega_\lambda$. The dataset in this case is expressed as $\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b, \mathcal{D}_u, \mathcal{D}_\lambda\}$, where $\mathcal{D}_u = \{x_i, u_i\}_{i=1}^{N_u}$, $\mathcal{D}_\lambda = \{x_i, \lambda_i\}_{i=1}^{N_\lambda}$, $u_i = u(x_i)$, $\lambda_i = \lambda(x_i)$, and $\mathcal{D}_f, \mathcal{D}_b$ are given as above.

The third scenario pertains to a mixed stochastic PDE (SPDE) problem, where $\mathcal{F}_{\lambda,\xi}$, $\mathcal{B}_{\lambda,\xi}$ are stochastic and known, while $\lambda(x;\xi), u(x;\xi)$ are partially unknown. The available data consists of N noisy paired realizations of f, b, u, and λ, each one associated with a different $\xi \in \Xi$. The realizations are sampled at random finite locations in the corresponding x domains. Specifically, the dataset in this case is expressed as $\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b, \mathcal{D}_u, \mathcal{D}_\lambda\}$, where $\mathcal{D}_f = \{F_i\}_{i=1}^{N}$, $\mathcal{D}_b = \{B_i\}_{i=1}^{N}$, $\mathcal{D}_u = \{U_i\}_{i=1}^{N}$, and $\mathcal{D}_\lambda = \{\Lambda_i\}_{i=1}^{N}$. The corresponding sub-datasets for each value of i are given as $F_i = \{x_j^{(i)}, f_j^{(i)}\}_{j=1}^{N_f}$, $B_i = \{x_j^{(i)}, b_j^{(i)}\}_{j=1}^{N_b}$, $U_i = \{x_j^{(i)}, u_j^{(i)}\}_{j=1}^{N_u}$, and $\Lambda_i = \{x_j^{(i)}, \lambda_j^{(i)}\}_{j=1}^{N_\lambda}$. The objective of this problem is to obtain the statistics, distribution and/or moments, of $u(x)$ at every $x \in \Omega$ and of $\lambda(x)$ at every $x \in \Omega_\lambda$.

The fourth scenario pertains to an operator learning problem, where $\mathcal{F}_{\lambda,\xi}$, $\mathcal{B}_{\lambda,\xi}$ are stochastic and unknown, while $\lambda(x;\xi), u(x;\xi)$ are partially unknown. The available data consists of paired realizations of f, b, u, and λ, each one associated with a different $\xi \in \Xi$. The realizations are considered noisy in general for u, and clean for f, b, and λ; see also discussion in Section 3.3. Further, the objective of this problem is to learn the operator mapping defined by Eq. (1). We refer to this procedure as *pre-training* and the required dataset is the same as in the mixed stochastic problem, except with clean data for f, b, and λ. For employing the learned operator during *inference* for an unseen $\xi' \in \Xi$, we are given noisy and limited data of f, b, u, and λ, and the objective is to obtain f, b, u, and λ at every x in the corresponding domains. The inference dataset in this case is expressed as $\mathcal{D}' = \{\mathcal{D}'_f, \mathcal{D}'_b, \mathcal{D}'_u, \mathcal{D}'_\lambda\}$, where $\mathcal{D}'_f = \{x_i, f_i\}_{i=1}^{N'_f}$, $\mathcal{D}'_b = \{x_i, b_i\}_{i=1}^{N'_b}$, $\mathcal{D}'_u = \{x_i, u_i\}_{i=1}^{N'_u}$, and

Table 1
Problem scenarios considered in this paper. Problems 1-3 relate to *neural (S)PDEs*, whereas problem 4 relates to *neural operators*. Problem 1 relates to a standard "forward" formulation. Problems 2-4 to "mixed" formulations, where both solution and parametric partially known fields, e.g., $\lambda(x)$, are sought. We refer to functions sampled at random finite locations containing *aleatoric uncertainty* as "noisy data". Similarly, we refer to stochastic realizations sampled at random finite locations containing aleatoric uncertainty as "noisy realizations". Operator learning consists of two phases, namely, pre-training and inference, as explained in Section 1.2; see also Table 11.

Considered problem scenarios corresponding to the following equation: $\mathcal{F}_{\lambda,\xi}[u(x;\xi)] = f(x;\xi),\ x \in \Omega,\ \xi \in \Xi$ and $\mathcal{B}_{\lambda,\xi}[u(x;\xi)] = b(x;\xi),\ x \in \Gamma$			
1) Forward deterministic PDE problem		***2) Mixed deterministic PDE problem***	
$\mathcal{F}_{\lambda,\xi}, \mathcal{B}_{\lambda,\xi}$	deterministic (fixed ξ) and known	$\mathcal{F}_{\lambda,\xi}, \mathcal{B}_{\lambda,\xi}$	deterministic (fixed ξ) and known
$f(x), b(x)$	noisy data	$f(x), b(x)$	noisy data
$u(x)$	unknown	$u(x)$	partially unknown (noisy data)
$\lambda(x)$	known	$\lambda(x)$	partially unknown (noisy data)
Dataset	$\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b\}$, where $\mathcal{D}_f = \{x_i, f_i\}_{i=1}^{N_f}$, $\mathcal{D}_b = \{x_i, b_i\}_{i=1}^{N_b}$	Dataset	$\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b, \mathcal{D}_u, \mathcal{D}_\lambda\}$, where $\mathcal{D}_f = \{x_i, f_i\}_{i=1}^{N_f}$, $\mathcal{D}_b = \{x_i, b_i\}_{i=1}^{N_b}$ $\mathcal{D}_u = \{x_i, u_i\}_{i=1}^{N_u}$, $\mathcal{D}_\lambda = \{x_i, \lambda_i\}_{i=1}^{N_\lambda}$
Objective	$u(x)$, $\forall x \in \Omega$	Objective	$u(x), \lambda(x)$, $\forall x \in \Omega, \Omega_\lambda$
3) Mixed stochastic PDE problem		***4) Operator learning problem***	
$\mathcal{F}_{\lambda,\xi}, \mathcal{B}_{\lambda,\xi}$	stochastic ($\xi \in \Xi$) and known	$\mathcal{F}_{\lambda,\xi}, \mathcal{B}_{\lambda,\xi}$	stochastic ($\xi \in \Xi$) and unknown
$f(x;\xi), b(x;\xi)$	noisy realizations	$f(x;\xi), b(x;\xi)$	partially unknown (clean realizations)
$u(x;\xi)$	partially unknown (noisy realizations)	$u(x;\xi)$	partially unknown (noisy realizations)
$\lambda(x;\xi)$	partially unknown (noisy realizations)	$\lambda(x;\xi)$	partially unknown (clean realizations)
Dataset	$\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b, \mathcal{D}_u, \mathcal{D}_\lambda\}$, where $\mathcal{D}_f = \{F_i\}_{i=1}^{N}$, $\mathcal{D}_b = \{B_i\}_{i=1}^{N}$ $\mathcal{D}_u = \{U_i\}_{i=1}^{N}$, $\mathcal{D}_\lambda = \{\Lambda_i\}_{i=1}^{N}$ $F_i = \{x_j^{(i)}, f_j^{(i)}\}_{j=1}^{N_f}$, $B_i = \{x_j^{(i)}, b_j^{(i)}\}_{j=1}^{N_b}$ $U_i = \{x_j^{(i)}, u_j^{(i)}\}_{j=1}^{N_u}$, $\Lambda_i = \{x_j^{(i)}, \lambda_j^{(i)}\}_{j=1}^{N_\lambda}$	Dataset (pre-training)	$\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b, \mathcal{D}_u, \mathcal{D}_\lambda\}$, where $\mathcal{D}_f = \{F_i\}_{i=1}^{N}$, $\mathcal{D}_b = \{B_i\}_{i=1}^{N}$ $\mathcal{D}_u = \{U_i\}_{i=1}^{N}$, $\mathcal{D}_\lambda = \{\Lambda_i\}_{i=1}^{N}$ $F_i = \{x_j^{(i)}, f_j^{(i)}\}_{j=1}^{N_f}$, $B_i = \{x_j^{(i)}, b_j^{(i)}\}_{j=1}^{N_b}$ $U_i = \{x_j^{(i)}, u_j^{(i)}\}_{j=1}^{N_u}$, $\Lambda_i = \{x_j^{(i)}, \lambda_j^{(i)}\}_{j=1}^{N_\lambda}$
Objective	statistics of $u(x), \lambda(x)$, $\forall x \in \Omega, \Omega_\lambda$	Objective	$f(x;\xi'), b(x;\xi'), u(x;\xi'), \lambda(x;\xi')$, $\forall x \in \Omega, \Gamma, \Omega, \Omega_\lambda$, given inference data
		Inference data	$\mathcal{D}' = \{\mathcal{D}'_f, \mathcal{D}'_b, \mathcal{D}'_u, \mathcal{D}'_\lambda\}$, where $\mathcal{D}'_f = \{x_i, f_i\}_{i=1}^{N'_f}$, $\mathcal{D}'_b = \{x_i, b_i\}_{i=1}^{N'_b}$ $\mathcal{D}'_u = \{x_i, u_i\}_{i=1}^{N'_u}$, $\mathcal{D}'_\lambda = \{x_i, \lambda_i\}_{i=1}^{N'_\lambda}$

$\mathcal{D}'_\lambda = \{x_i, \lambda_i\}_{i=1}^{N'_\lambda}$. Finally, note that a special case of inference data is where f, b and λ data are available at the complete set of locations used in pre-training, and the only sought outcome is the solution u. See also Table 11 in the comparative study for an overview of problem cases specific to operator learning.

1.3. Novel contributions of this work

In addition to the review and the comparative study we perform in this work, we also present several original contributions summarized below:

1. We test and integrate into physics-informed neural networks, neural operators, and SPDE solvers various methods for posterior inference, prior learning, data noise modeling, as well as for post-training calibration.
2. We demonstrate how to solve function approximation problems with heteroscedastic noise given historical data by using functional priors.
3. We propose a new technique for solving the deterministic forward PDE problem by combining Gaussian process regression with generative adversarial networks and compare with existing methods.
4. We solve mixed PDE problems with heteroscedastic noise in the source term, the problem parameters, and the solution data.
5. We solve mixed SPDE problems given noisy stochastic realizations, and we propose a new NN architecture for quantifying uncertainty using polynomial chaos.
6. We demonstrate how to deal with noisy and incomplete inference data given a pre-trained neural operator on clean data.
7. We propose methods to detect out-of-distribution data in neural operators, which is critical for risk-related applications.
8. We present a formulation towards a unified UQ framework that addresses diverse problems in scientific machine learning, by seamlessly combining physics with new and historical data, which could be contaminated with various types of noise.

1.4. Organization of the paper

We organize the paper as follows. In Section 2 we briefly present the PINN and DeepONet methods, which are the building blocks of all the SciML techniques we discuss in this paper. In Section 3 and the corresponding Appendix A, we

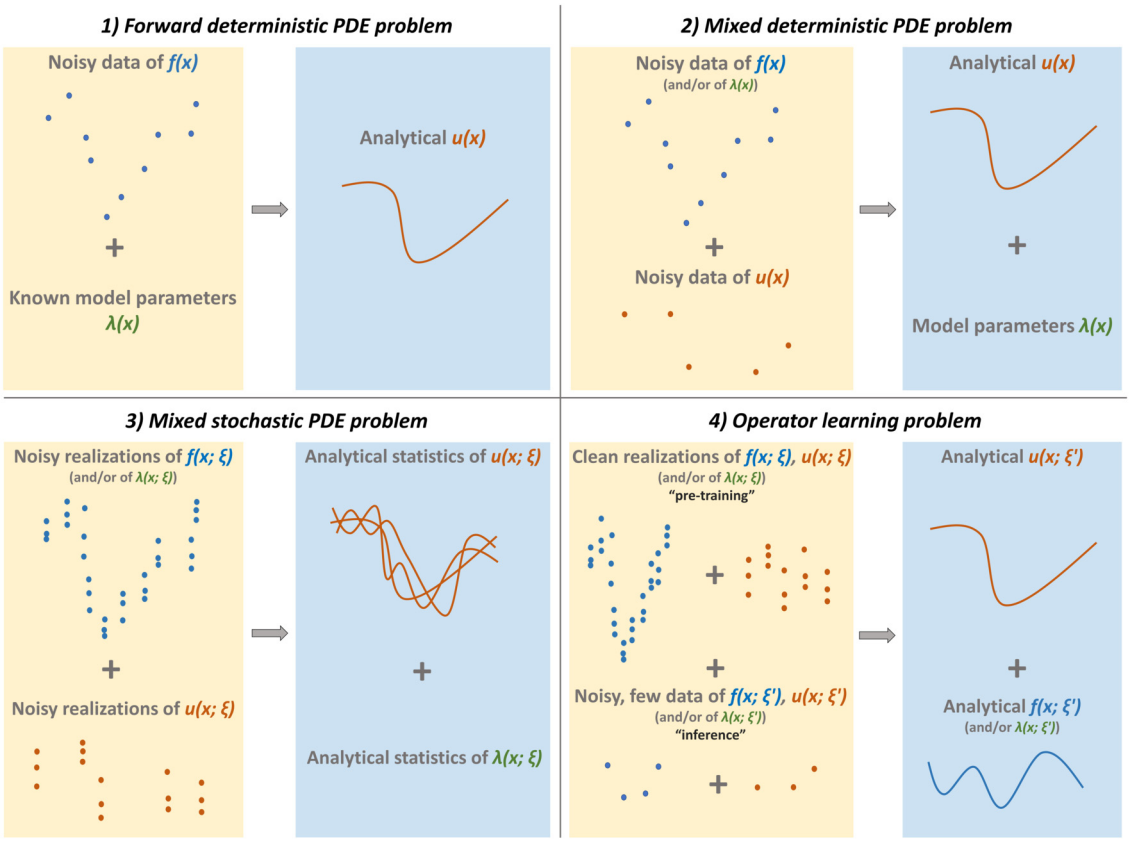


Fig. 1. Inputs (orange panels) - Outputs (blue panels) of the problems described in Table 1 corresponding to equation $\mathcal{F}_{\lambda,\xi}[u(x;\xi)] = f(x;\xi)$, $x \in \Omega$, $\xi \in \Xi$ and $\mathcal{B}_{\lambda,\xi}[u(x;\xi)] = b(x;\xi)$, $x \in \Gamma$. Problems 1-3 relate to *neural (S)PDEs*, whereas problem 4 relates to *neural operators*. Problem 1 relates to a standard "forward" formulation. Problems 2-4 to "mixed" formulations, where both solution and parametric partially known fields, e.g., $\lambda(x)$, are sought. We refer to outputs as "analytical" (or "analytical statistics") because they are expressed via a neural network for the corresponding entire continuous space-time domain. We refer to functions sampled at random finite locations containing *aleatoric uncertainty* as "noisy data". Similarly, we refer to stochastic realizations sampled at random finite locations containing aleatoric uncertainty as "noisy realizations". Operator learning consists of two phases, namely, pre-training and inference, as explained in Section 1.2; see also Table 11. Lastly, b is either known or treated similarly to f. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

develop an uncertainty modeling framework that includes data noise modeling, review of standard NN training, predictive distributions based on posterior samples, priors and hyperpriors, addressing model misspecification, as well as uncertainty modeling in PINN and DeepONet methods. In Section 4 and the corresponding Appendix B, we present two families of posterior inference methods, namely Bayesian methods and ensembles, we present functional priors for reducing the required amount of data and physics knowledge, as well as techniques for solving stochastic differential equations. In Section 4.5, we develop a unified model that represents all the problems of Table 1 and Fig. 1 and includes all solution methods in this paper as special cases. In Section 5 and the corresponding Appendix C, we propose a set of quality evaluation metrics for the discussed methods and we review and integrate three post-hoc quality improvement approaches into SciML. In Section 6 and the corresponding Appendix D, we perform a comparative study including a function approximation problem as well as various problem cases of Table 1 and Fig. 1. Lastly, in Section 7, we discuss the findings of this work and future research directions.

2. Neural PDEs and neural operators

In this paper, we obtain PDE solutions and operator mapping approximations that are endowed with uncertainty. In this regard, we solve the problems of Table 1 and Fig. 1 by combining available data with physical knowledge. In this section, we summarize the vanilla PINN and DeepONet methods for solving PDEs and learning operators, respectively. Because both methods utilize NNs, we refer to the former as neural PDEs and to the latter as neural operators. These are the building blocks for several extensions that we discuss in Section 4, such as functional priors for reducing the required amount of data and physics knowledge, as well as methods for solving SPDEs. A glossary of various terms we use in this section is provided in Table 15.

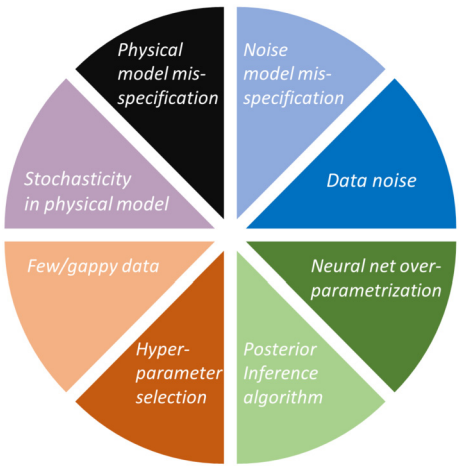


Fig. 2. A qualitative breakdown of *total* uncertainty describing the contributions from data (noisy, gappy); physical models (misspecification, stochasticity); neural networks (architecture, hyperparameters, overparametrization); and posterior inference. Aleatoric uncertainty is due to noisy data and cannot be reduced. Epistemic uncertainty is due to noisy and limited data, as well as neural network overparametrization. Note that all uncertainties in this figure are either modeled, addressed, or discussed in this work, except for model error due to physical model misspecification.

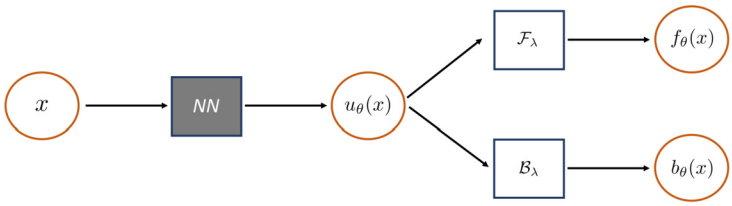


Fig. 3. The PINN architecture. PINNs consist of three NN approximators u_θ, f_θ, and b_θ that are connected via both the operators $\mathcal{F}_\lambda$ and $\mathcal{B}_\lambda$ of Eq. (1) and parameter sharing, i.e., $f_\theta = \mathcal{F}_\lambda[u_\theta]$ and $b_\theta = \mathcal{B}_\lambda[u_\theta]$, where θ are the shared parameters. The NN approximators u_θ, f_θ, and b_θ are trained simultaneously because of parameter sharing to fit the complete dataset. The model depicted in this figure can be construed as a special case of Fig. 10.

2.1. Solving forward and mixed PDE problems: overview of PINN method

The PINN method, developed by Raissi et al. [13] (first published in arxiv in 2017), addresses the forward deterministic PDE problem by constructing a NN approximator, parametrized by θ, for modeling the solution $u(x)$. The approximator $u_\theta(x)$ is substituted into Eq. (1) via authomatic differentiation for producing $f_\theta = \mathcal{F}_\lambda[u_\theta]$ and $b_\theta = \mathcal{B}_\lambda[u_\theta]$. PINN modeling for forward problems is illustrated in Fig. 3. Further, the NN parameters θ are optimized for fitting the dataset $\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b\}$ using the mean squared error (MSE) loss. For solving the mixed deterministic PDE problem, an additional NN can be constructed for modeling $\lambda(x)$. For notation simplicity, the parameters of both NNs are denoted by θ. Finally, the optimization problem for obtaining θ in the mixed problem scenario with the dataset $\mathcal{D} = \{\mathcal{D}_f, \mathcal{D}_b, \mathcal{D}_u, \mathcal{D}_\lambda\}$ is cast as

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\, \mathcal{L}(\theta), \text{ where } \mathcal{L}(\theta) = \frac{w_f}{N_f}\sum_{i=1}^{N_f} ||f_\theta(x_i) - f_i||_2^2 + \frac{w_b}{N_b}\sum_{i=1}^{N_b} ||b_\theta(x_i) - b_i||_2^2 + \frac{w_u}{N_u}\sum_{i=1}^{N_u} ||u_\theta(x_i) - u_i||_2^2 + \frac{w_\lambda}{N_\lambda}\sum_{i=1}^{N_\lambda} ||\lambda_\theta(x_i) - \lambda_i||_2^2. \quad (2)$$

In this equation, $\{w_f, w_b, w_u, w_\lambda\}$ are objective function weights for balancing the various terms in Eq. (2). The interested reader is directed to [129–131] for online and offline adaptive weight techniques. Following determination of $\hat{\theta}$, $u_{\hat{\theta}}$ and $\lambda_{\hat{\theta}}$ can be evaluated at any x in Ω, Ω_λ, respectively.

2.2. Learning operator mappings: overview of DeepONet method

The DeepONet method, developed by Lu et al. [7] (first published in arxiv in 2019), addresses the operator learning problem by constructing a NN approximator, parametrized by θ, for modeling the solution $u(x;\xi)$. In the following and without loss of generality, we consider f and b in Eq. (1) as known. Further, each value of $\xi \in \Xi$ in Eq. (1) corresponds to a different $\lambda(x;\xi)$ function, and thus a different solution $u(x;\xi)$. The NN $u_\theta(x;\xi)$ takes as input the $\lambda_j^{(i)}$ values from the

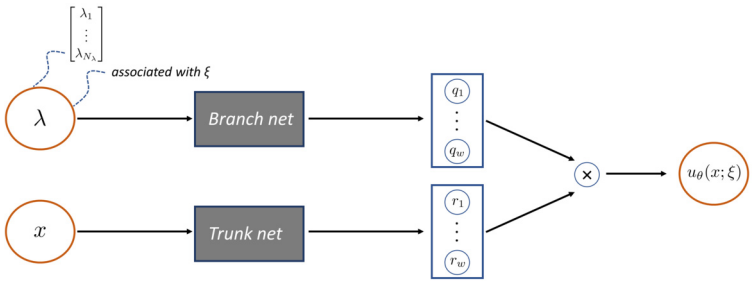


Fig. 4. The DeepONet architecture corresponding to output $u_\theta(x;\xi)$, where ξ is associated with an input function $\lambda(x;\xi)$. It consists of two sub-networks, one for the input x where the solution u is evaluated, called trunk net, and one for the input λ (a vector of size N_λ), called branch net. The vectors $[q_1,\dots,q_w]$ and $[r_1,\dots,r_w]$ denote the outputs of the branch and trunk nets, respectively, and "$\times$" denotes their inner product. The model depicted in this figure can be construed as a special case of Fig. 10.

sub-dataset $\Lambda_i = \{x_j^{(i)}, \lambda_j^{(i)}\}_{j=1}^{N_\lambda}$. The locations $\{x_j^{(i)}\}_{j=1}^{N_\lambda}$ are considered the same for all i values, without loss of generality. Inspired by the universal approximation theorem for operators, the DeepONet architecture consists of two sub-networks, one for the input x, called trunk net, and one for the input λ, called branch net. Considering a one-dimensional output $u_\theta(x;\xi)$, i.e., $D_u = 1$, each of the two networks produces a w-dimensional output. The two outputs are merged by computing their inner product for producing the final prediction $u_\theta(x;\xi)$; see Fig. 4. For learning the operator mapping from λ to u with the DeepONet method, the minimization problem with the MSE loss

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\, \mathcal{L}(\theta), \text{ where } \mathcal{L}(\theta) = \frac{1}{NN_u}\sum_{i=1}^{N}\sum_{j=1}^{N_u} ||u_\theta(x_j^{(i)};\xi_i) - u_j^{(i)}||_2^2 \tag{3}$$

is solved. We refer to this procedure as pre-training or just training, depending on whether the inference data is limited and noisy, and requires further processing, or clean and can be directly used for inference. Specifically, during inference for a new value $\xi' \in \Xi$, the vanilla DeepONet considers a clean dataset $\mathcal{D}'_\lambda = \{x_i, \lambda_i\}_{i=1}^{N'_\lambda}$. If the location set $\{x_i\}_{i=1}^{N'_\lambda}$ is not the same as the locations $\{x_j^{(i)}\}_{j=1}^{N_\lambda}$ used in pre-training, interpolation must be performed because the locations of the input λ to DeepONet are fixed; see Table 11 for an overview of different cases. The corresponding solution $u(x;\xi')$ can be obtained for each required $x \in \Omega$ by performing a forward DeepONet pass. Finally, note that in addition to supervised learning using data as described above, DeepONet can alternatively be trained by employing only the PINN loss during training. As shown in [8], the most effective training is the hybrid one, where both physics and data are used in the training loss; see also [9].

3. Modeling total uncertainty

In this section, we first show how to model and propagate total uncertainty in function approximation problems. Total uncertainty, which can be construed as the sum of the uncertainties depicted in Fig. 2, is quantified in this section using both a distribution, e.g., for obtaining confidence intervals, and first- as well as second-order statistics. Further, we integrate these concepts into SciML. A glossary of various terms we use in this section is provided in Table 15. The interested reader is also directed to Appendix A for supplementary material to this section.

3.1. Uncertainty in function approximation

Given a set of paired noisy observations $\mathcal{D} = \{x_i, u_i\}_{i=1}^{N}$, our goal is to construct a distribution $p(u|x,\mathcal{D},\mathcal{H})$ for predicting the value of u at any new location x. Although for brevity $\mathcal{H}$ is henceforth dropped, unless considered necessary, we emphasize that all probabilistic distributions discussed in this paper encapsulate a set of user assumptions and preferences (e.g., NN architecture, likelihood function, etc.) that we collectively denote by $\mathcal{H}$. For each x, it is commonly assumed that $u(x)$ produced by the data-generating process contains both a deterministic part $u_c(x)$ as well as some additive noise ϵ_u, i.e., $u(x) = u_c(x) + \epsilon_u$. This noise is typically called *data or aleatoric uncertainty*. For modeling this process, we assume a *likelihood function* denoted by $p(u|x,\theta)$, with parameters $\theta = [\theta_1,\dots,\theta_K]^T$ to be inferred from the available data. Further, for defining $p(u|x,\theta)$ we construct a model $u_\theta(x)$ that captures the deterministic part of $u(x)$ at some location x and assume a model for the noise. For example, the factorized Gaussian likelihood function, given as

$$p(u|x,\theta) = \mathcal{N}(u|u_\theta(x), diag(\Sigma_u^2)) = \prod_{d=1}^{D_u} \frac{1}{\sqrt{2\pi\sigma_u^2}} \exp(-\frac{(u_d - u_\theta(x)_d)^2}{2\sigma_u^2}), \tag{4}$$

with the subscript d denoting each of the D_u dimensions of u, is often used for multi-dimensional function approximation problems. In Eq. (4), the output vector $u_\theta(x)$ is the mean of the Gaussian distribution assumed for u at location x, $diag(\Sigma_u^2)$

is a diagonal covariance matrix with $\Sigma_u^2 = [\sigma_u^2, \dots, \sigma_u^2]$ in the diagonal, and σ_u^2 may either be known or assumed, or inferred from data. Furthermore, in Sections A.4 and D.2.2 we describe and apply, respectively, posterior tempering for addressing model misspecification cases where, for example, the employed likelihood function is incorrect.

The value of u at a location x given the data $\mathcal{D}$ is a random variable denoted by $(u|x, \mathcal{D})$. To obtain the density of $(u|x, \mathcal{D})$ we integrate out the model parameters θ, i.e.,

$$p(u|x, \mathcal{D}) = \int p(u|x, \theta) p(\theta|\mathcal{D}) d\theta. \tag{5}$$

This predictive probability density function (PDF) is commonly referred to as Bayesian model average (BMA). In Eq. (5), $p(\theta|\mathcal{D})$ denotes the *posterior density*, which is a distribution over the "plausible" parameter values based on the data. The uncertainty regarding the NN parameters θ is typically called *epistemic uncertainty*. Using Bayes' rule, the posterior $p(\theta|\mathcal{D})$ can be obtained via

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta) p(\theta)}{p(\mathcal{D})}. \tag{6}$$

In Eq. (6), $p(\mathcal{D}|\theta)$ is the likelihood of the data, i.e., $p(\mathcal{D}|\theta) = \prod_{i=1}^N p(u_i|x_i, \theta)$ for independent and identically distributed (i.i.d.) data; $p(\theta)$ is the prior probability of the parameters θ as defined by the model $\mathcal{H}$; and $p(\mathcal{D})$ is called *marginal likelihood* or *evidence* because it represents the probability that out of all possible datasets modeled by $\mathcal{H}$, it "happens" that we observe $\mathcal{D}$. The evidence $p(\mathcal{D})$ is given as

$$p(\mathcal{D}) = \int p(\mathcal{D}|\theta) p(\theta) d\theta, \tag{7}$$

i.e., given random samples drawn from the prior $p(\theta)$ and then used in conjuction with the likelihood function, $p(\mathcal{D})$ represents the probability that the dataset $\mathcal{D}$ is produced.

Obtaining the posterior exactly via Eq. (6) is computationally and analytically intractable. To address this issue, approximate inference aims to approximate the posterior by another distribution and/or obtaining samples from the posterior. "Standard NN training", which can be construed as approximating the posterior by a point estimate as described in Section A.2, does not take into account epistemic uncertainty. In this regard, in Sections 4.1-4.2 we review and integrate into SciML the most commonly used posterior inference approaches. All methods obtain a set of θ samples denoted as $\{\hat{\theta}_j\}_{j=1}^M$. Following the posterior inference phase, the BMA of Eq. (5) can be approximated using Monte Carlo (MC) estimation as

$$p(u|x, \mathcal{D}) = \mathbb{E}_{\theta|\mathcal{D}}[p(u|x, \theta)] \approx \bar{p}(u|x) = \frac{1}{M} \sum_{j=1}^M p(u|x, \hat{\theta}_j). \tag{8}$$

This equation provides the approximate *total uncertainty* of $(u|x, \mathcal{D})$ in the form of a predictive PDF $\bar{p}(u|x)$; see Fig. 5 for an illustration. The mean of $(u|x, \mathcal{D})$ is modeled via $\hat{u}(x) = \mathbb{E}[u|x, \mathcal{D}]$ of Eq. (A.1) and approximated by $\bar{\mu}(x)$ as

$$\hat{u}(x) \approx \bar{\mu}(x) = \frac{1}{M} \sum_{j=1}^M u_{\hat{\theta}_j}(x), \tag{9}$$

where $\{u_{\hat{\theta}_j}(x)\}_{j=1}^M$ is the set of NN predictions corresponding to the samples $\{\hat{\theta}_j\}_{j=1}^M$. For obtaining the variance of $(u|x, \mathcal{D})$, substituting the Gaussian likelihood of Eq. (4) into Eq. (8) yields a Gaussian mixture with the diagonal part of its covariance matrix derived analytically and given by

$$Var(u|x, \mathcal{D}) \approx \bar{\sigma}^2(x) = \underbrace{\Sigma_u^2}_{\bar{\sigma}_a^2(x)} + \underbrace{\frac{1}{M} \sum_{j=1}^M (u_{\hat{\theta}_j}(x) - \bar{\mu}(x))^2}_{\bar{\sigma}_e^2(x)}. \tag{10}$$

In this equation, $\bar{\sigma}^2(x)$ represents the approximate total uncertainty of $(u|x, \mathcal{D})$, while $\bar{\sigma}_a^2(x)$ and $\bar{\sigma}_e^2(x)$ denote the aleatoric and the epistemic parts of the total uncertainty, respectively. Note that up to a constant $\frac{M}{M-1}$, the term $\bar{\sigma}_e^2(x)$ is equal to the sample variance of $\{u_{\hat{\theta}_j}(x)\}_{j=1}^M$. An alternative derivation of Eq. (10) based on the law of total variance can be found in Eq. (A.2). Overall, after obtaining $\{\hat{\theta}_j\}_{j=1}^M$, a predictive model in the form of a PDF $\bar{p}(u|x)$ via Eq. (8) is available for each value of x; see Fig. 6. The corresponding cumulative distribution function (CDF) denoted by $\bar{P}(u|x)$ can also be obtained in a straightforward manner. In practice, however, instead of using Eq. (8), a Gaussian (or, e.g., Student-t) distribution is commonly fitted to the posterior samples and used for both PDF and CDF evaluations. For example, we denote as $\mathcal{N}(\bar{\mu}(x), \bar{\sigma}^2(x))$ the Gaussian approximation to the samples $\{u_{\hat{\theta}_j}(x)\}_{j=1}^M$ at an arbitrary location x, where $\bar{\mu}(x)$ and $\bar{\sigma}^2(x)$ are obtained via

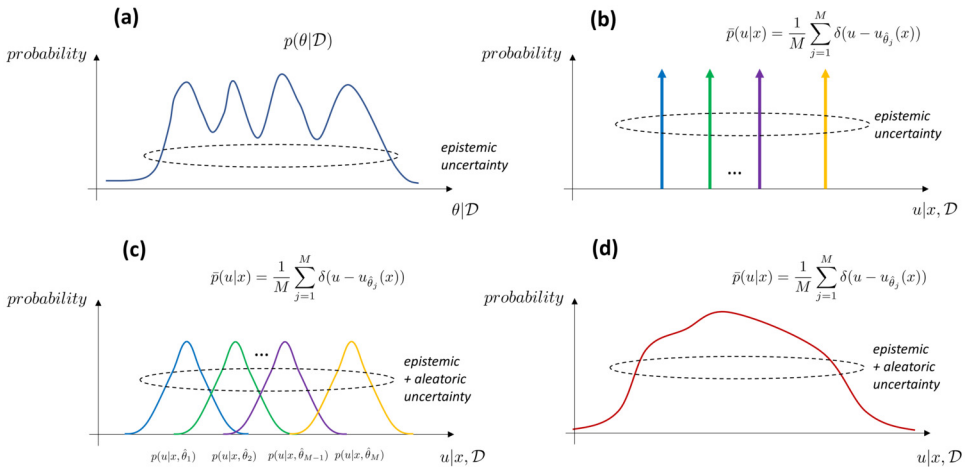


Fig. 5. Total uncertainty represented by the predictive distribution $\bar{p}(u|x)$: **(a)** Epistemic uncertainty is taken into account via the posterior $p(\theta|\mathcal{D})$, **(b)** $p(\theta|\mathcal{D})$ is approximated via representative samples $\{\hat{\theta}_j\}_{j=1}^M$, **(c)** aleatoric uncertainty is added to each sample from $p(\theta|\mathcal{D})$ via the distributions $\{p(u|x,\hat{\theta}_j)\}_{j=1}^M$, **(d)** $\bar{p}(u|x)$ approximates total uncertainty by adding up the distributions $\{p(u|x,\hat{\theta}_j)\}_{j=1}^M$. If each $p(u|x,\hat{\theta}_j)$ is a Gaussian distribution, $\bar{p}(u|x)$ corresponds to a Gaussian mixture. See Table 15 for a glossary of used terms.

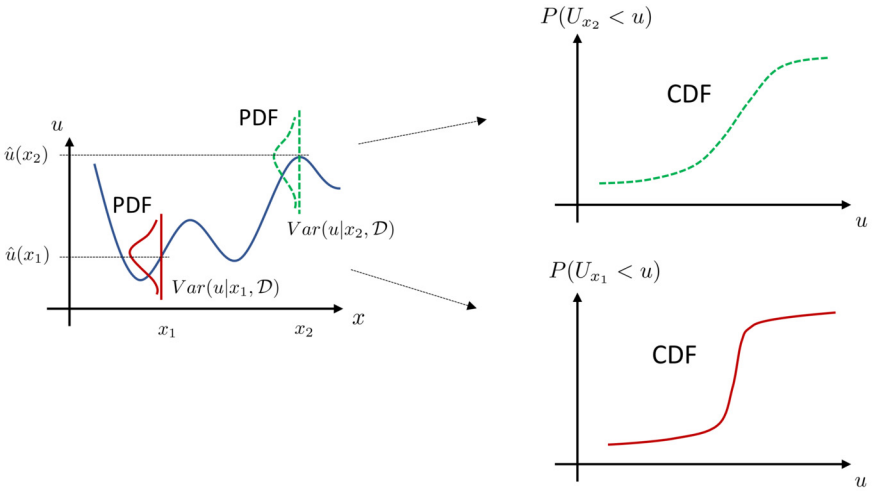


Fig. 6. Predictions for u corresponding to two arbitrary x locations, x_1 (red solid lines) and x_2 (green dashed lines): given data $\mathcal{D}$, all techniques presented in this paper output a predictive probability density function (PDF), or equivalently a cumulative distribution function (CDF), for each value of x via Eq. (5). The most straightforward way to make PDF/CDF evaluations is to fit a Gaussian (or, e.g., Student-t) distribution to the posterior samples $\{u_{\hat{\theta}_j}(x)\}_{j=1}^M$ for each x. First- and second-order statistics can be approximated via Eqs. (9) and (10), respectively. Note that, given left-out data referred to as calibration dataset, the post-training calibration techniques presented in Section 5.2 modify the predictive PDFs/CDFs to improve statistical consistency of the predictions. See Table 15 for a glossary of used terms.

Eqs. (9) and (10), respectively. In passing, note that similarly to computing marginal distribution statistics for each x separately, finite sets of x locations can be considered for computing joint statistics and confidence intervals; see, e.g., [132]. Information about joint predictive distributions can be found in [93].

For the Gaussian likelihood of Eq. (4), a homoscedastic data noise model has been considered so far in this paper, with a diagonal covariance matrix and constant, in terms of x, variance σ_u^2 for each dimension of $u(x)$. An extension that accounts for heteroscedastic noise relates to considering a location-dependent variance vector Σ_u^2. Specifically, Σ_u^2 can be modeled as part of the NN output by augmenting the NN output $u_\theta(x)$ by $\Sigma_{u,\theta}^2(x) = [\sigma_{u,\theta}^2(x)_1, \ldots, \sigma_{u,\theta}^2(x)_{D_u}]$, where $\sigma_{u,\theta}^2(x)$ can also be shared among dimensions (see also [43]). For making predictions given posterior samples $\{\hat{\theta}_j\}_{j=1}^M$, each $p(u|x,\hat{\theta}_j)$ in Eq. (8) not only has a different mean, but also a different variance, and Eq. (10) becomes

$$Var(u|x,\mathcal{D}) \approx \bar{\sigma}^2(x) = \underbrace{\frac{1}{M}\sum_{j=1}^{M}\Sigma_{u,\hat{\theta}_j}^2(x)}_{\bar{\sigma}_a^2(x)} + \underbrace{\frac{1}{M}\sum_{j=1}^{M}(u_{\hat{\theta}_j} - \bar{\mu}(x))^2}_{\bar{\sigma}_e^2(x)}. \tag{11}$$

Similarly, we can obtain the complete covariance matrix of $(u|x,\mathcal{D})$; see, e.g., [133].

3.2. Uncertainty in PINNs

The observations in the mixed PDE problem dataset $\mathcal{D}=\{\mathcal{D}_f,\mathcal{D}_b,\mathcal{D}_u,\mathcal{D}_\lambda\}$ are typically obtained with a data acquisition device and are noisy. In this regard, we employ four likelihood functions for approximating the f,b,u and λ distributions, denoted as $p(f|x,\theta)$, $p(b|x,\theta)$, $p(u|x,\theta)$, and $p(\lambda|x,\theta)$, respectively. The total likelihood of the dataset $\mathcal{D}$ becomes

$$p(\mathcal{D}|\theta)=p(\mathcal{D}_f|\theta)p(\mathcal{D}_b|\theta)p(\mathcal{D}_u|\theta)p(\mathcal{D}_\lambda|\theta), \tag{12}$$

where

$$\begin{aligned} p(\mathcal{D}_f|\theta)&=\prod_{i=1}^{N_f}p(f_i|x_i,\theta); \quad p(\mathcal{D}_b|\theta)=\prod_{i=1}^{N_b}p(b_i|x_i,\theta);\\ p(\mathcal{D}_u|\theta)&=\prod_{i=1}^{N_u}p(u_i|x_i,\theta); \quad p(\mathcal{D}_\lambda|\theta)=\prod_{i=1}^{N_\lambda}p(\lambda_i|x_i,\theta). \end{aligned} \tag{13}$$

Combining point estimation of θ (Section A.2) with the Gaussian likelihood function of Eq. (4) for all data categories leads to the PINN minimization problem of Eq. (2). The noise scales σ_u^2 in the assumed Gaussian likelihoods are in general different for each data category and are either known or learned together with θ during optimization. The PDF of the solution random variable $(u|x,\mathcal{D})$ contains only aleatoric uncertainty and is given as $p(u|x,\hat{\theta})$, where $\hat{\theta}$ is the optimal θ. For accounting for epistemic uncertainty as well, posterior inference (Sections 4.1-4.2) is required for obtaining multiple θ samples. In this case, the PDF of the solution random variable $(u|x,\mathcal{D})$ contains both epistemic and aleatoric uncertainties and is computed via Eq. (8).

3.3. Uncertainty in DeepONets

As in Section 2.2, we assume that f and b in Eq. (1) are known. Next, we assume that the dataset $\mathcal{D}_\lambda$ consists of clean data. Because the data of λ is used as input to DeepONet (Fig. 4), it is not straightforward to consider noisy data during the pre-training procedure; see, e.g., [134] for accounting for input uncertainty in standard NNs. Nevertheless, we assume that $\mathcal{D}_u$ consists of noisy data in general. In this regard, a likelihood function is employed for the u data, denoted by $p(u|x,\xi,\theta)$, and the total likelihood is given as

$$p(\mathcal{D}|\theta)=\prod_{i=1}^{N}\prod_{j=1}^{N_u}p(u_j^{(i)}|x_j^{(i)},\xi_i,\theta). \tag{14}$$

Combining point estimation of θ (Section A.2) with the Gaussian likelihood function of Eq. (4) for the u data leads to the DeepONet minimization problem of Eq. (3). The noise scale σ_u^2 in the assumed Gaussian likelihood is either known or learned together with θ during optimization. During inference for a new value $\xi'\in\Xi$, the vanilla DeepONet considers a clean dataset $\mathcal{D}'_\lambda=\{x_i,\lambda_i\}_{i=1}^{N'_\lambda}$; see Section 4.3 for overcoming this limitation. The PDF of the solution random variable $(u|x,\xi',\mathcal{D})$ contains only aleatoric uncertainty and is given as $p(u|x,\xi',\hat{\theta})$, where $\hat{\theta}$ is the optimal θ. For accounting for epistemic uncertainty as well, posterior inference (Sections 4.1-4.2) is required for obtaining multiple θ samples. In this case, the PDF of the solution random variable $(u|x,\xi',\mathcal{D})$ contains both epistemic and aleatoric uncertainties and is computed via Eq. (8). Note that concurrently with the present work, Markov chain Monte Carlo has been employed for posterior inference in conjunction with DeepONet-based operator learning [117].

4. Methods for uncertainty quantification

Following the uncertainty modeling presented in Section 3, a solution method is required for obtaining a set of NN parameter posterior samples $\{\hat{\theta}_j\}_{j=1}^M$. These samples are used in the approximate BMA of Eqs. (8) and (9)-(10) for obtaining total uncertainty predictions in the form of distribution and first-/second-order statistics, respectively. In this section, we briefly review and integrate into SciML the most commonly used methods for obtaining $\{\hat{\theta}_j\}_{j=1}^M$; see Table 2 for an overview. Next, we demonstrate how to extend these methods for improving predictions when historical data is available, as well as for representing stochastic processes and solving stochastic differential equations. Finally, we present a formulation towards a unified UQ framework that addresses the problems of Fig. 1 by seamlessly combining physics with novel and historical data, which could be contaminated with various types of noise. A glossary of various terms we use in this section is provided in Table 15. The interested reader is also directed to Appendix B for supplementary material to this section, including thorough descriptions of the presented methods. Note that, although partial, the list of methods summarized in Table 2 and described in Appendix B includes the most established and relevant to SciML UQ methods.

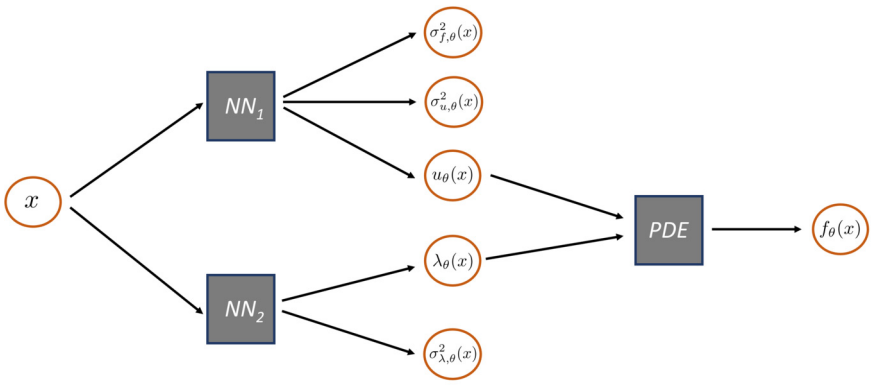


Fig. 7. PINN consisting of two NNs, one for the solution u and one for the parameter field λ. The parameters of the NNs are denoted by θ. We also consider heteroscedastic Gaussian noise in the data of f, u, and λ, whose variances are denoted by $\sigma^2_{f,\theta}$, $\sigma^2_{u,\theta}$, and $\sigma^2_{\lambda,\theta}$, respectively. We refer to this model as h-U-PINN in Table 2; i.e., PINN with heteroscedastic noise modeling and parameters θ obtained through posterior inference. Related computational results can be found in Section 6.2.2. The model depicted in this figure can be construed as a special case of Fig. 10 with additional NN outputs for the noise variances.

4.1. Bayesian methods

Bayesian methods approximate the posterior $p(\theta|\mathcal{D})$ of Eq. (6), either implicitly or explicitly. In the following, we provide an overview of the most widely used Bayesian methods (see also Table 2), as well as brief information regarding prior/data uncertainty optimization and integration into SciML. A detailed description of the considered methods can be found in Section B.2.

Markov-chain Monte Carlo (MCMC) techniques, reviewed in Section B.2.1, draw parameter θ samples from the posterior by using a Markov chain with $p(\theta|\mathcal{D})$ as its invariant, i.e., stationary, distribution. We employ two such methods, namely Hamiltonian Monte Carlo (HMC) and Langevin Dynamics (LD), in the comparative study of Section 6. Variational inference (VI) techniques, reviewed in Section B.2.2, approximate $p(\theta|\mathcal{D})$ by employing a variational distribution $q_\omega(\theta)$ parametrized by ω, that is optimized by maximizing a lower bound of the marginal likelihood of Eq. (7). Mean-field VI (MFVI) utilizes a factorized Gaussian variational distribution. Monte Carlo dropout (MCD), which is reviewed in Section B.2.2 and can be interpreted as a special case of VI, first requires standard training with dropout. Subsequently, at inference time θ samples are drawn by randomly dropping for each sample a fixed percentage of the NN parameters. Laplace approximation (LA), reviewed in Section B.2.3, involves standard NN training, and subsequently fitting a Gaussian distribution to the discovered mode $\hat{\theta}$, with a width determined by the Hessian of the training error.

For the techniques considered in this section, evaluations of the likelihood and prior terms, $p(\mathcal{D}|\theta)$ and $p(\theta)$, respectively, for different values of θ are required. In the PINN method, the required likelihood is given by Eqs. (12)-(13), whereas in the DeepONet method by Eq. (14). Evaluating the prior $p(\theta)$ does not differ from standard function approximation in either PINN or DeepONet cases. The obtained set of samples $\{\hat{\theta}_j\}_{j=1}^M$ is subsequently combined with the approximate BMAs in Eqs. (8)-(10). In this paper, we refer to PINN and DeepONet combined with Bayesian methods as U-PINN and U-DeepONet, where U stands for "uncertain".

For the case of Section A.3 pertaining to unknown prior and noise model, the considered Bayesian methods can be combined with alternating optimization schemes. We construe these approaches as *online learning methods*. Further, cross-validation or meta-learning, in conjunction with the evaluation metrics of Section 5 and validation data, can be utilized. These approaches are construed as *offline learning methods*. The interested reader can find detailed descriptions in Sections B.2.1, B.2.2, and B.2.3, pertaining to MCMC, VI, and LA, respectively. Lastly, aleatoric uncertainty can alternatively be considered as an additional output of the NN; see Section 3.1. In that case, the posterior $p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta)$ depends on $\sigma_{u,\theta}(x)$ through the likelihood term $p(\mathcal{D}|\theta)$, and thus $\sigma_{u,\theta}(x)$ is automatically taken into account by all the posterior inference techniques of this section. We refer to a posterior inference method denoted by X as h-X, when it is combined with heteroscedastic noise modeling. In Fig. 7, we illustrate how this idea can be applied to PINNs.

4.2. Ensembles

Instead of approximating the posterior $p(\theta|\mathcal{D})$ as performed with Bayesian methods, the aim of ensembles is to obtain multiple modes of $p(\theta|\mathcal{D})$, or equivalently, multiple minima of the NN parameter loss landscape. This set of parameters $\{\hat{\theta}_j\}_{j=1}^M$ can be used exactly as the posterior samples obtained with Bayesian techniques, i.e., in conjunction with an approximate BMA as delineated in Section 3.1. In the following, we provide an overview of the considered techniques (see also Table 2), as well as brief information regarding regularization/data uncertainty optimization and integration into SciML.

Approximate inference by deep ensembles (DEns) pertains to obtaining M different settings of parameters θ by standard NN training, independently M times using different NN parameter initializations. More information can be found in Sec-

Table 2
Overview of the UQ methods considered in this paper: The upper part corresponds to methods for obtaining NN parameter samples $\{\hat{\theta}_j\}_{j=1}^M$ (or $\{\hat{\beta}_j\}_{j=1}^M$). X represents one of the methods listed in the second column between entries "HMC" and "SWAG". The lower part corresponds to methods for learning (S)PDE solutions and neural operators by combining the methods in the upper part with PINNs and DeepONets. See also the conceptual model of Fig. 10. The numbers in the brackets correspond to the problems of Fig. 1, with "4s" referring to the special case of problem 4 (Section 1.2). The fourth column indicates the sections with the detailed descriptions of the methods. The right five columns serve to navigate through the comparative study of Section 6. For example, "All" in the first row of the fifth column indicates that HMC has been used in all subsections of 6.1, while "6.1.1" in the second row means that LD has been used only in 6.1.1.

Methods for obtaining NN parameter posterior samples $\{\hat{\theta}_j\}_{j=1}^M$ (or $\{\hat{\beta}_j\}_{j=1}^M$)

Method full name	Method	Short description	§	In § 6.1	In § 6.2	In § 6.3	In § 6.4	In § 6.5
Hamiltonian Monte Carlo	HMC	Two-step MCMC | simulates Hamiltonian dynamics	4.1	All	All		All	6.5.1
Langevin dynamics	LD	One-step MCMC | stochastic gradient descent with added noise	4.1	6.1.1				
Mean-field variational inference	MFVI	Approximates $p(\theta\|\mathcal{D})$ by a Gaussian variational distribution $q_\omega(\theta)$	4.1	All	6.2.1			
Monte Carlo dropout	MCD	Standard training with some NN parameters set to zero in each step	4.1	6.1.1	6.2.1			
Laplace approximation	LA	Standard training with a Gaussian fitted to the discovered mode $\hat{\theta}$	4.1	6.1.1, 6.1.2				
Deep ensemble	DEns	Standard training M independent times	4.2	6.1.1, 6.1.3	6.2.1	All		6.5.2
Snapshot ensemble	SEns	Snapshots during standard training with learning rate schedule	4.2	6.1.1				
Stochastic weight averaging-Gaussian	SWAG	SEns with a Gaussian distribution fitted to the snapshots	4.2	6.1.1				
Heteroscedastic-X	h-X	Combines above methods with heteroscedastic modeling of noise	3.1	6.1.3, D.2.2	6.2.2			
X+Bayesian neural network	X+BNN	Combines above methods with a BNN — *Default setting*	4	All	All	All	All	All
X+Functional prior	X+FP	Combines above methods with GAN functional prior instead of BNN	4.3	6.1.3	6.2.1			6.5.1

Methods for learning neural (S)PDEs and neural operators

Method full name	Method	Short description	§	In § 6.1	In § 6.2	In § 6.3	In § 6.4	In § 6.5
Uncertain PINN	U-PINN	PINN combined with X+BNN | *solves PDEs [1-2]*	4.5.1		All		All	
Heteroscedastic uncertain PINN	h-U-PINN	PINN combined with h-X+BNN | *solves PDEs (heteroscedastic) [1-2]*	4.1		6.2.2			
Physics-informed GAN functional prior	PI-GAN-FP	PINN combined with X+FP | *solves PDEs [1-2]*	4.5.1		6.2.1			
Gaussian process physics-informed GAN	GP+PI-GAN	Gaussian process combined with PINN and GAN | *solves PDEs [1]*	4.5.1				All	
Uncertain physics-informed GAN	U-PI-GAN	PINN combined with GAN and X+BNN | *solves SPDEs [3]*	4.5.1			All		
Uncertain NN polynomial chaos (+)	U-NNPC(+)	PINN combined with polynomial chaos and X+BNN | *solves SPDEs [3]*	4.5.1			All		
Uncertain DeepONet	U-DeepONet	DeepONet combined with X+BNN | *learns operators [4s]*	4.5.2					6.5.2
Physics-agnostic BNN functional prior	PA-BNN-FP	Pre-trained DeepONet combined with X+BNN | *learns operators [4]*	4.5.2					6.5.2
Physics-agnostic GAN functional prior	PA-GAN-FP	Pre-trained DeepONet combined with X+FP | *learns operators [4]*	4.5.2					6.5.1

tion B.3.1. On the contrary, snapshot ensembles (SEns) obtain $\{\hat{\theta}_j\}_{j=1}^M$ without incurring any additional cost as compared to standard training. To this end, standard algorithms are let to converge to M different local optima during a single optimization trajectory, and each optimum is used as a sample in $\{\hat{\theta}_j\}_{j=1}^M$. More information can be found in Section B.3.2. Stochastic weight averaging-Gaussian (SWAG) extends SEns by also fitting a Gaussian distribution to the aforementioned local optima. This distribution is used for drawing the samples in $\{\hat{\theta}_j\}_{j=1}^M$. More information can be found in Section B.3.3. Randomized prior methods, originally proposed for encouraging exploration and obtaining uncertainty in deep reinforcement learning, are also ensemble-based and have been extended recently to address deep operator learning with UQ [135]. Specifically, randomly initialized NNs are utilized and act as prior functions, providing regularization as well as predicted uncertainties. More information can be found in Section B.3.4.

For the techniques considered in this section, standard NN training is performed. To integrate ensemble techniques into SciML, standard training of PINNs and DeepONets, as described in Sections 2.1 and 2.2, respectively, is performed either multiple times (DEns) or once with an appropriate learning rate schedule (SEns and SWAG) for obtaining $\{\hat{\theta}_j\}_{j=1}^M$. This set of samples is, subsequently, combined with the approximate BMAs in Eqs. (8)-(10). As in the case of Bayesian methods, we refer to PINN and DeepONet combined with ensembles as U-PINN and U-DeepONet.

Similarly to Bayesian methods, online and offline methods can be used in the context of ensembles for identifying the constant noise scale σ_u^2. This requires using the negative log-likelihood as a loss function in NN training (Eqs. (A.5)-(A.8)), instead of the standard MSE loss (Eqs. (2) and (3)). Specifically, the loss function $\mathcal{L}(\theta)$ of Eqs. (A.5)-(A.8) is also a function of σ_u^2 through the likelihood term $\log p(\mathcal{D}|\theta)$. In this context, σ_u^2 can be identified *online* by alternating between update steps for the NN parameters θ and steps for σ_u^2. Clearly, these steps can also be combined into one step. Further, σ_u^2 can be identified *offline* by using cross-validation or meta-learning, in conjunction with the evaluation metrics of Section 5 and with validation data. Furthermore, the equivalent of prior in ensembles with standard NN training is regularization, which can be optimized offline. Lastly, aleatoric uncertainty can alternatively be considered as an additional output of the NN; see Section 3.1. In that case, the log likelihood loss function of Eqs. (A.5)-(A.8) is utilized for standard NN training, either once (SEns and SWAG) or M times (DEns). This loss function depends on $\sigma_{u,\theta}(x)$ through the likelihood term $\log p(\mathcal{D}|\theta)$.

4.3. Functional priors (FPs)

In this section we describe an approach for harnessing historical data in order to improve performance and in some cases reduce the computational cost of posterior inference in function approximation and SciML problems.

Consider Fig. 8a, where x and $u_\theta(x)$ are the input and the output of the NN, respectively, and θ are the NN parameters. Further, $\mathcal{M}$ is a black-box model that combines x and θ for producing $u_\theta(x)$. Typically, $\mathcal{M}$ is the NN architecture (connections and activation functions), which defines what operations are applied to x and θ. In this regard, the model of Fig. 8a defines a functional prior (FP); i.e., θ is drawn from a known prior probability distribution and it induces a distribution of corresponding functions $u_\theta(x)$. We refer to this as a Bayesian NN (BNN) or BNN-FP. In BNNs, posterior inference for θ may be performed either using the Bayesian methods of Section 4.1 or the ensembles of Section 4.2.

Note that both different $\mathcal{M}$ models and θ priors lead to different FPs. In this context, a FP can be constructed by training $\mathcal{M}$ in Fig. 8 such that it encodes historical data information, as proposed in [136]. To this end, we consider the dataset $\mathcal{D}_h = \{U_i\}_{i=1}^{N_h}$, with $U_i = \{(x_j^{(i)}, u_j^{(i)})\}_{j=1}^{N_u}$, containing N_h historical realizations. Specifically, $u_\theta(x)$ is the output of the generator of a pre-trained generative adversarial network (GAN) [137], whose distribution over θ matches the historical data distribution. We refer to this as GAN-FP. Following the pre-training phase, posterior inference for θ may be performed either using the standard Bayesian techniques of Section 4.1 or the ensembles of Section 4.2. We refer to a posterior inference method denoted by X as X+BNN[2] when it is combined with a BNN-FP (default setting), and as X+FP when it is combined with a GAN-FP. Note that inference for θ using the pre-trained GAN-FP can be performed faster, with less amount of data, and by using a significantly reduced dimension for the parameters θ. The interested reader is directed to Section B.4 for additional details.

The above concepts are integrated into SciML, as shown in Figs. 8b and c. Specifically, $\mathcal{M}$ in Fig. 8b is either a BNN, which is the case of standard U-PINNs, or the generator of a pre-trained GAN. We refer to the latter as physics-informed GAN-FP (PI-GAN-FP). The GAN is pre-trained using a dataset $\mathcal{D}_h$, N_h historical realizations of u, each one denoted as $U_i = \{(x_j^{(i)}, u_j^{(i)})\}_{j=1}^{N_u}$, and/or of f, each one denoted as $F_i = \{(x_j^{(i)}, f_j^{(i)})\}_{j=1}^{N_f}$. See the original paper [136] and Section B.4 for detailed information, as well as Section 6.2.1 for an example. Next, if $\mathcal{M}$ in Fig. 8c is a BNN and the DeepONet is pre-trained with noiseless data, the model can be used for treating cases with noisy data during inference; see Section 1.2 for problem formulation details. We refer to this as physics-agnostic BNN-FP (PA-BNN-FP) and note that it is a novel contribution of this paper. Further, if $\mathcal{M}$ is a pre-trained generator and the DeepONet is pre-trained with noiseless data, the model of Fig. 8c corresponds to a physics-agnostic GAN-FP (PA-GAN-FP). This is used for treating cases with noisy data during inference similarly to PA-BNN-FP. The historical dataset for training PA-GAN-FP can be the same as the one used for pre-training the DeepONet. After pre-training, PA-BNN-FP and PA-GAN-FP can be used during inference to infer θ given new (x, u) and/or

[2] We use "+" here to emphasize that the same posterior inference methods can be used regardless of the selected model, which can be a BNN or a GAN-FP.

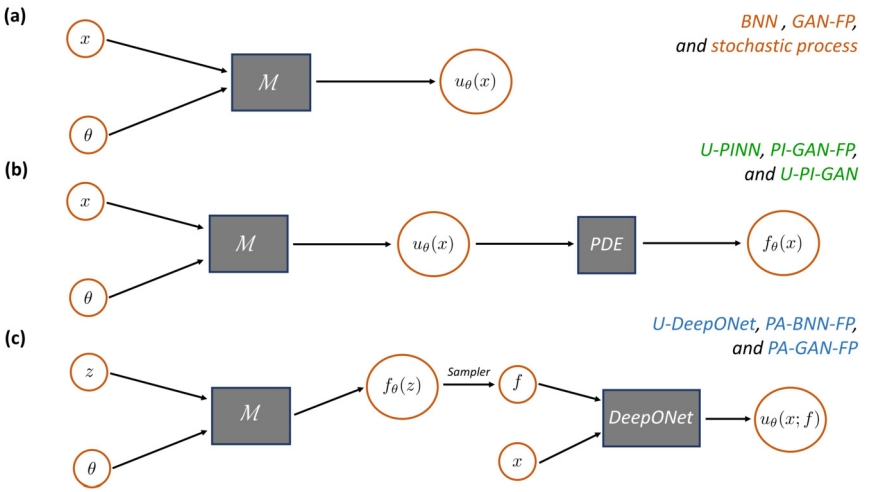


Fig. 8. **(a)** BNN, GAN functional prior (GAN-FP) and stochastic process representation. **(b)** U-PINN, PI-GAN-FP, and U-PI-GAN. **(c)** U-DeepONet, PA-BNN-FP, and PA-GAN-FP. $\mathcal{M}$ expresses how x and θ are combined to yield $u_\theta(x)$. In standard NN models (BNN, U-PINN, U-DeepONet), $\mathcal{M}$ is a BNN. In generator-based models (stochastic process, FP, PI-GAN-FP, U-PI-GAN, and PA-GAN-FP), $\mathcal{M}$ is the generator of a GAN that is trained using stochastic realizations or pre-trained using historical data. All models depicted in this figure can be construed as special cases of Fig. 10; note, however, the different meaning of θ for each technique. Further, in part (c), λ (or both f and λ) could be used as input to the DeepONet instead of f, as in Fig. 4. Furthermore, $u(x;f)$ is equivalent to $u(x;\xi)$, because each $\xi \in \Xi$ is associated with an input f. The additional input z is used to emphasize that the locations in the input domains of f and u can be different and the "sampler" that the function $f_\theta(z)$ must be sampled at pre-specified locations before entered as input to the DeepONet. See Table 2 for an overview of the UQ methods and Table 15 for a glossary of used terms.

(x, f) data. Further, in Fig. 8c, λ (or both f and λ) could be used as input to the DeepONet instead of f. For example, in Section 6.5.1 we solve an operator learning problem with λ as the only input to the DeepONet and we consider historical data of λ and u. See the original paper [136] and Section B.4 for more information. In passing, note that in Fig. 8c we have used an additional input z as well as a "sampler". The additional input z is used to emphasize that the locations in the input domains of f and u can be different, i.e., z and x, respectively. In addition, the "sampler" signifies that the function $f_\theta(z)$ must be sampled at pre-specified locations before entered as input to the DeepONet.

4.4. Solving stochastic PDEs (SPDEs)

In this section we extend the function approximation and PDE solution techniques of the previous sections for representing stochastic processes and for solving SPDEs.

The model in Fig. 8a can also be used for stochastic process representation if $\mathcal{M}$ is the generator of a GAN. In this regard, $\mathcal{M}$ is trained such that its distribution over θ matches the distribution of the available stochastic realizations. Specifically, the generator u_θ in Fig. 8a takes as input a random θ from a known distribution, e.g., Gaussian, and its parameters β are trained using the dataset $\mathcal{D} = \{U_i\}_{i=1}^N$ with $U_i = \{(x_j^{(i)}, u_j^{(i)})\}_{j=1}^{N_u}$. After training of β, we can generate samples of θ from its distribution, and thus samples of the stochastic process u_θ, which by construction approximately follow the same distribution as the data.

Next, consider the forward SPDE problem of Section 1.2 with known b in Eq. (1), without loss of generality. For known λ, unknown u, and stochastic realizations of f, the problem can be solved by training the model of Fig. 8b with $\mathcal{M}$ being the generator of a GAN. This GAN is used for modeling u, while f is given by substituting the output of the GAN into the PDE. The dataset for this forward case is expressed as $\mathcal{D} = \{F_i\}_{i=1}^N$, with $F_i = \{(x_j^{(i)}, f_j^{(i)})\}_{j=1}^{N_f}$, and contains N realizations of f. The same model of Fig. 8b can be used for solving a mixed SPDE problem with partially unknown u. If λ is also partially unknown, an additional GAN is used for modeling λ. The dataset in the latter case is expressed as $\mathcal{D} = \{\{F_i\}_{i=1}^N, \{U_i\}_{i=1}^N, \{\Lambda_i\}_{i=1}^N\}$, with $F_i = \{(x_j^{(i)}, f_j^{(i)})\}_{j=1}^{N_f}$, $U_i = \{(x_j^{(i)}, u_j^{(i)})\}_{j=1}^{N_u}$, and $\Lambda_i = \{(x_j^{(i)}, \lambda_j^{(i)})\}_{j=1}^{N_\lambda}$. After training the generator parameters β, we can generate new samples of the stochastic process $u_\theta(x)$ by generating samples of θ from its known distribution. We refer to this method as PI-GAN. Further, this technique can be combined with the posterior inference techniques of Sections 4.1-4.2 for obtaining multiple samples of the generator parameters β. This is referred to as U-PI-GAN in this paper. Overall, total uncertainty in this case refers to aleatoric and epistemic uncertainties, as well as uncertainty due to stochasticity. That is, for M posterior samples of the parameters β expressed as $\{\hat{\beta}_j\}_{j=1}^M$ and M' stochastic realizations produced from each one, the mean prediction for each x is given by

$$\bar{\mu}(x) = \frac{1}{MM'} \sum_{j=1}^{M} \sum_{k=1}^{M'} u_{\theta_k}(x; \hat{\beta}_j), \tag{15}$$

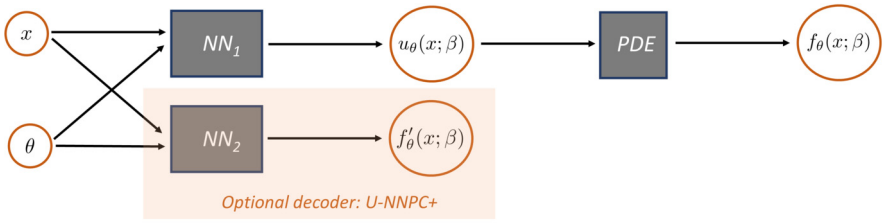


Fig. 9. NNPC, developed by [138], and NNPC+ (orange box), proposed herein, for solving SPDEs. NNs 1 and 2, parametrized by β, combine the location in the domain x to produce the modes of the stochastic representations of $u_\theta(x;\beta)$ and $f'_\theta(x;\beta)$, respectively. The model depicted in this figure can be construed as a special case of Fig. 10; note, however, the different meaning of θ in this technique.

and the total variance $\bar{\sigma}^2(x)$ for each x is given by

$$\bar{\sigma}^2(x) = \bar{\sigma}_a^2(x) + \underbrace{\frac{1}{MM'}\sum_{j=1}^{M}\sum_{k=1}^{M'}(u_{\theta_k}(x;\hat{\beta}_j) - \bar{\mu}(x))^2}_{\bar{\sigma}_e^2(x)+\bar{\sigma}_{stoc}^2(x)}. \tag{16}$$

For stochastic and partially unknown λ, the additional generator $\lambda_\theta(x;\beta)$ also parametrized by β for simplicity, takes the same random input θ and is trained simultaneously with $u_\theta(x;\beta)$. See the original paper [111] and Section B.4 for more information, as well as Section 6.3 for an example.

Further, PI-GAN can be combined with Gaussian process (GP) regression (Section B.1) for solving deterministic forward PDE problems. We refer to this approach as GP+PI-GAN[3] and note that it is a novel contribution of this paper. Specifically, first we fit a GP given the noisy f data (Table 1, part 1). Subsequently, we produce realizations of f using this GP, which are used for fitting a PI-GAN as described above and as if the problem was stochastic. This produces statistics of u that satisfy the PDE of Eq. (1) and include epistemic uncertainty due to limited and noisy data of f.

An alternative approach for solving SPDEs employs an arbitrary polynomial chaos expansion for the solution $u_\theta(x)$, and a PINN, parametrized by β, for approximating the corresponding stochastic modes. Fig. 9 provides an illustration of the model for the forward SPDE problem. For solving the mixed SPDE problem with partially unknown λ and u, an additional NN is used for modeling the stochastic modes of λ. This technique can also be combined with the posterior inference techniques of Sections 4.1-4.2 for obtaining multiple samples of the parameters β expressed as $\{\hat{\beta}_j\}_{j=1}^M$. In this case, the mean for each x and the total uncertainty including aleatoric, epistemic, and stochasticity uncertainties are given by Eqs. (15) and (16), respectively. This is referred to as U-NNPC in this paper. See the original papers [138,139] and Section B.5 for more information, as well as Section 6.3 for an example. An extension to U-NNPC that we propose in this paper and refer to as U-NNPC+ pertains to employing an additional decoder NN specific to f. This NN takes as input the encoding of the data of f, i.e., its low-dimensional representation expressed by θ, and outputs f'_θ in Fig. 9. More information regarding the training procedure of NNPC+ can be found in Section B.5.

4.5. Towards a unified view of the presented methods

In this section, we summarize our methodology. An overview of the presented methods for addressing the problems of Fig. 1 is provided in Fig. 10. The neural PDE methods utilize a PDE describing the underlying physics of the problem, whereas the neural operator methods utilize a DeepONet as a surrogate model for the physics. All methods utilize the posterior inference algorithms of Sections 4.1-4.2.

4.5.1. Neural PDEs: U-PINN, PI-GAN-FP, GP+PI-GAN, U-PI-GAN, and U-NNPC(+)

U-PINN (Sections 4.1-4.2) combines the PINN methodology with posterior inference for solving mixed PDE problems. U-PINN is depicted in Fig. 10 with "x" denoting the space-time input domain, "ϕ" the solution u, "ψ" the source term f, "physics" the PDE of Eq. (1), and "$\mathcal{M}$" a BNN parametrized by θ. The additional modeled field "χ" can be used to represent the model parameters λ in Eq. (1) and/or heteroscedastic noise; see h-U-PINN in Table 2 and Fig. 7. To solve a mixed PDE problem given f, u, and λ data, we perform posterior inference for the PINN parameters θ.

PI-GAN-FP (Section 4.3) combines the PINN methodology with a GAN-FP and with posterior inference for solving mixed PDE problems. PI-GAN-FP is depicted in Fig. 10 with "x", "ϕ", "ψ", and "physics" as in U-PINN. Further, "$\mathcal{M}$" is the generator of a GAN with random input θ that has a known distribution, e.g., Gaussian. PI-GAN-FP is pre-trained using historical data. Subsequently, given new limited and noisy f, u, and λ data, the goal is to reconstruct f, u, and λ. To solve the problem, we perform posterior inference for the GAN generator input θ.

[3] We use "+" here to emphasize that a GP is first employed for fitting f, and subsequently, a PI-GAN is *separately* employed for fitting u by treating the problem as stochastic.

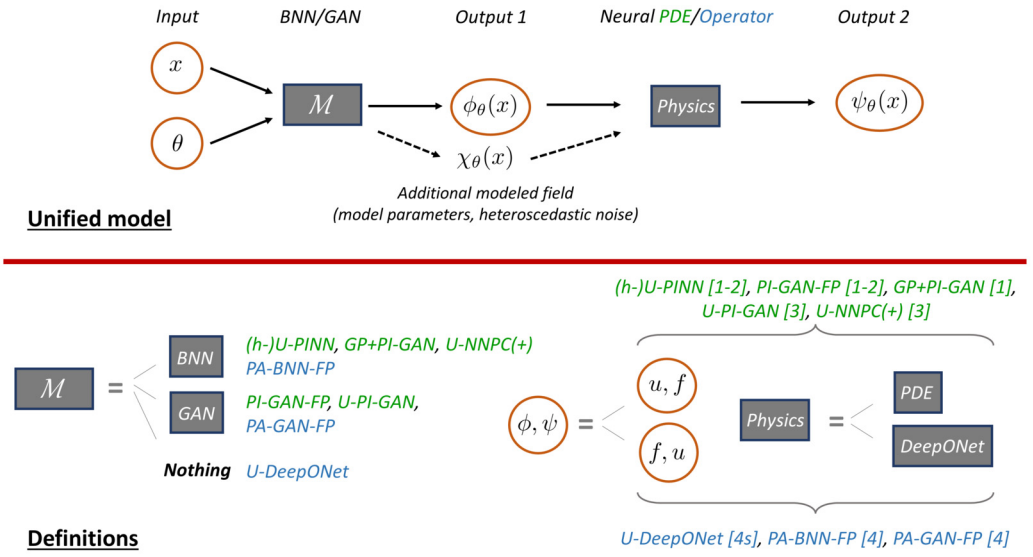


Fig. 10. A conceptual model towards a unified view of the presented techniques for addressing the problems of Fig. 1. We divide the methods into the ones that utilize a PDE describing the underlying physics *(neural PDEs)* and the ones that utilize a DeepONet as a surrogate model for the physics *(neural operators)*. U-PINN and PI-GAN-FP address problems 1-2; GP+PI-GAN addresses problem 1; U-PI-GAN and U-NNPC(+) address problem 3; U-DeepONet, PA-BNN-FP, and PA-GAN-FP address problem 4. The **model** $\mathcal{M}$ can be a BNN or the generator of a GAN, or nothing. The **physics** can be a PDE or a pre-trained DeepONet. The **quantities** ϕ, ψ can be the solution u and the source f, or vice-versa; x represents the space-time input domain; and θ represents the parameters of the BNN or the input noise to the GAN generator. Further, "χ" can represent the model parameters λ in Eq. (1) and/or heteroscedastic noise. The numbers in the brackets correspond to the problems of Fig. 1, with "4s" referring to the special case of problem 4 (Section 1.2).

U-PI-GAN (Section 4.4) combines the PINN methodology with the GAN-based stochastic process representation of Section 4.4 and with posterior inference for solving mixed SPDE problems. U-PI-GAN is depicted in Fig. 10 with "x", "ϕ", "ψ", "physics", "$\mathcal{M}$", and "θ" as in PI-GAN-FP. U-PI-GAN is given stochastic realizations of f, and potentially u, and the goal is to obtain the statistics of u and of any unknown parameters. To solve the problem, we perform posterior inference for the parameters β of U-PI-GAN. Subsequently, we produce samples of u by drawing θ samples from its known distribution and calculate the sought statistics.

GP+PI-GAN (Section 4.4) combines a PI-GAN with GP regression for solving forward PDE problems. Specifically, it fits a GP using noisy data of f, and subsequently produces realizations of f and treats the problem as stochastic using PI-GAN.

Finally, U-NNPC(+) (Section 4.4) combines the PINN methodology with polynomial chaos and with posterior inference for solving mixed SPDE problems. U-NNPC(+) is depicted in Fig. 10 with "x", "ϕ", "ψ", and "physics" as in U-PINN. Further, "$\mathcal{M}$" is a BNN parametrized by β and combines the location in the domain x to produce the stochastic modes of $u_\theta(x;\beta)$. Furthermore, θ represents a low-dimensional representation of the f-data. U-NNPC(+) is given stochastic realizations of f, and potentially u, and the goal is to obtain the statistics of u and of any unknown parameters. To solve the problem we perform posterior inference for the parameters β of U-NNPC(+). Subsequently, we calculate the statistics of the reconstructed u realizations and of the unknown parameters.

4.5.2. Neural operators: U-DeepONet, PA-BNN-FP and PA-GAN-FP

U-DeepONet (Sections 4.1-4.2) combines the DeepONet methodology with posterior inference for learning neural operators. U-DeepONet is depicted in Fig. 10 with "x" denoting the space-time input domain, "ϕ" the source term f, "ψ" the solution u, and "physics" the DeepONet parametrized by θ. Further, "$\mathcal{M}$" is discarded. To solve the problem given a dataset of paired u and f realizations, we perform posterior inference for the DeepONet parameters θ.

PA-BNN-FP (Section 4.3) combines the DeepONet methodology with an additional BNN and with posterior inference for learning neural operators. PA-BNN-FP is depicted in Fig. 10 with "x", "ϕ", and "ψ" as in U-DeepONet, while "physics" is a pre-trained DeepONet and "$\mathcal{M}$" is a BNN parametrized by θ. After pre-training DeepONet with noiseless data, we are given new limited and noisy data of f and u, and the goal is to reconstruct f and u. To solve the problem, we perform posterior inference for the BNN parameters θ.

Finally, PA-GAN-FP (Section 4.3) combines the DeepONet methodology with a GAN-FP and with posterior inference for learning neural operators. PA-GAN-FP is depicted in Fig. 10 with "x", "ϕ", "ψ", and "physics" as in PA-BNN-FP. Further, "$\mathcal{M}$" is the generator of a GAN with random input θ. After pre-training the DeepONet and the GAN-FP separately with noiseless data, we are given new limited and noisy data of f and u, and the goal is to reconstruct f and u. To solve the problem, we perform posterior inference for the GAN generator input θ.

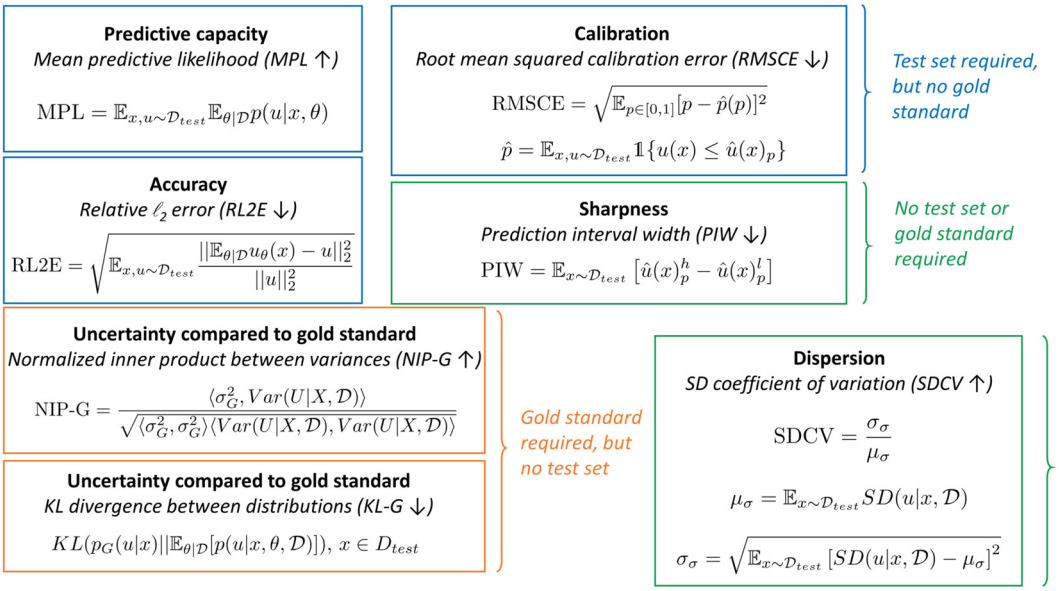


Fig. 11. Accuracy and uncertainty quality evaluation metrics used in this paper. Blue boxes correspond to evaluation metrics that require a test set, orange boxes to metrics that require a "gold standard", e.g., for testing novel techniques, and green boxes to metrics that require neither a test set nor a gold standard. MPL measures general predictive capability, RL2E accuracy, and RMSCE statistical consistency between predictions and data. Further, the secondary metrics PIW and SDCV, presented in Section C.1.1, measure prediction sharpness and dispersion across x of uncertainty predictions, respectively. Finally, NIP-G and KL-G, presented in Section C.1.2 (see also [145,146]), measure the inner product between the predicted variance and the variance of gold standard, and the average distance between the predictive and gold standard distributions, respectively. A gold standard can be an accurate but computationally expensive technique or the exact solution if the problem is amenable to one. The up and down arrows next to the abbreviated names of the metrics indicate whether larger or smaller metric values correspond to better performance, respectively.

5. Accuracy and uncertainty quality evaluation and improvement

5.1. Evaluation: accuracy and uncertainty quality evaluation

In this section we introduce a set of evaluation metrics, summarized in Fig. 11, to be used for model selection, e.g., prior/architecture optimization, comparison between UQ methods, and overall quality evaluation of the UQ design procedure. Each of the metrics presented in this section may correspond to first- or second-order statistics of the predictive model, or to the whole distribution (PDF or CDF). Subsets of our evaluation metrics have been used in various comparative studies in the literature, e.g., [80,85,92,95,140–144], where also a few other metrics, such as the Area Under Sparsification Error [144], have been used. We provide additional information about metrics in Appendix C.

5.1.1. Accuracy and predictive capacity

As explained throughout the present paper, the BMA not only provides uncertainty estimates in the form of $p(u|x,\mathcal{D})$, but can also improve the accuracy of the mean predictions. The mean for an arbitrary location x is expressed as $\hat{u}(x)$ in Eq. (A.1) and is approximated by $\bar{\mu}(x)$ via Eq. (9) using samples $\{u_{\hat{\theta}_j}(x)\}_{j=1}^M$. In contrast, standard NNs predict $\hat{u}(x)$ as $u_{\hat{\theta}}$, where $\hat{\theta}$ is obtained through a point estimate as presented in Section A.2. For evaluating the accuracy of the mean we use the relative ℓ_2 error averaged over validation/test data, i.e.,

$$\mathrm{RL2E} = \sqrt{\mathbb{E}_{x,u\sim\mathcal{D}_{test}}\frac{||\mathbb{E}_{\theta|\mathcal{D}}u_\theta(x)-u||_2^2}{||u||_2^2}} \approx \sqrt{\frac{\sum_{i=1}^{N_{test}}(\bar{\mu}(x_i)-u_i)^2}{\sum_{i=1}^{N_{test}}u_i^2}}. \tag{17}$$

The validation/test data in Eq. (17) can either be clean or noisy.

Next, for predictive capacity evaluation a widely used metric is *mean predictive likelihood* given and approximated as

$$\mathrm{MPL} = \mathbb{E}_{x,u\sim\mathcal{D}_{test}}\mathbb{E}_{\theta|\mathcal{D}}p(u|x,\theta) \approx \frac{1}{N_{test}}\sum_{i=1}^{N_{test}}\mathcal{N}(\bar{\mu}(x_i),\bar{\sigma}^2(x_i))[u_i]. \tag{18}$$

In Eq. (18) and for each $i\in\{1,\dots,N_{test}\}$, $\mathcal{N}(\bar{\mu}(x_i),\bar{\sigma}^2(x_i))[u_i]$ denotes the PDF value of the tested datapoint u_i in the bracket, based on the Gaussian approximation $\bar{p}(u_i|x_i)\approx\mathcal{N}(\bar{\mu}(x_i),\bar{\sigma}^2(x_i))$ at the location x_i. High MPL values can be interpreted as the validation/test data being highly probable under the predictive distribution.

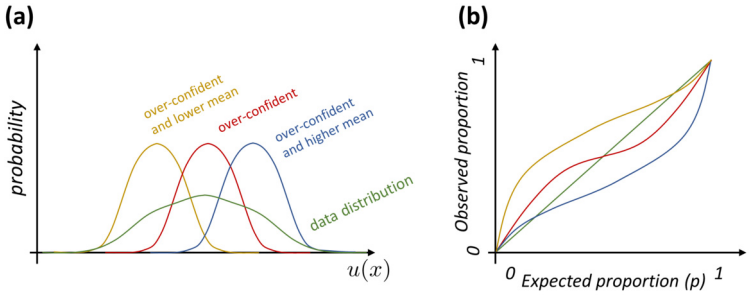


Fig. 12. Sketch of a data-generating distribution and indicative over-confident predictive models (a), and the corresponding calibration plots (b), for a given x. The calibration plot, for a given x, can be used to visualize the relationship between "expected proportion" of datapoints inside a statistical interval, which is p, and "observed proportion", based on the predictive model, which is $P(U_x \leq \bar{P}_x^{-1}(p))$. Note, however, that in practice for each x at most one sample $u(x)$ is available, and thus the calibration plot is produced "on average" for different values of x. The observed proportions can be approximated via $\mathbb{E}_{x,u\sim\mathcal{D}_{test}}\mathbb{1}\{u \leq \hat{u}(x)_p\}$, where $\hat{u}(x)_p$ is the inverse CDF value of the predictive model at x. An "ideal" predictive model matches the data distribution and its calibration plot is a straight line (green). An over-confident model (red) is typically above the green line for small values of p, and below the green line for larger values of p. The opposite is typically true for an under-confident model. Predictive models with different means compared to the data distribution (yellow, blue), may yield calibration plots that do not follow the aforementioned trends.

5.1.2. Statistical consistency

For evaluating the whole predictive distribution, we adopt a calibration metric that evaluates the statistical consistency between our predictions and data. In the following, we assume that data is generated as $X, U_X \sim \mathbb{P}$, where X, U_X are the random variables corresponding to the samples x, $u(x)$, and $\mathbb{P}$ is the data-generating distribution. A model that produces a CDF denoted by $\bar{P}(U_X|X)$ and abbreviated as $\bar{P}_X$ is called "ideal" if it matches the data-generating process, i.e., $\bar{P}(U_X|X) = \mathbb{P}(U|X)$ [147]. If $\bar{P}(U_X|X) \neq \mathbb{P}(U|X)$, the model may be over- or under-confident, depending on whether it predicts smaller or larger confidence intervals, respectively, as compared to the data-generating distribution. However, because $\mathbb{P}$ is not known, a set of calibration conditions can be used in practice. The most commonly used condition is probabilistic calibration, which requires that $P(U_X \leq \bar{P}_X^{-1}(p)) = p$ is satisfied for any $p \in [0,1]$ [147,148]. Indicatively, for a given x and for $p = 20\%$, this condition requires that 20% of the observed datapoints $u(x)$ fall within the 20th percentile of the predictive model. For a given x, the calibration plot can be used to visualize the relationship between "expected proportion" of datapoints inside a statistical interval, which is p, and "observed proportion", based on the predictive model. The observed proportion is $P(U_x \leq \bar{P}_x^{-1}(p))$, where U_x is the random variable of the sample $u(x)$, for a fixed x. In Fig. 12, we provide a sketch of indicative over-confident predictive models, for a given x, and the corresponding calibration plots. Note, however, that in practice for each x at most one sample $u(x)$ is available. The observed proportions can be approximated via $\mathbb{E}_{x,u\sim\mathcal{D}_{test}}\mathbb{1}\{u \leq \hat{u}(x)_p\}$, where $\hat{u}(x)_p \approx \mathcal{N}^{-1}(\bar{\mu}(x), \bar{\sigma}^2(x))[p]$, and $\mathcal{N}^{-1}(\bar{\mu}(x), \bar{\sigma}^2(x))[p]$ denotes the inverse CDF value of the Gaussian predictive distribution $\mathcal{N}(\bar{\mu}(x), \bar{\sigma}^2(x))$ at a location x. As a result, the calibration plot is not produced for each x separately, but "on average" for different values of x. As a metric to measure calibration, we adopt the root mean squared calibration error (RMSCE) given and approximated as

$$\text{RMSCE} = \sqrt{\mathbb{E}_{p\in[0,1]}(p - \hat{p}(p))^2} \approx \sqrt{\frac{1}{N_p}\sum_{j=1}^{N_p}\left[p_j - \frac{1}{N_{test}}\sum_{i=1}^{N_{test}}\mathbb{1}\{u_i \leq \hat{u}(x_i)_{p_j}\}\right]^2}, \tag{19}$$

where $\hat{p}(p)$ denotes the proportion of points falling inside the interval defined by p [148]. The interested reader is directed to Section C.1.1 for secondary metrics evaluating statistical consistency and to Section C.1.2 for metrics comparing with a "gold standard" solution, e.g., an accurate but computationally expensive technique.

5.2. Post-training improvement: calibration

Suppose that an additional dataset $\mathcal{D}_r = \{x_i, u_i\}_{i=1}^{N_r}$ is available after training. This can be a left-out calibration dataset, similar to validation and test sets. In this section, we present three approaches for improving the calibration of function approximation predictive models after training, using the calibration dataset. Further, we extend these approaches for addressing SciML problems. These approaches can also prove useful for addressing model misspecification cases, e.g., when a wrong data noise model has been utilized (see also Section A.4). More information on the active area of research related to calibration approaches for function approximation can be found in [54,143,148–153] and Appendix C. Computational results can be found in Sections 6.1, 6.3, and 6.5.

A calibration approach proposed by Levi et al. [149] utilizes the calibration data for re-weighting optimally all predicted variances. Specifically, given a Gaussian predictive distribution $\mathcal{N}(\bar{\mu}(x), \bar{\sigma}^2(x))$ for every x using samples $\{u_{\hat{\theta}_j}(x)\}_{j=1}^{M}$, the optimal constant value of s to multiply the uncalibrated total uncertainty prediction is sought for. This is achieved by optimizing one of the related metrics of Section 5.1. As a result, the calibrated predictive distribution for each x becomes

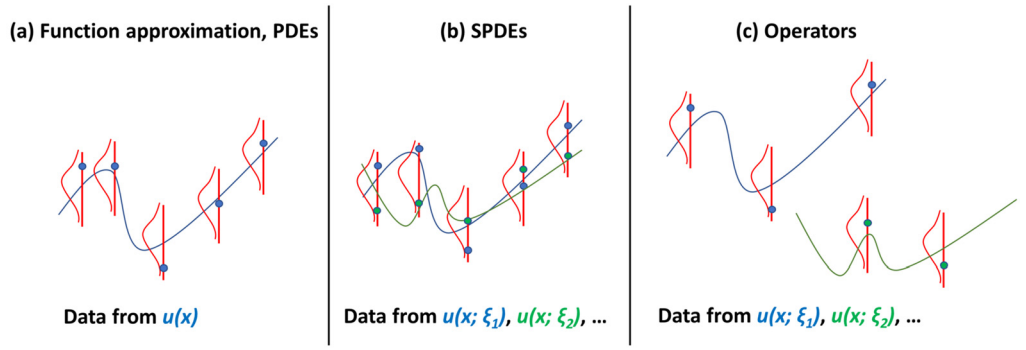


Fig. 13. Post-training calibration of uncertainty predictions: (a) for function approximation and deterministic PDE problems, calibration can be applied using data from u at different locations; (b) for solving SPDEs, using data from realizations (denoted with ξ) of u at different locations; and (c) for neural operator learning, using data from u functions evaluated at different locations and corresponding to different λ inputs (or f, depending on which quantity is varying).

$\mathcal{N}(\bar{\mu}(x), s^2\bar{\sigma}^2(x))$. Note that by using this approach only the variance is altered for each x and not the mean prediction. Next, an alternative approach that has become standard in function approximation calibration has been proposed by Kuleshov et al. [148]. Assume that the obtained predictive model is miscalibrated, i.e., it satisfies $Q(p) = P(U_X \leq \bar{P}_X^{-1}(p)) \neq p$ for every $p \in [0, 1]$. It can be shown that by approximating Q using calibration data, we can calibrate our model by applying the approximate Q to all the predictive CDFs. Clearly, this approach modifies both the mean and the variance of the predicted u at each x. More details can be found in Section C.2.1. Finally, recall that when a Gaussian distribution $\mathcal{N}(\bar{\mu}(x), \bar{\sigma}^2(x))$ is used for making predictions as described in Section 3.1, the scaled residuals $\epsilon_u(x) = (u - \bar{\mu}(x))/\bar{\sigma}(x)$ for arbitrary x, u from the data-generating distribution are implicitly assumed to follow $\mathcal{N}(0, 1)$. In this context, according to a calibration approach proposed by Zelikman et al. [152], referred to as CRUDE, an empirical distribution is fitted to the scaled residuals that correspond to the calibration dataset. By doing so, the mean and the variance of all the predicted distributions can be updated. More details can be found in Section C.2.2. Clearly, all of the above methods modify the predictions of all values of x in the same way, i.e., they calibrate the predictions for all points in the domain "on average". Also the calibration dataset is sampled from a given data distribution, which is usually the same as the training data distribution. For these reasons, there are cases in which performance may deteriorate for either in-distribution evaluation, i.e., for test data following the training data distribution, or out-of-distribution evaluation, i.e., for test data following a different distribution. See, for example, computational results in Table D.1.

For the case of solving mixed PDE problems with U-PINN, the calibration approaches can be separately applied to any of the quantities f, b, u, and λ for which a left-out calibration dataset is available (Fig. 13). Next, for the case of solving mixed stochastic problems with U-PI-GAN or U-NNPC(+), the predictive distribution for each x includes aleatoric and epistemic uncertainties as well as uncertainty from stochasticity of the modeled quantities, as explained in Sections 4.4 and B.5. In this case, the calibration dataset includes data for different x locations and from different stochastic realizations $\xi \in \Xi$. The calibration approaches can be separately applied to any of the quantities f, b, u, and λ for which a calibration dataset containing left-out stochastic realizations is available; see Section 6.3 for an example. Finally, we consider two cases of neural operator learning as explained in Section 1.2. First, for the case where f, b, and λ data are available at the complete set of locations used in pre-training, and the only sought outcome is the solution u, U-DeepONet produces a different u predictive distribution for each x and for each input λ (for fixed and known f, b). The calibration dataset must include u data for various x locations and corresponding to different inputs λ. Second, for the case where limited noisy data is available for f, b, u, and λ, PA-BNN-FP and PA-GAN-FP produce separate predictive distributions for each of the f, b, u, and λ quantities. In this case, the calibration approaches can be separately applied to any of the quantities f, b, u, and λ for which a left-out calibration dataset is available; see Section 6.5.1 for an example.

6. Comparative study

In this section, we test the presented UQ methods for solving prototype problems in computational science and engineering. Although there exist various recent studies, such as [93,140], providing and comparing methods on benchmark UQ problems, they are mostly focused on regression and classification tasks. In this regard, we construct five benchmark problems that are relevant to SciML application scenarios, such as function approximation, solving differential equations as well as learning operators, and test the selected methods on several sub-cases for each scenario. Specifically, in Section 6.1, we consider a discontinuous function approximation problem, which we solve with more than ten methods. This pedagogical example serves as a testbed for demonstrating various features of our UQ methods, such as prior learning, heteroscedastic noise modeling, posterior tempering, functional priors, and post-training calibration. In Section 6.2, we consider a mixed PDE problem involving a time-dependent diffusion-reaction equation. To solve this problem, we compare five different methods as well as four diverse problem setups, including extrapolation and heteroscedastic noise. In Section 6.3, we consider a

Table 3
Problems and corresponding sections of the comparative study. For each example, we provide the sections that include the problem description, results, and hyperparameters, as well as the sections that include the various problem cases that we considered and the corresponding sub-sections, while "single case" corresponds to examples with only one problem case. This table in conjunction with Table 2 serves to navigate through the comparative study. In Appendix D, we include additional results, implementation details, as well as the hyperparameters and NN architectures that we used.

Comparative study contents					
Problem	Function approximation	Mixed PDE	Mixed SPDE	Forward PDE	Operator learning
Sections	6.1 & D.2	6.2 & D.3	6.3 & D.4	6.4 & D.5	6.5 & D.6-D.7
Problem cases	Known/homoscedastic Gaussian noise 6.1.1 & D.2.1	Standard case $\sigma_u = \sigma_f = \sigma_\lambda = 0.05$ 6.2.1 & D.3.1	Single case	15 datapoints of source term f 6.4	Noisy/limited input data 6.5.1 & D.7
	Unknown/homoscedastic Gaussian noise 6.1.2	Heteroscedastic noise $\sigma_u = \sigma_f = \sigma_\lambda = 0.1\lvert x\rvert$ 6.2.2		6 datapoints of source term f D.5	Clean/complete input data 6.5.2
	Heteroscedastic Gaussian noise 6.1.3	Large noise $\sigma_u = \sigma_f = \sigma_\lambda = 0.1$ D.3.2			OOD data detection 6.5.3
	Heteroscedastic Student-t noise D.2.2	Extrapolation case D.3.3			
		Steep boundary layers D.3.4			

mixed problem involving a stochastic elliptic equation. To solve this problem we compare and calibrate three methods, including the herein proposed U-NNPC+. In Section 6.4, we consider a forward PDE problem. To solve this problem we compare two methods, including the herein proposed GP+PI-GAN. In Section 6.5, we consider an operator learning problem with two different problem setups, as described in Section 1.2. We use and calibrate various methods depending on the problem case. We also propose methods to detect *out-of-distribution data*, which is critical for risk-related applications. Finally, in Section 6.6, we briefly discuss the accuracy versus cost trade-off of the presented methods and also provide quantitative results pertaining to the function approximation example. The hyperparameters we used as well as additional problem cases we tested can be found in Appendix D. In all problems, unless otherwise stated, a standard feed-forward NN architecture is used; see Section D.9 for architecture details. Tables 2 and 3 serve to navigate through the comparative study.

To evaluate the performance of the employed UQ methods using the metrics of Section 5, we use either in-distribution (ID) data, i.e., data sampled from the same distribution as the one used for training, or out-of-distribution (OOD) data, i.e., data sampled from an unseen distribution during training. Unless otherwise stated, the evaluation data is ID. In addition, we present epistemic, aleatoric, and total uncertainties using the corresponding standard deviations and the 95% confidence intervals (CIs), which correspond to approximately two standard deviations for Gaussian predictive distributions. Lastly, we use calibration plots as illustrated in Fig. 12, and refer to UQ methods as over- or under-confident, depending on whether they predict smaller or larger confidence intervals compared to the data-generating distribution, respectively.

6.1. Discontinuous function approximation

We consider the following one-dimensional discontinuous function

$$u(x) = \begin{cases} \frac{1}{2}[\sin^3(2\pi x) - 1], & -1 \le x < 0, \\ \frac{1}{2}[\sin^3(3\pi x) + 1], & 0 \le x \le 1. \end{cases} \tag{20}$$

The training dataset $\mathcal{D} = \{x_i, u_i\}_{i=1}^N$ consists of $N = 32$, unless otherwise specified, equidistant measurements of Eq. (20) at $x \in [-0.8, -0.2] \cup [0.2, 0.8]$. The data is contaminated with zero-mean noise. In Section 6.1.1, the data noise is considered Gaussian with constant along x (homoscedastic) and known scale; in Section 6.1.2, Gaussian with unknown constant scale; in Section 6.1.3, Gaussian with varying along x (heteroscedastic) and unknown scale; and in Section D.2.2, Student-t with unknown varying scale. The hyperparameters and the NN architectures that we used are summarized in Section D.8 and D.9, respectively.

6.1.1. Known homoscedastic noise

In this section, we solve the function approximation problem of Eq. (20) with known and constant noise that follows $\mathcal{N}(0, 0.1^2)$. The considered techniques include HMC, LD, MFVI, and LA from Section 4.1 with fixed prior distribution $\mathcal{N}(0, 1)$ for all NN parameters; MCD with tuned dropout rate; as well as DEns, SEns, and SWAG from Section 4.2 with tuned weight decay constant, learning rate, and cycle length, where applicable. For obtaining a form of reference solution, we consider the "analogous" GP that corresponds to BNN with $\mathcal{N}(0, 1)$ as prior distribution, i.e., we numerically obtain a GP kernel that induces a FP that approximates the BNN-FP (see, e.g., [24]). Using this derived kernel we can perform exact Bayesian inference using GP regression with the available data; see Section B.1. Note, however, that this does not mean that a GP with

Table 4
Function approximation problem of Eq. (20) — *Known homoscedastic noise*: the most competitive techniques are the GP, HMC, MCD, DEns and SEns. Post-training calibration makes all techniques approximately equally calibrated. To evaluate the performance of the employed UQ methods we split the x domain into in-distribution (ID) data, i.e., a union of sub-domains $[-0.8, -0.2] \cup [0.2, 0.8]$ that was used for training, and out-of-distribution (OOD) data, i.e., $[-1.0, -0.8] \cup [-0.2, 0.2] \cup [0.8, 1.0]$ that was unseen during training. Here we evaluate ID performance of nine UQ methods based on the metrics of Section 5, using noisy test data and uncalibrated predictions, except for calibrated (Cal) RMSCE. See also Figs. 15 and D.4 for the disadvantages of MCD compared to HMC, as well as Table D.1 for OOD evaluation. The up and down arrows next to the names of the metrics indicate whether larger or smaller metric values correspond to better performance, respectively.

Metric ($\times 10^2$)	GP	HMC	LD	MFVI	MCD	LA	DEns	SEns	SWAG
RL2E (↓)	22.1	22.9	23.5	29.2	**21.9**	23.1	23.1	22.1	26.2
MPL (↑)	228.3	225.8	103.3	121.6	237.8	209.5	235.0	**241.6**	109.0
RMSCE (↓)	**6.9**	7.2	15.5	8.8	8.6	7.1	8.3	7.2	13.6
Cal RMSCE (↓)	6.8	6.7	6.4	**3.8**	8.0	6.5	6.5	6.9	5.7
PIW (↓)	45.8	45.2	142.0	116.2	42.5	51.9	**40.5**	41.9	132.0
SDCV (↑)	3.3	4.6	2.8	**38.2**	4.6	29.6	6.0	6.3	8.8

Table 5
Function approximation problem of Eq. (20) — *Known homoscedastic noise*: MCD performs better than HMC for some magnitudes of noise and dataset sizes based on the evaluation metrics. However, unlike HMC, MCD does not provide consistently larger epistemic uncertainties for increasing noise or decreasing dataset size, as shown in Figs. 15 and D.4. See also [92,142] regarding the limitations of the evaluation metrics. Here we evaluate ID performance of HMC and MCD in conjunction with three noise scales (0.1, 0.3, 0.5) and two dataset sizes (32, 16 points), using noisy test data and uncalibrated predictions. Comparisons are made in the vertical direction.

	(N, σ_u)					
	(32, 0.1)	(32, 0.3)	(32, 0.5)	(16, 0.1)	(16, 0.3)	(16, 0.5)
Metric ($\times 10^2$)	HMC					
RL2E (↓)	22.9	**45.2**	**63.6**	**23.9**	**53.1**	**72.8**
MPL (↑)	225.8	91.8	55.8	216.8	83.9	52.2
RMSCE (↓)	**7.2**	4.2	4.3	**3.9**	3.6	4.2
Metric ($\times 10^2$)	MCD					
RL2E (↓)	**21.9**	46.1	65.2	24.5	53.7	73.4
MPL (↑)	**237.8**	**94.9**	**56.9**	**222.0**	**89.9**	**56.7**
RMSCE (↓)	8.6	**3.0**	**3.5**	4.6	**2.9**	**2.0**

a carefully selected kernel can replace BNNs in all cases. For example, BNNs can provide improved performance by learning a representation of the data and result in significant computational savings for large datasets; see, e.g., [69,154,155] for extended discussions. Further, BNNs can be integrated into SciML in a straightforward manner without linearization of the problem; see [156] and Section 4, as well as indicative literature about NNGPs [68–70].

In Fig. 14 we present the mean and total uncertainty predictions obtained by nine UQ methods. In Table 4 we also provide the corresponding evaluation metrics for ID evaluation, i.e., for values of $x \in [-0.8, -0.2] \cup [0.2, 0.8]$, whereas OOD evaluation, i.e., for values of $x \in [-1.0, -0.8] \cup [-0.2, 0.2] \cup [0.8, 1.0]$, is provided in Table D.1. The most competitive techniques for this problem are the GP, HMC, MCD, DEns, and SEns. However, the uncertainties of MCD and SEns do not increase consistently with OOD data. This is important in risk-sensitive applications, where computational methods should be able to distinguish between parts of the input domain corresponding to interpolation and parts corresponding to extrapolation. OOD evaluation as well as predictions following post-training calibration can be found in Section D.2.1. In addition, the secondary metrics PIW and SDCV of Section C.1.1 can be used for comparisons between equally calibrated techniques. For example, the RMSCEs of MFVI and MCD are 8.8 and 8.6, respectively, while their SDCVs are 38.2 and 4.6. That is, the uncertainty predictions of MFVI vary more along x compared to the predictions of MCD. This is used as a diagnostic tool for assessing which method is expected to perform better for OOD predictions (see, e.g., [147,149]). In this regard, by comparing parts (d) and (e) of Fig. 14, we note that the uncertainty of MFVI increases for OOD data, whereas the uncertainty of MCD does not. Similarly, LD has low SDCV score and its uncertainty is the same for both ID and OOD data. Nevertheless, MFVI has a much larger PIW score compared to MCD, which indicates that its predictions may be under-confident. This property may be considered as safe and desirable in some applications, or as wasteful in some others.

Next, we compare HMC, which is in general computationally expensive, with MCD, which is as costly as standard NN training, for three difference noise scale values (0.1, 0.3, and 0.5) and two dataset sizes (32 and 16 points). In general, epistemic uncertainty is expected to increase with increasing aleatoric uncertainty and decreasing dataset size (see also Section B.1 for related discussion in GP regression). As shown in Figs. 15 and D.4, unlike HMC, epistemic uncertainty of MCD does not increase with increasing noise magnitude and with decreasing dataset size.

Nevertheless, this is not depicted in the corresponding evaluation Table 5, where MCD performs better than HMC for some magnitudes of noise and dataset sizes based on the used metrics. In fact, MCD performs better than HMC in terms of predictive likelihood (MPL) for all noise and dataset cases. This can be explained by the fact that HMC, having large

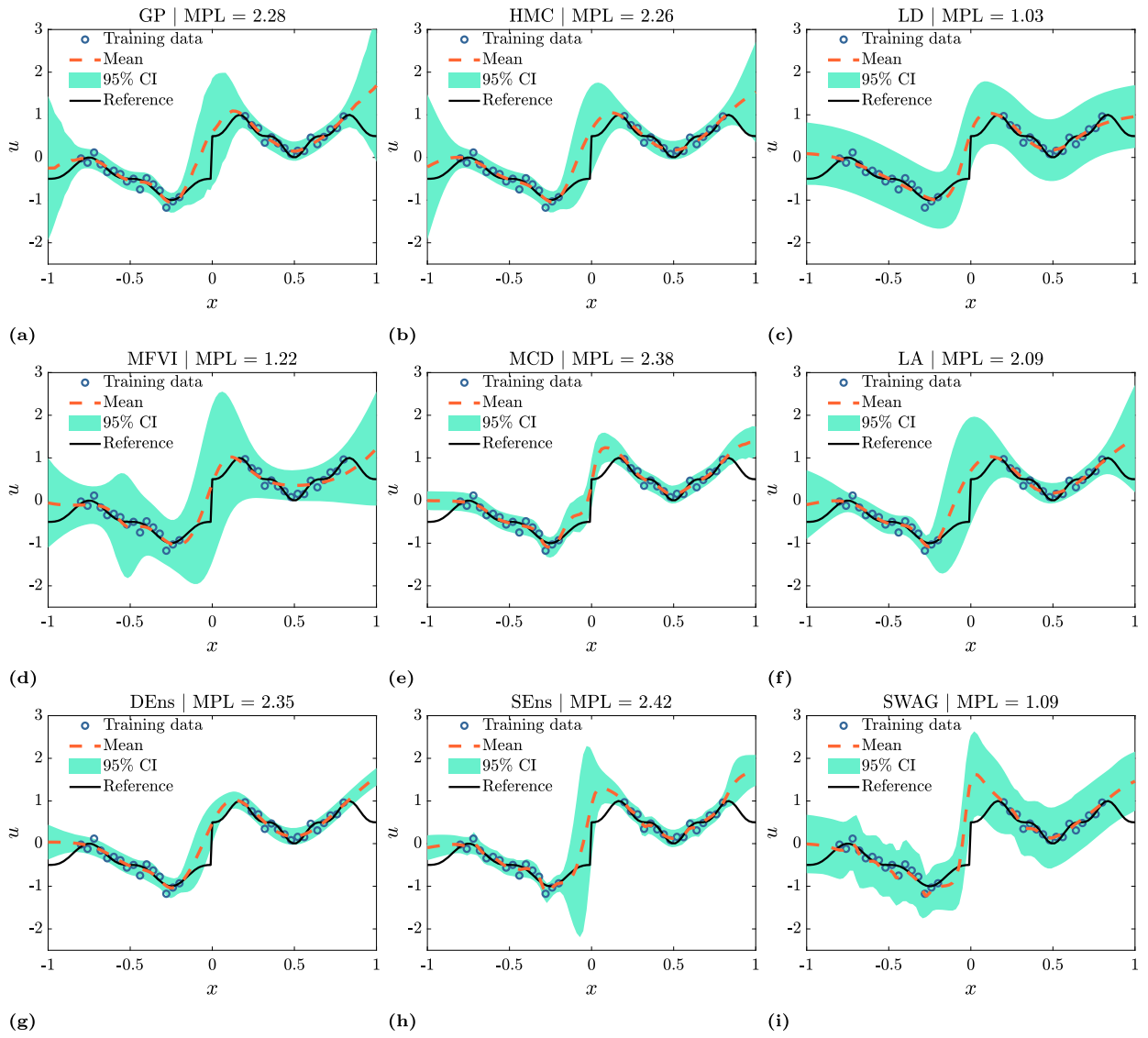


Fig. 14. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: the most competitive techniques are the GP, HMC, MCD, and SEns. However, the uncertainties of MCD and SEns do not increase consistently with OOD data. Shown here are the training data and exact function, as well as the mean and total uncertainty (95% confidence interval, CI) predictions without post-training calibration, as well as corresponding MPL values (higher is better). The results correspond to the nine posterior inference methods of Table 2. Note that the total uncertainty estimates cover in most cases the point-wise errors within the 95% CIs (approximately two standard deviations). See also accompanying Figs. D.1–D.3 including calibration plots and predictions with post-training calibration.

epistemic uncertainty, is quite under-confident in terms of total uncertainty predictions. However, this lack of confidence of HMC is desirable in this case: more noise and less data should make the predictions less confident. This fact shows that a plurality of evaluation metrics is necessary for comparing different methods. The interested reader is directed to [92,142] for discussions related to the limitations of evaluation metrics.

In Fig. 16, we compare the HMC results for different NN architectures; namely, with widths (number of neurons in each layer) in the set $\{50, 100\}$ and with depths (number of layers) in the set $\{2, 4\}$. It is shown that epistemic uncertainty increases with increasing NN expressivity, i.e., larger number of trainable parameters. Nevertheless, this effect can be partially reverted by learning the prior. Specifically, as we demonstrate in the right column of Fig. 16, the distributions of $u_\theta(x = 0)$ with θ drawn from the priors corresponding to the different architectures, concentrate to similar distributions after learning the prior. Having similar learned priors, the different architectures yield similar posteriors, as shown in the middle column of Fig. 16.

In Fig. 17, we demonstrate the effect of different post-training calibration approaches and dataset sizes on calibration error (RMSCE). It is shown that post-training calibration, even with only a few calibration datapoints (2-14), can reduce

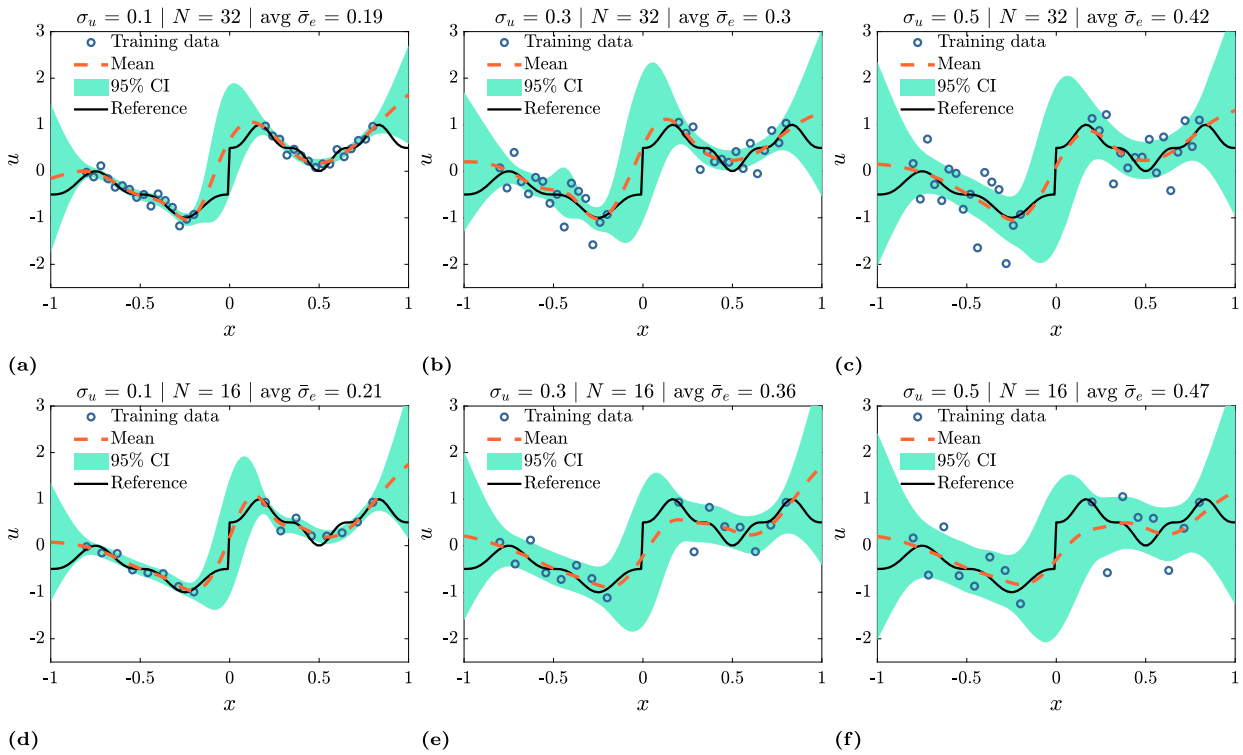


Fig. 15. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: epistemic uncertainty of HMC (avg $\bar{\sigma}_e^2$ along x) increases with increasing noise magnitude and with decreasing dataset size. Shown here are the training data and exact function, as well as the mean and epistemic uncertainty (95% CI) predictions, as obtained by HMC in conjunction with three noise scales ($\sigma_u = 0.1$: left, 0.3: middle, 0.5: right) and two dataset sizes ($N = 32$: top, 16: bottom).

Table 6
Function approximation problem of Eq. (20) — *Unknown homoscedastic noise*: HMC, MFVI, and LA with learned priors and noise scales are comparable in terms of accuracy (RL2E) and predictive capacity (MPL) for ID evaluation. Here we evaluate ID and OOD performance of HMC, MFVI, and LA, using noisy test data and uncalibrated predictions (part a), and the accuracy of the learned noise scale (part b).

	Metric ($\times 10^2$)	ID			OOD		
		HMC	MFVI	LA	HMC	MFVI	LA
a	RL2E (↓)	23.7	25.3	**22.9**	56.4	**51.7**	66.3
	MPL (↑)	**187.3**	130.2	162.4	**100.9**	61.4	57.2
	RMSCE (↓)	**6.1**	11.0	8.3	17.0	**15.6**	21.9
b	Reference	Learned noise (aleatoric uncertainty)					
	$\sigma_u = 0.1$	HMC		MFVI		LA	
		0.15		**0.1**		0.14	

significantly RMSCE, especially for initially less calibrated methods, such as SWAG. It is also shown that calibration using the training dataset typically leads to overfitting; i.e., RMSCE after calibration is higher, and thus a separate calibration dataset should be used. Lastly, note that calibration is performed using ID data, hence OOD performance may deteriorate; see, e.g., SWAG in Table D.1.

6.1.2. Unknown homoscedastic noise

In this section, the data noise follows $\mathcal{N}(0, 0.1^2)$ and the noise scale is assumed to be unknown. In Fig. 18, we present the mean and total uncertainty predictions obtained by HMC, MFVI and LA, as well as the corresponding calibration plots. In this figure, we include results before ("uncalibrated") and after ("calibrated") post-training calibration. To obtain these results, the noise scale σ_u and the prior $p(\theta)$ are learned online, as explained in Section 4.1. In Table 6 we also provide the corresponding evaluation metrics for ID evaluation. Overall, in this case computationally cheaper techniques, such as MFVI and LA, perform comparably with HMC, if their priors are learned. Nevertheless, HMC remains the method with the smallest calibration error for ID data evaluation, even following post-training calibration of the cheaper methods.

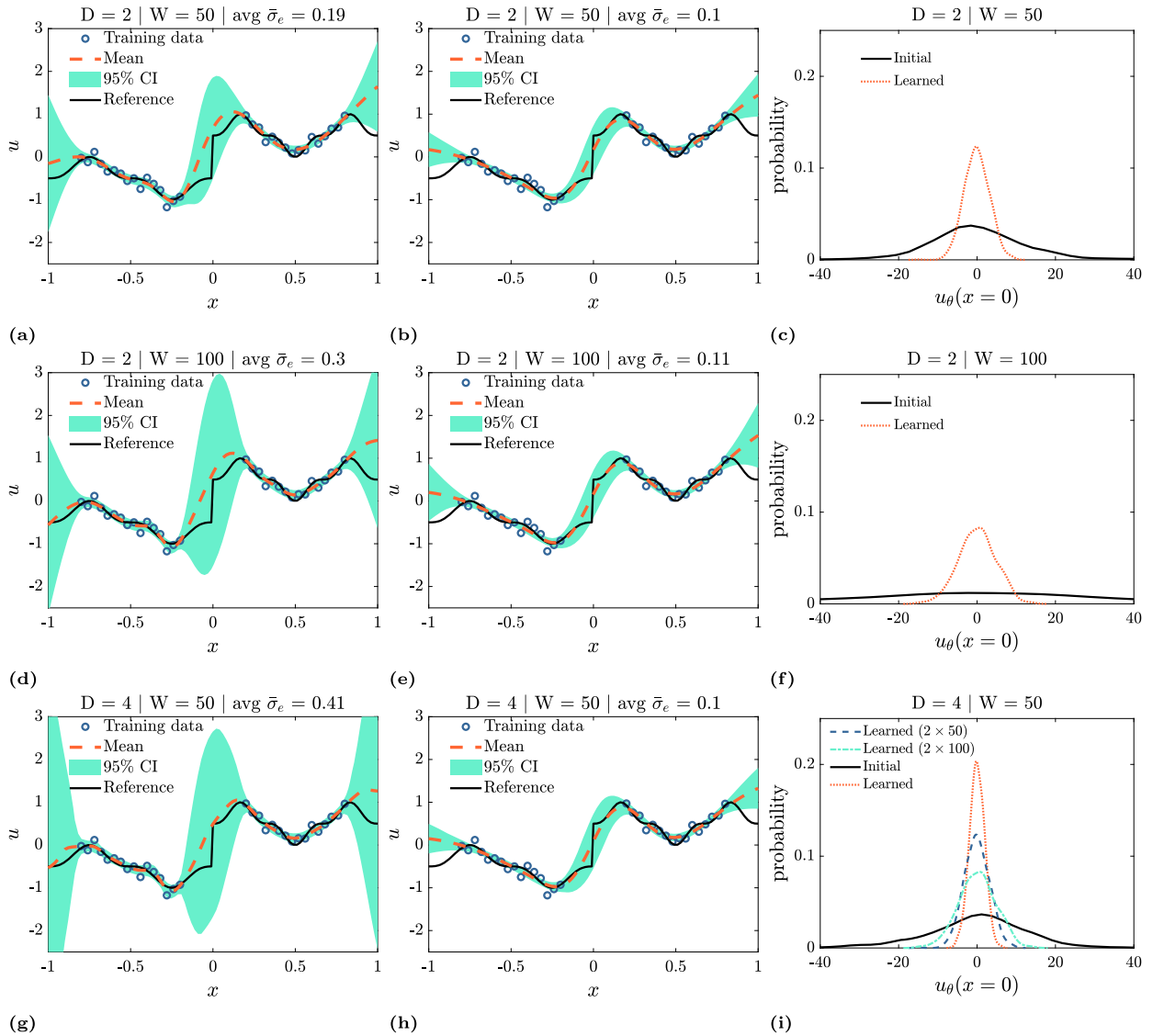


Fig. 16. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: epistemic uncertainty (avg σ_e^2 along x) increases with increasing NN expressivity. By learning the prior this effect can be partially reverted. This reduces the burden of thoroughly selecting the NN architecture, as long as the selected one is expressive enough for the considered problem. Shown here are the training data and exact function, as well as the mean and epistemic uncertainty (95% CI) predictions, as obtained by HMC in conjunction with three NN architectures defined by their depth (D) and width (W). Cases considered are (a)-(c) D, W = 2, 50; (d)-(f) D, W = 2, 100; (g)-(i) D, W = 4, 50. **Left:** HMC with fixed prior $\mathcal{N}(0, 1)$. **Middle:** HMC with prior learning. **Right:** effect of prior of learning on the distribution of $u_\theta(0)$ with θ following the initial and learned prior distribution.

Next, Fig. 19 shows the prior and posterior distributions of the NN parameters θ, of the prediction $u_\theta(x = 0)$, and of the predicted noise scale. They are obtained using HMC combined with Gibbs sampling for prior and noise scale learning (Section B.2.1). In all cases, the initially spread out prior and noise scale distributions concentrate while the corresponding quantities are learned during HMC combined with Gibbs sampling. Further, as explained in Section B.2.1 and corroborated in Fig. 19a, the learned prior approximates the "stacked" posterior during prior learning with Gibbs sampling, which is an one-dimensional distribution of the NN parameters stacked together. Detailed descriptions of the plots in Fig. 19 can be found in Section D.1.

6.1.3. Unknown Gaussian heteroscedastic noise

In this section, the data noise for each x follows $\mathcal{N}(0, (0.5|x|)^2)$; see Fig. 20. For implementing h-HMC, h-MFVI, and h-DEns, the NN approximator has two outputs; one for the mean $u_\theta(x)$ and one for the location-dependent variance $\sigma_u^2(x)$ of the Gaussian likelihood. Subsequently, we perform posterior inference using each one of HMC, MFVI, DEns to obtain samples $\{\hat{\theta}_j\}_{j=1}^M$, which we use in Eq. (11) to obtain total uncertainty including heteroscedastic noise. Further, for implementing h-

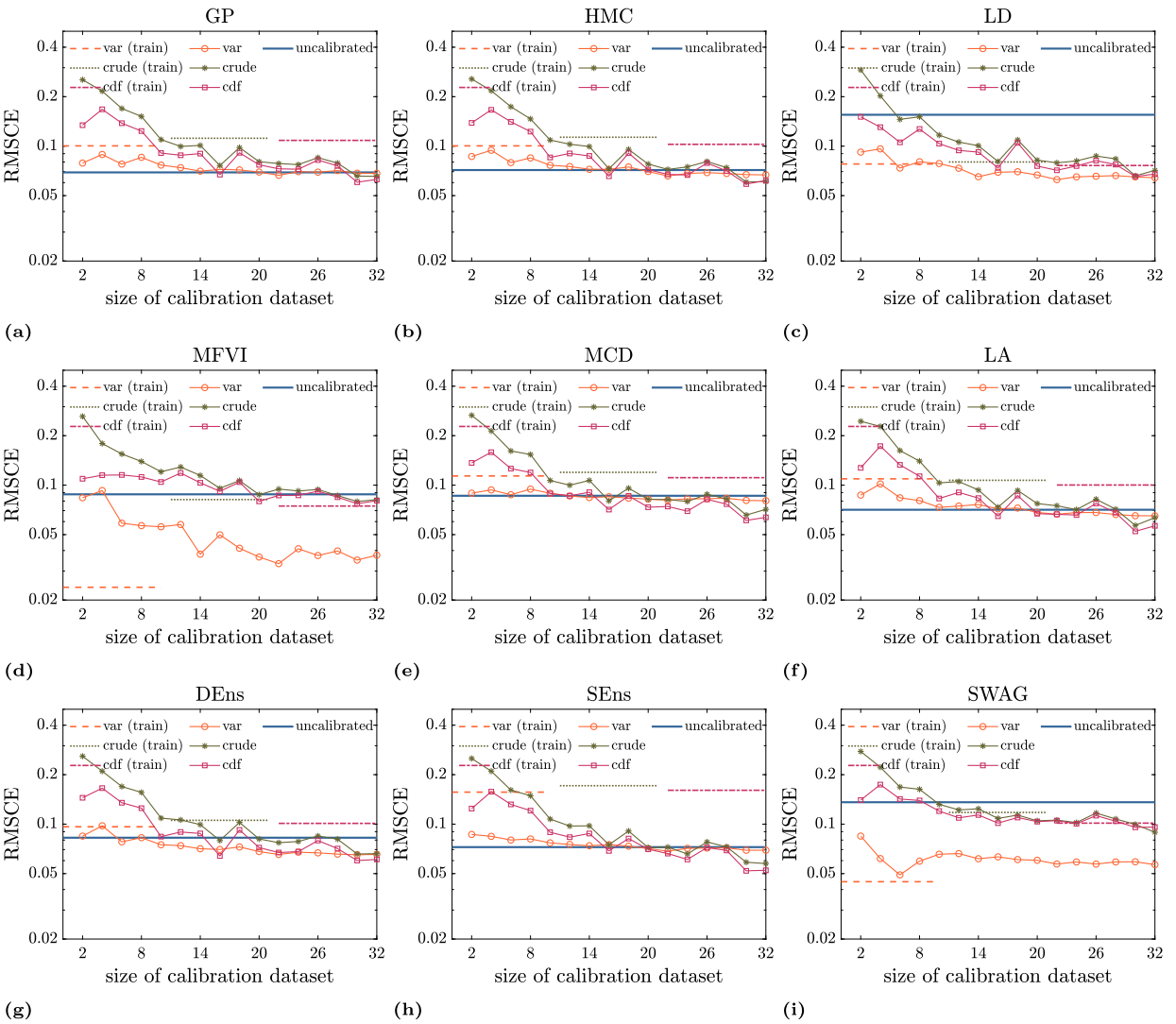


Fig. 17. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: post-training calibration, even with only a few calibration datapoints (2-14), can reduce significantly RMSCE; see Figs. D.1–D.3 for the effect on the predictions. Here we demonstrate the effect of different calibration approaches and calibration dataset sizes on RMSCE for all considered UQ methods. Variance re-weighting (var), CDF modification using an auxiliary model (cdf) and CRUDE are considered (Section 5.2), whereas the calibration set is either the training set or a separate set with varying size. Calibration using the training dataset typically leads to overfitting; see, e.g., part (a). Note that calibration is performed using ID data, hence OOD performance may deteriorate; see, e.g., SWAG in Table D.1.

HMC+FP, we pre-train a GAN-FP that learns, using historical data, the heteroscedastic aleatoric uncertainty. In this case, the historical data comprise of realizations from the BNN-FP, each one contaminated with a different realization from the noise following $\mathcal{N}(0, (0.5|x|)^2)$. For each x and for each random input θ, the GAN generator outputs a random output that depends on both x and θ, and a deterministic output that depends on x only; see Section D.9 for more details. The random and deterministic parts are combined and compared with the historical data for training the GAN. The GAN-FP learns the clean FP via the random output, and the heteroscedastic noise via the deterministic output. Subsequently, the NN approximator in h-HMC+FP has only one output, i.e., $u_\theta(x)$, and posterior inference for θ is performed using HMC.

In Fig. 21 and Table 7, we present the results obtained with h-HMC, h-MFVI, h-DEns, and h-HMC+FP. For ID data evaluation, it is shown that h-HMC+FP, which learns the heteroscedastic noise using a GAN-FP and historical data, outperforms the UQ methods that use a BNN-FP, in terms of accuracy of mean predictions (RL2E). Further, h-DEns has the highest predictive capacity (MPL), while h-HMC is the most calibrated one (RMSCE). Furthermore, the aleatoric uncertainty predictions of h-HMC, h-MFVI, h-DEns are satisfactory, especially given the limited amount of data for such task. For example, note that the data on the left side "happen" to be more noisy and this is reflected in the noise predictions. Note that the epistemic uncertainty of DEns is quite small. In our experiments, we found that this uncertainty depends on the weight

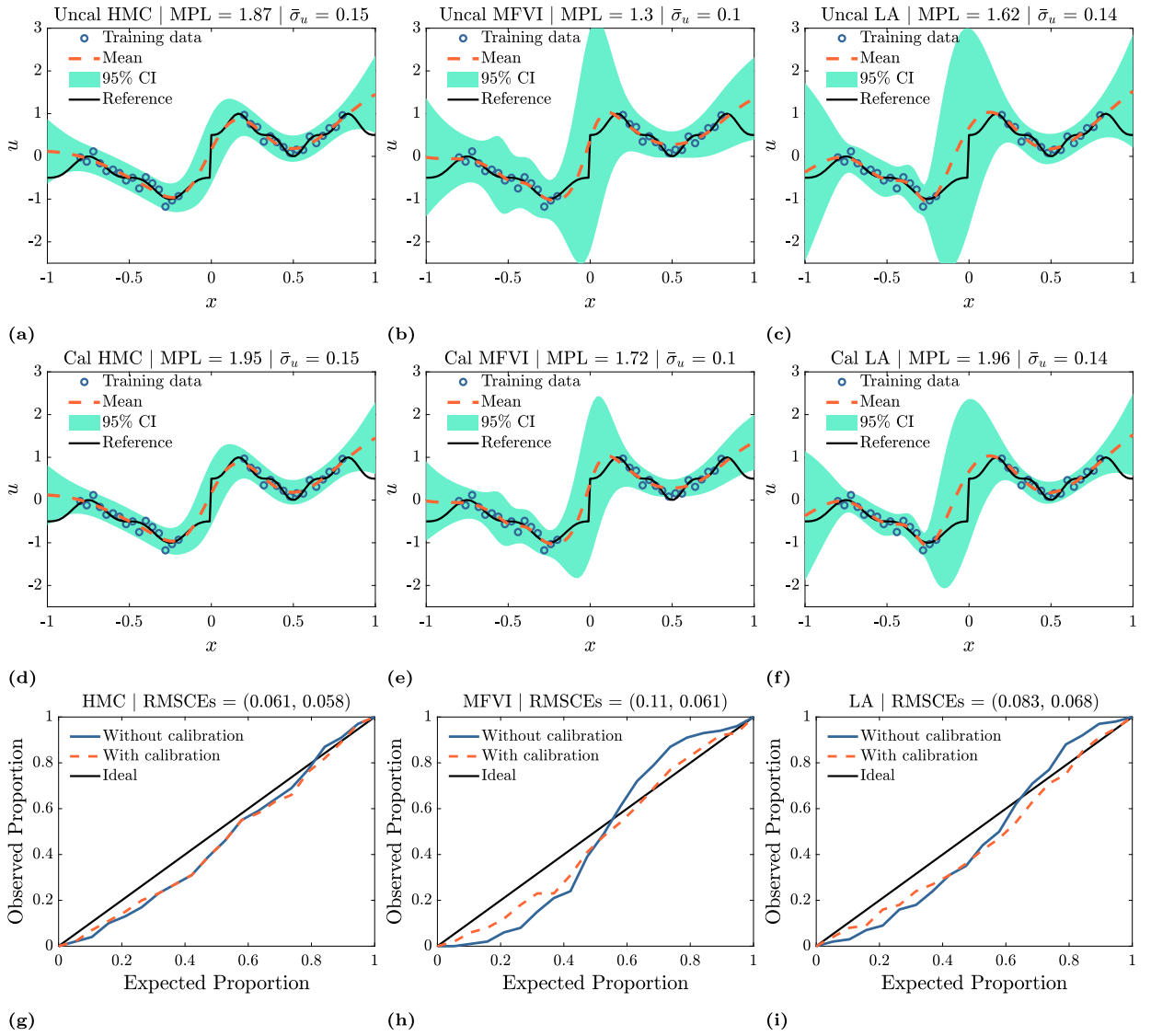


Fig. 18. Function approximation problem of Eq. (20) — *Unknown homoscedastic noise*: computationally cheaper techniques (MFVI and LA) perform comparably with HMC if their priors are learned. HMC remains the most calibrated candidate even following post-training calibration of the cheaper methods. Shown here are the training data and exact function, as well as the mean and total uncertainty (95% CI) predictions of HMC, MFVI, and LA with learned prior and noise scale. **Top row:** uncalibrated predictions. **Middle row:** calibrated predictions (Section 5.2). **Bottom row:** calibration plots (see also Fig. 12) and RMSCEs before and after post-training calibration (in the parentheses). Learned noise scales are denoted by $\bar{\sigma}_u$, while the reference is $\sigma_u = 0.1$. "Uncal" and "Cal" refer to results obtained before and after post-training calibration, respectively.

Table 7
Function approximation problem of Eq. (20) — *Unknown Gaussian heteroscedastic noise*: combining HMC with GAN using historical data to learn the heteroscedastic noise (h-HMC+FP) outperforms the UQ methods that use a BNN prior (h-HMC, h-MFVI, h-DEns). Here we evaluate ID and OOD performance using noisy test data and uncalibrated predictions. "h" indicates that heteroscedastic noise modeling has be used. See also Section D.2.2 for computational results pertaining to data with Student-t noise.

Metric	ID				OOD			
($\times 10^2$)	h-HMC	h-MFVI	h-DEns	h-HMC+FP	h-HMC	h-MFVI	h-DEns	h-HMC+FP
RL2E (↓)	48.9	49.1	48.5	**47.9**	59.0	73.1	**86.1**	66.8
MPL (↑)	96.6	110.1	**133.8**	96.1	60.5	**96.6**	57.7	68.4
RMSCE (↓)	**6.1**	8.5	9.5	6.9	17.5	26.3	40.1	**13.5**

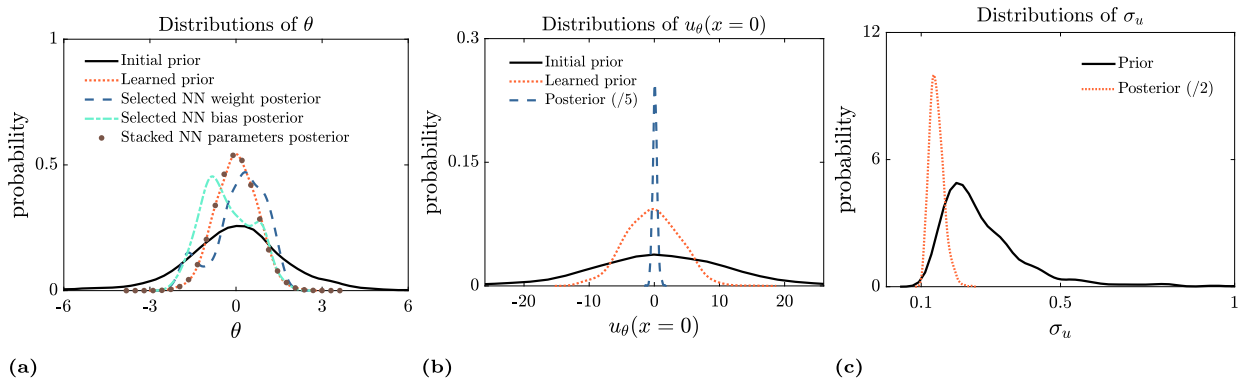


Fig. 19. Function approximation problem of Eq. (20) — *Unknown homoscedastic noise*: the initially spread out prior and noise scale distributions concentrate while the corresponding quantities are learned during HMC combined with Gibbs sampling. **(a)** Initial and learned prior $p(\theta)$, as well as posterior $p(\theta|\mathcal{D})$. For visualization, $p(\theta|\mathcal{D})$ is shown both for two randomly "picked" parameters, and for all dimensions of θ "stacked" together. Learned prior approximates the "stacked" posterior as explained in Section B.2.1. **(b)** Distributions of $u_\theta(0)$ based on prior and posterior distributions of θ from part (a) of the figure. **(c)** Prior $p(\sigma_u)$ and learned posterior $p(\sigma_u|\mathcal{D})$ of the noise scale.

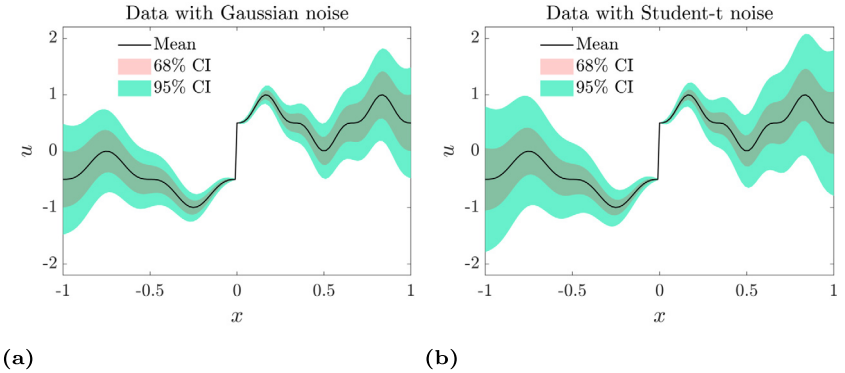


Fig. 20. Function approximation problem of Eq. (20) — *Unknown heteroscedastic noise*: data-generating process for Gaussian (a) and Student-t with 5 degrees of freedom (b) distributions. Although 68% noise CIs is comparable for the Gaussian and Student-t cases, the 95% intervals differ considerably because of the heavy tails of Student-t distribution. See Section D.2.2 for computational results pertaining to data with Student-t noise.

decay parameter. In this case, higher weight decay parameter values led to less epistemic uncertainty, but better predictions for the noise. Finally, see also Section D.2.2 for computational results pertaining to data with heteroscedastic Student-t noise.

6.1.4. Summary

In this section, we presented a comparative study pertaining to function approximation. For constant aleatoric uncertainty along x (homoscedastic noise) with known scale and Gaussian distribution, we compared nine methods, considered various noise scales and dataset sizes, and showed the effect of NN architecture on epistemic uncertainty. For homoscedastic noise with unknown scale, we compared three methods with learned priors, for approximating the function and learning the noise scale. For unknown aleatoric uncertainty with varying along x (heteroscedastic noise) Gaussian distribution, we compared four methods, including a method involving a GAN-FP, for approximating the function and learning the heteroscedastic noise. For Student-t heteroscedastic noise, we compared two methods for approximating the function and learning the noise using also posterior tempering.

Overall, we found that UQ methods with smaller computational cost and learned priors (e.g., MFVI and LA), can yield comparable performance with more expensive methods (e.g., HMC and DEns). Further, based on our experiments, post-training calibration using a small left-out dataset can reduce significantly the calibration error. We also showed that learning the NN parameter prior during posterior inference helps to reduce the burden of thorough NN architecture selection. An additional finding that has also been reported in other studies, e.g., [92,142], is that performance evaluation and comparison of different UQ methods requires a plurality of evaluation metrics. Further, we demonstrated that heteroscedastic noise modeling and learning with online or offline methods, e.g., functional priors, exhibits satisfactory accuracy.

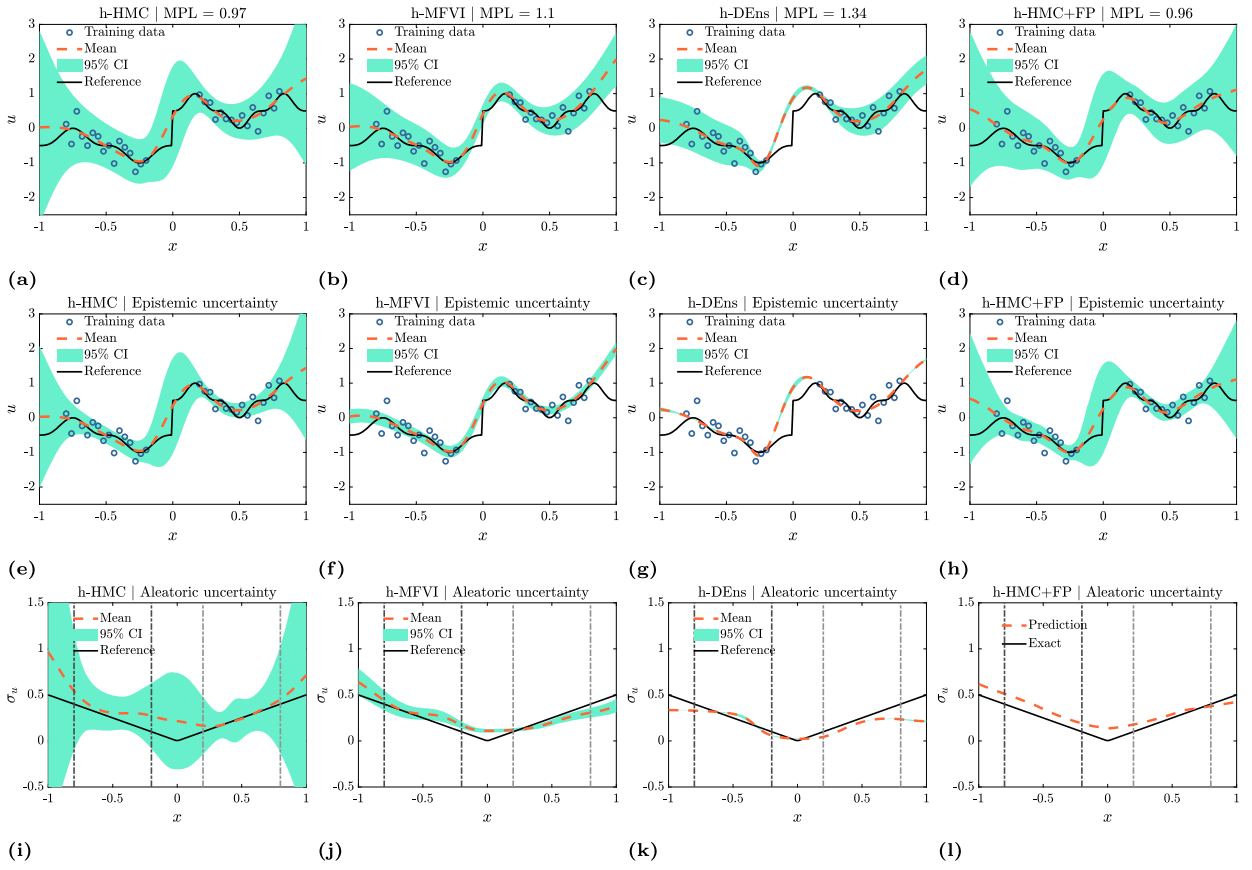


Fig. 21. Function approximation problem of Eq. (20) — *Unknown Gaussian heteroscedastic noise*: combining HMC with GAN using historical data to learn the heteroscedastic noise (h-HMC+FP) outperforms the UQ methods that use a BNN prior (h-HMC, h-MFVI, h-DEns). **Top row:** mean and total uncertainty of $u(x)$. **Middle row:** mean and epistemic uncertainty of $u(x)$. **Bottom row:** mean and epistemic uncertainty of predicted aleatoric uncertainty $\sigma_u(x)$. In the top two rows the training datapoints are also included, while in the bottom row the black and gray dot-dashed lines mark the training data intervals. Note that the aleatoric uncertainty predictions of h-HMC, h-MFVI, h-DEns are satisfactory, especially given the limited amount of data for such task. For example, note that the data on the left side happen to be more noisy and this is reflected in the noise predictions. "h" indicates that heteroscedastic noise modeling has been used.

6.2. Mixed deterministic PDE: nonlinear time-dependent diffusion-reaction equation

Next, we consider the deterministic 1D nonlinear time-dependent diffusion-reaction equation

$$\partial_t u = D\partial_x^2 u - \lambda(x)u^3 + f(x),\ t \in [0,1],\ x \in [-1,1], \tag{21a}$$

$$u(t,-1) = u(t,1) = 0.5,\ \text{and}\ u(0,x) = 0.5\cos^2(\pi x), \tag{21b}$$

$$\lambda(x) = 0.2 + \exp(x)\cos^2(2x), \tag{21c}$$

$$f(x) = \exp(-\frac{(x-0.25)^2}{2l^2})\sin^2(3x),\ l = 0.4, \tag{21d}$$

where t and x are the time and space coordinates, respectively, $u(t,x)$ is the sought solution, $\lambda(x)$ and $f(x)$ are the space-dependent reaction rate and source term, respectively, and $D = 0.01$ is the diffusion coefficient. Eq. (21) can be viewed as a special case of Eq. (1) with ξ fixed and dropped; $\mathcal{F}$ and $\mathcal{B}$ given by Eqs. (21a) and (21b), respectively, and with x given by (t,x). The objective of this example is to compare different UQ methods for solving a mixed problem involving Eq. (21), given noisy data of f, u, and λ. To this end, we consider the dataset of problem 2 in Table 1, consisting of $N_f = 15$ and $N_\lambda = 6$ measurements of $f(x)$ and $\lambda(x)$, respectively, as well as $N_u = 66$ measurements of $u(t,x)$ (11 locations along x and 6 locations along t). For the data of u, the locations along t are equidistant and the boundary/initial conditions of Eq. (21b) are considered as noisy measurements of u at the respective locations. Further, all of f, u, and λ data are contaminated with the same noise distribution, which is zero-mean Gaussian with different scales depending on the cases presented below. We emphasize that the UQ methods we study in this paper do not distinguish between data of u inside the domain and on the boundaries. That is, the simple boundary/initial conditions of Eq. (21b) are treated in exactly the same way as the rest

Table 8
Mixed PDE problem of Eq. (21) — *Standard case*: combining HMC with GAN using historical data to learn a FP for the reaction rate $\lambda(x)$ (HMC+FP) outperforms the UQ methods that use a BNN prior (HMC, MFVI, MCD, DEns) for learning $\lambda(x)$. Among HMC, MFVI, MCD, and DEns, the more expensive techniques (HMC and DEns) perform better. Here we evaluate the accuracy (RL2E) using clean test (reference) data, as well as the predictive capacity (MPL) and calibration error (RMSCE) using noisy test data, of the predictions for u (part a), f (part b), and λ (part c). We also evaluate the aleatoric uncertainty (noise) predictions (part c); "known" indicates that the noise was considered as known during training.

Standard case						
a	Metric ($\times 10^2$) for $u(t,x)$	HMC	MFVI	MCD	DEns	HMC+FP
	RL2E ($\downarrow$)	3.7	6.1	13.5	4.1	**1.6**
	Metric ($\times 10^2$) for $u(t=1,x)$	HMC	MFVI	MCD	DEns	HMC+FP
	RL2E ($\downarrow$)	2.5	4.2	6.8	2.8	**1.9**
	MPL ($\uparrow$)	545.5	435.7	386.5	**612.8**	568.4
	RMSCE ($\downarrow$)	**2.5**	7.0	6.1	3.9	3.1
b	Metric ($\times 10^2$) for $f(x)$	HMC	MFVI	MCD	DEns	HMC+FP
	RL2E ($\downarrow$)	8.3	10.0	10.9	8.2	**4.8**
	MPL ($\uparrow$)	474.4	459.2	441.0	**578.8**	554.4
	RMSCE ($\downarrow$)	10.5	8.8	5.9	12.3	**1.6**
c	Metric ($\times 10^2$) for $\lambda(x)$	HMC	MFVI	MCD	DEns	HMC+FP
	RL2E ($\downarrow$)	17.3	15.3	19.5	14.8	**2.6**
	MPL ($\uparrow$)	242.0	380.3	310.0	395.1	**559.8**
	RMSCE ($\downarrow$)	9.7	8.1	11.0	11.1	**2.5**
d	Reference	Learned noise (aleatoric uncertainty)				
	$\sigma_u = 0.05$	HMC	MFVI	MCD	DEns	HMC+FP
		0.051	0.052	known	0.041	known

Table 9
Mixed PDE problem of Eq. (21) — *Additional cases*: accuracy of the mean predictions (RL2E) deteriorates in the more challenging cases, as compared to the standard case of Table 8. However, using UQ we also obtain uncertainty estimates that cover in most cases the point-wise errors within the 95% CIs (approximately two standard deviations).

RL2E ($\times 10^2$) of quantity ($\downarrow$)	Additional case solved with HMC			
	Heteroscedastic §6.2.2	Large noise §D.3.2	Extrapolation §D.3.3	Steep boundary layers §D.3.4
$u(t,x)$	4.7	7.6	3.9	4.2
$u(t=1,x)$	3.2	5.1	2.8	4.1
$f(x)$	9.0	15.1	11.1	40.3
$\lambda(x)$	13.1	19.4	15.9	22.3

of the noisy measurements of u. As a result, these measurements can be noisy, limited, or even not available. This flexibility enables these methods to be used in multiple contexts and problem scenarios. Clearly, however, the amount of available data and the locations in the domain affect the well-posedness of the problem, as well as the decision about what method should ultimately be employed. Here, we use the *Matlab PDE toolbox* with a uniform grid $x \times t = 101 \times 101$ to obtain the reference solutions as well as the training data that we present in the following results.

We solve all problem cases described below using a U-PINN with different functional priors and posterior inference methods. In Section 6.2.1, we solve the mixed problem, where the noise is homoscedastic (constant) with scales $\sigma_u = \sigma_f = \sigma_\lambda = 0.05$, with five methods of Table 2. We refer to this case as "standard" because the additional considered cases have equivalent setups, except for various modifications that make them more challenging to treat. Specifically, we consider a "heteroscedastic" case in Section 6.2.2 with space-dependent noise scale; a "large noise" case in Section D.3.2 with a larger noise scale compared to the standard case; an "extrapolation" case, where the measurements of $\lambda(x)$ are concentrated in the interval $x \in [0,1]$ and we extrapolate for $x \in [-1,0]$; and a "steep boundary layers" case in Section D.3.4, where the functions $u(t,x)$ and $\lambda(x)$ have large gradients for $x \approx -1$ and $x \approx 1$. The hyperparameters and the NN architectures that we used are summarized in Section D.8 and D.9, respectively.

6.2.1. Standard case: homoscedastic noise and uniform measurements in space and time

In this section, we solve the mixed PDE problem of Eq. (21) with data noise scales equal to $\sigma_u = \sigma_f = \sigma_\lambda = 0.05$. For distributing uniformly the measurements along x, we have considered equidistant points for each of u, f and λ, containing small perturbations in the respective coordinate x values. The objective of this section is to demonstrate and compare the capabilities of five UQ methods of Table 2 in a problem where the noise scale and the NN parameter prior are unknown. In this regard, we consider two NN approximators, one for u and one for λ, as in Fig. 7 (without the outputs for the noise). The outputs of the two NNs are substituted into Eq. (21a) for producing the approximated f. In U-PINN with HMC, we learn the prior and the noise scale using Gibbs sampling (Section B.2.1). In U-PINN with MFVI, we fix the prior, learn the mean

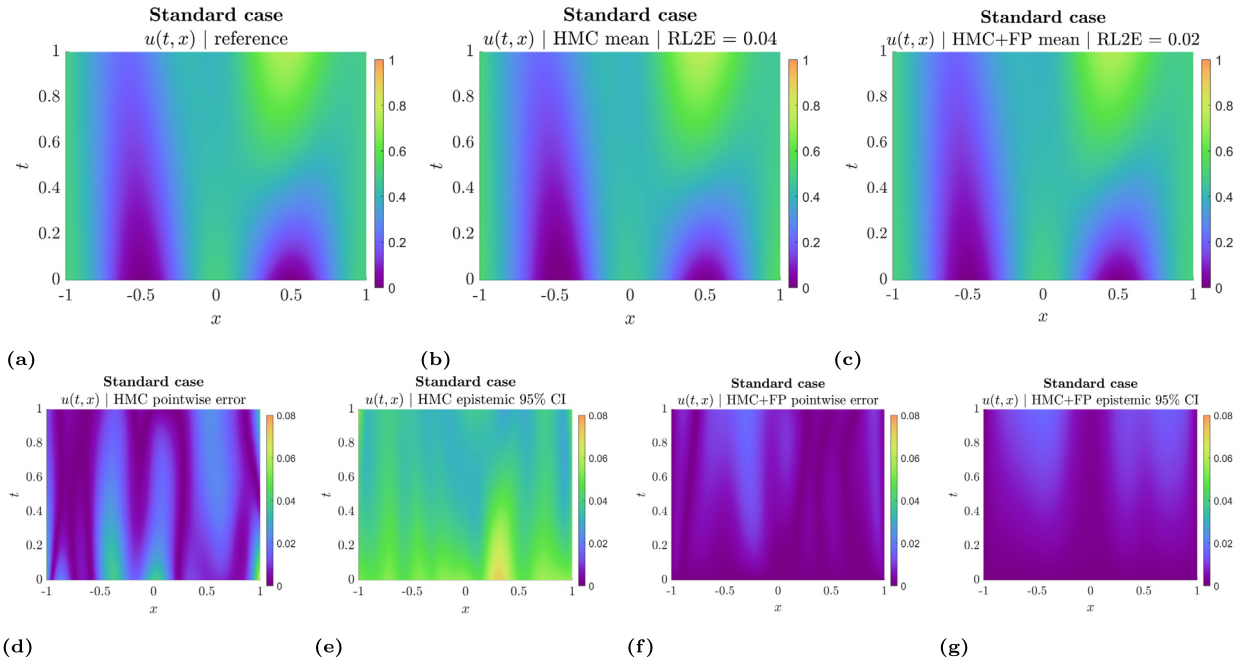


Fig. 22. Mixed PDE problem of Eq. (21) — *Standard case*: for predicting $u(t,x)$, HMC (BNN prior) and HMC+FP (GAN prior) yield similar mean predictions, whereas HMC is less confident than HMC+FP (larger epistemic uncertainty). In both cases, however, the absolute error for each x is within the 95% confidence interval (CI) of total (epistemic + aleatoric) uncertainty. **Top row:** reference solution $u(t,x)$, and mean prediction for $u(t,x)$ obtained by HMC and HMC+FP. **Bottom row:** point-wise absolute error and epistemic uncertainty obtained by HMC and HMC+FP.

of posterior Gaussian distribution with its standard deviation fixed and tuned, and learn the noise scale by optimization. In U-PINN with MCD, we tune the dropout rate and assume known noise scale; for unknown noise scale the results were not meaningful. In U-PINN with DEns, we tune the weight decay constant and learn the noise scale. In U-PINN with HMC+FP, i.e., PI-GAN-FP of Table 2, we first pre-train a PI-GAN with synthetic historical data of $\lambda(x)$ and $f(x)$. In particular, we generate the historical data using

$$\lambda(x) = 0.2 + \exp(r_{\lambda,1}x)\cos^2(r_{\lambda,2}x),\ r_{\lambda,1} = \frac{1}{2}(r_{k,1}+1),\ r_{\lambda,2} = r_{k,2}+3, \tag{22}$$

$$r_{k,i} \sim \mathcal{U}[0,1],\ i=1,2, \tag{23}$$

$$f(x) = \exp(-\frac{(x-0.25)^2}{2l^2})\sin^2(r_f x),\ l = 0.2 + 0.8r_{f,1},\ r_f = 1 + 3r_{f,2}, \tag{24}$$

$$r_{f,i} \sim \mathcal{U}[0,1],\ i=1,2, \tag{25}$$

and use 1,000 samples for λ and f in the training of PI-GANs. Subsequently, we use the PI-GAN-FP to perform HMC posterior inference for θ, which is the random input of the GAN generator.

Table 8 summarizes the performance evaluation of the considered UQ methods using the metrics of Section 5. As expected, HMC+FP outperforms the rest of the methods in most comparisons. Among the methods that use a BNN prior, i.e., HMC, MFVI, MCD, and DEns, the more expensive techniques (HMC and DEns) perform better.

Further, in Fig. 22 we provide the space- and time-dependent predictions for $u(t,x)$ and compare with the reference solution. As shown in Fig. 22e and g, although HMC and HMC+FP yield similar mean predictions for u, HMC is less confident than HMC+FP, i.e., it has larger epistemic uncertainty. This is due to the fact that HMC+FP has been pre-trained using pertinent historical data.

Next, in Figs. 23–25 (and D.8–D.9) we present the results obtained for $u(t=1,x)$, $f(x)$, and $\lambda(x)$, using HMC, DEns, and HMC+FP (and MFVI as well as MCD). It is shown that DEns estimates the mean of $\lambda(x)$ more accurately than HMC, although it is over-confident, i.e., it has point-wise errors that are not covered by the total uncertainty. On the contrary, HMC+FP estimates $f(x)$ and $\lambda(x)$ more accurately than both HMC and DEns, and it is almost perfectly calibrated (not over- or under-confident). This is reflected in the calibration error in Table 8, where it is shown that the RMSCE of HMC+FP is at least six times smaller than the RMSCE of the rest of the methods. Finally, for all UQ methods considered here, the absolute error for u at each x is, in most cases, within the 95% CI of total (epistemic+aleatoric) uncertainty, i.e., within approximately two standard deviations.

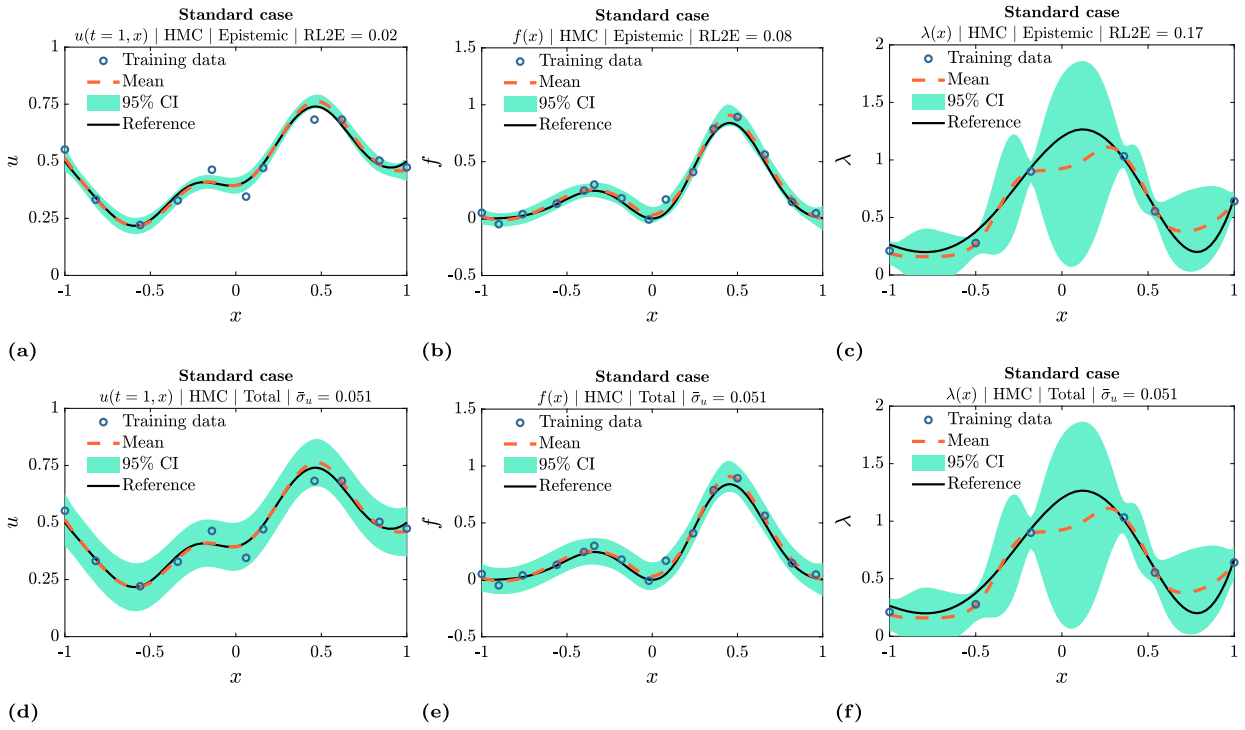


Fig. 23. Mixed PDE problem of Eq. (21) — *Standard case*: epistemic uncertainty of HMC increases with more noise and with less amount of data, and is affected by the interdependence of u, f, and λ through Eq. (21a). Shown here are the training data, reference functions, as well as the mean and uncertainty (95% CI) predictions of HMC for u, f, and λ. We learn the prior via Gibbs sampling; see Section 4.1. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *learned* amount of aleatoric uncertainty, $\bar{\sigma}_u = 0.051$.

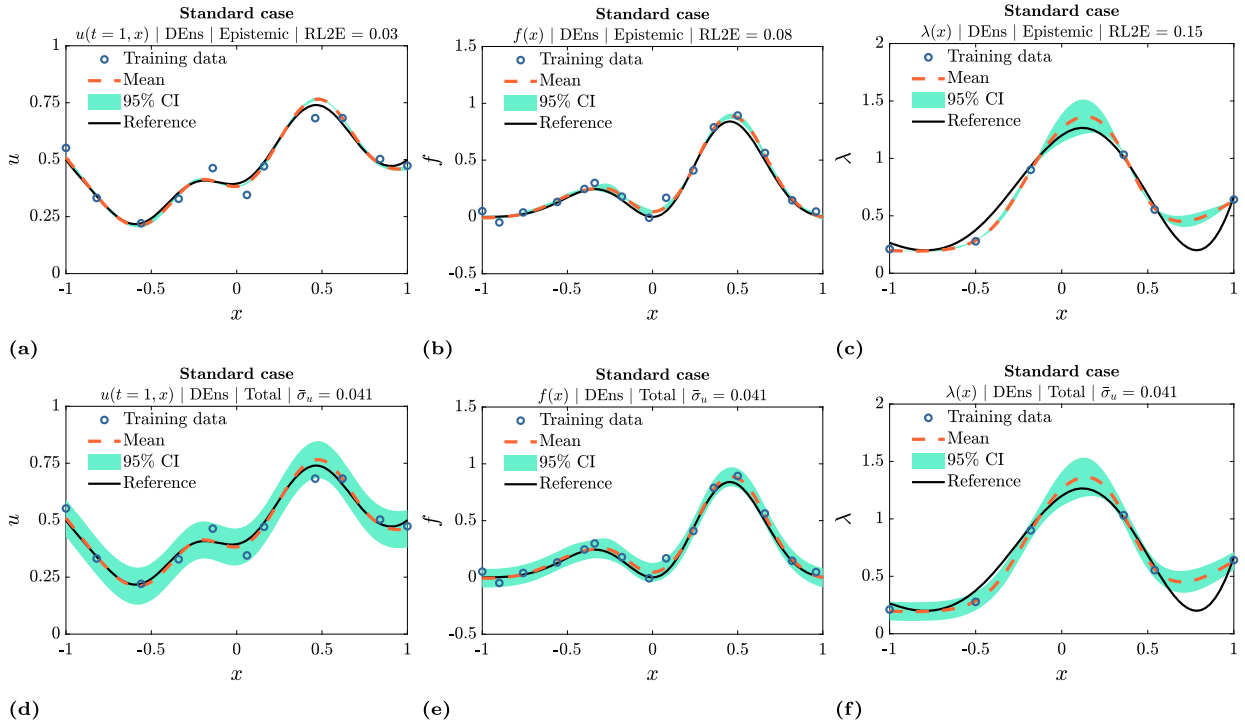


Fig. 24. Mixed PDE problem of Eq. (21) — *Standard case*: DEns estimates $\lambda(x)$ more accurately than HMC, although it is over-confident (small epistemic uncertainty). Shown here are the training data, reference functions, as well as the mean and uncertainty (95% CI) predictions of DEns for u, f, and λ. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *learned* amount of aleatoric uncertainty, $\bar{\sigma}_u = 0.041$.

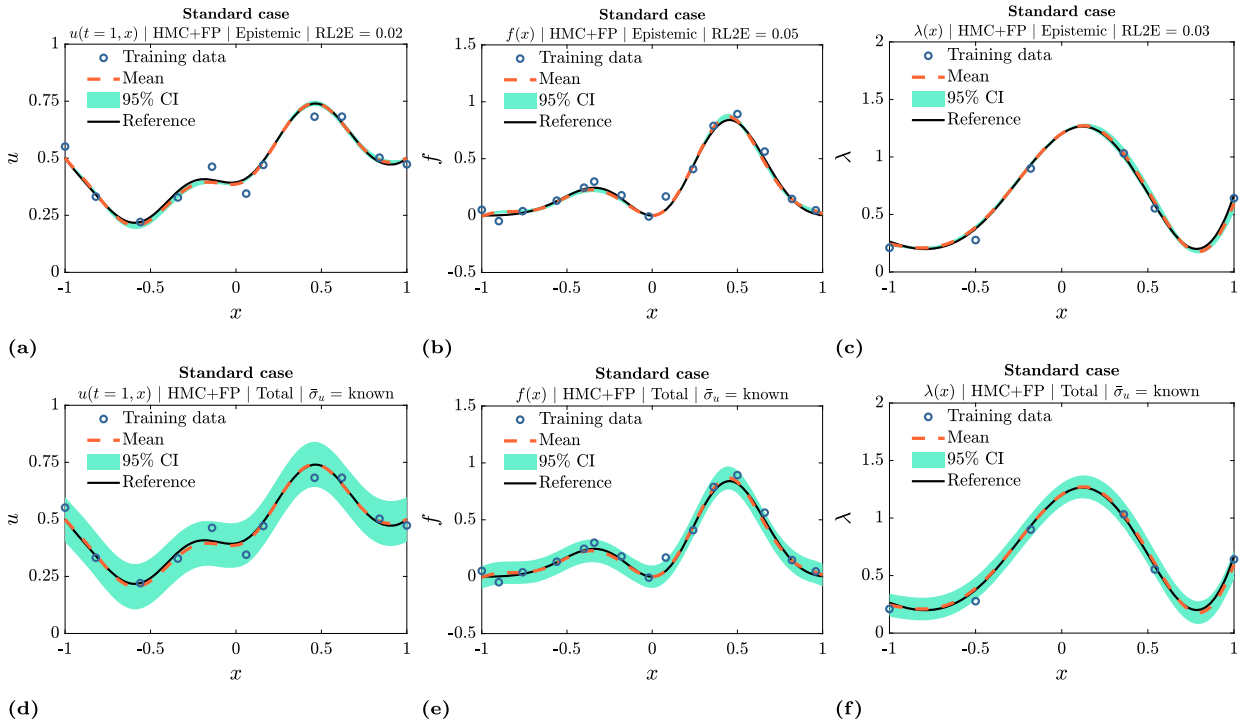


Fig. 25. Mixed PDE problem of Eq. (21) — *Standard case*: HMC+FP estimates $\lambda(x)$ more accurately than both HMC and DEns, and it is almost perfectly calibrated (not over- or under-confident). Shown here are the training data, reference functions, as well as the mean and uncertainty (95% CI) predictions of HMC+FP for u, f, and λ. Results were obtained by pre-training a GAN-FP using historical data of $\lambda(x)$, and subsequently performing HMC posterior inference given the training data shown here; see Section 4.3. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the considered as *known* amount of aleatoric uncertainty, $\bar{\sigma}_u = 0.05$.

6.2.2. Heteroscedastic noise case: larger noise scales close to the boundaries of the space domain

In this section, we solve the mixed PDE problem of Eq. (21a) with space-dependent, i.e., heteroscedastic, noise with noise scales given as $\sigma_u = \sigma_f = \sigma_\lambda = 0.1|x|$. That is, data is more noisy close to the boundaries of the space domain. The training dataset locations are the same as in Section 6.2.1 and we solve the problem using a U-PINN combined with HMC for posterior inference. We refer to this method as h-U-PINN-HMC or just h-HMC, for simplicity; see Table 2. In this regard, the output of the NN of λ that we considered in Section 6.2.1 is augmented by an additional output for the aleatoric uncertainty, as in Fig. 7. We perform posterior inference using HMC to obtain samples $\{\hat{\theta}_j\}_{j=1}^M$, which we use in Eq. (11) to obtain total uncertainty including heteroscedastic noise. In Fig. 26, we present the mean predictions as well as epistemic and total uncertainties, as obtained for $u(t=1,x)$, $f(x)$, and $\lambda(x)$. In Fig. 26a-c we notice that epistemic uncertainty of h-HMC increases with more noise, i.e., close to the boundaries of the space domain. It also increases with less amount of data, i.e., between datapoints and especially for $\lambda(x)$ in Fig. 26c for which the amount of data is small. Further, the learned noise is relatively accurate, whereas its respective point-wise error is within the 95% CI of its corresponding uncertainty (see Fig. 26d). In Table 9, we evaluate the accuracy of the mean predictions (RL2E) for all additional cases, including the heteroscedastic noise case of this section as well as the cases of Sections D.3.2-D.3.4. Clearly, accuracy of the mean predictions deteriorates in the more challenging cases, as compared to the standard case of Table 8. However, using UQ we also obtain uncertainty estimates that cover in most cases the point-wise errors within the 95% CIs.

6.2.3. Summary

In this section, we presented a comparative study pertaining to a mixed PDE problem. For the "standard" case, pertaining to constant noise and equidistant measurements, we compared five methods, including a method involving a GAN-FP, termed PI-GAN-FP, for solving the problem and learning the noise. For the "additional cases", pertaining to heteroscedastic noise, large constant noise, extrapolation and steep boundary layers, we presented results obtained by HMC. Overall, we showed that employing a PI-GAN-FP, pre-trained with historical data of the source term and the problem parameters, exhibits higher accuracy than the BNN-based UQ methods (HMC, MFVI, MCD, and DEns). Among HMC, MFVI, MCD, and DEns, the more expensive techniques (HMC and DEns) perform better, while we estimated the unknown noise satisfactorily with HMC, MFVI, and DEns. For challenging cases involving large noise, data concentrated on one side of the space domain that require extrapolation, and functions with large gradients, the proposed UQ methods still perform satisfactorily, with uncertainties that cover the errors in most cases.

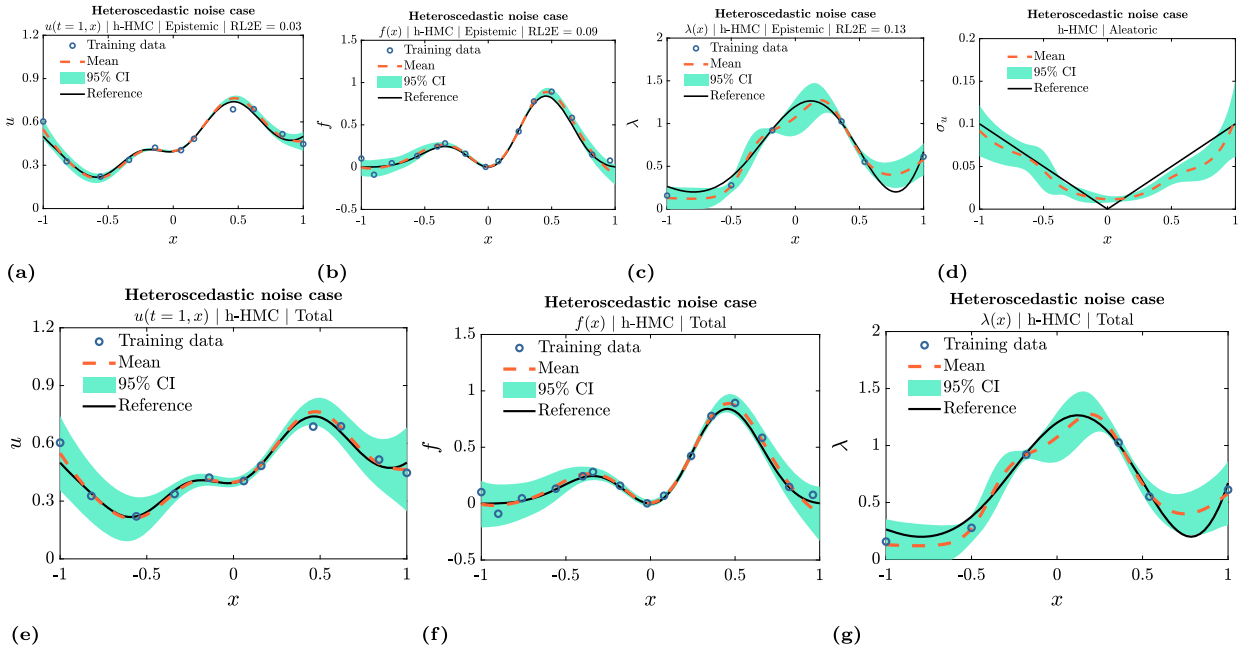


Fig. 26. Mixed PDE problem of Eq. (21) — *Heteroscedastic noise case*: epistemic uncertainty of h-HMC increases with more noise ($x \approx -1$ and $x \approx 1$) and with less amount of data, and is affected by the interdependence of u, f, and λ through Eq. (21a). Shown here are the training data, reference functions, as well as the mean and uncertainty (95% CI) predictions of h-HMC for u, f, and λ. We fix the prior to $\mathcal{N}(0,1)$ for all parameters, and use an additional NN output for learning the space-dependent aleatoric uncertainty; see Fig. 7. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *learned* amount of space-dependent aleatoric uncertainty, shown in part (d). The learned noise is relatively accurate, whereas the respective point-wise error is within the 95% CI of its corresponding uncertainty.

6.3. Mixed stochastic elliptic equation

In this section, we consider the stochastic elliptic equation

$$-\frac{1}{10}\frac{d}{dx}\left(\lambda(x;\xi)\frac{d}{dx}u(x;\xi)\right) = f(x,\xi), \tag{26}$$

where

$$f(x,\xi) \sim GP\left(\frac{1}{2}, \frac{9}{400}\exp\left(-(x_1-x_2)^2\right)\right), \tag{27a}$$

$$\lambda(x;\xi) = \exp(\frac{1}{5}\sin(\frac{3}{2}\pi(x+1)) + \tilde{\lambda}), \tag{27b}$$

$$\tilde{\lambda} \sim GP\left(0, \frac{1}{100}\exp\left(-(x_1-x_2)^2\right)\right). \tag{27c}$$

Eq. (26) can be viewed as a special case of Eq. (1) without the boundary term of Eq. (1b), with $\mathcal{F}$ given by Eq. (26), and with each $\xi \in \Xi$ being a different random event corresponding to a different sample from the GPs of f and λ. The objective of this example is to compare different UQ methods for solving a mixed problem involving Eq. (26), given clean or noisy stochastic realizations of f, u, and λ. To this end, we consider the dataset of problem 3 in Table 1, consisting of $N = 1{,}000$ realizations of f, u, and λ, with each of the three quantities sampled at a different number of locations. Specifically, $N_f = 13$ sensors are considered for f, $N_u = 9$ for u, and $N_\lambda = 5$ for λ. Further, there is no boundary term b and the boundary data are included in the aforementioned datasets. For the noisy data case, the data is contaminated with Gaussian homoscedastic (constant) noise with scales $\sigma_u = \sigma_f = \sigma_\lambda = 0.01$. Next, we implement the three methods of Section 4.4, namely U-PI-GAN, U-NNPC, and U-NNPC+, to solve the problem. All methods are combined with DEns of Section 4.2 for performing posterior inference of the β parameters of each method. That is, we train each model $M = 3$ times independently and with different NN initializations for obtaining three predictive models that we use in Eqs. (15)-(16). To obtain the reference solutions that we present in the following results, we use the finite difference method. The hyperparameters and the NN architectures that we used are summarized in Section D.8 and D.9, respectively.

For obtaining the first- and second-order statistics of the sought processes u and λ, consider as an example the U-PI-GAN for predicting u. For each of the three PI-GANs that we have trained, we first produce 500 new samples of u. Using these 500 samples, we can compute the mean and standard deviation for each x. That is, given the three trained PI-GANs,

Table 10
Mixed stochastic problem of Eq. (26): the herein proposed U-NNPC+ performs better in this case and is robust to noisy data. Here we evaluate the accuracy (RL2E) of U-PI-GAN, U-NNPC, and U-NNPC+ in terms of mean and standard deviation (std) predictions corresponding to the partially unknown stochastic processes $u(x;\xi)$ and $\lambda(x;\xi)$ of Eq. (26). The training data consist of 1,000 clean or noisy realizations of f, u, and λ, while the test (reference) data we use to calculate RL2E are clean in all cases.

a	Metric ($\times 10^2$)	Clean data			Noisy data		
	corresponding to u	U-PI-GAN	U-NNPC	U-NNPC+	U-PI-GAN	U-NNPC	U-NNPC+
	mean RL2E (↓)	2.1	**0.2**	**0.2**	3.5	**3.2**	**3.2**
	std RL2E (↓)	6	**0.6**	0.7	8.2	**3.8**	**3.8**
b	Metric ($\times 10^2$)	Clean data			Noisy data		
	corresponding to λ	U-PI-GAN	U-NNPC	U-NNPC+	U-PI-GAN	U-NNPC	U-NNPC+
	mean RL2E (↓)	**1.5**	2.2	1.6	2.3	4.3	**1.3**
	std RL2E (↓)	6.7	5.7	**4.3**	7.7	4.8	**4**

we obtain three predictions for the mean and for the standard deviation of u. We use them to compute the mean of the mean and standard deviation predictions. The variability among these three predictions for the mean and the standard deviation corresponds to epistemic uncertainty. We compare with the corresponding statistics from a set of 500 new, unseen during training, samples from f and λ and corresponding reference solutions u. Finally, by discretizing the domain of x into 201 points, we also compute the covariance matrix of the predictions of each PI-GAN evaluated on the discrete points. Subsequently, we obtain the 10 dominant eigenvalues for each of the three trained PI-GANs. Clearly, the eigenvalues also have variability due to epistemic uncertainty; i.e., due to different optimizations of PI-GAN. Note that U-NNPC and U-NNPC+ cannot produce new stochastic samples by sampling from a known distribution, as done by U-PI-GAN. U-NNPC and U-NNPC+ require a new sample from the stochasticity-inducing functions, i.e., from f and λ, for producing a new sample for u. For this reason, we can obtain the statistics of the predicted stochastic processes based on the training realizations. This is equivalent to Monte Carlo simulation, as performed using traditional solvers; see, e.g., [157]. Nevertheless, in this paper we use the 500 unseen samples of f and λ for producing the corresponding predictions of u and λ using the trained U-NNPC and U-NNPC+.

Table 10 summarizes the performance of the considered UQ methods for predicting the mean and the standard deviation (std) of the partially unknown stochastic processes $u(x;\xi)$ and $\lambda(x;\xi)$ of Eq. (26). In addition, Fig. 27 presents the errors of the mean and standard deviation predictions of the UQ methods, as obtained by training with noisy stochastic realizations. Although, the herein proposed U-NNPC+ performs better in this case and is robust to noisy data, all three methods approximate relatively accurately the sought stochastic processes $u(x;\xi)$ and $\lambda(x;\xi)$. Finally, in Fig. 28 we present the spectra of the learned stochastic processes. Specifically, we provide the first 10 eigenvalues of the covariance matrices of $u(x;\xi)$ and $\lambda(x;\xi)$, as obtained by the unseen realizations of λ, the corresponding reference solutions u, and the predictions of U-PIGAN, U-NNPC, and U-NNPC+. The results were obtained using noisy training realizations. For each eigenvalue, the mean and the epistemic uncertainty 95% CI are provided. It is seen that all methods approximate relatively accurately the most significant parts of the spectra of u and λ. In passing, note that in D.13 we present the results corresponding to a post-training calibration experiment. In some cases, even with a few (2-10) left-out stochastic realizations, we can reduce the calibration error (RMSCE) by a factor of 3; see, e.g., Fig. D.13c.

6.3.1. Summary

In this section we solved a mixed SPDE problem. We considered three methods, namely U-PI-GAN, U-NNPC, and the herein proposed U-NNPC+, which were combined with an ensemble of three independent training sessions of the involved NNs. We observed that U-NNPC+ can outperform existing methods for treating cases with noisy stochastic realizations. Nevertheless, we found in our experiments that epistemic uncertainty obtained by combining deep ensembles with the considered methods may not cover the errors in predicting the first- and second-order statistics of the quantities of interest. Further, the epistemic uncertainty of U-NNPC+ was reduced, as compared to U-NNPC, leading to over-confident predictions.

6.4. Forward deterministic PDE: nonlinear static diffusion-reaction equation

In this section, we consider the deterministic 1D nonlinear static diffusion-reaction equation

$$D\partial_x^2 u - \lambda u^3 = f,\ x \in [-1, 1], \tag{28}$$

where $D = 0.01$ is the diffusion coefficient, λ is the constant reaction rate, while $f(x)$ and $u(x)$ are the source term and the sought solution, respectively. We set the exact solution as $u(x) = \sin^3(6x)$, and $f(x)$ can then be derived given the exact $u(x)$. Similarly to Section 6.2, Eq. (28) can be viewed as a special case of Eq. (1), with ξ fixed and dropped, and $\mathcal{F}$ given by Eq. (28). In this case, however, the model parameter λ is known and equal to 0.7. The objective of this example is to compare U-PINN with the herein proposed GP+PI-GAN for solving a forward PDE problem involving Eq. (28), given noisy data of f as well as noisy boundary conditions for u. To this end, we consider the dataset of problem 1 in Table 1, consisting of $N_f = 15$ or 6 measurements of $f(x)$, and 2 measurements on $u(x)$ at $x = -1$ and 1. The data for $f(x)$ and

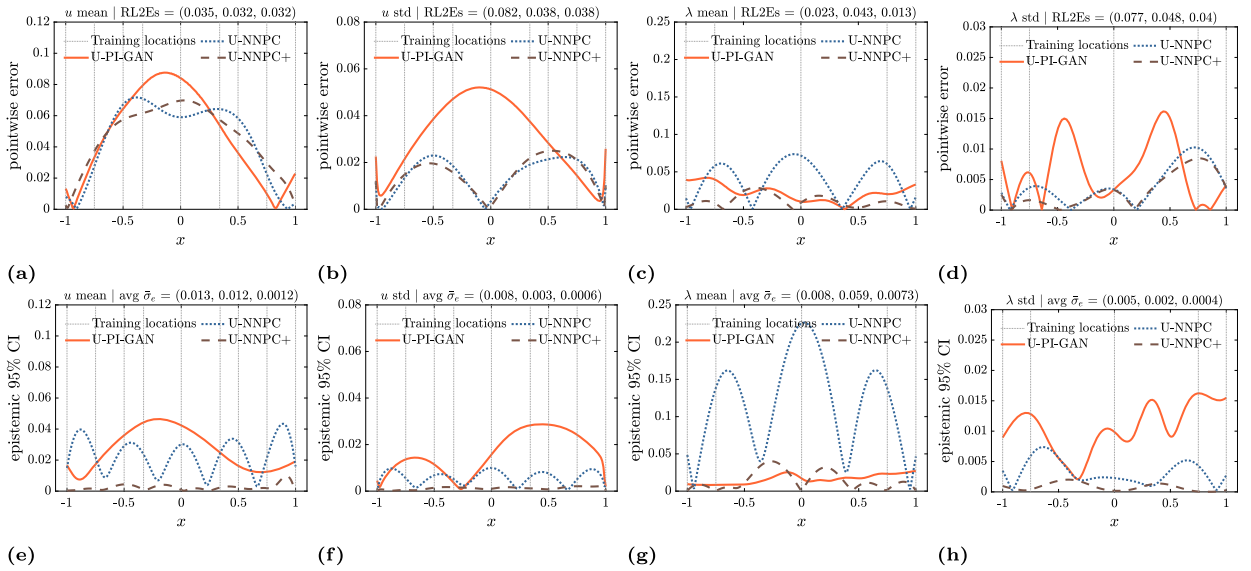


Fig. 27. Mixed stochastic problem of Eq. (26): the herein proposed U-NNPC+ performs better in this case and is robust to noisy data. **Top row:** Absolute point-wise errors of the mean and standard deviation (std) of $u(x;\xi)$ and $\lambda(x;\xi)$, as obtained by the unseen realizations of λ (reference), the corresponding solutions u (reference), and the predictions of U-PIGAN, U-NNPC, and U-NNPC+ (trained with noisy stochastic realizations). The RL2E values are also shown in the parentheses for U-PIGAN, U-NNPC, and U-NNPC+. **Bottom row:** Corresponding epistemic uncertainty (95% CI - approximately two standard deviations) of the mean and std of $u(x;\xi)$ and $\lambda(x;\xi)$, as obtained by combining the methods with DEns of Section 4.2. The average along x epistemic standard deviation (avg $\bar{\sigma}_e$) is also shown in the parentheses for U-PIGAN, U-NNPC, and U-NNPC+. In both rows, the sensor locations for the noisy data of u and λ are shown with vertical dashed lines. For both u and λ, for all UQ methods employed, and for most x locations, the errors are not covered within the 95% CIs. Note that the epistemic uncertainty of U-NNPC+ is considerably smaller than the other two methods.

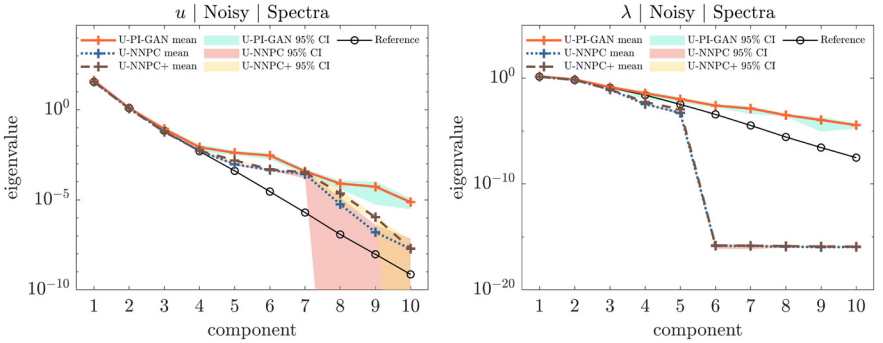


Fig. 28. Mixed stochastic problem of Eq. (26): U-PI-GAN, U-NNPC, and U-NNPC+ approximate relatively accurately the most significant parts of the spectra of u and λ. Shown here are the first 10 eigenvalues of the covariance matrices of $u(x;\xi)$ and $\lambda(x;\xi)$, as obtained by the unseen realizations of λ (reference), the corresponding solutions u (reference), and the predictions of U-PIGAN, U-NNPC, and U-NNPC+. The results were obtained using noisy training realizations. For each eigenvalue, the mean and the epistemic uncertainty 95% CI are provided, as obtained by combining the methods with DEns of Section 4.2. All covariance matrices were obtained using 201 points for x.

$u(x)$ are both contaminated with noise following $\mathcal{N}(0, 0.1^2)$. The hyperparameters and the NN architectures that we used are summarized in Section D.8 and D.9, respectively.

We solve the problem using a standard U-PINN, i.e., with a BNN-FP, and HMC for posterior inference, similarly to Section 6.2. The mean and total uncertainty of $f(x)$ are shown in Figs. 29a ($N_f = 15$) and D.14a ($N_f = 6$), while the corresponding results for $u(x)$ are shown in Figs. 29c ($N_f = 15$) and D.14c ($N_f = 6$). To implement GP+PI-GAN, we first fit the 15 or 6 measurements of f using a GP with a squared exponential kernel (Section B.1). Based on this GP, we produce 10,000 realizations of $f(x)$ at 101 equidistant locations in the domain of x. The statistics of these realizations are shown in Figs. 29d ($N_f = 15$) and D.14d ($N_f = 6$). Subsequently, we treat the problem as stochastic and employ a PI-GAN to solve it (Section 4.4). Following the training of PI-GAN, we produce 1,000 realizations of $u(x)$ and plot the statistics in Figs. 29f ($N_f = 15$) and D.14f ($N_f = 6$). The uncertainty of u in Figs. 29 and D.14 can also be construed as epistemic uncertainty, because it includes the uncertainty due to the limited and noisy data of f. However, it does not include the uncertainty due to fitting a PI-GAN; i.e., we trained PI-GAN only once, in contrast to U-PI-GAN of Section 6.3, where we trained and combined three PI-GANs. Further, in the middle panels of Figs. 29 and D.14 we present the prior and posterior distributions of f evaluated at $x = 1$, i.e., at the right edge of the input domain where there is no available data for f. For U-PINN,

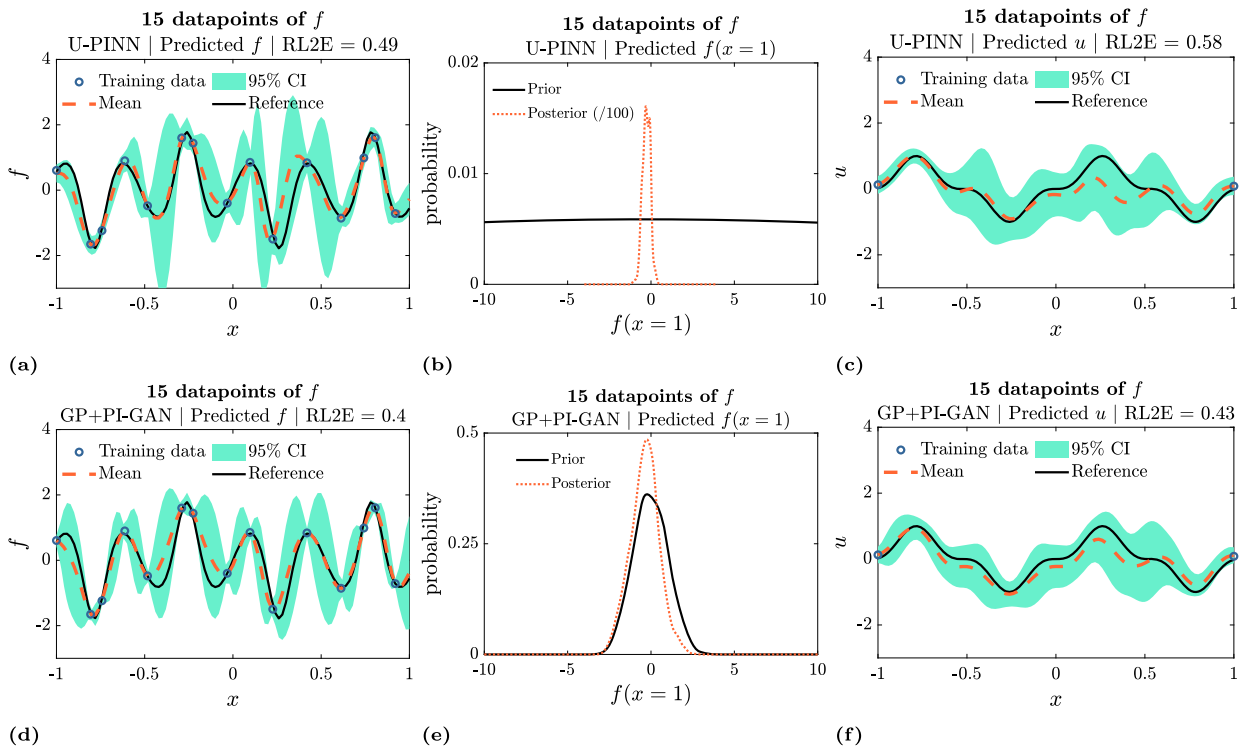


Fig. 29. Forward PDE problem of Eq. (28) — $N_f = 15$ *datapoints of* f: we compare the U-PINN, which fits f and u simultaneously, with the herein proposed GP+PI-GAN, which first fits f and then u. Note that because there is no f datapoint at $x = 1$ and fitting f does not use the PDE of Eq. (28), the GP prior used for f is similar to the posterior in the case of GP+PI-GAN. In contrast, the posterior of U-PINN concentrates significantly because u and f are coupled. Shown here are the training data, reference functions, as well as the mean and epistemic uncertainty (95% CI) predictions of U-PINN (top) and GP+PI-GAN (bottom) for u and f. The middle panels include the prior and posterior distributions of f evaluated at $x = 1$, i.e., at the right edge of the input domain where there is no available data for f.

the prior and posterior distributions are obtained using samples from the prior $\mathcal{N}(0, 1)$ for the parameters θ of the NN and HMC samples from the posterior, respectively. For GP+PI-GAN, they are obtained using samples from the GP prior and posterior.

In Fig. 29 pertaining to $N_f = 15$ datapoints of f, GP+PI-GAN outperforms U-PINN in terms of accuracy of the mean (RL2E). In Fig. 29e pertaining to GP+PI-GAN, it is observed that because there is no f datapoint at $x = 1$ and fitting f does not use the PDE of Eq. (28), the GP prior used for f is similar with the posterior. In contrast, the U-PINN posterior in Fig. 29b concentrates significantly because u and f are coupled. In Fig. D.14 pertaining to $N_f = 6$ datapoints of f, we observe that the uncertainty of u is underestimated by GP+PI-GAN, because the approximations of u and f are not coupled through the PDE of Eq. (28). In contrast, in U-PINN the approximation of u affects the approximation of f and in this case the uncertainties of both u and f increase together.

Overall, in this section we compared two techniques, namely U-PINN and the herein proposed GP+PI-GAN, for solving a forward PDE problem. We found that GP+PI-GAN, which first fits the source term data and subsequently propagates the uncertainty to the solution, can outperform U-PINNs in some cases. Nevertheless, because it does not use the PDE for fitting the source term data, it can be over-confident in its predictions. Further study is required to establish these new results. To illustrate the fact that, in addition to data, U-PINN also uses the PDE to fit the source term, we provided the prior and posterior distributions of the source term evaluated at a location in the domain without available data, using both methods. Although the GP posterior is similar to the prior in the case of GP+PI-GAN, the posterior of U-PINN concentrates significantly because of the coupling of the solution and the source term.

6.5. Operator learning and out-of-distribution data detection: 2D flow in heterogeneous porous media

Here we consider a two-dimensional Darcy flow in porous media, which is described by

$$\nabla \cdot (\lambda(x, y; \xi)\nabla u(x, y; \xi)) = f, \ 0 \leq x, y \leq 1, \xi \in \Xi, \tag{29}$$

where x, y are the space coordinates, the sought solution $u(x, y; \xi)$ denotes the hydraulic head, and we consider $f = 50$. Further, $\lambda(x, y; \xi)$ denotes the hydraulic conductivity field and is determined by the pore structure. Furthermore, $\xi \in \Xi$ is an instance of the problem corresponding to a given field $\lambda(x, y, \xi)$. We consider the boundary conditions

Table 11
Overview of problem cases during pre-training (a) and inference (b) phases of operator learning using DeepONet, as well as pertinent solution approaches. The sections for each problem case and solution approach that are studied in this paper are provided in the third column. In the pre-training phase, we learn an operator mapping from λ to u. To pre-train (U-)DeepONet, for each λ in the pre-training dataset u can be clean or noisy, whereas λ must be clean and available on a pre-specified grid ("complete"). Interpolation is performed if λ is not available on a grid ("limited"). A future research direction pertains to extending DeepONet for treating cases of noisy λ during pre-training. In the inference phase, we employ the learned operator to make inferences given new data. Different approaches can be used depending on the quantity and quality of new data. See Section 1.2 for more information regarding formulation of the operator learning problem. Note that the cases marked with an asterisk correspond to the special case described in Section 1.2.

	Pre-training phase of operator learning		
a	**Problem case**	**Solution approach**	**Tested in §**
	Clean λ and clean u	(U-)DeepONet	6.5.1-6.5.2
	Clean λ and noisy u	(U-)DeepONet	-
	Noisy λ and noisy u	Future work	-
	Inference phase of operator learning		
b	**Problem case**	**Solution approach**	**Tested in §**
	Clean λ only (complete* or limited)	Perform simple interpolation to reconstruct λ, if limited. Then pass λ to (U-)DeepONet.	6.5.2
	Noisy λ only (complete* or limited)	Perform Bayesian inference using a GP, a BNN, or a GAN to obtain realizations of λ. Then pass them to (U-)DeepONet.	-
	Both λ and u (limited and noisy)	PA-BNN-FP or PA-GAN-FP: Reconstruct λ and u, together. Works with DeepONet only, not U-DeepONet.	6.5.1

$$u(x=0,y;\xi)=1,\ u(x=1,y;\xi)=0, \tag{30a}$$

$$\partial_{\boldsymbol{n}}u(x,y=0;\xi)=\partial_{\boldsymbol{n}}u(x,y=1;\xi)=0,\forall\xi\in\Xi, \tag{30b}$$

where $\boldsymbol{n}$ denotes the unit normal vector of the boundary. In this example, we consider a stochastic model for $\lambda(x,y;\xi)$, accounting for different porous media. In particular, $\lambda(x,y;\xi)=\exp(\tilde{\lambda}(x,y;\xi))$, where $\tilde{\lambda}(x,y;\xi)$ is a truncated Karhunen-Loève expansion of a GP with zero mean and kernel given by

$$k_{\tilde{\lambda}}(x,y,x',y')=\exp(-\frac{(x-x')^2}{2l^2}-\frac{(y-y')^2}{2l^2}), \tag{31a}$$

$$0\le x,y,x',y'\le 1,\ l=0.25. \tag{31b}$$

In the following, we only keep the first 100 leading terms of the expansion.

Eqs. (29)-(30) can be viewed as a special case of Eq. (1), with each $\xi\in\Xi$ being a different random event corresponding to a different sample from the GP of λ; with $\mathcal{F}$ and $\mathcal{B}$ both considered unknown and given by Eqs. (29) and (30), respectively; and with x given by (x,y). The objective of this example is to compare different DeepONet-based UQ methods for solving an operator learning problem involving Eqs. (29)-(30). That is, for learning the operator mapping from λ to u. To this end, we consider the pre-training dataset of problem 4 in Table 1, consisting of N = 1,000 realizations of λ and u. Instead of using the original data of λ, we transform them using $\tilde{\lambda}=\log\lambda$, which corresponds to the GP of Eq. (31). We also standardize the data for each (x,y) and for each of the quantities $\tilde{\lambda}$ and u, by subtracting the mean and dividing by the standard deviation, computed for each (x,y) using the N = 1,000 realizations. The modified pre-training dataset is, thus, expressed as $\mathcal{D}=\{\mathcal{D}_{\hat{\lambda}},\mathcal{D}_{\hat{u}}\}$, where $\mathcal{D}_{\hat{\lambda}}=\{\hat{\Lambda}_i\}_{i=1}^N$, $\mathcal{D}_{\hat{u}}=\{\hat{U}_i\}_{i=1}^N$, each $\hat{\Lambda}_i$ is a transformed and standardized realization of λ, and each $\hat{U}_i$ is a standardized realization of u. Note that all plots in this section correspond to the results obtained for the standardized versions of $\tilde{\lambda}=\log\lambda$ and u.

We consider two problem scenarios, as described in Section 1.2, depending on the application of the learned operator for making inferences given new data, i.e., given inference data. These are: a) a general case, which pertains to limited and noisy inference data of both λ and u; and b) a special case, which pertains to complete data of only λ for predicting the corresponding u. Recall that DeepONet takes as input the function λ in the branch net in the form of values on a pre-specified grid, as well as (x,y) for predicting the corresponding $u(x,y)$ (see Fig. 4). Therefore, a complete or limited dataset refers to the case where λ is available or not on the aforementioned pre-specified grid, on which the DeepONet has been pre-trained on, respectively. We summarize the problem cases as well as the corresponding solution approaches in Table 11. In passing, note that to obtain the reference solutions and the training data presented in the following, we use the *Matlab PDE Toolbox*. The hyperparameters and the NN architectures that we used are summarized in Section D.8 and D.9, respectively.

6.5.1. Physics-agnostic functional prior: combining a pre-trained DeepONet with BNN and GAN priors for treating noisy inference data

In this section, we first use a DeepONet to learn the operator mapping from λ to u, given the pre-training dataset $\mathcal{D}=\{\mathcal{D}_\lambda,\mathcal{D}_u\}$ and following Section 2.2. Next, during inference, we assume that new data arrives corresponding to an unseen random event $\xi'\in\Xi$, containing limited and noisy samples from λ and u. This can be construed as a case of dataset

Table 12
Operator learning problem of Eqs. (29)-(30) — *Limited and noisy inference data of λ and u (ID or OOD)*: combining a pre-trained DeepONet with a pre-trained GAN using historical data performs better than combining with a BNN. Note, however, that combining with a BNN does not incur any additional computational cost and also the results for u are more calibrated (smaller RMSCE), even for ID data. This can be attributed to the fact that we pre-trained GAN using historical data of λ only. Shown here is the performance evaluation of BNN-FP (proposed herein) and GAN-FP, referred to as PA-BNN-FP and PA-GAN-FP, respectively, when combined with a pre-trained DeepONet. R2LE was calculated based on clean reference data, whereas MPL and RMSCE based on noisy data.

Metric ($\times 10^2$)	$\log\lambda$ | ID data		u | ID data		$\log\lambda$ | OOD data		u | OOD data	
	BNN	GAN	BNN	GAN	BNN	GAN	BNN	GAN
RL2E (↓)	26.7	**16.3**	14.6	**7.7**	49	**42.8**	48	**35.5**
MPL (↑)	91	**154.8**	168.8	**217**	65.6	**131.2**	99	**127.4**
RMSCE (↓)	**8**	**8**	**2**	8.6	**5.1**	11.4	**9.7**	20.3

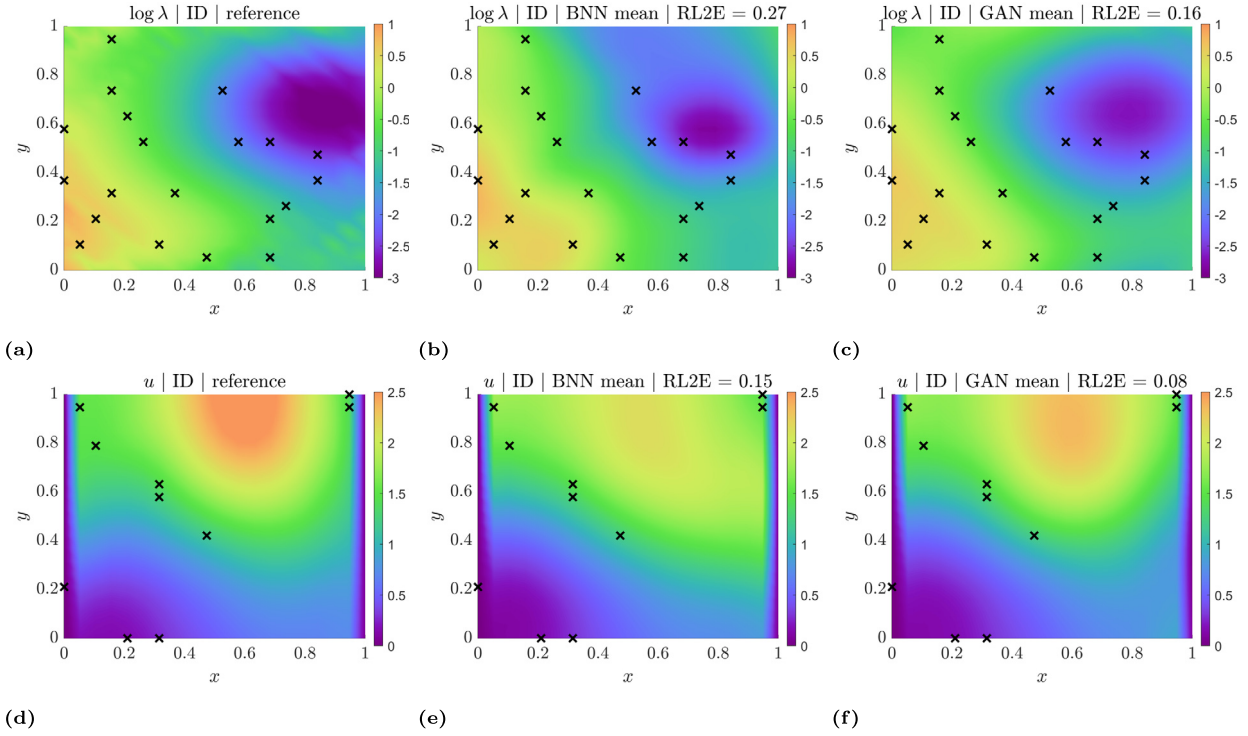


Fig. 30. Operator learning problem of Eqs. (29)-(30) — *Limited and noisy inference data of λ and u (ID)*: the mean predictions of PA-GAN-FP for λ and u are more accurate than the predictions of the herein proposed PA-BNN-FP. Note, however, that PA-BNN-FP does not incur any additional computational cost and its prediction can be more calibrated as shown in Fig. 31. Shown here are the reference input $\tilde{\lambda} = \log\lambda$ and solution u, the corresponding inference data locations (x markers), as well as the mean predictions by PA-BNN-FP and PA-GAN-FP. Top row results correspond to input $\log\lambda$, whereas bottom row to solution u.

shift [95] or covariate shift [158]. We denote these two noisy datasets by $\mathcal{D}'_\lambda = \{x_i, y_i, \lambda_i\}_{i=1}^{N'_\lambda}$ and $\mathcal{D}'_u = \{x_i, y_i, u_i\}_{i=1}^{N'_u}$. The numbers of datapoints are $N'_\lambda = 20$ and $N'_u = 10$. Clearly, the points (x, y) in the space domains of λ and u are in general different. The objective of this section is to evaluate two alternative techniques for reconstructing u and λ based on the new data. Note that this section can be construed as an extension of a similar problem considered in [136].

We employ an approximator λ_θ for fitting the transformed and standardized data of λ, which is denoted by $\hat{\lambda}$. The output of λ_θ becomes the input to the pre-trained DeepONet and produces an approximator u_θ for fitting the standardized data of u, denoted by $\hat{u}$. This is illustrated in Fig. 8c, where λ_θ is denoted by $\mathcal{M}$. Because of data standardization (and transformation), the modified inference dataset is expressed as $\mathcal{D}' = \{\mathcal{D}'_{\hat{\lambda}}, \mathcal{D}'_{\hat{u}}\}$, where $\mathcal{D}'_{\hat{\lambda}} = \{x_i, y_i, \hat{\lambda}'_i\}_{i=1}^{N'_\lambda}$ and $\mathcal{D}'_{\hat{u}} = \{x_i, y_i, \hat{u}'_i\}_{i=1}^{N'_u}$. As explained in Section D.6, the transformation $\tilde{\lambda} = \log\lambda$ corresponds to a log-normal noise model for λ and to a Gaussian noise model for $\tilde{\lambda}$. Because we also standardize the data, according to Section D.6 the noise for the transformed and standardized $\hat{\lambda}$ is Gaussian and space-dependent, and is given by Eq. (D.5). However, for ease of implementation we consider a constant variance for the data of $\hat{\lambda}$ with value equal to 0.01. This corresponds to approximately 10% of noise in the original data of λ for all (x, y) points (in a log-normal noise model). We also consider the same constant variance for the data of $\hat{u}$,

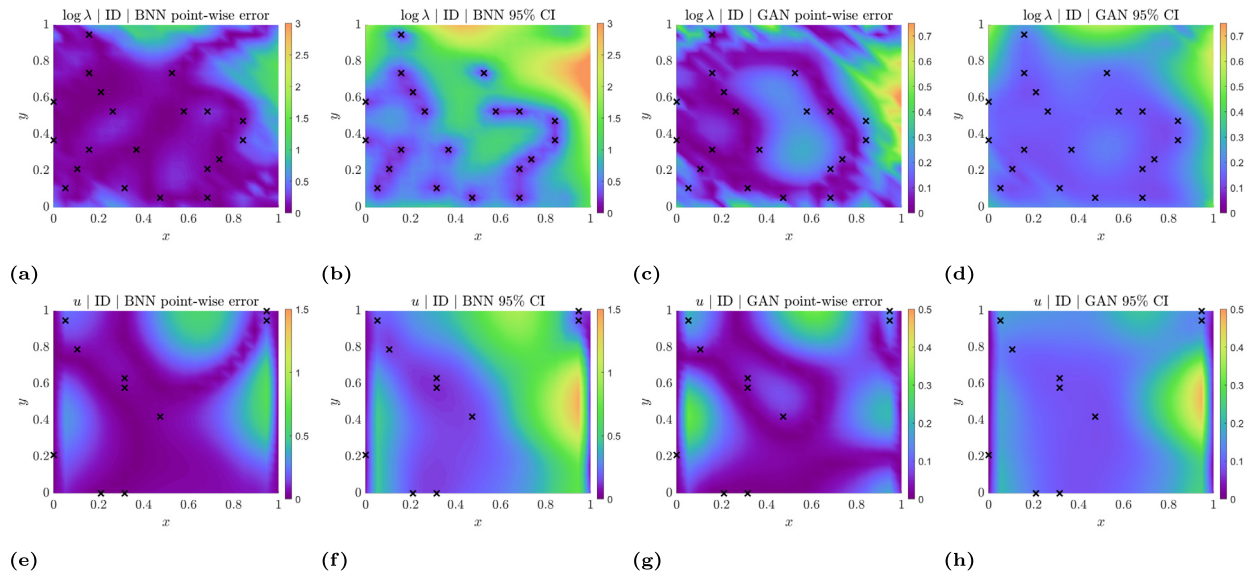


Fig. 31. Operator learning problem of Eqs. (29)-(30) — *Limited and noisy inference data of λ and u (ID)*: although the mean of PA-GAN-FP is more accurate than the mean of the herein proposed PA-BNN-FP (Table 12), PA-GAN-FP is over-confident (less calibrated) for predicting u; compare, e.g., left sides of parts (g) and (h) and note that the error is not covered within the 95% CI. This can be attributed to the fact that we pre-trained GAN using historical data of λ only. Shown here are the absolute point-wise error and epistemic uncertainty predictions corresponding to input $\tilde{\lambda} = \log\lambda$ (a-d) and solution u (e-f), as obtained by comparing PA-BNN-FP and PA-GAN-FP with reference solutions, as well as the corresponding inference data locations (x markers).

which corresponds to variance approximately equal to 0.01 in the original data of u for all (x, y) points (in a Gaussian noise model).

Next, to solve the problem we consider two versions of the model of Fig. 8c, namely the herein proposed PA-BNN-FP and the PA-GAN-FP proposed in [136]. In PA-BNN-FP, $\mathcal{M}$ in Fig. 8c is assumed to be a BNN; this corresponds to a BNN-induced functional prior. In PA-GAN-FP, $\mathcal{M}$ in Fig. 8c is assumed to be the generator of a GAN as described in Section 4.3. We pre-train the GAN using clean historical data of λ, which in this case is the same as the data used for pre-training the DeepONet; i.e., the truncated GP of Eq. (31). In both cases, we perform HMC posterior inference for obtaining samples $\{\hat{\theta}_j\}_{j=1}^{M}$, which corresponds to BNN parameters in the case of PA-BNN-FP, and to GAN generator random input in the case of PA-GAN-FP (see Fig. 35). The likelihood function used in both cases is exactly the same, i.e., Gaussian with mean being the output of each model and variance as described above.

In Table 12, we evaluate the performance of the two methods based on the metrics of Section 5. We perform the evaluation for ID and OOD data, i.e., for a field λ from the same and from a different distribution, respectively, as compared to the one used for pre-training the DeepONet and the GAN. Specifically, the OOD λ corresponds to a sample from the truncated GP of Eq. (31) with correlation length $l = 0.2$ (100 leading terms). It is shown that PA-GAN-FP performs better than PA-BNN-FP in terms of accuracy (RL2E) and predictive capacity (MPL) for both ID and OOD data. This is also shown in Fig. 30, where we plot the mean predictions for λ and u and compare with reference solutions. Clearly, however, the performance of both methods deteriorates significantly for OOD data with RL2E values close to 40-50% (see also Figs. D.15–D.16). Note also that the calibration error of PA-BNN-FP for u is smaller for ID data. This can be attributed to the fact that we pre-trained GAN using historical data of λ only. In Fig. 31 we plot the absolute errors of the mean predictions as well as the epistemic uncertainties provided by PA-BNN-FP and PA-GAN-FP. It is shown that the errors of both methods in most areas of the space domain are covered within the epistemic uncertainty 95% CIs (approximately two standard deviations). Nevertheless, PA-GAN-FP is over-confident (less calibrated) for predicting u in some areas of the space domain, as also indicated by its higher RMSCE value. For example, by comparing the left sides of the plots in parts (g) and (h) of Fig. 31, where there is not much available data, we notice that the error is not covered with the 95% CI. In passing, note that because comparisons in this example are made with clean reference solutions in Figs. 30–31, we only plot epistemic uncertainty. If the reference solutions used for comparisons are noisy, total uncertainty should be used instead. In addition, we note that PA-BNN-FP is more calibrated than PA-GAN-FP also for the case of OOD data (see Table 12 and Figs. D.15–D.16)

Finally, in Fig. D.17 we plot the RMSCE corresponding to $\tilde{\lambda} = \log\lambda$ and u predictions, after post-training calibration with varying sizes of calibration datasets and calibration approach. We see that performing post-training calibration even with a few left-out noisy datapoints (2-10) can reduce the calibration error by half in some cases. An interesting future research direction is to perform a cost-effectiveness study for comparing post-training calibration with active learning, i.e., with techniques that re-adjust the mean and uncertainty predictions in light of new data; see, e.g., [89,159]. Although the computational cost of post-training calibration is approximately zero, there are cases in which it can make the predictions worse (e.g., Fig. D.17c), because it calibrates the predictions "on average", as explained in Section 5.2. On the other hand,

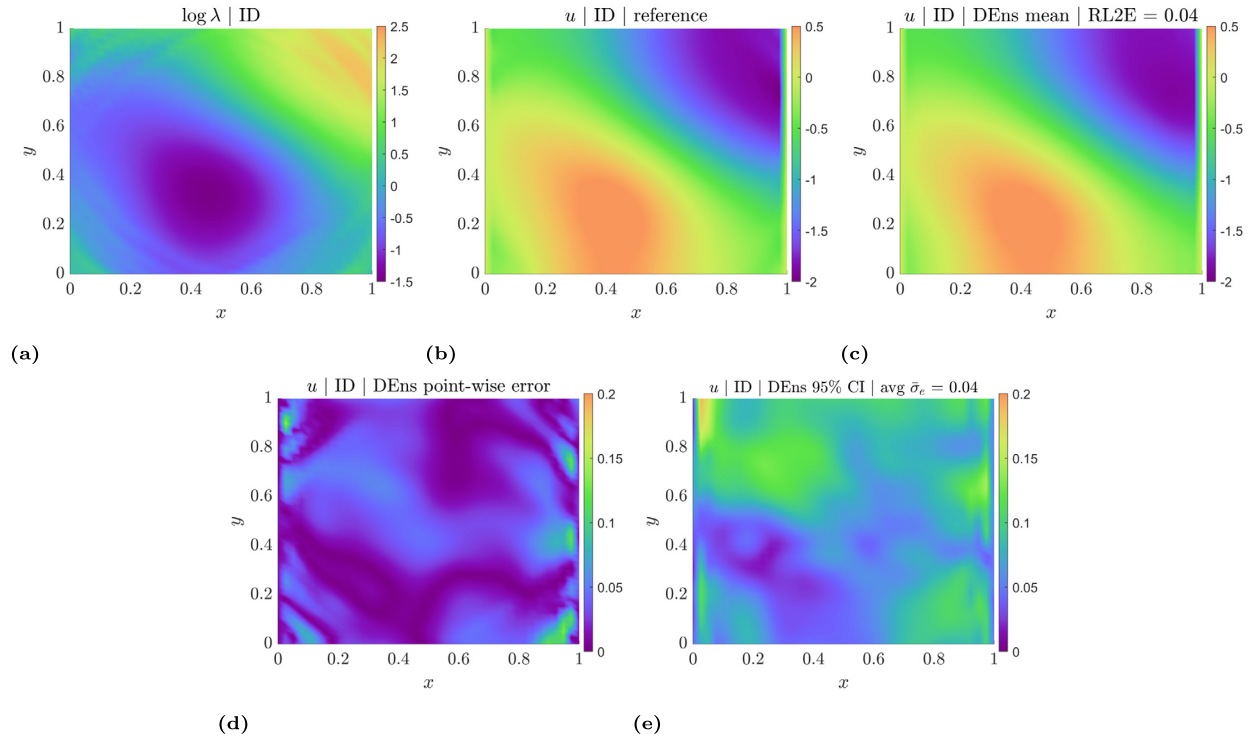


Fig. 32. Operator learning problem of Eqs. (29)-(30) — *Complete and clean inference data of λ (ID data)*: the mean prediction of U-DeepONet, combined with DEns, approximates the solution u accurately with RL2E = 4%. Using each DeepONet separately, instead of a deep ensemble, would yield approximately the same error for ID data. Further, the 95% CI (approximately two standard deviations) of the epistemic uncertainty covers in all areas of the domain and follows spatially the absolute point-wise error of the mean. Here shown are the ID input $\tilde{\lambda} = \log\lambda$ (a), the reference solution u (b), the mean of U-DeepONet (c), the absolute point-wise error of the mean (d), and the 95% CI of epistemic uncertainty (e).

active learning may incur significant computational cost, but it is expected to perform better. It is worth mentioning that the quality of predicted uncertainties is crucial in active learning, and it is strongly affected by the choice of priors as well as the choice of the posterior inference method. Interested readers may refer to [93,140] for more information.

6.5.2. U-DeepONet: combining DeepONet with deep ensemble for incorporating epistemic uncertainty

In this section, we pre-train a DeepONet to learn the operator mapping from λ to u, given the pre-training dataset $\mathcal{D} = \{\mathcal{D}_\lambda, \mathcal{D}_u\}$. In contrast to Section 6.5.1, here we employ a deep ensemble of five trained DeepONets; see DEns in Table 2. This corresponds to U-DeepONet of Table 2. Next, during inference, we assume that new data arrives corresponding to an unseen random event $\xi' \in \Xi$, containing complete and clean samples from λ. This corresponds to the special case we described in Section 1.2; see also Table 11. We denote the clean dataset as $\mathcal{D}'_\lambda = \{x_i, y_i, \lambda_i\}_{i=1}^{N'_\lambda}$. The locations and the number of points are exactly the same as those of the points used in training of U-DeepONet, i.e., $N'_\lambda = 40 \times 40$ on a uniform grid. Note that this is an ill-posed problem because there is no available data of the solution u, not even in the boundaries of its domain. The objective of this section is to demonstrate the advantage of U-DeepONet as compared to standard DeepONet, which does not provide epistemic uncertainty estimates. Concurrently with the present work, Lin et al. [117] employed replica-exchange MCMC in conjunction with DeepONet for operator learning. Their model is a version of U-DeepONet, with MCMC as the posterior inference algorithm.

In Figs. 32–33, we present the neural operator predictions for representative examples of ID and OOD data, as obtained by U-DeepONet. Specifically, for ID data, λ corresponds to a sample from the same stochastic process used for pre-training the U-DeepONet, i.e., the truncated GP of Eq. (31). For OOD data, λ corresponds to a sample from a truncated GP that follows Eq. (31) with correlation length $l = 0.15$, instead of the value 0.25 used in pre-training. As shown in Fig. 32, the RL2E value for ID data is 4% and is approximately the same for each DeepONet of the ensemble separately. However, U-DeepONet also provides epistemic uncertainty. By comparing parts (d) and (e) of Fig. 32 we observe that the 95% CI (approximately two standard deviations) of the epistemic uncertainty covers in all areas of the domain and follows spatially the absolute point-wise error of the mean. Nevertheless, U-DeepONet flourishes in cases of OOD data; i.e., data that have not been seen during pre-training. Specifically, as shown in Fig. 33 for OOD data, the RL2E of U-DeepONet is approximately 19%, whereas the error of each DeepONet of the ensemble used separately is approximately 40%. Further, similarly to the ID case, the 95% CI of the epistemic uncertainty covers the absolute point-wise error of the mean in all areas of the domain. We also note that the overall epistemic uncertainty is higher for the case of OOD data, as compared to ID data. Specifically, the average

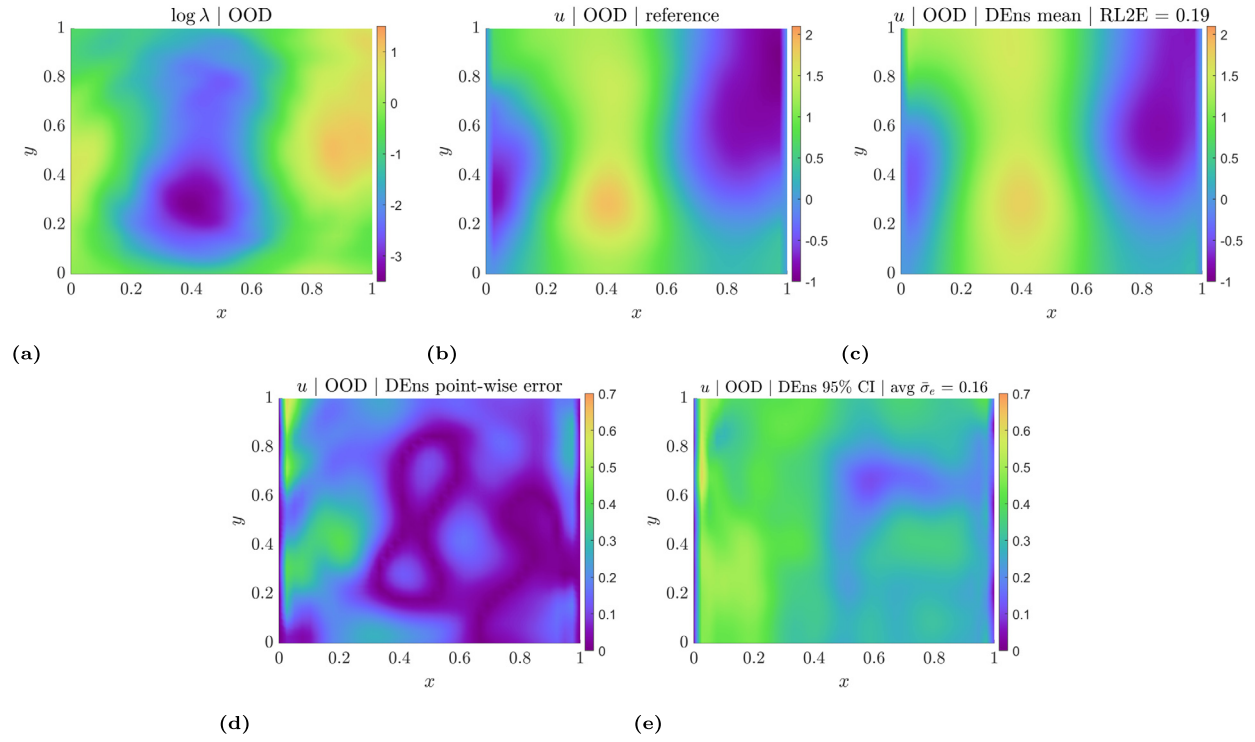


Fig. 33. Operator learning problem of Eqs. (29)-(30) — *Complete and clean inference data of λ (OOD data)*: although the mean prediction of U-DeepONet, combined with DEns, has RL2E = 19%, the value of the same metric would be 40% if a deep ensemble were not used. This fact demostrates the potency of U-DeepONet, as compared to standard DeepONet, for treating OOD data. Further, the 95% CI (approximately two standard deviations) of the epistemic uncertainty covers the absolute error of the mean in all areas of the domain. Here shown are the OOD input $\tilde{\lambda} = \log \lambda$ (a), the reference solution u (b), the mean of U-DeepONet (c), the absolute point-wise error of the mean (d), and the 95% CI of epistemic uncertainty (e).

along (x, y) standard deviation of u is 0.04 for ID data and 0.16 for OOD data. This can be attributed to the fact that the five independently trained DeepONets "disagree" more on their predictions when the data has not been seen during training. This additional property of U-DeepONet is harnessed in Section 6.5.3 for detecting cases of OOD inputs to the learned neural operator.

In passing, note that a GP, or the BNN-FP and GAN-FP models of Section 6.5.1 can be combined with the U-DeepONet of this section for treating noisy, limited or complete, input data of λ during inference, without access to u data. Specifically, given a noisy inference dataset $\mathcal{D}'_\lambda = \{x_i, y_i, \lambda_i\}_{i=1}^{N'_\lambda}$, we can first perform posterior inference for obtaining, for instance, $M =$ 1,000 θ samples from the BNN, or GAN prior that explain the dataset. In the BNN case, θ denotes the BNN parameters, and in the GAN case, θ denotes the GAN generator random input. Subsequently, we use the θ samples to produce realizations of λ. These realizations are, finally, forward-passed through each of the five pre-trained DeepONets of the ensemble. This results to $5 \times 1{,}000$ realizations of u. The uncertainty of these realizations corresponds to epistemic uncertainty due to DeepONet training (five models in the ensemble), as well as to uncertainty due to the noisy input λ. This is similar in spirit to the herein proposed GP+PI-GAN model that we tested for solving forward PDE problems in Section 6.4. An interesting future research direction would be to extend the models we study in this paper for treating cases with limited and noisy input and output data, during both pre-training and inference in operator learning problems. We summarize the various problem cases and respective solution approaches in Table 11.

6.5.3. Out-of-distribution (OOD) data detection

In Section 6.5.1, we pre-trained one DeepONet and combined it with BNN-FP and GAN-FP for treating limited and noisy inference data of both λ and u. In Section 6.5.2, we pre-trained an ensemble of five DeepONets for treating complete and clean inference data of λ only. In the end of Section 6.5.2, we explained that the methods of Sections 6.5.1-6.5.2 can be combined for treating limited and noisy inference data of λ only. Nevertheless, these two methods can also be applied for OOD data detection, which can be construed as a case of dataset distribution shift [95]. Specifically, suppose that we have a pre-trained DeepONet (or an ensemble). The question that we ask in this section is the following: *given inference data of λ, which is the input of the DeepONet, what metric could be used for detecting whether this new data has been seen during pre-training?* Such a metric evaluates quantitatively the reliability of the DeepONet output based on the new data, and is useful for decision-making. As an example, consider Fig. 34. The RL2E of the U-DeepONet output is approximately 1% on average based on 100 ID inputs λ; i.e., for $l = 0.25$ in Eq. (31). However, the average RL2E increases significantly for OOD

Table 13
OOD data detection metrics proposed herein. GAN MSE corresponds to the normalized final training MSE, following optimization of θ using a GAN-FP and new data of λ. GAN NLL corresponds to the negative log-likelihood of optimal θ obtained via optimization as above. DEns std corresponds to the average (along x, y in the domain) standard deviation of the output u, as obtained via U-DeepONet given new data of λ.

Proposed OOD data detection metrics	
GAN MSE	$\frac{\sum_{i=1}^{N'_\lambda} \|\|\lambda_i - \lambda_{\hat{\theta}}(x_i, y_i)\|\|_2^2}{\sum_{i=1}^{N'_\lambda} \|\|\lambda_i\|\|_2^2}$
GAN NLL	$\|\|\hat{\theta}\|\|_2^2 + 0.5K \log 2\pi$
DEns std	$\mathbb{E}_{x,y}[\bar{\sigma}_e(x, y)]$, with $\bar{\sigma}_e$ given in Eq. (10)

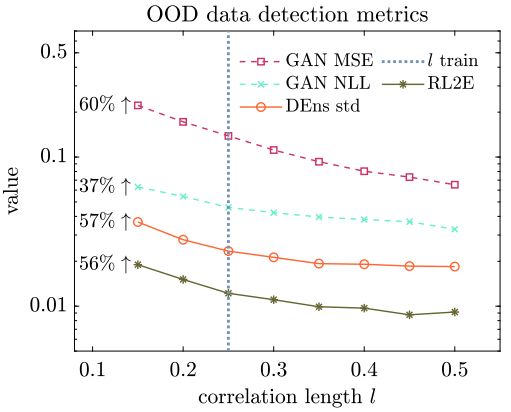


Fig. 34. Operator learning problem of Eqs. (29)-(30) — *OOD data detection*: the three proposed OOD data detection metrics of Table 13, namely, GAN MSE, GAN NLL, and DEns std, follow the RL2E trend. Here shown are the metric values for inputs λ associated with a truncated GP of the form of Eq. (31) with different correlation lengths. The RL2E of U-DeepONet as well as the correlation length used in the U-DeepONet pre-training phase are also included for comparisons. All values are averaged over 100 λ inputs.

data with $l < 0.25$ and the error for $l = 0.15$ is approximately 60% higher than the error of $l = 0.25$. Nevertheless, for OOD data with $l > 0.25$ the RL2E decreases. That is, although U-DeepONet was not trained with λ fields drawn from a GP with correlation lengths higher than 0.25, it performs satisfactorily. Obviously, this a special case. Potential explanations are that a) the inputs λ for $l = 0.25$ used in training contain most of the information of the unseen inputs for $l > 0.25$; and b) the outputs u for $l = 0.25$ have similar characteristics with the unseen outputs for $l > 0.25$. Next, note that the RL2E values shown in Fig. 34 are not available in practice; i.e., we only have one shot for predicting the solution u based on a single realization of the input λ. The objective of this section is to propose metrics that utilize only the available information, which is a single input λ, and follow the RL2E trend in Fig. 34. Indicatively, more information on OOD detection can be found in [60,160–164].

In this regard, we assume that we have a pre-trained GAN based on historical data of λ, exactly as in Section 6.5.1. This GAN learns the distribution of the data that the DeepONet has been trained based on. Next, we consider inference data of λ, limited or complete and noisy or clean. We can perform posterior inference for obtaining θ samples that explain the data, where θ is the input to the GAN generator. Because in this use case we are not interested in the uncertainty of θ, we can obtain a point estimate $\hat{\theta}$ using standard optimization according to Section A.2, instead of costly posterior inference according to Sections 4.1-4.2. Following optimization, an optimal value $\hat{\theta}$ is available, as well as the final MSE between the approximator $\lambda_{\hat{\theta}}$ evaluated on the data locations and the data of λ. The first metric that we propose is the normalized final training MSE, which we refer to as *GAN MSE*, and is given in the first row of Table 13. Clearly, if we work with standardized and/or transformed data as explained in the beginning of Section 6.5, the respective modified values are used instead of the λ values. The rationale of this metric is that if we are unable to obtain a $\hat{\theta}$ through optimization that yields a small training MSE, the GAN-FP is unable to fit the given data, and thus it is expected that the GAN has not seen this data during pre-training. As shown in Fig. 34, the GAN MSE metric follows the same trend as RL2E and the metric value for $l = 0.15$ is 60% higher than the metric value for $l = 0.25$. In practice, we can obtain the average GAN MSE for 10, for example, ID inputs λ, and subsequently given a new input λ we can compare its GAN MSE with the average GAN MSE from the ID data.

The second metric we propose, which we refer to as *GAN NLL*, is the negative log-likelihood of $\hat{\theta}$ based on the known distribution of the GAN generator random input, and is given in the second row of Table 13. In this paper, this distribution is denoted by p_{GAN} and is a factorized Gaussian distribution, with $\mathcal{N}(0, 1)$ in each dimension. Thus, GAN NLL is given as $-\log p_{GAN}(\hat{\theta}) = ||\hat{\theta}||_2^2 + 0.5K \log 2\pi$, where K represents the size of $\hat{\theta}$. Note that the GAN has been trained such that drawing samples of θ from this Gaussian distribution produces λ fields that follow the historical data distribution. For

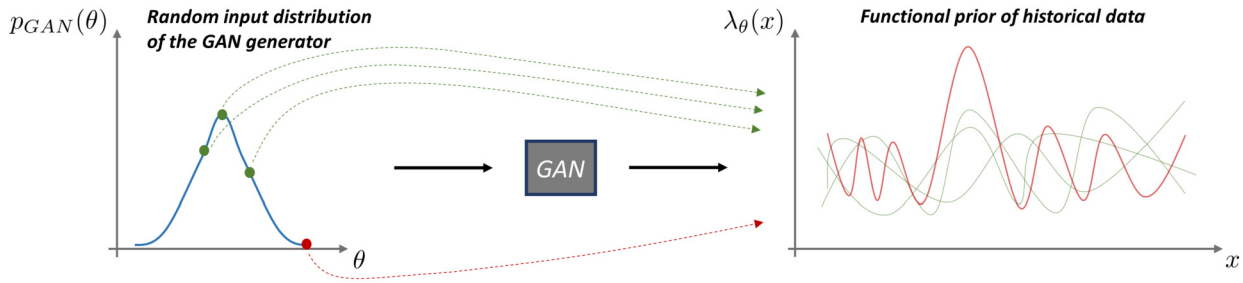


Fig. 35. The GAN-FP is trained such that drawing samples of θ from $p_{GAN}(\theta)$ produce λ functions that follow the historical data distribution. For OOD detection, we utilize the inverse of this relationship: a λ function that is far from the historical data distribution is expected to correspond to a θ value that is far from the GAN input distribution. Values of θ shown with green color on the left side are more probable under the distribution p_{GAN} and correspond to functions λ on the right side that have similar characteristics. On the other hand, the θ value shown with red color is less probable and corresponds to a function λ that is different from the rest.

OOD data detection, we utilize the inverse of this relationship: a λ field that is far from the historical data distribution is expected to correspond to a θ value that is far from the GAN input distribution. This is illustrated in Fig. 35, where values of θ with green color on the left side are more probable under the input distribution p_{GAN} and correspond to functions λ on the right side that have similar characteristics. On the other hand, the θ value shown with red color is less probable and corresponds to a function λ that is quite different from the rest. If the function λ shown with red is used in the optimization as explained above, it is expected to yield an optimal $\hat{\theta}$ that belongs in the tails of p_{GAN}, and thus it is associated with a high value of GAN NLL. As shown in Fig. 34, the metric GAN NLL follows the same trend as RL2E and the metric value for $l = 0.15$ is 37% higher than the metric value for $l = 0.25$.

Next, consider a pre-trained ensemble of DeepONets, i.e., a U-DeepONet combined with DEns of Section 4.2. As shown in Figs. 32–33 for a random ID or OOD input λ, respectively, the epistemic uncertainty of the output u of U-DeepONet for OOD input λ is higher (0.04 vs 0.16). In this regard, the third metric we propose, which we refer to as *DEns std*, is the average along (x, y) standard deviation of u obtained via U-DeepONet. It is given in the third row of Table 13. As shown in Fig. 34, the metric DEns std follows the same trend as RL2E and the metric value for $l = 0.15$ is 57% higher than the metric value for $l = 0.25$.

6.5.4. Summary

In this section, we presented a comparative study pertaining to neural operator learning. Following the pre-training phase of DeepONet, we considered two inference cases, namely a case with noisy and limited inference data, and a case with clean and complete data. In the first case, we used the PA-GAN-FP and the herein proposed PA-BNN-FP. In the second case, we used a U-DeepONet combined with an ensemble of five independent training sessions of DeepONet. Further, we proposed and tested three metrics for detecting OOD inference input data to U-DeepONet. This can be used to raise a "red flag" that the DeepONet output given a new input may not be accurate.

Overall, for treating limited and noisy inference data, combining a pre-trained DeepONet with a GAN-FP (PA-GAN-FP) performs better than the herein proposed BNN-FP (PA-BNN-FP), as expected. However, PA-BNN-FP does not incur any additional computational cost and is more calibrated in our experiments. For treating complete and clean inference data, U-DeepONet is more accurate (in terms of mean predictions) than standard DeepONet for OOD data, while the provided epistemic uncertainty covers the errors for ID and OOD data. The epistemic uncertainty also increases for OOD data, a fact that is harnessed for OOD data detection. For OOD data detection, the three proposed detection metrics follow the errors of performing inference with OOD data. Therefore, in practice, for a new input to the DeepONet, we can use the detection metrics to evaluate whether the input has been seen during training.

6.6. Performance versus cost

This section provides qualitative comments and quantitative results regarding the computational cost of the employed UQ methods. In all cases, we compare the computational cost with that of standard training of NNs. Note that we only compare the cost of obtaining the samples, and not the cost of using them for obtaining the sought statistics. The latter cost depends on the number of samples M that each method collects and for the problems we considered herein is negligible. It becomes, however, important for larger NNs and number of evaluation points.

For obtaining posterior samples $\{\hat{\theta}_j\}_{j=1}^M$ (or $\{\hat{\beta}_j\}_{j=1}^M$), we considered eight methods, as summarized in Table 2, excluding GP regression. In the following, we provide qualitative comments regarding their computational cost. HMC is a two-step MCMC approach that simulates the Hamiltonian dynamics of a fictitious particle by using a number of leapfrog steps and rejection sampling. Indicatively, for the function approximation problem and for 50 leapfrog steps the computational cost of HMC is 4-5 times higher than that of standard training. LD is a one-step MCMC approach that does not use rejection sampling and resembles stochastic gradient descent with added noise to the gradient. Its cost is approximately the same as that of standard training, although because of noise it may require a larger number of steps to converge. Unlike standard

Table 14
Indicative computational time for performing posterior inference in the function approximation problem of Section 6.1.1 with each of the methods of Table 2, by using CPU and GPU. CPU details: 2 Intel(R) Xeon(R) CPU E-2643 @ 3.30 GHz (4 cores, total 8 threads); GPU details: Nvidia's TITAN Xp. Note that the computational time of DEns is the total time required for $M = 10$ independent training sessions performed serially. This can also be done in parallel. Although, clearly, NN training on GPU is generally faster than on CPU, in this case the computational cost for all methods is higher on GPU. For HMC and LD, based on our experiments one reason that makes CPU more efficient is that we need to generate random numbers for the momentum at each iteration, which is more efficient on a simple architecture, such as CPU, than on GPU for *tensorflow probability* [165]. Further, the rejection sampling of HMC includes an if/else statement, which is more efficient on CPU than on GPU. For the remaining approaches, which are trained using the Adam optimizer, using GPU is typically preferable, especially in cases with big data. Specifically, matrix-vector multiplication, which is a heavy-lifter for NN training, has spatial locality of $O(n)$ and naturally maps to GPU architecture. However, for cases with small data, a CPU may have comparable performance to a GPU. The reason is that we need to transfer the dataset from the CPU memory to the GPU, and then transfer it back to the CPU memory. Transferring the data from CPU to GPU and vice versa is facilitated by PCI Express, which in our case is 3.0 and can provide only 8 GT/s (Giga Transmission per second). Thus, the computational time in small data cases is dominated by the memory latency. Overall, this table is not intended for performance comparison between CPU and GPU, but only for reporting the respective elapsed times on both architectures.

	HMC	LD	MFVI	MCD	LA	DEns	SEns	SWAG
CPU (s)	131.5	44.4	19.5	41.7	60.3	359.8	31.9	36.6
GPU (s)	322.7	269.7	31.2	55.4	77.6	494.7	56.1	61.5

training that updates the NN parameters only, MFVI updates the mean and standard deviation of a NN variational distribution in each training step. Its cost is approximately the same as that of standard training, although in this study we used early stopping and the resulting cost can vary and be less than standard training without early stopping. MCD corresponds to standard training including dropout and its cost is approximately the same as that of standard training. LA consists of standard training and, subsequently, Hessian approximation. Depending on the method employed for Hessian approximation, the cost of LA is slightly higher than that of standard training. DEns consists of M standard training sessions and its cost is either the same (if performed in parallel) or M times higher (if performed serially) than that of standard training. SEns corresponds to standard training with an appropriate learning rate schedule, and thus its cost is approximately the same as that of standard training. SWAG consists of SEns and, subsequently, fitting and sampling a Gaussian distribution using the optimization iterates. Its cost is slightly higher than that of standard training. In Table 14, we provide the indicative computational time required for performing posterior inference in the function approximation problem of Section 6.1.1 using each of the methods described above.

Next, h-X combines the above methods with heteroscedastic noise modeling, i.e., the involved NNs have one additional output modeling the noise. If the same NN architecture is used as above, the computational costs remain approximately the same as described above. All the above methods assume by default a BNN-FP, i.e., a standard NN with unknown parameters following a prior distribution. If a GAN-FP is used instead of a BNN-FP, the cost of posterior inference can be smaller, because we draw samples of θ that corresponds to the GAN generator input. This input commonly has a significantly lower dimension compared to the parameters θ of a NN. However, GAN-FP requires pre-training using historical data. Indicatively, training the GAN-FP used in the function approximation problem with heteroscedastic noise (Fig. 21) takes approximately one hour on one GPU. Finally, these computational costs generalize accordingly when we combine the posterior inference methods with PINNs and DeepONets, as summarized in Table 2.

In terms of accuracy of mean predictions, predictive capacity, and statistical consistency (calibration), the most computationally expensive methods, namely HMC and DEns, are expected to exhibit the highest performance. Their performance can significantly be enhanced if the prior is learned in HMC or the hyperparameters of DEns are tuned. This has been corroborated with our experiments throughout the present study. In terms of the computationally cheaper methods, we found that they can provide accurate mean predictions and relatively calibrated predictions, especially if their priors (or hyperparameters) are learned (or tuned). However, with the exception of LA, they are less calibrated than HMC and DEns and their epistemic uncertainties do not increase consistently with increasing noise and decreasing dataset size (see Section 6.1.1). Nevertheless, we found that LA, MFVI, and SEns can provide *a good trade-off between performance and cost*. We suggest to potential users of the presented methods to (a) compare various methods for solving a problem at hand; (b) use a plurality of evaluation metrics; (c) learn (or tune) the priors (or hyperparameters) using validation data (where applicable); and (d) calibrate, after training, the predictions using a small left-out calibration dataset. Lastly, we found that by incurring some additional pre-training computational cost using a GAN-FP, we can reduce the computational cost of posterior inference and improve performance.

In terms of code implementation, MFVI, MCD, LA, DEns, SEns, and SWAG can be implemented by extending available standard NN training codes in a straightforward manner. For LA, an off-the-shelf Hessian approximation implementation can be used; see, e.g., [166]. For HMC and LD the open-source package *tensorflow probability* [165] can be used, with the implementation of HMC being more involved, especially for learning the prior and noise scale. For implementing h-X, we simply include an additional NN output modeling the noise. For GAN-FP, we used our own implementation following Section B.4. For post-training calibration, we used the package of Chung et al. [143]. Overall, to help facilitate the deployment of UQ in SciML research and practice, we have developed in [1] an open-source Python library (*github.com/Crunch-UQ4MI/neuraluq*), termed NeuralUQ, that is accompanied by an educational tutorial and additional computational experiments.

Table 15
Glossary of terms frequently used in this paper. The upper part corresponds to machine learning terms, whereas the lower part to scientific machine learning terms.

Glossary of terms frequently used in the paper			
Terms related to Machine Learning (ML)			
NN	Neural Network with parameters θ (or β)	**Standard NN training**	Point estimates $\hat{\theta}$ (or $\hat{\beta}$) according to §A.2
FP	Functional Prior induced by a model with random parameters θ (or β)	**Posterior inference**	Samples $\{\hat{\theta}_j\}_{j=1}^M$ (or $\{\hat{\beta}_j\}_{j=1}^M$) according to §4.1-4.2
BMA	Bayesian Model Average of Eq. (5) (exact) or Eqs. (8)-(10) (approximate)	**Predictive PDF/CDF**	Predictive Probability/Cumulative Density/Distribution Function (Fig. 6)
BNN	Bayesian NN - FP induced by a NN with θ (or β) following prior distribution	**Post-training calibration**	Modified predictive PDF/CDF to improve statistical consistency (§5.2)
GAN	Generative Adversarial Network	**MCMC**	Markov chain Monte Carlo
GAN-FP	FP induced by a GAN generator with random input θ	**Epistemic uncertainty**	Posterior uncertainty of θ (or β) quantified via $\{\hat{\theta}_j\}_{j=1}^M$ (or $\{\hat{\beta}_j\}_{j=1}^M$)
h-X	Model X with heteroscedastic modeling of aleatoric uncertainty	**Aleatoric uncertainty**	Irreducible data uncertainty considered known or learned
U-X	Model X with epistemic and aleatoric uncertainties taken into account	**Total uncertainty**	Epistemic + Aleatoric uncertainty (PDF/CDF or moments)
Terms related to Scientific Machine Learning (SciML)			
(S)PDE	(Stochastic) Partial Differential Equation	**Operator**	Mapping between infinite-dimensional function spaces
Neural (S)PDE	Solves (S)PDE with a NN	**Neural operator**	Learns operator with a NN
PI-X	Physics-Informed method X - Uses (S)PDE during training	**PA-X**	Physics-Agnostic method X - Ignores/replaces (S)PDE w/ surrogate
PINN	Physics-Informed Neural Network for solving PDEs	**DeepONet**	Deep Operator Network (PI or PA) for learning operators

7. Summary

We have presented UQ methods for neural networks to address function approximation, and for solving PDEs and learning operator mappings between infinite-dimensional function spaces. In particular, we have presented a formulation towards a unified and comprehensive framework for UQ in SciML, which is summarized in Fig. 10. Further, we have integrated various uncertainty modeling, posterior inference, prior learning, and post-training calibration approaches into neural PDEs, SPDEs, and neural operators. An overview of the presented solution methods is provided in Table 2. Furthermore, we have performed an extensive comparative study, where we addressed a function approximation problem as well as the problems of Fig. 1, by employing the related UQ methods of Table 2. The comparative study serves to demonstrate the applicability and reliability of the presented methods for treating limited and mixed input-output data that are contaminated with various types of noise; for seamlessly combining these data with available physics knowledge; and for harnessing historical data to reduce cost while increasing performance. An overview of the considered problems in the comparative study is provided in Table 3. In the following, we summarize our contributions, our findings, as well as proposed future research directions.

7.1. Novel contributions

For each of the problems considered herein, we have also presented original contributions. For *function approximation*, we have demonstrated how to learn the heteroscedastic noise given historical data by using functional priors; see Section 6.1.3 for the method and Fig. 21 for results. For *forward PDEs*, we have proposed a new technique, termed GP+PI-GAN, which we compared with U-PINNs; see Section 4.4 for the GP+PI-GAN method and Figs. 29 and D.14 for results. For *mixed PDEs*, we have trained and combined with existing methods a GAN-based functional prior, termed PI-GAN-FP, given historical data of the source term and the PDE space-dependent parameters for solving the problem faster and more accurately; see Section 4.3 for the method and Table 8 for results. Further, we have demonstrated how to treat heteroscedastic noise in the source term, problem parameters, and solution data; see Fig. 7 for the method and Fig. 26 for results. For *mixed SPDEs*, we have proposed a new NNPC architecture, termed NNPC+, which performs better, and we have considered stochastic problems with noisy realizations; see Fig. 9 for the method and Fig. 27 for results. For *operator learning*, we have demonstrated how to deal with noisy and incomplete inference data given a pre-trained neural operator on clean data; see Section 4.3 for the method and Figs. 30–31 for results. In addition, we have proposed different metrics for detecting OOD data in neural operators, which is critical for risk-related applications; see Table 13 for the metrics and Fig. 34 for results.

7.2. Findings

Based on the comparative study of Section 6.1, we have found that UQ methods with smaller computational cost (e.g., MFVI and LA), can yield comparable performance with more expensive methods (e.g., HMC and DEns); see Fig. 18. This is true especially if the NN parameter priors are learned during posterior inference and, clearly, depends on the problem. Further, based on our experiments, post-training calibration using a small left-out dataset can reduce significantly the calibration error; see, e.g., Fig. 17. This is achieved by calibrating the produced uncertainties so that they are neither under- or over-confident. We have also shown that learning the NN parameter prior during posterior inference helps to reduce the burden of thorough NN architecture selection. Specifically, we have shown that different architectures yield similar results if the priors are learned; see Fig. 16. An additional finding that has also been reported in other studies, e.g., [92,142], is that performance evaluation and comparison of different UQ methods requires a plurality of evaluation metrics. In particular, we have shown that there are cases in which MCD performs much better than HMC in terms of metrics employed, although it may lead to over-confident predictions, especially for OOD data; compare Fig. 15 (HMC) with D.4 (MCD). Further, we have demonstrated that heteroscedastic noise modeling and learning with online or offline methods, e.g., functional priors, exhibits satisfactory performance. This is true even for computationally less expensive methods, e.g., MFVI; see Fig. 21.

In *solving forward PDEs*, we found that the herein proposed GP+PI-GAN method, which first fits the source term data and subsequently propagates the uncertainty to the solution, can outperform U-PINNs in some cases; see Fig. 29. Nevertheless, because it does not use the PDE for fitting the source term data, it can be over-confident in its predictions; see Fig. D.14. Further study is required to establish these new results. In *solving mixed PDEs*, we have shown that employing a PI-GAN-FP, pre-trained with historical data of the source term and the problem parameters, exhibits higher performance than the BNN-based UQ methods; compare Fig. 25 (PI-GAN-FP) with Fig. 23 (standard U-PINN with BNN-FP). Among HMC, MFVI, MCD, and DEns, the more expensive techniques (HMC and DEns) perform better, while we estimated the unknown noise satisfactorily with HMC, MFVI, and DEns. For challenging cases involving large noise, data concentrated on one side of the space domain that require extrapolation, and functions with large gradients, the proposed UQ methods still perform satisfactorily, with uncertainties that cover the errors in most cases; see Table 9. In *solving mixed SPDEs*, the herein proposed U-NNPC+ can outperform existing methods for treating cases with noisy stochastic realizations; see Table 10. Nevertheless, we found in our experiments that epistemic uncertainty obtained by combining deep ensembles with the considered methods may not cover the errors in predicting the first- and second-order statistics of the quantities of interest; see Fig. 27.

In *operator learning and treating limited and noisy inference data*, combining a pre-trained DeepONet with a GAN-FP, termed PA-GAN-FP, performs better than the herein proposed PA-BNN-FP, as expected; see Table 12. However, the PA-BNN-FP does not incur any additional computational cost and is more calibrated in our experiments. In *operator learning and for treating complete and clean inference data*, U-DeepONet, i.e., DeepONet combined with deep ensembles, is more accurate (in terms of mean predictions) than standard DeepONet for OOD data, while the provided epistemic uncertainty covers the errors for ID and OOD data. The epistemic uncertainty also increases for OOD data, a fact that is harnessed for OOD data detection; see Figs. 32–33. In *operator learning and OOD data detection*, the three proposed detection metrics follow the errors of performing inference with OOD data. Therefore, in practice, for a new input to the DeepONet, we can use the detection metrics to evaluate whether the input has been seen during training; see Fig. 34. This can be used to raise a "red flag" that the DeepONet output given a new input may not be accurate.

7.3. Outlook and future research directions

Neural networks are currently revolutionizing the computational science field by combining physics with data for solving problems of ever-increasing complexity, and are the building blocks in developing digital twins of physical and biological systems. In such problems, the physics may be available only partially, the data may be limited, noisy, and contaminated with heteroscedastic noise of an unknown distribution, and extrapolation with OOD data may be required. In this context, reliability assessment by UQ is essential for employment of the NN-based methods in risk-related applications and for decision-making. To achieve this, a unified framework is required that classifies the problems of interest, and could provide methods, hyperparameter selection approaches, evaluation metrics, and post-training calibration approaches for avoiding making under- or over-confident predictions. Herein, we have proposed a formulation towards such a unified framework, based on which we have presented several novel contributions for addressing some of the aforementioned challenges of SciML. We have shown that by modifying, only slightly in some cases, standard optimization algorithms, output predictions can be endowed with uncertainty estimates. These estimates follow the trend of the prediction error and can be used for evaluation of the NN methods and for detecting unseen data during training. Nevertheless, and despite the plurality of the methods studied herein, there are still several open challenges in the field. These pertain, indicatively, to solving problems with large levels and unknown distributions of noise, limited number of measurements, approximating multi-modal posterior distributions, detecting out-of-distribution data and thoroughly assessing the quality of uncertainty predictions; see more details in Section 1. In this paper, we have addressed or at least explored the extent to which most of these challenges affect the problem solutions and have left as future work the study of rare events, active learning, and joint predictions, among other topics.

In terms of future research, a direction that would be useful for practical applications is UQ scalability, i.e., extending the presented UQ methods for treating high-dimensional problems as well as large amounts of data. In such cases, the increased

computational cost can be addressed by using cheaper methods that are also combined with stochastic gradients; see, e.g., [36,91]. This could be combined with a thorough cost-effectiveness study of the various posterior inference methods we discussed in this paper, as well as of the neural (S)PDE and operator methods. Preliminary results included in Section 6.6 suggest that even UQ methods with associated computational cost slightly higher than standard NN training, such as MFVI, LA, and SEns, can improve mean predictions and provide reliable uncertainty estimates, if properly combined with prior learning approaches, hyperparameter tuning, and post-training calibration. Clearly, however, the levels of accuracy and confidence that can be tolerated are discipline- and application-dependent. Further, it would be of interest to also perform a cost-effectiveness study for comparing post-training calibration with active learning approaches. The latter re-adjust the predictions in light of left-out or novel data and are expected to improve performance by incurring some additional computational cost; see, e.g., [89,159]. Active learning is particularly important for UQ of rare or extreme events for which very limited data may exist, and hence output-weighted criteria should be exploited as in [145]. More broadly, UQ methods for multi-fidelity training of neural networks as well as for diverse data augmentation techniques that have to be used in these cases should be investigated in the near future. This proposed research area is also important for operator learning, but in addition, a future research direction pertains to extending DeepONet for treating noisy input data during the pre-training phase, as shown in Table 11. Furthermore, alternative UQ approaches that could be employed in the context of SciML are based on the evidential framework, e.g., [57–65], the variational information bottleneck, e.g., [167–169], and the conformal prediction framework, e.g., [73,74]. Lastly, the capabilities of the presented UQ methods should be tested in the context of digital twins of multi-scale and multi-physics systems, where a plurality of methods could be used for different sub-systems, and it would be interesting to assess the synergistic or antagonistic effects of all UQ methods employed together in such a complex environment.

Code availability

The code used to produce a sample of the computational results in this work is described in [1] and can be accessed at *github.com/Crunch-UQ4MI/neuraluq*.

CRediT authorship contribution statement

Apostolos F. Psaros: Conceptualization, Methodology, Investigation, Coding, Writing - original draft, Writing – review & editing, Visualization. **Xuhui Meng:** Conceptualization, Methodology, Investigation, Coding, Writing - original draft, Writing – review & editing, Visualization. **Zongren Zou:** Coding, Writing – original draft, Writing – review & editing. **Ling Guo:** Coding, Writing – review & editing. **George Em Karniadakis:** Conceptualization, Methodology, Investigation, Writing – original draft, Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by OSD/AFOSR MURI grant FA9550-20-1-0358 and DOE PhILMs project no. DE-SC0019453. The authors would like to thank Dr. Khemraj Shukla of Brown University for reviewing parts of the paper and providing useful comments.

Appendix A. Supplementary material to Section 3: additional topics in total uncertainty modeling

A.1. Statistics of the predictive distribution

The mean of the random variable $(u|x,\mathcal{D})$ is denoted by $\mathbb{E}[u|x,\mathcal{D}]$ and is given by

$$\hat{u}(x) = \mathbb{E}[u|x,\mathcal{D}] = \int u_\theta(x)\, p(\theta|\mathcal{D}) d\theta. \tag{A.1}$$

Clearly, even if a single output value $\hat{u}$ is required instead of a predictive distribution, its determination involves an integration over all plausible values of θ given (conditioned on) the data. Next, utilizing the law of total variance we obtain the diagonal part of the covariance matrix of $(u|x,\mathcal{D})$ as (e.g., [170])

$$Var(u|x,\mathcal{D}) = \underbrace{\mathbb{E}_{\theta|\mathcal{D}}\left[Var(u|x,\theta)\right]}_{\text{aleatoric}} + \underbrace{Var_{\theta|\mathcal{D}}(\mathbb{E}[u|x,\theta])}_{\text{epistemic}}, \tag{A.2}$$

which simplifies to

$$Var(u|x,\mathcal{D}) = \underbrace{\mathbb{E}_{\theta|\mathcal{D}}\left[\Sigma_u^2\right]}_{=\Sigma_u^2} + Var_{\theta|\mathcal{D}}(u_\theta(x)) \tag{A.3}$$

for the Gaussian likelihood function of Eq. (4). Eqs. (A.2)-(A.3) provide the *total uncertainty* of $(u|x,\mathcal{D})$ in the form of total variance. Note that in Eq. (A.2) there are two distinct random variables pertaining to u; i.e., $(u|x,\mathcal{D})$, which represents the sought value of $u(x)$, is conditioned on the dataset $\mathcal{D}$, and does not depend on θ, and $(u|x,\theta)$ which is the means for obtaining $(u|x,\mathcal{D})$ and depends on θ. Further, it is straightforward to express similarly the non-diagonal terms of the covariance matrix of $(u|x,\mathcal{D})$; see, e.g., [133]. Furthermore, we emphasize that aleatoric uncertainty affects $Var(u|x,\mathcal{D})$ not only via the corresponding added term in Eq. (A.3), but also via epistemic uncertainty, which is expected to increase with more noise. Nevertheless, it is not generally straightforward to completely disentangle aleatoric and epistemic uncertainties; see the comparative study in Section 6 for related experiments as well as Section B.1 for a related discussion pertaining to GP regression. Finally, if no aleatoric uncertainty in the prediction of $(u|x,\mathcal{D})$ is considered, Eq. (5) becomes

$$p(u|x,\mathcal{D}) = \int \delta(u - u_\theta(x))\, p(\theta|\mathcal{D}) d\theta. \tag{A.4}$$

Note that this equation is still a probability density with the diagonal part of the covariance matrix of the random variable $(u|x,\mathcal{D})$ given by Eqs. (A.2)-(A.3) without the aleatoric uncertainty part.

A.2. Point estimates (standard neural network training)

Here we provide a brief and partial overview of standard NN training that serves for making our paper self-contained. By maximizing the data likelihood $p(\mathcal{D}|\theta)$, the model parameters θ can be inferred. This approach is widely known as maximum likelihood estimation (MLE) and when combined with a Gaussian likelihood of the form of Eq. (4) it is expressed as

$$\hat{\theta} = \underset{\theta}{\text{argmin}}\ \mathcal{L}(\theta), \text{ where } \mathcal{L}(\theta) = \sum_{i=1}^{N} ||u_i - u_\theta(x_i)||_2^2. \tag{A.5}$$

In standard function approximation, Eq. (A.5) is referred to as mean squared error (MSE) minimization without regularization. Combining MLE with a different likelihood function, e.g., Laplace distribution, yields a different minimization objective in Eq. (A.5), e.g., mean absolute error minimization. See also [131] for meta-learning the loss function in Eq. (A.5).

Alternatively, if we consider the approximation

$$p(\theta|\mathcal{D}) \approx \delta(\theta - \hat{\theta}), \tag{A.6}$$

we can infer θ by maximizing the posterior $p(\theta|\mathcal{D})$, i.e.,

$$\hat{\theta} = \underset{\theta}{\text{argmax}}\ p(\theta|\mathcal{D}), \tag{A.7}$$

or, equivalently,

$$\hat{\theta} = \underset{\theta}{\text{argmax}}\ \mathcal{L}(\theta), \text{ where } \mathcal{L}(\theta) = \log p(\mathcal{D}|\theta) + \log p(\theta). \tag{A.8}$$

This is widely known as maximum a posteriori (MAP) estimation. In conjunction with the Gaussian likelihood of Eq. (4), Eq. (A.8) corresponds to regularized MSE minimization. For example, a Gaussian prior $p(\theta)$ gives rise to ℓ_2 regularization (also called weight decay in NN literature; e.g., [5]). Utilizing either one of Eqs. (A.5) and (A.8), we end up with a deterministic estimate of θ, and thus $\hat{u}$ is also deterministically estimated as $\hat{u} \approx u_{\hat{\theta}}(x)$.

Finally, if gradient descent with data sub-sampling is used, i.e., stochastic gradient descent (SGD), the update step of MAP at t-th iteration is given by

$$\theta^{(t+1)} = \theta^{(t)} + \epsilon \left[\nabla_\theta \sum_{i \in S} \frac{N}{|S|} \log p(u_i|x_i,\theta^{(t)}) + \nabla_\theta \log p(\theta^{(t)}) \right], \tag{A.9}$$

where only a mini-batch, i.e., datapoints in the index set S of size $|S|$, are considered and ϵ denotes the learning rate. Note that by modifying Eq. (A.9) slightly, we show in Section 4 how epistemic and aleatoric uncertainties can be incorporated. An overview of gradient descent optimization methods can be found in [171].

A.3. Hyperparameters and hyperpriors

We have assumed in Section 3.1 that the prior $p(\theta)$ and the parameters of the noise model (i.e., σ_u^2 for the Gaussian likelihood of Eq. (4)) are known and fixed. If they are not known, according to the Bayesian framework, they should be integrated out, similarly to Eq. (5) involving θ. For convenience, assume that the prior is parametrized by σ_θ^2; e.g., σ_θ^2 can be the variance of each normally distributed component of the parameter vector θ. Other prior choices are the Laplace, Cauchy, and horseshoe distributions, where σ_θ^2 can be construed as an unknown scale parameter; see relevant discussions in [155,172,173]. Next, by integrating over σ_θ^2 and σ_u^2, Eq. (5) becomes

$$p(u|x,\mathcal{D}) = \int p(u|x,\theta,\sigma_u^2)p(\theta,\sigma_\theta^2,\sigma_u^2|\mathcal{D})d\theta d\sigma_\theta^2 d\sigma_u^2, \tag{A.10}$$

where $p(\theta,\sigma_\theta^2,\sigma_u^2|\mathcal{D}) = p(\theta|\mathcal{D},\sigma_\theta^2,\sigma_u^2)p(\sigma_\theta^2,\sigma_u^2|\mathcal{D})$,

$$p(\theta|\mathcal{D},\sigma_\theta^2,\sigma_u^2) = \frac{p(\mathcal{D}|\theta,\sigma_u^2)p(\theta|\sigma_\theta^2)}{p(\mathcal{D}|\sigma_\theta^2,\sigma_u^2)}, \tag{A.11}$$

and

$$p(\sigma_\theta^2,\sigma_u^2|\mathcal{D}) = \frac{p(\mathcal{D}|\sigma_\theta^2,\sigma_u^2)p(\sigma_\theta^2)p(\sigma_u^2)}{p(\mathcal{D})}. \tag{A.12}$$

This procedure is called hierarchical modeling, σ_θ^2 and σ_u^2 are called hyperparameters in order to be distinguished from the NN parameters θ, and $p(\sigma_\theta^2)$, $p(\sigma_u^2)$ are called hyperpriors. Clearly, exact integration in Eq. (A.10) for real world problems is infeasible, and thus approximation techniques are typically utilized. A standard approximation is type-II maximum likelihood (also known as empirical Bayes), in which, in the same spirit of MLE of Section A.3, best hyperparameters are not integrated out, but are selected based on maximizing the likelihood $p(\mathcal{D}|\sigma_\theta^2,\sigma_u^2)$ of Eq. (A.11). For each of the BNN methods presented in Section B.2, we demonstrate how the hyperparameters can be optimized in practice.

So far in the discussion of this section we have considered a fixed model $\mathcal{H}$; e.g., a fixed NN architecture. Similarly to Eq. (A.10), different models can also, in principle, be integrated out. As was the case with the hyperparameters σ_θ^2 and σ_u^2, exact integration is not feasible and type-II maximum likelihood can be considered as an approximate way for selecting between competing models. In Section 5 we provide additional metrics for hyperparameter optimization and model design.

A.4. Posterior tempering for model misspecification

In general, for overparametrized NNs that are trained by utilizing finite datasets, the Bayesian posterior $p(\theta|\mathcal{D})$ is the optimal candidate distribution, with respect to θ, to be used in the model average of Eq. (5). Specifically, for a fixed prior, the Bayesian posterior minimizes the Probably Approximately Correct (PAC)-Bayes generalization bound [88,174]. This is also corroborated in practice, where the BMA approximated via Eq. (8) outperforms point estimate procedures (see, e.g., [175] and references therein). Nevertheless, this result requires perfect model specification; i.e., that there exists a θ such that the predictive model $p(u|x,\mathcal{D})$ induced by $p(\theta|\mathcal{D})$ matches exactly the data-generating process [88]. Indicatively, it requires that the approximated function belongs to the function space of the selected NN architecture and the correct data noise model is used.

In this regard, it has been shown that in many cases the BMA performs worse than a point estimate; see, e.g., [175, 176]. This is often attributed to model misspecification and is also studied in theoretical works [88]. A case of model misspecification is when the Gaussian likelihood of Eq. (4) is used in calculations, but the data follows, for example, a Student-t distribution (see Section D.2.2 for an example). The prior $p(\theta)$, which controls together with the NN architecture the function space of our approximation, may also lead to sub-optimal results using the Bayesian paradigm [173].

In this regard, a technique that is often used in practice is posterior tempering, i.e., sampling θ values from $p(\theta|\mathcal{D})^{1/\tau}$ instead of the true posterior ($\tau = 1$), where τ is called temperature. Specifically, it has been reported in the literature that "cold" posteriors, $\tau < 1$, perform better [175,177–179], although using a more informed prior can potentially remove this effect [173]. Cold posteriors can be interpreted as over-counting the available data using $1/\tau$ replications of it, thus, making the posterior more concentrated. This is also related to calibration discussed in Section 5.2. Note that posterior tempering may lead to posterior distributions that are harder to approximate, and thus users should carefully select the posterior inference method in such cases. Finally, in Section B.2 we explain how posterior tempering can be used for each of the presented posterior inference techniques. In passing, note that the post-training calibration techniques of Section 5.2 can also prove helpful for addressing model misspecification.

Appendix B. Supplementary material to Section 4: additional details of employed UQ methods

B.1. Gaussian processes (GPs)

Consider for simplicity a one-dimensional function approximation problem solved with GP regression. We denote the noisy data as $u = u_c + \epsilon_u$, where u_c is the function to be approximated and ϵ_u is a Gaussian noise with zero mean and variance σ_u^2. Then, the data $U = [u_1, \dots, u_N]^T$ on locations $X = [x_1, \dots, x_N]^T$ together with the predicted function $U_c^* = [u_{c1}^*, \dots, u_{cN'}^*]^T$ on test locations $X^* = [x_1^*, \dots, x_{N'}^*]^T$ follow a joint Gaussian distribution written as

$$\begin{bmatrix} U \\ U_c^* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K_u(X,X) + \sigma_u^2 I & K_u(X,X^*) \\ K_u(X^*,X) & K_u(X^*,X^*) \end{bmatrix}\right), \tag{B.1}$$

where $K_u(\cdot,\cdot)$ denotes the GP covariance function evaluated on each dataset pair and I the identity matrix. Consequently, the mean vector and covariance matrix of U_c^* are given as

$$mean(U_c^*) = K_u(X^*,X)\left[K_u(X,X) + \sigma_u^2 I\right]^{-1} U, \tag{B.2}$$

and

$$cov(U_c^*) = K_u(X^*,X^*) - K_u(X^*,X)\left[K_u(X,X) + \sigma_u^2 I\right]^{-1} K_u(X,X^*), \tag{B.3}$$

respectively. Eq. (B.3) provides the epistemic uncertainty of u evaluated at X^*. Further, the total uncertainty of u at X^*, i.e., the covariance matrix of $U^* = [u_1^*, \dots, u_{N'}^*]^T$, is given by adding $\sigma_u^2 I$ to Eq. (B.3).

Overall, exactly as in Eq. (A.3), aleatoric uncertainty is added to epistemic uncertainty. However, in GP regression we also have a relationship between epistemic and aleatoric uncertainties given by Eq. (B.3): higher values of σ_u^2 make the negative right-hand side term decrease, and thus make $cov(U_c^*)$ increase. For large σ_u^2 values, the negative right-hand side term becomes zero and epistemic uncertainty is given exclusively by the prior uncertainty; i.e., no new information is acquired from a dataset that is too noisy. Detailed information on GPs can be found in [180]; see also [66–68,70–72] for combining NNs with GPs.

B.2. Bayesian methods

B.2.1. Markov chain Monte Carlo (MCMC)

Markov chain sampling. Parameter θ samples from the posterior can be generated using a Markov chain with $p(\theta|\mathcal{D})$ as its invariant (or stationary) distribution, an idea that is foundational for a family of techniques known as Markov chain Monte Carlo (MCMC) techniques [27,32]. First, a Markov chain with initial state $\theta^{(0)}$ (following some initial distribution) and transition probability $Q_t(\theta^{(t+1)}|\theta^{(t)})$ for each state t is defined. If Q_t is independent of t, the Markov chain is called homogeneous with transition probability Q. An invariant distribution q is one for which if $\theta^{(t)}$ follows a distribution q, then any $\theta^{(t')}$ with $t' > t$ will also follow the distribution q. Equivalently, any transition Q_t applied to the invariant distribution q leaves q invariant. Further, an ergodic Markov chain is one that has a unique invariant distribution, called equilibrium distribution, and regardless of the initial state $\theta^{(0)}$ the same equilibrium distribution is reached in stationarity.

The goal of MCMC techniques is to construct an ergodic Markov chain with equilibrium distribution $q(\theta) = p(\theta|\mathcal{D})$, and subsequently sample from $p(\theta|\mathcal{D})$ by sampling from $q(\theta)$; this can be performed by simulating a realization of the Markov chain and discarding a number of "burn-in" states until equilibrium is reached. After sampling a state, there is no need to start over the Markov chain because of ergodicity, i.e., regardless the sampled state, after a few steps the equilibrium distribution q is reached again. We can thus resample from the same realization after a number of "lag" steps.

One way to construct an ergodic Markov chain is by applying a set of base transitions $B_1, \dots, B_K$ in turn (e.g., by changing only a subset of the dimensions of $\theta^{(t)}$ in each transition). To this end, q is required to remain invariant with respect to each transition, so that it remains invariant with respect to the overall transition $Q = B_1 \cdots B_K$, where the center dot denotes sequential transitions. Gibbs sampling is an example of the above scheme. Specifically, for a K-dimensional θ vector, K transitions $B_1, \dots, B_K$ are considered; in each transition only one dimension of the t-th state $\theta^{(t)}$ is changed using the conditional distribution (under $p(\theta|\mathcal{D})$) of each dimension given the current values of all the rest. Although Gibbs sampling is widely used in the context of latent variable models (e.g., [181]), in deep learning the conditional distributions under the posterior are typically not available. Nevertheless, Gibbs sampling is useful in the context of the stochastic dynamics method and of hyperparameter optimization.

A more general technique that unlike Gibbs sampling does not require sampling from complex distributions is the Metropolis algorithm. Specifically, a new state $\theta^{(t+1)}$ is generated from $\theta^{(t)}$ by first generating a candidate state using a proposal distribution $p_p(\theta^*|\theta^{(t)})$ and then deciding whether to accept the candidate θ^* or not. If it is not accepted, $\theta^{(t+1)}$ is taken to be previous state $\theta^{(t)}$. The above scheme is called global Metropolis, whereas if each component of $\theta^{(t)}$ is separately modified the scheme is called local Metropolis. Denoting $\log p(\theta|\mathcal{D})$ by $\mathcal{L}(\theta)$, the steps of global Metropolis are:

1. Generate candidate θ^* from a proposal distribution $p_p(\theta^*|\theta^{(t)})$,

2. If $\frac{p(\theta^*|\mathcal{D})}{p(\theta^{(t)}|\mathcal{D})} = \exp(-\mathcal{L}(\theta^*) + \mathcal{L}(\theta^{(t)})) \geq 1$, then accept θ^*, otherwise accept θ^* with probability $\exp(-\mathcal{L}(\theta^*) + \mathcal{L}(\theta^{(t)}))$,
3. If θ^* is accepted, then set $\theta^{(t+1)} = \theta^*$, otherwise set $\theta^{(t+1)} = \theta^{(t)}$.

The acceptance criterion at Step 2 is called Metropolis acceptance function and can be shown that, under certain conditions on p_p, leads to $p(\theta|\mathcal{D})$ as the invariant distribution. The proposal distribution $p_p(\theta^*|\theta^{(t)})$, among many choices, can be a Gaussian distribution centered at $\theta^{(t)}$; see [27]. The Metropolis algorithm is an important step of Hamiltonian Monte Carlo presented below.

The stochastic dynamics method. MCMC techniques have been widely used in statistical physics, and thus the terminology originating from their use in statistical physics often accompanies them even when used in domains other than physics. For example, in the MCMC literature $p(\theta|\mathcal{D}) = \exp(-\mathcal{L}(\theta))/Z$ is called canonical distribution over "positions" θ, $\mathcal{L}(\theta)$ is the energy of the system defined for each θ, and Z, the normalization constant, is known as the partition function. For formulating the stochastic dynamics method, a "momentum" variable m is also introduced for which there is one-to-one correspondence with the components of positions θ. The canonical distribution for the phase space of θ and m is defined as

$$p(\theta, m) \propto \exp(-H(\theta, m)), \tag{B.4}$$

where $H(\theta, m) = \mathcal{L}(\theta) + \mathcal{K}(m)$ is the total energy of the system and is known as the Hamiltonian function, and $\mathcal{K}$ denotes the "kinetic" energy. In this regard, θ and m are independent, i.e., the marginal of θ under $p(\theta, m)$ is equal to $p(\theta|\mathcal{D})$, which is the distribution we seek to sample from. Thus, a Markov chain that converges to $p(\theta, m)$ can first be constructed, and subsequently θ samples can be collected by sampling from $p(\theta, m)$ and discarding the m samples.

Overall, the stochastic dynamics method consists of two tasks: (1) sampling states (θ, m) with fixed energy H, and (2) sampling states with different values of H. In Task 1, we obtain different samples of θ by following the Hamiltonian dynamics of the system without visiting states of different probability. This is done by simulating the Hamiltonian dynamics of the system, in which the state (θ, m) evolves in fictitious time τ. It can be shown that, in theory, H remains the same for different states, and thus $p(\theta, m)$ is invariant with respect to transitions related to following a trajectory based on the Hamiltonian dynamics equations. Next, in Task 2, we visit states with different probability by changing only the state of m. This is performed by Gibbs sampling updates of the momentum via its density which is proportional to $\exp(-\mathcal{K}(m))$. If, for example, $\mathcal{K}(m) = \|m\|_2^2/2$, the conditional distributions required by Gibbs sampling are readily available (the components of m have independent Gaussian distributions).

In practice, however, Hamiltonian dynamics are simulated using a discretization scheme and finite time steps (e.g., leapfrog method). To follow the dynamics for some period of fictitious time $\Delta\tau$, $T = \Delta\tau/\epsilon$ iterations are applied, with ϵ being the time step; this is the main computational bottleneck of the algorithm. As a result, H does not stay exactly the same and MC estimates obtained via the stochastic dynamics method will have some systematic error that goes to zero as ϵ goes to zero. The systematic error is eliminated in the hybrid MC method (HMC; also known as Hamiltonian MC) by merging the stochastic dynamics method with the Metropolis algorithm. In contrast to the stochastic dynamics method, in HMC the state reached is only a candidate; it is accepted based on the change in total energy (as in the Metropolis algorithm). These rejections eliminate the bias introduced by inexact simulation. Because of its high degree of accuracy, HMC is often used as the "ground truth" for comparing different approximate inference techniques. As reported in [35], HMC with an acceptance rate around 0.6 exhibits satisfactory performance, and thus in this study we tune the time/leapfrog step with the objective of yielding an acceptance rate close to 0.6 in our computations. For more theoretical and practical information on HMC the interested reader is directed to [24,27,34,182].

Next, the Langevin dynamics (LD) method is a computationally cheap version of HMC with $T = 1$, i.e., with a single leapfrog iteration. In fact, in the uncorrected version of LD (all candidates accepted) there is no need for the two-step approach described above and for explicitly representing the momentum at all; one stochastic differential equation, the Langevin equation, is used for sampling new values of θ. Specifically, for the t-th state, the Langevin equation is given as

$$\theta^{(t+1)} - \theta^{(t)} = -\frac{\zeta^2}{2}\nabla_\theta \mathcal{L}(\theta^{(t)}) + \zeta n, \tag{B.5}$$

where $n = [n_1, \ldots, n_K]^T$ and $n_j \sim \mathcal{N}(0, 1)$ for all $j \in \{1, \ldots, K\}$. By substituting $\zeta = \sqrt{\epsilon}$ and $\mathcal{L}(\theta^{(t)}) = -\log p(\mathcal{D}|\theta^{(t)}) - \log p(\theta^{(t)})$, Eq. (B.5) becomes

$$\theta^{(t+1)} = \theta^{(t)} + \frac{\epsilon}{2}\left[\nabla_\theta \sum_{i=1}^{N} \log p(u_i|x_i, \theta^{(t)}) + \nabla_\theta \log p(\theta^{(t)})\right] + \eta, \tag{B.6}$$

where $\eta = [\eta_1, \ldots, \eta_K]^T$ and $\eta_j \sim \mathcal{N}(0, \epsilon)$ for all $j \in \{1, \ldots, K\}$. The interested reader is directed to [183] for a Metropolis-adjusted LD algorithm. Note that all the MCMC methods described so far require computations over the whole dataset at every iteration, resulting in high computational cost for large datasets. Motivated by SGD, in which at each iteration only a mini-batch of the available data is used, [184] proposed a technique that combines LD and SGD, referred to as stochastic

gradient Langevin dynamics (SGLD). Specifically, the resulting parameter update is computed using only a subset of the data and is the same as the update in SGD (Eq. (A.9)), except with added Gaussian noise, i.e.,

$$\theta^{(t+1)} = \theta^{(t)} + \frac{\epsilon}{2}\left[\nabla_\theta \sum_{i\in S}\frac{N}{|S|}\log p(u_i|x_i,\theta^{(t)}) + \nabla_\theta \log p(\theta^{(t)})\right] + \eta. \tag{B.7}$$

Further, also motivated by the scalability issues of MCMC techniques, [185] introduced stochastic gradients within the HMC framework; see also [36] for a general framework for constructing MCMC samplers with data sub-sampling as well as [91] for a recent review. Furthermore, for including posterior tempering, presented in Section A.4, the term inside the brackets in Eq. (B.6) can be divided by τ (the temperature), while the variance of η remains unchanged; see [175] for more information. The optimal value of τ can be obtained via cross-validation using one of the metrics of Section 5 and validation data.

Finally, see [186–188] for a replica exchange MCMC technique that considers two θ particles that follow Langevin dynamics with two different temperatures. Specifically, the sampler swaps between the two particle chains with a certain ratio and at each step samples from the current chain. Concurrently with the present work, the replica-exchange MCMC technique has been employed in conjunction with DeepONet for operator learning [117]. Additional UQ methods based on MCMC can be found, indicatively, in [36,178,189–191].

Prior and data uncertainty optimization. If the likelihood function hyperparameter σ_u^2 is unknown, a deterministic way to optimize it is by alternating between MCMC steps for the parameters θ and maximum likelihood $p(\mathcal{D}|\theta,\sigma_u^2)$ steps for σ_u^2; this is commonly referred to as MC expectation-maximization (e.g., [192]). Further, a Bayesian approach that can be used also if both the prior and likelihood function hyperparameters σ_θ^2 and σ_u^2, respectively, are unknown, involves using Eq. (A.10) and integrating out σ_θ^2 and σ_u^2. In this context, the objective of MCMC becomes to sample from $p(\theta,\sigma_\theta^2,\sigma_u^2|\mathcal{D})$ instead of $p(\theta|\mathcal{D})$. A straightforward way to do this is by sampling the parameters and hyperparameters simultaneously according to the posterior expressed as $p(\theta,\sigma_\theta^2,\sigma_u^2|\mathcal{D}) \propto p(\mathcal{D}|\theta,\sigma_\theta^2,\sigma_u^2)p(\theta|\sigma_\theta^2)p(\sigma_\theta^2)p(\sigma_u^2)$, where $p(\sigma_\theta^2), p(\sigma_u^2)$ are hyperpriors; see, e.g., [32,112]. Note that, following this approach, the likelihood term $p(\mathcal{D}|\theta,\sigma_\theta^2,\sigma_u^2)$ does not depend explicitly on σ_θ^2, and thus σ_θ^2 only appears in the prior part of the posterior.

An alternative way used in this paper is based on Gibbs sampling; see [29,32,155,193]. That is, for each step the parameters are first sampled based on the current hyperparameters, and subsequently the hyperparameters are sampled based on the current parameters. Specifically, in step t, $\theta^{(t)}$ can be sampled based on $p(\theta^{(t)}|\mathcal{D},\sigma_\theta^{2(t-1)},\sigma_u^{2(t-1)})$ (exactly as done so far with HMC, for example), and subsequently $\sigma_\theta^{2(t)},\sigma_u^{2(t)}$ can be sampled based on $p(\sigma_\theta^{2(t)},\sigma_u^{2(t)}|\mathcal{D},\theta^{(t)}) = p(\sigma_\theta^{2(t)}|\theta^{(t)})p(\sigma_u^{2(t)}|\mathcal{D},\theta^{(t)})$. Next, assuming Gaussian prior and likelihood functions, we utilize an inverse-Gamma hyperprior for each of $\sigma_\theta^2,\sigma_u^2$; note that the inverse-Gamma distribution is the conjugate prior for Gaussian distribution with unknown variance. In more detail, $p(\sigma_\theta^2)$ is expressed as

$$p(\sigma_\theta^2) = \frac{\sigma_\theta^{-2(h_1+1)}}{\Gamma(h_1)h_2^{h_1}}\exp\left(-\frac{1}{\sigma_\theta^2 h_2}\right), \tag{B.8}$$

where h_1, h_2 are hyperparameters, and the conditional distribution $p(\sigma_\theta^2|\theta)$ is given as

$$\begin{aligned} p(\sigma_\theta^2|\theta) &\propto p(\sigma_\theta^2)p(\theta|\sigma_\theta^2) \\ &\propto \sigma_\theta^{-2(h_1+1)-K}\exp\left(-\frac{1}{\sigma_\theta^2}\left(\frac{1}{h_2}+\frac{\sum_{j=1}^K\theta_j^2}{2}\right)\right), \end{aligned} \tag{B.9}$$

where K is the size of θ. Using standard transformation of random variables rules, it can be shown that a sample from $p(\sigma_\theta^2|\theta)$ can be drawn using a Gamma distribution with shape/scale parametrization according to the following steps:

1. A sample is drawn from a Gamma distribution with shape parameter value equal to $h_1 + K/2$ and scale parameter value equal to $(h_2^{-1} + 0.5\sum_{j=1}^K\theta_j^2)^{-1}$,
2. The reciprocal of the Gamma sample is used as a sample from $p(\sigma_\theta^2|\theta)$.

Overall, the initial prior of θ, i.e., before performing Gibbs sampling, is expressed as

$$p(\theta) = \int p(\theta|\sigma_\theta^2)p(\sigma_\theta^2)d\sigma_\theta^2, \tag{B.10}$$

which, for the selected inverse-Gamma hyperprior, is a Student-t distribution (see, e.g., [155]). Note that although $p(\theta)$ is nowhere used in the computations within the Gibbs sampling scheme, it is useful for visualization purposes (see Fig. 19) as well as for selecting the hyperparameters h_1, h_2. Further, the learned prior of θ, i.e., after performing Gibbs sampling, is denoted as $\tilde{p}(\theta)$ and given by

$$\tilde{p}(\theta) = \int p(\theta|\sigma_\theta^2) p(\sigma_\theta^2|\mathcal{D}) d\sigma_\theta^2, \tag{B.11}$$

where

$$p(\sigma_\theta^2|\mathcal{D}) = \int p(\sigma_\theta^2|\theta) p(\theta|\mathcal{D}) d\theta. \tag{B.12}$$

That is, the learned prior can be viewed as a simple Gaussian fit, parametrized by σ_θ^2, to the posterior samples (see Fig. 19). Overall, during the alternating steps of Gibbs sampling, on the one hand $\tilde{p}(\theta)$ attempts to approximate the posterior $p(\theta|\mathcal{D})$ by a simple Gaussian fit, and on the other hand posterior θ samples attempt to both maximize likelihood, but also stay close to the continuously updated prior $\tilde{p}(\theta)$; see, e.g., Eq. (B.7) for the latter.

Similarly, for drawing a sample from $p(\sigma_u^2|\mathcal{D}, \theta)$, the reciprocal of a sample from a Gamma distribution with shape parameter value equal to $h_3 + N/2$ and scale parameter value equal to $(h_4^{-1} + 0.5 \sum_{i=1}^{N} ||u_i - u_\theta(x_i)||_2^2)^{-1}$ is used. The hyperparameters h_3, h_4 define the inverse-Gamma hyperprior $p(\sigma_u^2)$ and N is the number of datapoints. The interested reader is directed to [29] and [155] for extensions considering alternative distributions for the prior and the likelihood function.

B.2.2. Variational inference (VI)

Theoretical concepts. In variational inference (VI) the posterior is approximated by a so-called variational distribution $q_\omega(\theta)$ parametrized by ω [194]. The objective used for optimizing ω is marginal likelihood maximization; i.e., $p(\mathcal{D})$ of Eq. (7), which is the probability of observing the training dataset $\mathcal{D}$ under the model $\mathcal{H}$. In this regard, noting that $\log p(\mathcal{D})$ is constant with respect to $q_\omega(\theta)$, $\log p(\mathcal{D})$ is expressed as

$$\log p(\mathcal{D}) = \mathbb{E}_{q_\omega(\theta)} \left[\log p(\mathcal{D}) \right], \tag{B.13}$$

and using $p(\mathcal{D}, \theta) = p(\mathcal{D}) p(\theta|\mathcal{D})$ as

$$\log p(\mathcal{D}) = \mathbb{E}_{q_\omega(\theta)} \left[\log \left(\frac{p(\mathcal{D}, \theta)}{p(\theta|\mathcal{D})} \frac{q_\omega(\theta)}{q_\omega(\theta)} \right) \right], \tag{B.14}$$

or

$$\log p(\mathcal{D}) = \text{ELBO}(\omega) + KL\left(q_\omega(\theta)||p(\theta|\mathcal{D})\right), \tag{B.15}$$

where $KL\left(q_\omega(\theta)||p(\theta|\mathcal{D})\right)$ denotes the KL divergence between the true and the approximate posterior, and the evidence lower bound (ELBO) is given as

$$\text{ELBO}(\omega) = \mathbb{E}_{q_\omega(\theta)} \left[\log \left(\frac{p(\mathcal{D}, \theta)}{q_\omega(\theta)} \right) \right] \leq \log p(\mathcal{D}). \tag{B.16}$$

Obviously, $KL\left(q_\omega(\theta)||p(\theta|\mathcal{D})\right)$ cannot be computed since $p(\theta|\mathcal{D})$ is the sought distribution. However, by maximizing the ELBO, simultaneously the evidence of the model is maximized and the KL divergence between the true and the approximate posterior is minimized. Note, also, that Eq. (B.16) can equivalently be written as

$$\text{ELBO}(\omega) = \int \log\left(p(\mathcal{D}|\theta)\right) q_\omega(\theta) d\theta - KL\left(q_\omega(\theta)||p(\theta)\right), \tag{B.17}$$

i.e., maximizing the ELBO encourages $q_\omega(\theta)$ to explain the data, while being as close to the prior as possible (penalizing overcomplex models). Note, finally, that the quality of the approximation depends on the expressiveness of $q_\omega(\theta)$. That is, the richer the family of q distributions considered, the better the resulting bound will be; see, e.g., [30] for discussion and [195] for theoretical results. Relevant discussions and methods considering various distributions can be found, indicatively, in [33,35,37,47,196–203]. Overall, developing computationally efficient VI-based methods that approximate sufficiently the highly multi-modal posterior remains a challenge.

Having determined the optimal parameters ω, $p(\theta|\mathcal{D})$ in Eq. (5) is substituted by $q_\omega(\theta)$, which is straightforward to sample from. A MC estimate of the predictive distribution can be obtained via Eq. (8) by drawing M samples from the approximate posterior $q_\omega(\theta)$.

Computational efficiency and tractability. Eq. (B.17) can also be written as

$$\mathcal{L}_{VI}(\omega) = -\sum_{i=1}^{N} \int \log\left(p(u_i|x_i, \theta)\right) q_\omega(\theta) d\theta + KL\left(q_\omega(\theta)||p(\theta)\right), \tag{B.18}$$

where $\mathcal{L}_{VI}(\omega) = -\text{ELBO}(\omega)$ is the minimization objective. In this section, we present an estimator of the integral term in Eq. (B.18) and, more specifically, for its gradient with respect to ω in order to optimize it [37]. This derivative has the general form

$$I(\omega) = \nabla_\omega \int G(\theta) q_\omega(\theta) d\theta. \tag{B.19}$$

Expressing

$$q_\omega(\theta) = \int q_\omega(\theta, \eta) d\eta = \int q_\omega(\theta|\eta) p(\eta) d\eta, \tag{B.20}$$

where $q_\omega(\theta|\eta) = \delta(\theta - F(\omega, \eta))$ for some transformation $\theta = F(\omega, \eta)$, we obtain

$$\int G(\theta) q_\omega(\theta) d\theta = \int G(F(\omega, \eta)) p(\eta) d\eta, \tag{B.21}$$

which we can differentiate with respect to ω. Specifically, $I(\omega)$ becomes

$$I(\omega) = \int \nabla_\theta G(\theta)|_{\theta=F(\omega,\eta)} \, J_{F,\omega} p(\eta) d\eta, \tag{B.22}$$

where $J_{F,\omega}$ is the Jacobian of the transformation $\omega \mapsto F(\omega, \eta)$. Therefore, for a one-sample MC estimate of the integral of Eq. (B.22), we can draw a sample of η from $p(\eta)$ and estimate the derivative $I(\omega)$ by $\nabla_\theta G(\theta)|_{\theta=F(\omega,\eta)} \, J_{F,\omega} F(\omega, \eta)$. This is also known as path-wise estimator; see [37] for a detailed discussion.

If, for computational efficiency, only a mini-batch, i.e., datapoints in the index set S of size $|S|$ are considered, Eq. (B.18) is approximated as

$$\mathcal{L}_{VI}(\omega) = -\frac{N}{|S|} \sum_{i \in S} \int \log\left(p(u_i|x_i, \theta)\right) q_\omega(\theta) d\theta + KL\left(q_\omega(\theta)||p(\theta)\right). \tag{B.23}$$

The update step for ω at t-th iteration in a gradient descent optimization scheme, with ϵ as learning rate, is given by

$$\omega^{(t+1)} = \omega^{(t)} + \epsilon \left[\nabla_\omega \sum_{i \in S} \frac{N}{|S|} \log p(u_i|x_i, F(\omega, \eta_i)) - \nabla_\omega KL\left(q_\omega(\theta)||p(\theta)\right) \right], \tag{B.24}$$

where $\{\eta_i\}_{i=1}^{|S|}$ are samples from $p(\eta)$. In Eq. (B.24) the one-sample estimator of Eq. (B.22) has been considered, while the first derivative with respect to ω can be evaluated based on the integrand of Eq. (B.22). Overall, as seen in Eq. (B.24), the update steps of VI resemble the ones of SGD and SGLD (see Eqs. (A.9) and (B.7), respectively). Finally, for including posterior tempering, presented in Section A.4, the KL term in Eq. (B.24) is commonly multiplied by τ in practice (e.g., [175]). Similarly to posterior tempering in MCMC techniques, the optimal value of τ can be obtained via cross-validation.

Factorized Gaussian as a variational distribution. A common choice for $q_\omega(\theta)$ is a factorized Gaussian posterior given as [204]

$$q_\omega(\theta) = \mathcal{N}(\theta|\mu, diag(\Sigma^2)) = \prod_{j=1}^{K} \mathcal{N}(\theta_j|\mu_j, \Sigma_j^2), \tag{B.25}$$

where μ, Σ are vectors of size K. The reparametrization for this case is given as

$$\theta = F(\omega, \eta) = \mu + \Sigma \odot \eta, \tag{B.26}$$

and $\omega = \{\mu, \Sigma\}$. Note that, in general, an approximation in which the variational distribution factorizes over the parameters is often referred to as mean-field approximation (MFVI). Further, in our experiments we found that early stopping can help to improve performance. Specifically, in Section 6 we employ validation data and the negative log-likelihood metric to determine the optimal number of training steps for early stopping; see, e.g., [205] for a similar implementation.

Dropout as a Bernoulli variational distribution. Stochastic regularization relates to a family of techniques that are used to regularize NNs through the injection of stochastic noise into the model. One such technique, dropout, follows exactly standard NN training (including standard weight-decay regularization), but in every iteration some of the parameters are set to zero when computing the gradient descent step [206]. Equivalently, the weight matrices are multiplied by diagonal matrices with 1's and 0's in the diagonal for randomly deleting parameters in each iteration.

Gal [37] has shown that by expressing the above matrix multiplication as reparametrization, dropout is equivalent with BNNs trained with VI, given that the prior over the parameters and the approximate posterior in VI have a specific form. NNs trained with dropout can thus be used exactly as BNNs; they can be used for making predictions using Eq. (8) (multiple forward passes), where $\{\hat{\theta}_j\}_{j=1}^{M}$ are obtained by multiplying the optimal parameters by binary vectors drawn from a Bernoulli distribution. Equivalently, a set of optimal parameters are first obtained by training using dropout and then samples from the approximate posterior are just noisy versions of these parameters. This is called MC-dropout (MCD) in order to be distinguished from "standard" dropout. Standard dropout approximates the model average of Eq. (8) using only one forward

pass with the obtained set of parameters multiplied by a deterministic factor. Finally, different reparametrizations lead to different versions of dropout, e.g., Gaussian dropout. See [207] for relevant discussion and an extension of MCD.

Prior and data uncertainty optimization. The prior $p(\theta)$ enters VI in the KL term of Eq. (B.17), whereas the data uncertainty enters in the likelihood term. As explained in Section A.3, the prior likelihood function hyperparameters σ_θ^2 σ_u^2 can be optimized using type-II maximum likelihood. Within the context of VI, marginal likelihood is approximated with the ELBO, which can be used for hyperparameter optimization as well. Employing an alternating optimization scheme, in each step of the VI optimization, the variational parameters ω can be updated based on ELBO with fixed σ_θ^2 σ_u^2, and subsequently σ_θ^2 σ_u^2 can be updated based on ELBO with fixed ω.

B.2.3. Laplace approximation (LA)

Theoretical concepts. The Laplace method for approximating the posterior of a BNN was first proposed by [208]. For fixed hyperparameters, it involves optimizing the NN parameters according to Section A.2, and subsequently fitting a Gaussian distribution to the discovered mode $\hat{\theta}$, with width determined by the Hessian of the training error. This is equivalent to using a quadratic approximation for the logarithm of the probability density. In this regard, the covariance matrix of the fitted distribution is the inverse of the Hessian A of the loss at $\hat{\theta}$, which is expressed as $A = -\nabla^2_{\theta\theta}\mathcal{L}(\hat{\theta})$, where $\mathcal{L}(\hat{\theta}) = \log p(\hat{\theta}|\mathcal{D})$. Further, the posterior $p(\theta|\mathcal{D})$ is approximated as (see, e.g., [209]))

$$q(\theta) = \frac{1}{Z}\exp(-\mathcal{L}(\hat{\theta}) - \frac{1}{2}(\theta-\hat{\theta})^T A(\theta-\hat{\theta})), \tag{B.27}$$

where Z is a normalization constant. Based on the approximation of Eq. (B.27), the log-evidence $\log p(\mathcal{D}|\sigma_\theta^2, \sigma_u^2)$ of Eq. (7) can be approximated as

$$\log p(\mathcal{D}|\sigma_\theta^2, \sigma_u^2) \approx \mathcal{L}(\hat{\theta}) - \frac{1}{2}\log\det A + \frac{K}{2}\log 2\pi - \log Z_\theta - \log Z_u, \tag{B.28}$$

where $\log Z_\theta$, $\log Z_u$ are the normalization constants of the prior $p(\theta|\sigma_\theta^2)$ and the data likelihood $p(\mathcal{D}|\theta, \sigma_u^2)$, respectively; in practice $\log Z_\theta$, $\log Z_u$ are omitted (see e.g., [166,210]).

Because obtaining the exact Hessian is computationally expensive, the Fisher information matrix and the generalized Gauss-Newton matrix are commonly utilized as Hessian approximations. Indicatively, according to the generalized Gauss-Newton approximation, the Hessian $A = -\nabla^2_{\theta\theta}\mathcal{L}(\hat{\theta})$ is approximated using only first-order information as

$$A \approx -\nabla_\theta \mathcal{L}(\hat{\theta})\left[\nabla_\theta \mathcal{L}(\hat{\theta})\right]^T. \tag{B.29}$$

Nevertheless, A in Eq. (B.29) is a full matrix that must be inverted for computing the covariance matrix. To reduce the determinant and matrix inversion computational cost, in practice a diagonal factorization of the above approximations is considered. Alternative approximations can be obtained by using block-diagonal factorizations (typically known as KFAC, standing for Kronecker-factored approximate curvature). More information regarding Hessian approximations can be found in [166,210–215]. In the comparative study of Section 6, we adopt the approximation introduced in [212]. In passing, it is noted that an additional step to reduce the computational cost is to consider the approximate posterior of Eq. (B.27) only for a subset of the NN parameters (e.g., only the last layer) and use the MAP estimate for the rest [166,216].

For making predictions, a MC estimate of the predictive distribution can be obtained by drawing M samples from the approximate posterior $q(\theta)$, which is a Gaussian distribution with mean $\hat{\theta}$ and covariance matrix A^{-1}. The obtained samples $\{\hat{\theta}_j\}_{j=1}^M$ can be used for making predictions according to Eq. (8). However, it has been reported in the literature that the accuracy of the MC estimate deteriorates if an approximation of the Hessian A is considered (see, e.g., [166]). This can be attributed to the fact that the new approximate posterior corresponds to a linearized NN model, different from u_θ. Therefore, as proposed in [215], this linearized model should instead be used for predictions. Following [215], for one-dimensional outputs u, the epistemic part of the total variance of Eq. (10) is computed based on the analytical expression

$$\bar{\sigma}_e^2(x) = \left[\nabla_\theta u_{\hat{\theta}}(x)\right]^T A^{-1} \nabla_\theta u_{\hat{\theta}}(x). \tag{B.30}$$

In the present study, we compute the full matrix of A using Eq. (B.29) to achieve optimal performance and also use Eq. (B.30) for obtaining epistemic uncertainty. Finally, for including posterior tempering, presented in Section A.4, the Hessian A in Eq. (B.27) can be divided by τ; i.e., small values of τ make the posterior more concentrated. Similarly to posterior tempering in MCMC and VI techniques, the optimal value of τ can be obtained via cross-validation.

Prior and data uncertainty optimization. Type-II maximum likelihood based on Eq. (B.28) and as presented in Section A.3 can be used to optimize the prior and the likelihood function hyperparameters σ_θ^2 and σ_u, respectively. This approach leads to an alternating optimization scheme similar to the one presented for VI in Section B.2.2, where the hyperparameters are optimized online. As an approximation to marginal likelihood, the ELBO of Eq. (B.17) can be used by replacing q_ω with q of Eq. (B.27). However, in both cases computation of the Hessian A is required for updating σ_θ^2 and σ_u^2, which renders this approach too computationally expensive for high-dimensional θ. To address this issue, approximations can be considered

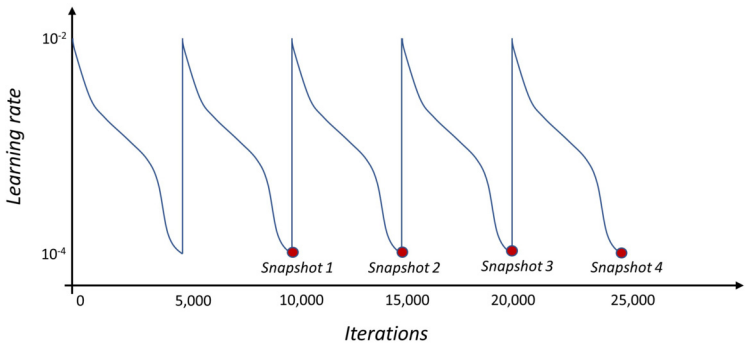


Fig. B.1. Learning rate schedule example for snapshot ensembles (SEns). The learning rate is initially decreased and after converging to a local minimum it is increased abruptly, letting the algorithm escape the current local minimum, and then decreased again for convergence at a different local minimum. After a certain number of iterations (e.g., 10,000 in the figure), a snapshot of NN parameters is captured before every restart to be used in the ensemble.

for A or the hyperparameter updates can be performed only every several iterations and not at every iteration. Instead of updating the hyperparameters alternately with the parameters θ, a different approach involves optimizing the hyperparameters after completing MAP optimization. In this case, the Hessian A makes use of the optimized hyperparameters for making predictions. Clearly, always hyperparameters can also, alternatively, be optimized in a cross-validation manner using validation data and one of the metrics of Section 5, e.g., predictive log-likelihood.

B.3. Ensembles

B.3.1. Deep ensembles (DEns)

Posterior inference by deep ensembles (DEns) pertains to obtaining M different settings of parameters θ by training the approximator NN (Section A.2) independently M times using different parameter initializations [43]. Because the philosophies of DEns and the Bayesian framework are similar [39], DEns can be interpreted as approximating the BMA of Eq. (5). Precisely, DEns approximates the posterior $p(\theta|\mathcal{D})$ using a mixture of delta distributions located at $\{\hat{\theta}_j\}_{j=1}^{M}$, which are different MAP estimates arising from random initialization of the NN. Therefore, Eq. (5) is approximated by Eq. (8) with M values of $\hat{\theta}$ which are not anymore samples from an approximate posterior, but rather M modes of the true posterior obtained by M MAP optimization runs. In passing, it can be shown that for linear models DEns provides samples that follow the posterior, unlike for standard NNs [217]. Finally, Pearce et al. [218] have proposed a modification to standard DEns that leads to approximate Bayesian inference, i.e., approximate samples from the posterior, even for NN models.

B.3.2. Snapshot ensembles (SEns)

The purpose of snapshot ensembles (SEns) is to perform BMA without incurring any additional cost as compared to standard NN training. Instead of training M different networks, standard training algorithms (Section A.2) are let to converge to M different local optima (see, e.g., [44]). The learning rate is initially decreased and after converging to a local minimum it is increased abruptly, letting the algorithm escape the current local minimum, and then decreased again for convergence at a different local minimum. Before every restart a snapshot is captured. A convenient way to implement this cyclical learning rate is by using a cosine annealing learning rate schedule; the range of learning rates and the number of epochs until the next restart can be treated as hyperparameters. Note that a larger restart learning rate leads to a stronger perturbation of the NN parameters and possibly to increased model diversity. The total training budget (e.g., 25,000 iterations) is divided into M cycles (e.g., $M = 5$) and a snapshot is collected at the end of each cycle. Fig. B.1 provides an illustration of the SEns training procedure. Note that DEns and SEns can be combined for forming a multi-snapshot ensemble (Multi-SEns) of $M \times \tilde{M}$ models, which consists of $\tilde{M}$ training trajectories, each one consisting of M snapshots.

B.3.3. Stochastic weight averaging (SWA) and SWA-Gaussian (SWAG)

According to stochastic weight averaging (SWA) proposed by Izmailov et al. [219], parameter snapshots captured during standard training (Section A.2) are averaged for producing the final prediction; this is in contrast with SEns that involve ensembling on the prediction level. Similarly to SEns, training involves a cyclical learning rate (but not necessarily) and the computational cost is the same as in standard NN training. Clearly, the main goal of SWA is to improve accuracy through ensembling, although uncertainty estimates can, in principle, be obtained by using the various captured parameter settings as in SEns.

An extension to SWA, called SWA-Gaussian (SWAG), that also provides uncertainty estimates has been proposed by Maddox et al. [49]. In SWAG, a Gaussian distribution is fitted to the snapshots of SEns. Specifically, SWA is used for estimating the mean of the Gaussian distribution and two different approaches for estimating the covariance matrix can be used. The first approach involves fitting a diagonal covariance matrix; a running average of the parameters and of the squares of the

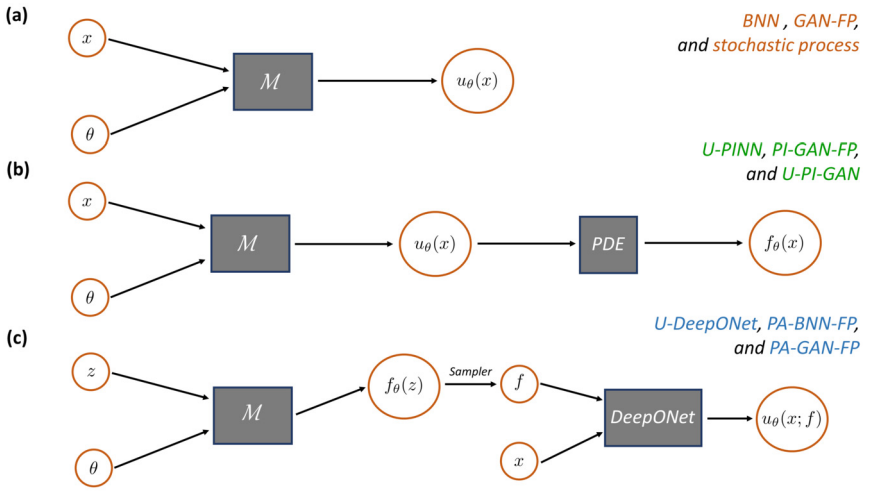


Fig. B.2. Repeated Fig. 8 from main text for convenience of the reader. **(a)** BNN, GAN functional prior (GAN-FP) and stochastic process representation. **(b)** U-PINN, PI-GAN-FP, and U-PI-GAN. **(c)** U-DeepONet, PA-BNN-FP, and PA-GAN-FP. $\mathcal{M}$ expresses how x and θ are combined to yield $u_\theta(x)$. In standard NN models (BNN, U-PINN, U-DeepONet), $\mathcal{M}$ is a BNN. In generator-based models (stochastic process, FP, PI-GAN-FP, U-PI-GAN, and PA-GAN-FP), $\mathcal{M}$ is the generator of a GAN that is trained using stochastic realizations or pre-trained using historical data. All models depicted in this figure can be construed as special cases of Fig. 10; note, however, the different meaning of θ for each technique. Further, in part (c), λ (or both f and λ) could be used as input to the DeepONet instead of f, as in Fig. 4. Furthermore, $u(x;f)$ is equivalent to $u(x;\xi)$, because each $\xi \in \Xi$ is associated with an input f. The additional input z is used to emphasize that the locations in the input domains of f and u can be different and the "sampler" that the function $f_\theta(z)$ must be sampled at pre-specified locations before entered as input to the DeepONet. See Table 2 for an overview of the UQ methods and Table 15 for a glossary of used terms.

parameters is maintained (updated once per iterations) and the covariance matrix is computed at the end of training. The second approach involves estimating the covariance as the sum of the diagonal covariance and a low-rank matrix given as

$$\Sigma = \frac{1}{Q-1}\sum_{j=1}^{Q}(\theta_j - \bar{\theta}_j)(\theta_j - \bar{\theta}_j)^T, \tag{B.31}$$

where $\bar{\theta}$ is the running average of the parameters. Note that Q defines the rank of the matrix and can be regarded as a hyperparameter of the method. In this study we used $Q = M$, i.e., the rank is equal to the number of snapshots obtained by using SEns. After fitting a covariance matrix it is straightforward to draw samples of the NN parameters and perform BMA using the MC estimate of Eq. (8) for making predictions. By combining SWAG with DEns, referred to as Multi-SWAG, i.e., by performing the above fitting procedure to each DEns trajectory, we can obtain even better uncertainty estimates.

B.3.4. Randomized priors

Randomized priors [55,56,220] typically use a weighted sum of two NNs to approximate a function, i.e., $u(x) \approx u_\theta(x) + \beta u^\star_\Theta(x)$, which is conventionally approximated by a single NN by all other methods. These two NNs generally share the same hyperparameters, including NN architecture, activation function, initialization method and prior distributions, and β is a fixed constant to balance their contributions to the output. In each training, θ and Θ are both randomly initialized and only the former is updated to optimize the loss function, whereas the latter remains the same throughout training. The term $u^\star_\Theta(x)$ is referred to as prior function. To perform BMA and obtain uncertainty, the set of parameters becomes $\{\hat{\theta}_j, \hat{\Theta}_j\}_{j=1}^M$ for training NNs independently M times. Hence, randomized priors do not approximate the posterior distribution $p(\theta|D)$ of the NN parameters, and the uncertainty arises from both updating randomly the initialized parameters and the randomly generated prior functions.

Randomized priors are similar to DEns in certain cases. In [55], it was shown that for function regression with linear models, randomized priors are equivalent to a variant of DEns, which, during NN training, regularizes the parameters to values randomly generated specifically for this training. Therefore, it follows that for regression with linear models, the main difference between randomized prior methods and deep ensemble methods lies in the regularization target. That is, deep ensemble methods utilize the same regularization target for all training sessions, whereas randomized prior methods regularize the parameters to different values, each of which is randomly generated from their joint prior distribution for each training session. Further, Bayesian ensembling, proposed in [218] as a variant of DEns, could also be considered as a variant of randomized prior methods. To the best of our knowledge, the similarity between randomized priors and DEns for SciML problems has not been investigated yet.

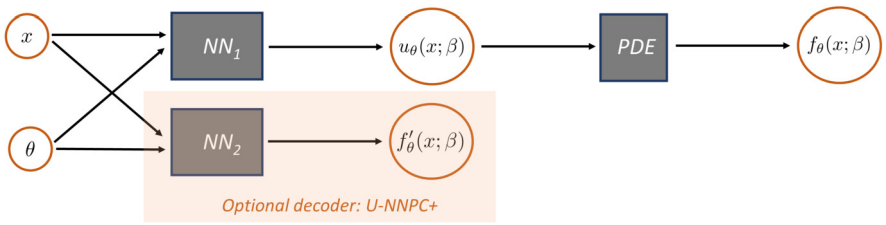


Fig. B.3. Repeated Fig. 9 from main text for convenience of the reader. NNPC, developed by [138], and NNPC+ (orange box), proposed herein, for solving SPDEs. NNs 1 and 2, parametrized by β, combine the location in the domain x to produce the modes of the stochastic representations of $u_\theta(x;\beta)$ and $f'_\theta(x;\beta)$, respectively. The model depicted in this figure can be construed as a special case of Fig. 10; note however the different meaning of θ in this technique.

B.4. Functional priors (FPs)

Consider Fig. B.2a. For given $\mathcal{M}$ and paired data of (x, u), the aim of standard NN training is to obtain optimal parameters θ to be used for making predictions $u_\theta(x)$. Next, for making training more efficient, meta-learning [221] can be used to obtain, among other choices, a better optimization algorithm, or a better $\mathcal{M}$ (architecture), or a better initialization of θ. In cases where the procedure is performed offline, i.e., one of the above components is pre-trained, the objective of meta-learning is to maximize predictive performance at "test-time", i.e., for solving a novel problem; see also [131] for a detailed view of meta-learning from a PINN perspective. For example, a better $\mathcal{M}$ may correspond to a pre-trained black-box function that takes as input x and θ, and outputs $u_\theta(x)$. Such a function has its own parameters, denoted herein by β, and can make learning θ at test-time more efficient.

Next, in the context of posterior inference according to Sections 4.1-4.2, meta-learning relates to obtaining either a better posterior approximation algorithm, or a better $\mathcal{M}$, or a better prior distribution $p(\theta)$, indicatively. For example, the online prior optimization procedures we presented in Sections B.2.1, B.2.2, and B.2.3 seek to make the learning procedure more efficient and the obtained predictive model better performing. As for offline procedures, indicatively, the techniques presented in [222–224] construct various different FPs by constructing $p(\theta)$ so that the BNN-induced FP (BNN-FP) behaves like a GP, or it assigns high probability to the data and to perturbations of it, or it constraints the values that the function is allowed to take in certain parts of the input space, respectively. See also [90,225,226] for additional techniques and a review.

In the following, suppose that we have a dataset $\mathcal{D}_h = \{U_i\}_{i=1}^{N_h}$, with $U_i = \{(x_j^{(i)}, u_j^{(i)})\}_{j=1}^{N_u}$, containing N_h historical realizations. The main idea of Meng et al. [136] is to utilize, as $\mathcal{M}$ in Fig. B.2, a pre-trained model parametrized by β instead of just a NN architecture. Specifically, it has been shown that if $u_\theta(x)$ is the output of a pre-trained generator $u_\theta(x;\beta)$, whose distribution over θ matches the historical data distribution P_{data}, posterior inference of θ at test-time can be performed faster and with less amount of data. Additionally, such a procedure allows the use of much lower-dimensional θ, even of size 10-100, as opposed to the high-dimensional θ corresponding to a typical NN architecture. For example, by using a type of generative adversarial network (GAN), called Wasserstein GAN with gradient penalty (WGAN-GP), we can train a generator NN parametrized by β whose underlying distribution P_{gen} approximates P_{data} by minimizing

$$\mathcal{L}_G(\beta) = -\mathbb{E}_{\theta\sim p(\theta)}\left[D_\gamma(u_\theta(X;\beta))\right]. \tag{B.32}$$

In this equation, $u_\theta(X;\beta)$ denotes the output of the generator for fixed θ and β evaluated on $X = [x_1, \dots, x_{N_u}]$ and can be seen as a point in an N_u-dimensional vector space. Further, in Eq. (B.32), $D_\gamma(\cdot)$ denotes a discriminator NN parametrized by γ, which aims to approximate the Wasserstein-1 distance between P_{data} and P_{gen}. This is achieved by minimizing the objective

$$\mathcal{L}_D(\gamma) = \mathbb{E}_{\theta\sim p(\theta)}\left[D_\gamma(u_\theta(X;\beta))\right] - \mathbb{E}_{S\sim P_{data}}\left[D_\gamma(S)\right] + \zeta\mathbb{E}_{\hat{S}\sim P_i}\left(\|\nabla_{\hat{S}} D_\gamma(\hat{S})\|_2 - 1\right)^2, \tag{B.33}$$

where P_i is the distribution induced by uniform sampling on interpolation lines between independent samples of $S \sim P_{data}$ and $u_\theta(X;\beta)$, and ζ is the gradient penalty coefficient, set as equal to 10 in the present study; see more information in [227]. Because $u_\theta(\cdot;\beta)$ is assumed to be noiseless, in order for P_{gen} to match P_{data}, we have to synthetically contaminate the functions produced by $u_\theta(\cdot;\beta)$ with the assumed amount of noise in the historical data, denoted by σ_h^2. If σ_h^2 is unknown, it can be optimized simultaneously with the generator and discriminator parameters; see Fig. 21 for an example. Finally, regardless of the amount of noise in the historical data used to optimize β, at test-time the new dataset $\mathcal{D} = \{x_i, u_i\}_{i=1}^N$ can have a different amount of noise. This is facilitated by the fact that the generator output $u_\theta(\cdot;\beta)$ is noiseless and the fact that pre-training of the generator is a separate procedure than learning θ at test-time.

B.5. Solving stochastic PDEs (SPDEs) with U-NNPC(+)

NNPC, depicted in Fig. B.3, was proposed in [138]; see also [139]. For the forward problem, given the dataset $\mathcal{D} = \{F_i\}_{i=1}^N$ with $F_i = \{x_j^{(i)}, f_j^{(i)}\}_{j=1}^{N_f}$, Principal Component Analysis (PCA) is first performed for the f data. This yields the low dimensional representation of the realizations, which is expressed as $\theta = \Phi^T \sqrt{Q}^{-1} f = \{\theta_j\}_{j=1}^K$, where Φ and the diagonal matrix Q contain the eigenvectors and the eigenvalues of the covariance matrix of the N realizations of f, respectively; and a measurement f is a vector of size N_f. Further, given a value of θ, $u_\theta(x;\beta)$ is expressed as

$$u_\theta(x;\beta) = \sum_{p=0}^{P} u_p(x;\beta)\rho_p(\theta), \tag{B.34}$$

where each ρ_p represents one of the $P+1$ considered polynomial basis functions and each $u_p(\cdot;\beta)$ represents one of the $P+1$ arbitrary polynomial chaos modes of u. Each $u_p(\cdot;\beta)$ is one of the outputs of a NN (specific to u) parametrized by β. See [138] for implementation information. Finally, β is trained such that $f_\theta(x;\beta)$ in Fig. B.3 approximates the data of f.

As an extension, a Karhunen-Loève expansion in the form

$$f'_\theta(x;\beta) = f'_0(x;\beta) + \sum_{j=1}^{S} \sqrt{q_j} f'_j(x;\beta)\theta_j, \tag{B.35}$$

can also be utilized for f. In this equation, each $f'_j(\cdot;\beta)$ represents one of the mode functions of f and is one of the outputs of an additional NN (specific to f), also parametrized by β for notation simplicity. This NN can be construed as a decoder NN, e.g., [5], because it takes as input the PCA encoding of f, which is expressed by θ, and outputs $f'_\theta(x;\beta)$ in Fig. B.3, which is trained to approximate the data of f. Finally, β is also trained such that $f_\theta(x;\beta)$ in Fig. B.3 approximates $f'_\theta(x;\beta)$. Note that when performing PCA for obtaining θ, some information of f is lost. Therefore, the rationale of NNPC+ is that the output f_θ of the PDE is not compared, in a loss function during training, with the data of f. Instead, it is compared with the output of a decoder that is trained to approximate the data of f given the low-dimensional input θ. Clearly, this added expressivity through NN_2 in Fig. B.3 may deteriorate performance, and thus regularization should be used during training. We refer to this technique as NNPC+ and provide computational results in Section 6.3.

For a mixed SPDE problem where λ is also stochastic and partially unknown, we consider the dataset $\mathcal{D} = \{\{F_i\}_{i=1}^N, \{U_i\}_{i=1}^N, \{\Lambda_i\}_{i=1}^N\}$, with $F_i = \{(x_j^{(i)}, f_j^{(i)})\}_{j=1}^{N_f}$, $U_i = \{(x_j^{(i)}, u_j^{(i)})\}_{j=1}^{N_u}$, and $\Lambda_i = \{(x_j^{(i)}, \lambda_j^{(i)})\}_{j=1}^{N_\lambda}$. Within NNPC(+), this additional stochastic term can be modeled similarly to Eq. (B.35) with a Karhunen-Loève expansion as

$$\lambda_{\theta_2}(x;\beta) = \lambda_0(x;\beta) + \sum_{j=1}^{S_2} \sqrt{q_j}\lambda_j(x;\beta)\theta_{2j}, \tag{B.36}$$

where $\theta_2 = \Phi_2^T \sqrt{Q_2}^{-1}\lambda = \{\theta_{2j}\}_{j=1}^{K_2}$; Φ_2 and the diagonal matrix Q_2 contain the eigenvectors and the eigenvalues of the covariance matrix of the N realizations of λ, respectively; and a measurement λ is a vector of size N_λ. Similarly to Eq. (B.35), each $\lambda_j(\cdot;\beta)$ represents one of the mode functions of λ and is one of the outputs of a NN (specific to λ) parametrized by β. Finally, $u_\theta(x;\beta)$ is given by Eq. (B.34), with $\theta = [\theta_1;\theta_2]$, where θ_1, θ_2 are the low dimensional representations of f and λ, respectively. See Section 6.3 for an example.

Appendix C. Supplementary material to Section 5: additional metrics and calibration details

C.1. Additional evaluation metrics

C.1.1. Secondary metrics evaluating statistical consistency

Because Eq. (19) averages predictions from different locations x, it is possible for a predictive model to be probabilistically calibrated but not ideal; e.g., a model with constant uncertainty for every x, although useless, may satisfy Eq. (19) if the predicted uncertainty is accurate for the entire data distribution. In this context, Gneiting et al. [147] have shown that a metric measuring sharpness, which is independent of any data, can be used. Specifically, between two models with the same RMSCE, we prefer the sharper one, where sharpness can be defined as $\text{PIW} = \mathbb{E}_{x\sim\mathcal{D}_{test}}\left[\hat{u}(x)_p^h - \hat{u}(x)_p^l\right]$ for some value of p, e.g., 95%. Finally, a model with homogeneous across x uncertainty predictions may still be performing well in terms of RMSCE and PIW, but be far from ideal. In this regard, references [147,149] have proposed to use a metric measuring dispersion in uncertainty predictions. The rationale for this metric is that a more disperse model can be more robust in terms of predictions away from training data. We denote the metric as SDCV and is given as the coefficient of variation of the predicted standard deviation for different x values, i.e., $\text{SDCV} = \frac{\sigma_\sigma}{\mu_\sigma}$, where μ_σ, σ_σ are given and approximated as

$$\mu_\sigma = \mathbb{E}_{x\sim\mathcal{D}_{test}} SD(u|x,\mathcal{D}) \approx \frac{1}{N_{test}}\sum_{i=1}^{N_{test}} \bar{\sigma}(x_i),$$
$$\sigma_\sigma = \sqrt{\mathbb{E}_{x\sim\mathcal{D}_{test}} [SD(u|x,\mathcal{D}) - \mu_\sigma]^2} \approx \sqrt{\frac{1}{N_{test}}\sum_{i=1}^{N_{test}} (\bar{\sigma}(x_i) - \mu_\sigma)^2}, \tag{C.1}$$

where $SD(u|x,\mathcal{D}) = \sqrt{Var(u|x,\mathcal{D})}$. Both PIW and SDCV cannot be used as metrics by themselves. Instead, for a given calibration value RMSCE, we prefer models with smaller PIW and larger SDCV values.

C.1.2. Comparison to gold standard

Further, when novel techniques are developed, they are expected to be more efficient or more accurate than alternatives. In those cases, usually a representative problem is solved with both the novel technique and a gold standard technique for comparison purposes. A gold standard can be an accurate but computationally expensive technique (e.g., HMC of Section B.2.1) or the exact solution if the problem is amenable to one. A comparison metric to be used in such cases is the normalized inner product between the predicted variances (see also [145,146]), i.e.,

$$\text{NIP-G} = \frac{\langle \sigma_G^2, Var(U|X,\mathcal{D})\rangle}{\sqrt{\langle \sigma_G^2, \sigma_G^2\rangle \langle Var(U|X,\mathcal{D}), Var(U|X,\mathcal{D})\rangle}}, \tag{C.2}$$

where $\sigma_G^2(X)$, $Var(U|X,\mathcal{D}) \approx \bar{\sigma}^2(X)$ are the predicted variances of a set of values of u, denoted by $U = [u_1,\dots,u_{N_{test}}]^T$, at locations $X = [x_1,\dots,x_{N_{test}}]^T$, obtained by the gold standard and the technique to be tested, respectively. Of course, NIP-G of Eq. (C.2) is connected through a linear relationship with the distance $\langle \sigma_G^2 - Var(U|X,\mathcal{D}), \sigma_G^2 - Var(U|X,\mathcal{D})\rangle$, but it is advantageous in terms of interpretability because it takes values only between 0 and 1. Nevertheless, NIP-G compares only the second-order statistics between the gold standard and the predictive distributions. A more comprehensive metric is the KL divergence between the two predictive distributions $p_G(u|x)$ and $\mathbb{E}_{\theta|\mathcal{D}}[p(u|x,\theta,\mathcal{D})]$ at an arbitrary location x. This is written as $KL(p_G(u|x)||\mathbb{E}_{\theta|\mathcal{D}}[p(u|x,\theta,\mathcal{D})])$ and denoted as KL-G in this paper. Once again, $\mathbb{E}_{\theta|\mathcal{D}}[p(u|x,\theta,\mathcal{D})]$ is approximated by $\bar{p}(u|x)$ via Eq. (8), and $\bar{p}(u|x)$ can be approximated by $\mathcal{N}(\bar{\mu}(x), \bar{\sigma}^2(x))$ given samples $\{u_{\hat{\theta}_j}(x)\}_{j=1}^M$.

C.2. Post-training calibration

C.2.1. CDF modification by fitting an auxiliary model

Following [148], assume that our predictive model is miscalibrated and does not satisfy $P(U_X \le \bar{P}_X^{-1}(p)) = p$, but instead satisfies $Q(p) = P(U_X \le \bar{P}_X^{-1}(p)) \neq p$ for every $p \in [0,1]$. Subsequently, note that $p = Q^{-1}(p')$ is satisfied for some arbitrary $p' \in (0,1)$ and

$$p' = Q \circ Q^{-1}(p') = P(U_X \le \bar{P}_X^{-1} \circ Q^{-1}(p')). \tag{C.3}$$

Using the property $(g\circ f)^{-1} = f^{-1}\circ g^{-1}$ we obtain

$$p' = P(U_X \le (Q\circ \bar{P}_X)^{-1}(p')), \tag{C.4}$$

which shows that $Q\circ\bar{P}_X$ yields a perfectly calibrated model; i.e., $P(U_X \le (Q\circ\bar{P}_X)^{-1}(p)) = p$ for any $p\in[0,1]$. Therefore, we can calibrate our model by applying Q to the output CDFs.

Clearly, Q is not known, but it can be approximated. Given calibration data $\mathcal{D}_r = \{x_i, u_i\}_{i=1}^{N_r}$ and the CDFs $\bar{P}(u_i|x_i)$ for every i, we can construct an estimate of $Q(p)$ as

$$Q_{N_r}(p) = \frac{1}{N_r}\sum_{i=1}^{N_r} \mathbb{1}(\bar{P}(u_i|x_i) \le p), \tag{C.5}$$

where the fact that every CDF is an increasing function has been used. We can, subsequently, train an auxiliary model that approximates Q using the dataset

$$\mathcal{D}_Q = \{\bar{P}(u_i|x_i), Q_{N_r}(\bar{P}(u_i|x_i))\}_{i=1}^{N_r}. \tag{C.6}$$

For example, isotonic regression, which is suitable for monotonic function fitting, can be used; an open-source implementation can be found in [143]. Note that if the model is calibrated, the dataset of Eq. (C.6) is simply a straight line (see Fig. 12). Finally, clearly this approach modifies both the mean and the variance of the predicted u at each x; the new mean and variance for each x can be obtained using

$$\bar{\mu}_{\text{new}}(x) = \int_0^\infty \left[1 - Q\left(\bar{P}(u|x)\right)\right] du - \int_{-\infty}^0 Q\left(\bar{P}(u|x)\right) du,$$
$$\bar{\sigma}^2_{\text{new}}(x) = \int_0^\infty \left[1 - \left[Q\left(\bar{P}(\sqrt{z}|x)\right) - Q\left(\bar{P}(-\sqrt{z}|x)\right)\right]\right] dz - \bar{\mu}^2_{\text{new}}(x), \tag{C.7}$$

where the integrals can be evaluated numerically.

C.2.2. CRUDE: CDF modification by fitting an empirical distribution

Zelikman et al. [152] have proposed a calibration approach according to which an empirical distribution is fitted to the scaled residuals that correspond to the calibration dataset. This is performed by simply creating a vector of sorted residuals expressed as $E_u = sorted\{\epsilon_u(x_i)\}_{i=1}^{N_r}$. The sorted residuals E_u can, subsequently, be combined with $\bar{\mu}(x)$ and $\bar{\sigma}(x)$ for each x for defining a new predictive distribution that follows the empirical one. Specifically, for any $p \in [0, 1]$, the corresponding percentiles can be obtained using

$$\hat{u}(x)^h_p \approx \bar{\mu}(x) + \bar{\sigma}(x) E_u[int(p_h N_r)],$$
$$\hat{u}(x)^l_p \approx \bar{\mu}(x) + \bar{\sigma}(x) E_u[int(p_l N_r)], \tag{C.8}$$

where $int(\cdot)$ returns the closest integer to a scalar. This approach also modifies both the mean and the variance of the predicted u at each x. The new values can be obtained as

$$\bar{\mu}_{\text{new}}(x) = \bar{\mu}(x) + \bar{\sigma}(x)\bar{\mu}_\epsilon,$$
$$\bar{\sigma}^2_{\text{new}}(x) = \bar{\sigma}^2(x)\bar{\sigma}^2_\epsilon, \tag{C.9}$$

where

$$\bar{\mu}_\epsilon = \frac{1}{N_r}\sum_{i=1}^{N_r} \epsilon_u(x_i),$$
$$\bar{\sigma}^2_\epsilon = \frac{1}{N_r}\sum_{i=1}^{N_r} (\epsilon_u(x_i) - \bar{\mu}_\epsilon)^2. \tag{C.10}$$

Appendix D. Supplementary material to Section 6: details and additional computational results

D.1. Details regarding Fig. 19 for the function approximation problem of Section 6.1

In all parts of Fig. 19, we obtain the histograms as described below and use kernel density estimation for visualization. In Fig. 19a, we plot the "initial prior" of θ, i.e., the histogram of θ values drawn from the Gaussian prior $p(\theta|\sigma^2_\theta)$, where σ^2_θ is the prior variance drawn from the Gamma hyperprior $p(\sigma^2_\theta)$ (see Section B.2.1); the "learned prior" of θ, i.e., the histogram of θ values drawn from $p(\theta|\sigma^2_\theta)$, where σ^2_θ is learned and drawn from $p(\sigma^2_\theta|\mathcal{D})$ (see Eq. (B.11)); the posterior distributions of two randomly picked NN parameters, i.e., the histograms of samples of these two parameters obtained during posterior inference; and the "stacked" posterior of θ, i.e., the histogram of all posterior samples θ stacked together. In Fig. 19b, we plot the initial prior, the learned prior, and the posterior of $u_\theta(x = 0)$, i.e., the histograms of $u_\theta(x = 0)$ given θ samples according to Fig. 19a. Finally, in Fig. 19c, we plot the prior and the posterior of the predicted noise σ_u, i.e., the histograms of the σ_u values drawn from the Gamma distributions $p(\sigma^2_u)$ and $p(\sigma^2_u|\mathcal{D})$; see Section B.2.1.

D.2. Additional results for the function approximation problem of Section 6.1

In this section, we provide additional results pertaining to the known homoscedastic noise (Section D.2.1) and to the unknown Student-t heteroscedastic noise (Section D.2.2) cases.

D.2.1. Known homoscedastic noise and post-training calibration

Here we evaluate the employed methods using OOD data (Table D.1), and provide the results of post-training calibration (Figs. D.1–D.3) and of MCD for various noise scales and dataset sizes (Fig. D.4).

D.2.2. Unknown student-t heteroscedastic noise

In this section, each data noise sample for x is given as $\epsilon_u = 0.5|x|\epsilon_t$, where ϵ_t is a sample from a standard Student-t distribution with $\nu = 5$ degrees of freedom; see Fig. 20(b). The variance of ϵ_u is given as $0.25|x|^2\frac{\nu}{\nu-2}$. Similarly to Section 6.1.3,

Table D.1
Function approximation problem of Eq. (20) — *Known homoscedastic noise*: although GP and HMC are expected to be the most accurate methods, final performance depends on the specific dataset considered and on tuning each technique individually. Further, techniques with large uncertainties for OOD data, although desirable, may perform worse in terms of MPL because of being conservative. See also [92,142] regarding the limitations of the evaluation metrics. Lastly, because calibration is performed using ID data, OOD performance may deteriorate. For example, compare the RMSCE and Cal RMSCE values corresponding to SWAG. Here we evaluate OOD performance of nine UQ methods based on the metrics of Section 5 Noisy test data was used for all metrics. All values were obtained using noisy test data and they correspond to uncalibrated predictions, except for calibrated (Cal) RMSCE.

Metric ($\times 10^2$)	GP	HMC	LD	MFVI	MCD	LA	DEns	SEns	SWAG
RL2E (↓)	68.1	67.4	51.4	**45.9**	61.6	69.6	64.0	82.4	88.1
MPL (↑)	99.3	100.5	78.7	70.5	86.7	84.0	**109.0**	73.3	65.1
RMSCE (↓)	26.9	28.7	17.3	**12.8**	40.9	28.7	35.7	35.0	22.0
Cal RMSCE (↓)	24.3	25.8	24.3	**11.4**	36.1	27.2	31.7	31.1	27.0
PIW (↓)	165.1	142.3	150.6	218.0	63.5	165.5	**61.9**	113.3	151.2
SDCV (↑)	48.6	48.7	9.9	35.9	27.5	41.9	47.4	**91.9**	19.6

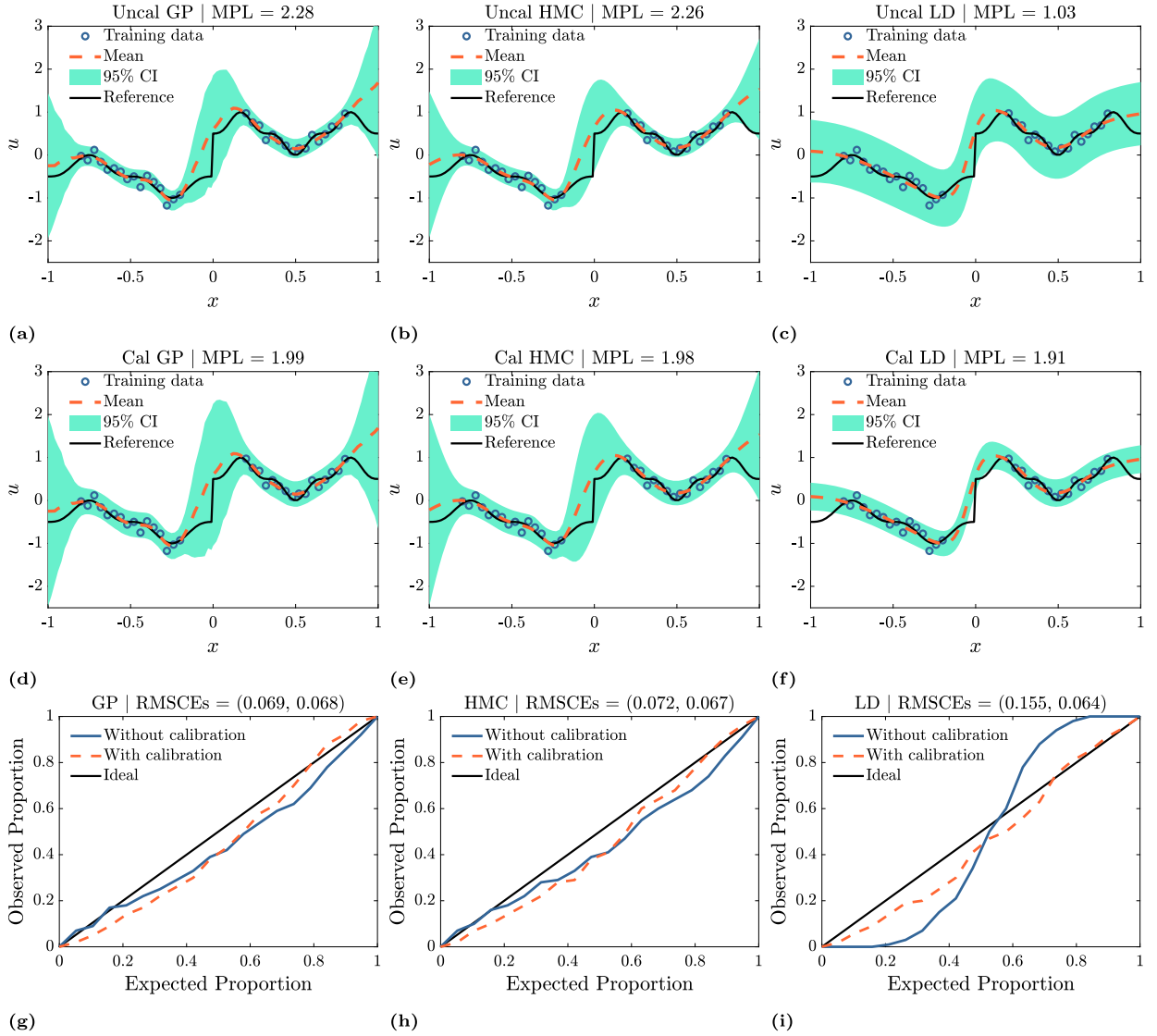


Fig. D.1. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: training data and exact function, as well as the mean and total uncertainty (95% CI) predictions of GP, HMC, and LD. **Top row:** uncalibrated predictions. **Middle row:** calibrated predictions (Section 5.2). **Bottom row:** calibration plots (see also Fig. 12) and RMSCEs before and after post-training calibration (in the parentheses).

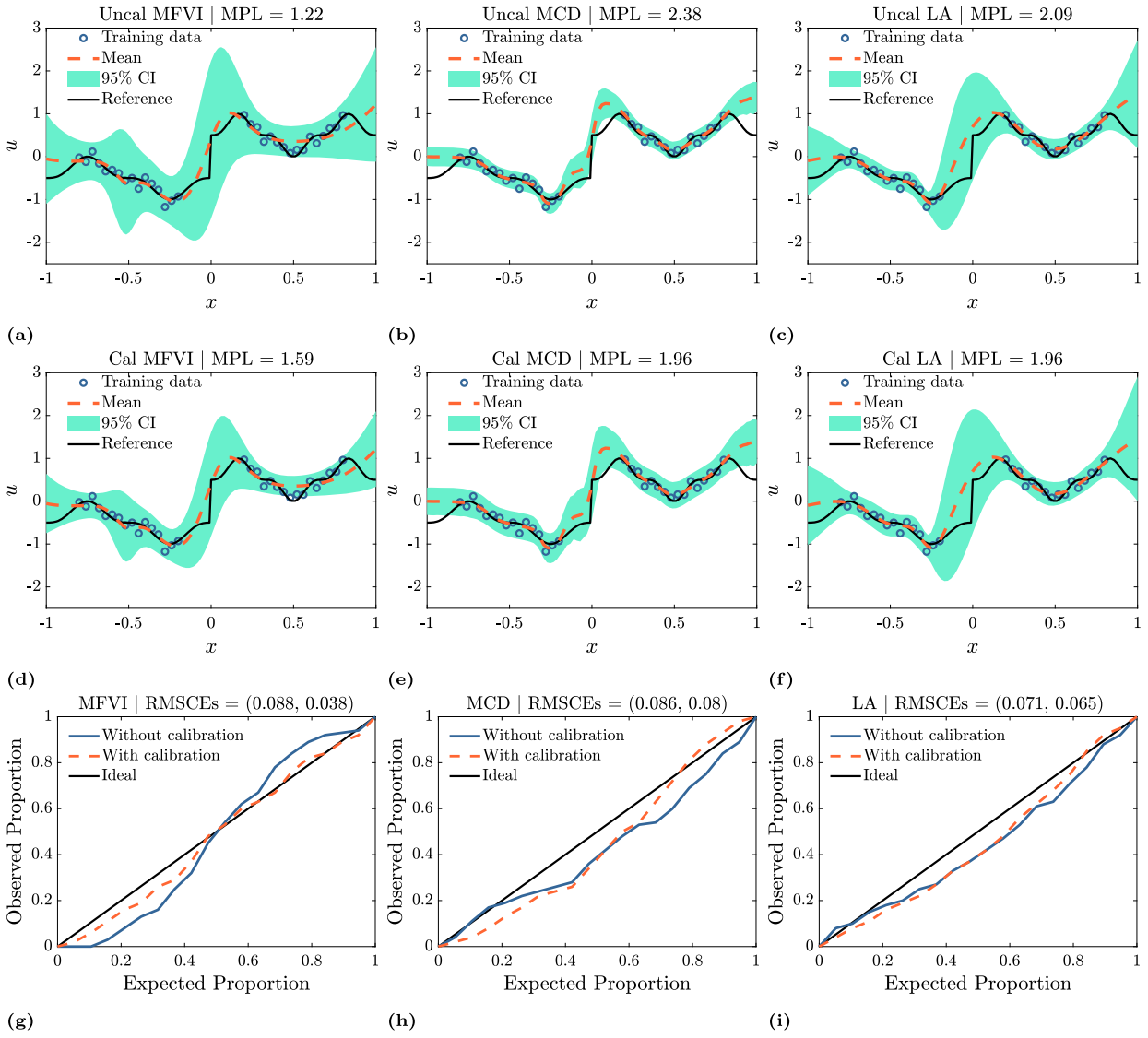


Fig. D.2. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: training data and exact function, as well as the mean and total uncertainty (95% CI) predictions of MFVI, MCD, and LA. **Top row:** uncalibrated predictions. **Middle row:** calibrated predictions (Section 5.2). **Bottom row:** calibration plots (see also Fig. 12) and RMSCEs before and after post-training calibration (in the parentheses).

the approximator NN has two outputs, one for the mean $u_\theta(x)$ and one for the variance $\sigma_u^2(x)$ of the Gaussian likelihood, and the results are obtained by h-HMC and h-MFVI. This corresponds to a model misspecification case (Section A.4), because we employ a Gaussian likelihood, although the data distribution is Student-t. For h-HMC and h-MFVI the results without posterior tempering for addressing this case are juxtaposed with the posterior tempering ones.

In Table D.2 we evaluate h-HMC and h-MFVI with and without posterior tempering, based on the metrics of Section 5. In Figs. D.5–D.6, we provide the corresponding mean predictions and predicted aleatoric uncertainties. By modifying the "temperature" τ we can control how conservative the predictions are. For this case of model misspecification, a cold posterior performs better in terms of predictive capacity (MPL). Cold posteriors have been found to perform well also in other studies [175,177,178]. In Fig. D.7, we employ posterior tempering also for Gaussian known homoscedastic and unknown heteroscedastic noise cases. In general, posterior tempering can be construed as including one additional hyperparameter in the training procedure. Thus, it can be used both for cases where the assumed model is correct and for model misspecification cases.

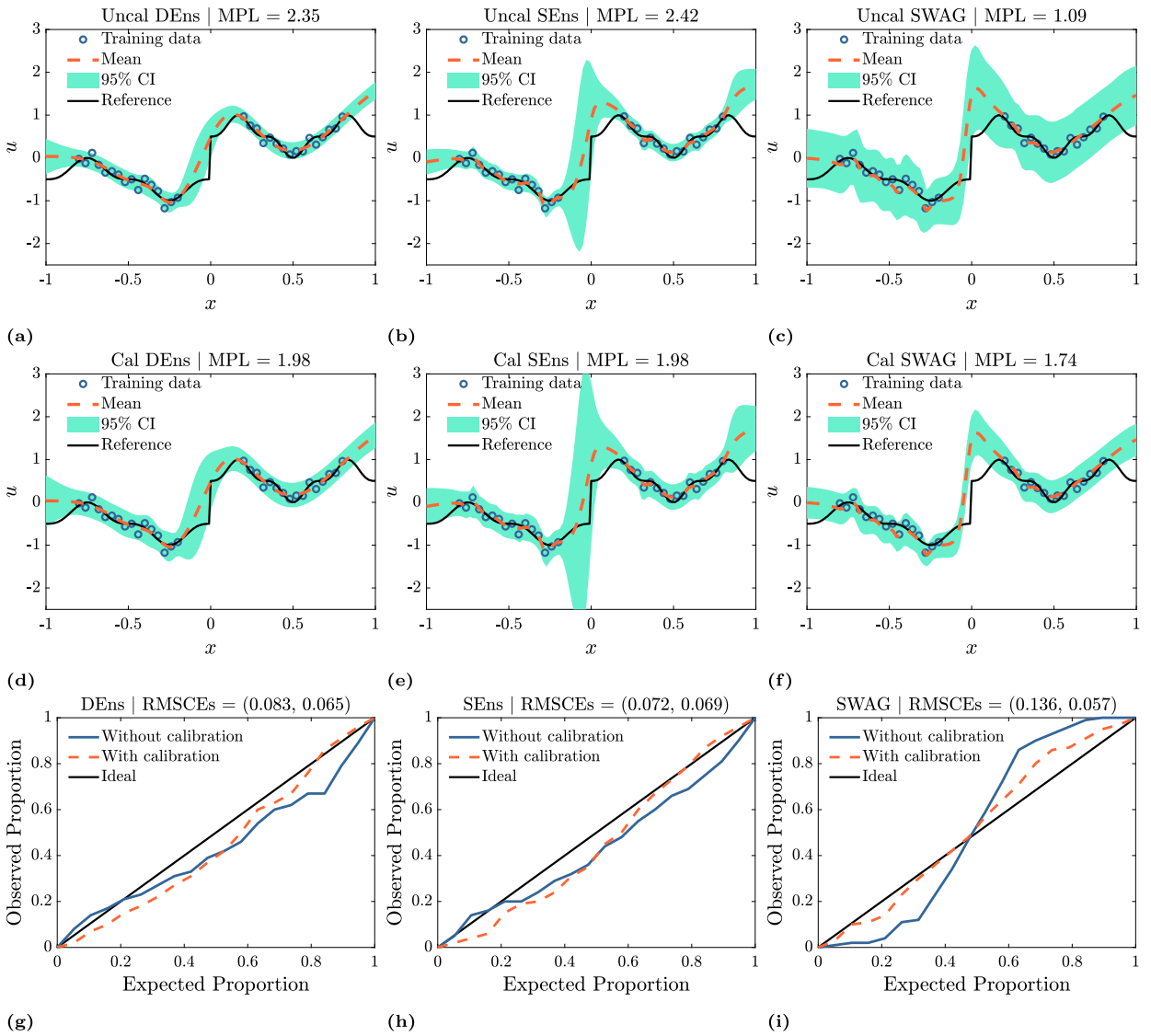


Fig. D.3. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: training data and exact function, as well as the mean and total uncertainty (95% CI) predictions of DEns, SEns, and SWAG. **Top row:** uncalibrated predictions. **Middle row:** calibrated predictions (Section 5.2). **Bottom row:** calibration plots (see also Fig. 12) and RMSCEs before and after post-training calibration (in the parentheses).

Table D.2
Function approximation problem of Eq. (20) — *Unknown Student-t heteroscedastic noise*: ID performance evaluation of HMC and MFVI without and with posterior tempering (PT - Section A.4). All values were obtained using noisy test data and they correspond to uncalibrated predictions. Both techniques perform similarly based on RL2E, while HMC performs better based on MPL. Nevertheless, note that according to Figs. D.5–D.6, the uncertainty of HMC increases for OOD data, whereas the uncertainty of MFVI does not consistently. This important disadvantage of MFVI compared to more accurate techniques, such as HMC, may not be captured by the evaluation metrics.

Metric	Without PT		With PT ($\tau = 0.5$)	
($\times 10^2$)	h-HMC	h-MFVI	h-HMC	h-MFVI
RL2E (↓)	**52.3**	52.4	53.3	**52.8**
MPL (↑)	**112.5**	83.5	**128.5**	89.6
RMSCE (↓)	**4.2**	6.2	5.3	**5.1**

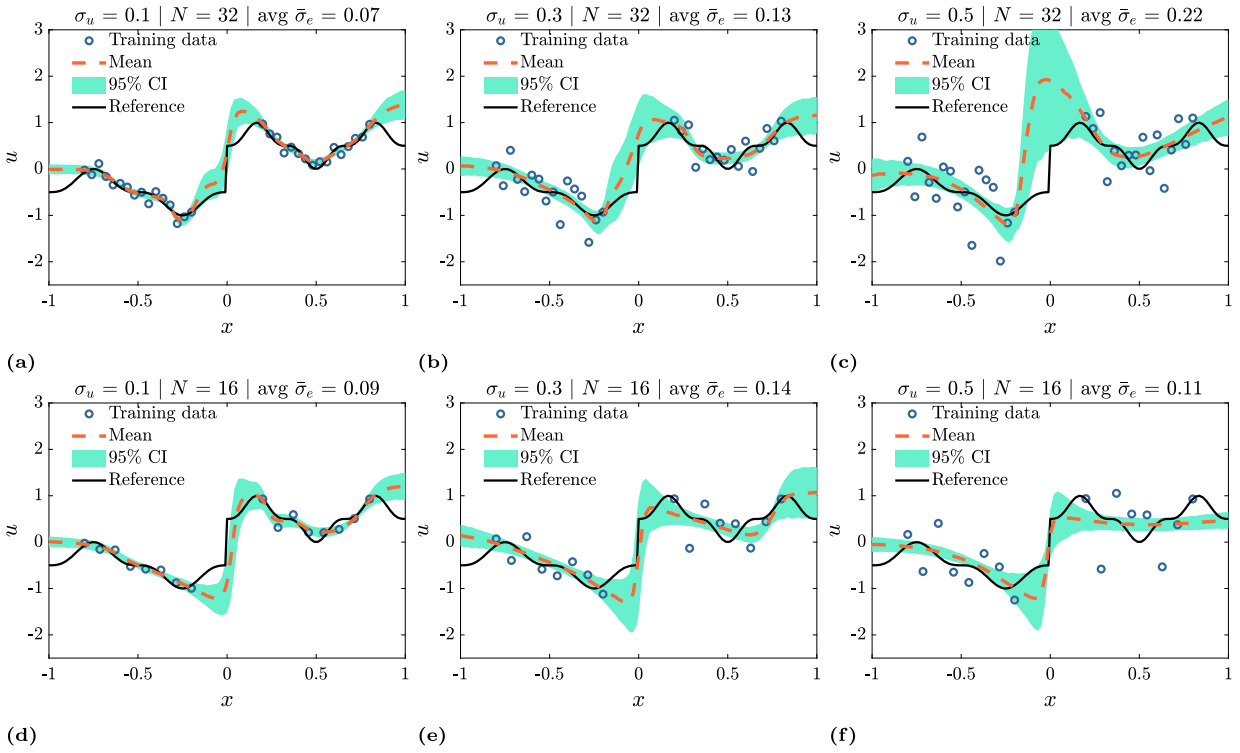


Fig. D.4. Function approximation problem of Eq. (20) — *Known homoscedastic noise*: epistemic uncertainty of MCD (avg σ_e^2 along x) does not increase with increasing noise magnitude and with decreasing dataset size. Shown here are the training data and exact function, as well as the mean and epistemic uncertainty (95% CI) predictions, as obtained by MCD in conjunction with three noise scales ($\sigma_u = 0.1$: left, 0.3: middle, 0.5: right) and two dataset sizes ($N = 32$: top, 16: bottom).

D.3. Additional results for the mixed PDE problem of Section 6.2

In this section, we present results pertaining to four more cases for the mixed PDE problem, i.e., (1) results from MFVI and MCD for the standard case of Section 6.2.1; (2) a "large noise" case in Section D.3.2 with a larger noise scale compared to the standard case; (3) an "extrapolation" case Section D.3.3, where the measurements of $\lambda(x)$ are concentrated in the interval $x \in [0, 1]$ and we extrapolate for $x \in [-1, 0]$; and (4) a "steep boundary layers" case in Section D.3.4, where the functions $u(t, x)$ and $\lambda(x)$ have large gradients for $x \approx -1$ and $x \approx 1$.

D.3.1. Standard case: homoscedastic noise and uniform measurements in space and time

The results obtained by MFVI (with learned noise) and MCD (with known noise) for the standard case of Section 6.2.1 are provided in Figs. D.8 and D.9. Although the predictions for u and f are satisfactory, the predictions for λ are over-confident in some areas of the domain.

D.3.2. Large noise case: homoscedastic noise with large noise scale

In this case, we keep the locations of all the training data the same as in Section 6.2.1 and only change the noise scale in the data from 0.05 to 0.1. The rest of the setup for this problem as well as the architecture of the BNN are the same as in Section 6.2.1. The results from HMC are illustrated in Fig. D.10. As shown, (1) we obtain larger predicted total uncertainty, as expected, when comparing the results with the ones in the standard case in Section 6.2.1, (2) the computational errors for u, f, and λ are all within the 95% CIs.

D.3.3. Extrapolation case: data concentrated on one side of the space domain

Here we use the same training data for u and f as in Section 6.2.1. In addition, we assume that we have the same number of observations on λ, but all the data are concentrated in $x \in [0, 1]$, as shown in Fig. D.11. In Fig. D.11 we present the results obtained by HMC. As shown, (1) we can still obtain accurate predictions for λ for $x \in [-1, 0]$, where we do not have any training data. This is because λ is coupled with u and f through the PDE. Thus, information on u and f is leveraged to help the inference of λ, (2) the computational errors for u, f, and λ are all within the 95% CIs.

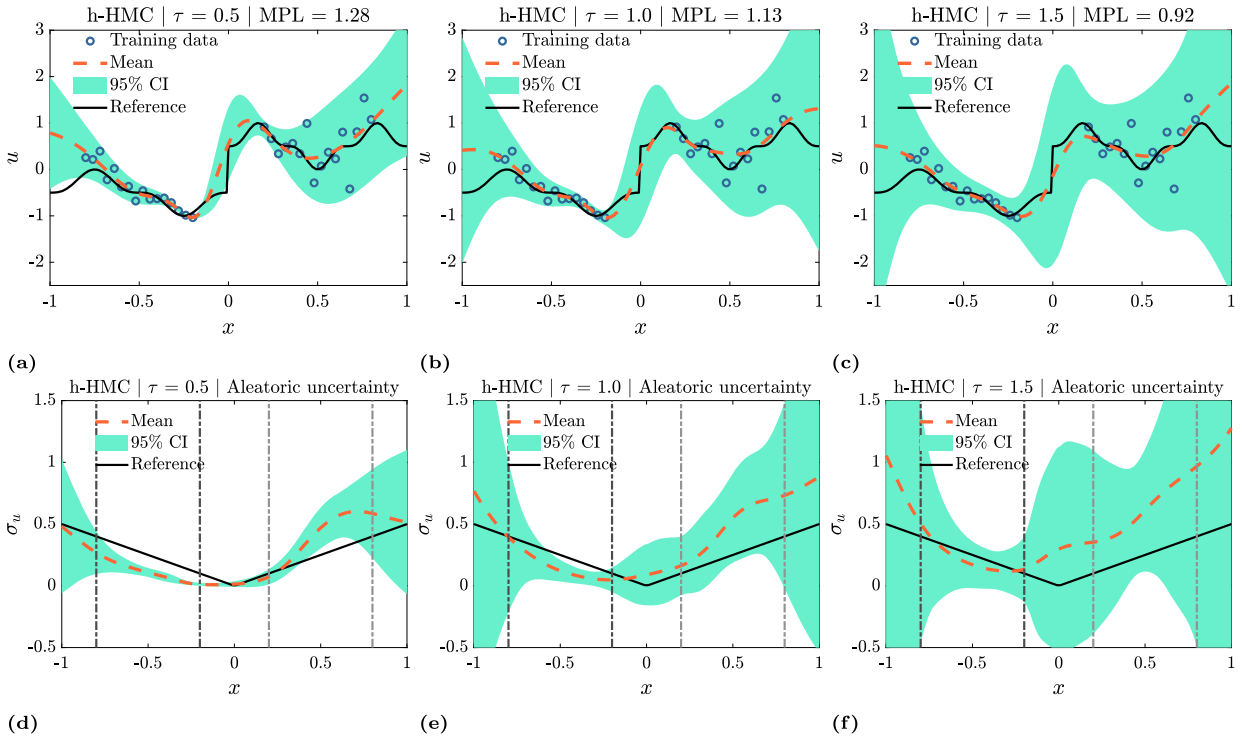


Fig. D.5. Function approximation problem of Eq. (20) — *Unknown Student-t heteroscedastic noise*: by modifying the "temperature" τ we can control how conservative the predictions are. For this case of model misspecification, a cold posterior performs better in terms of predictive capacity (MPL); see also Table D.2. Note that data on the right side "happen" to be more noisy and this is reflected in the noise predictions. Here shown are h-HMC predictions for three values of τ (see Section A.4). **Top row:** mean and total uncertainty of $u(x)$. **Bottom row:** mean and epistemic uncertainty of predicted aleatoric uncertainty $\bar{\sigma}_u(x)$. In the top row the training datapoints are also included, while in the bottom row the black and gray dot-dashed lines mark the training data intervals.

D.3.4. Steep boundary layers case: large gradients close to the boundaries of the space domain

In this section, we consider a more challenging case, in which we change the reaction rate λ to make u more sharp at the two boundaries. The equations for the problem considered here are expressed as:

$$\partial_t u = D\partial_x^2 u - \lambda(x)u^3 + f(x), t \in [0,1],\ x \in [-1,1], \tag{D.1}$$

$$u(-1,t) = u(1,t) = 1,\ u(x,0) = \cos^2(\pi x), D = 0.01, \tag{D.2}$$

$$\lambda(x) = 0.2 + \exp(x^2)\cos^2(3x), \tag{D.3}$$

$$f(x) = \exp(-\frac{(x-0.25)^2}{2l^2})\sin^2(3x),\ l = 0.4. \tag{D.4}$$

We solve the above equations using the *Matlab PDE toolbox* as in Section 6.2.1, and use the numerical results as the reference solution as well as to generate the training data. Specifically, we keep the locations of measurements for u and λ the same as in Section 6.2.1, and for f we assign more measurements near the two boundaries and further delete some measurements away from boundaries. The total number of measurements of f is $N_f = 13$. The noise scales for the training data are $\sigma_u = \sigma_f = \sigma_\lambda = 0.05$.

In Fig. D.12 we present the results obtained by HMC. As shown, (1) the computational errors for u, f, and λ are in most parts of the domains within the 95% CIs, (2) small fluctuations on the predictions of f at the left part are observed, which may be caused by the steep boundary layer of u, since f is related to the second-order derivative of u, and (3) no fluctuations on the predicted f at the right part are observed, which is reasonable since we have a lot of data on f near the right boundary, and the noise in the training data near the right boundary happens to be smaller than near the left boundary.

D.4. Additional results for the mixed stochastic problem of Section 6.3

In this section, we present the results corresponding to a post-training calibration experiment for the mixed SPDE problem of Section 6.3. First, we explain how to compute the calibration error (RMSCE) for this case. Consider one clean test realization containing values of u sampled at 101 points x. Using one of U-PI-GAN, U-NNPC, or U-NNPC+, we produce 100

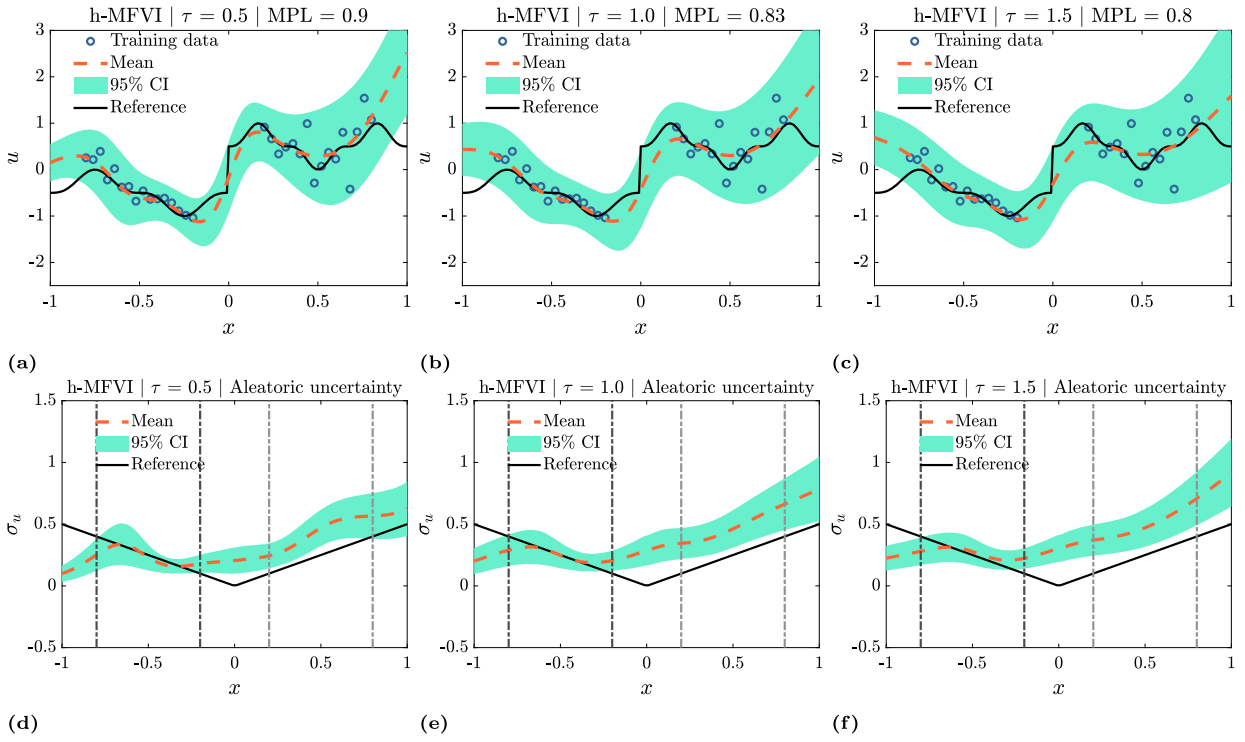


Fig. D.6. Function approximation problem of Eq. (20) — *Unknown Student-t heteroscedastic noise*: by modifying the "temperature" τ we can control how conservative the predictions are. Here shown are h-MFVI predictions for three values of τ (see Section A.4). **Top row:** mean and total uncertainty of $u(x)$. **Bottom row:** mean and epistemic uncertainty of predicted aleatoric uncertainty $\bar{\sigma}_u(x)$. In the top row the training datapoints are also included, while in the bottom row the black and gray dot-dashed lines mark the training data intervals.

realizations of u, sampled at 101 points, with each one of the three ensemble samples. This results to 300 realizations. Equivalently, for each of the $1 \times 101 = 101$ test points of $u(x)$, there are 300 prediction samples and it is straightforward to compute the RMSCE according to Eq. (19). If more than one test realizations are available, e.g., 100, we can use the same 300 predictions also for the additional realizations and compute the RMSCE based on $100 \times 101 = 10100$ test points. By doing so, each realization of $u(x)$ is using the same 300 predictions for computing the RMSCE. Similarly for λ. Because in this case the test realizations are clean, we only consider epistemic uncertainty in the predictions (3 ensemble samples) and uncertainty due to stochasticity (100 realizations). We refer to this as total uncertainty in this case, although it does not include aleatoric uncertainty (see also Eq. (16)).

Next, for performing post-training calibration, we assume that we have 2-50 available realizations of u and λ, not used during training. The data locations and noise are the same as in training. To perform calibration according to Section 5.2, we use the same 100 prediction realizations as we did for computing RMSCE, sampled at the training data locations. Fig. D.13 shows the effect of calibration approach and calibration dataset (number of left-out realizations) on RMSCE for U-PIGAN, U-NNPC, and U-NNPC+. In some cases, even with a few (2-10) noisy left-out stochastic realizations, we can reduce RMSCE by a factor of 3; see, e.g., Fig. D.13c.

D.5. Additional results for the forward PDE problem of Section 6.4

Here we present the results pertaining to the case of $N_f = 6$ datapoints for f. Specifically, we do not have any measurements of f at the two boundaries, as shown in Fig. D.14. It is shown that the predicted uncertainties for f by GP+PI-GAN at the two boundaries are larger than those by U-PINN, since the PDE is not used during training. Further, the predicted uncertainty of u is underestimated by GP+PI-GAN as compared to the results by U-PINN.

D.6. Details regarding the DeepONet input for the operator learning problem of Section 6.5

Assume that λ takes only positive values. For this reason, we consider that the noisy data of λ at each x, y follow a log-normal distribution with $\mathbb{E}[\lambda(x,y)] = \lambda_c(x,y)$ and $Var(\lambda(x,y)) = \alpha^2\lambda_c^2(x,y)$. That is, the mean of the data is equal to the true ("clean") value $\lambda_c(x,y)$ and the data contains an amount of noise proportional to $\lambda_c(x,y)$ at each (x,y). In order to follow the modeling procedure of Section 3, we consider $\tilde{\lambda} = \log\lambda$, which follows a Gaussian distribution with mean $\tilde{\lambda}_c = \log\lambda_c - \log(\sqrt{1+\alpha^2})$ and variance $\sigma^2 = \log(1+\alpha^2)$. Thus, although the original dataset is expressed as $\mathcal{D}_\lambda = \{x_i, y_i, \lambda_i\}_{i=1}^{N_\lambda}$,

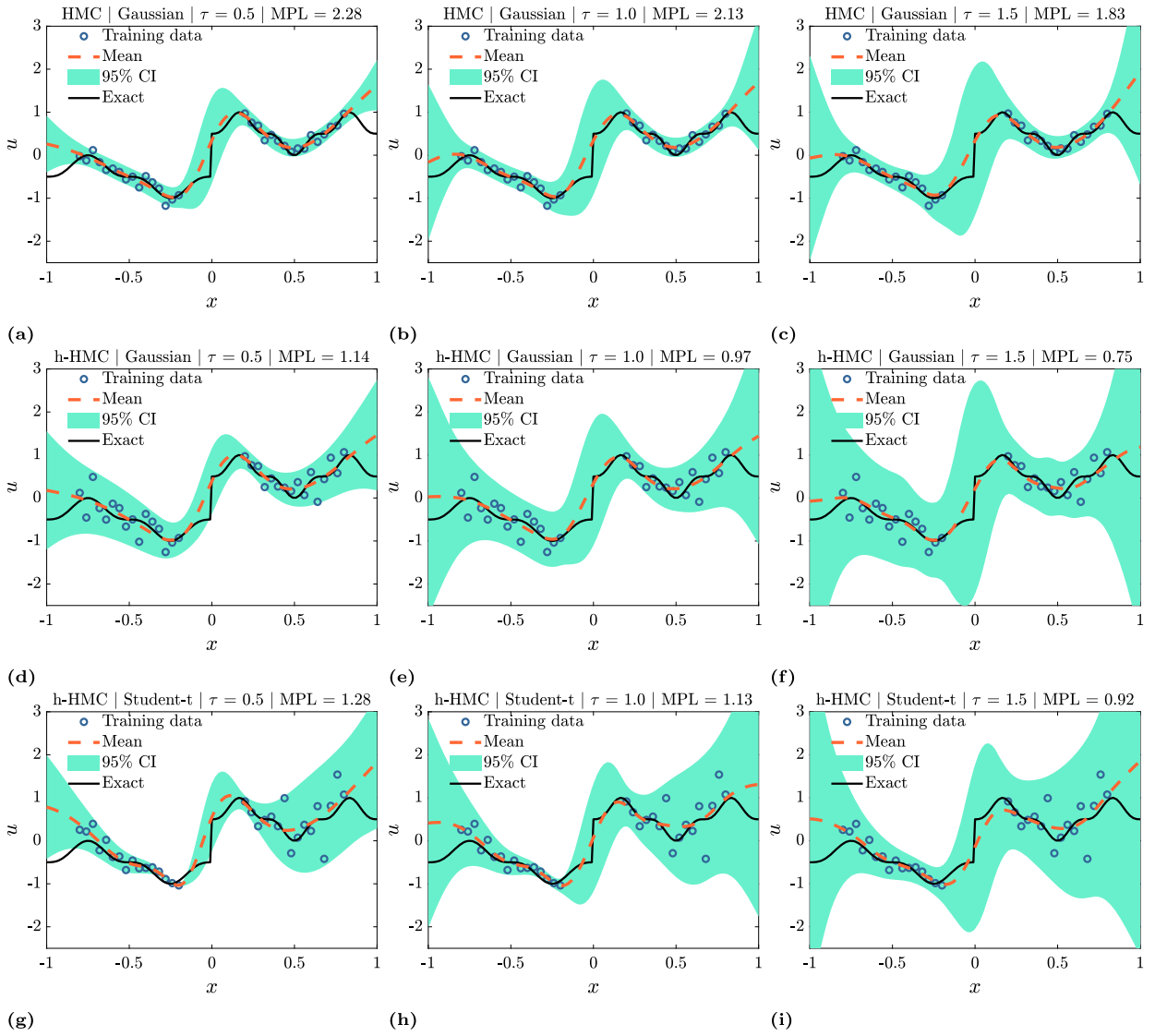


Fig. D.7. Function approximation problem of Eq. (20) — *Gaussian or Student-t noise*: posterior tempering (Section A.4) can be construed as including one additional hyperparameter in the training procedure. Thus, it can be used both for cases where the assumed model is correct (a-c) and for model misspecification cases (d-i). Here shown are the mean and total uncertainty predictions of uncalibrated HMC for different posterior temperatures. **Top row:** known homoscedastic Gaussian noise. **Middle row:** unknown heteroscedastic Gaussian noise. **Bottom row:** unknown heteroscedastic Student-t noise.

we utilize $\mathcal{D}_{\tilde{\lambda}} = \{x_i, y_i, \tilde{\lambda}_i\}_{i=1}^{N_\lambda}$, which contains the logarithms of the data. Suppose also that we standardize the new data using the sample means and standard deviations of the data used for training the DeepONet, on which the logarithm has been applied as well. Specifically, we subtract from each $\tilde{\lambda}_i$ the sample mean $\mu_d(x, y)$ and divide by the sample standard deviation $\sigma_d(x, y)$ for each (x, y). Then the standardized datapoint $\hat{\lambda}(x, y)$ for each (x, y) follows a Gaussian distribution with

$$\begin{aligned} \mathbb{E}[\hat{\lambda}] &= \frac{\tilde{\lambda}_c - \mu_d}{\sigma_d} = \frac{\log \lambda_c - \log(\sqrt{1+\alpha^2}) - \mu_d}{\sigma_d}, \\ Var(\hat{\lambda}) &= \frac{\sigma^2}{\sigma_d^2} = \frac{\log(1+\alpha^2)}{\sigma_d^2}, \end{aligned} \tag{D.5}$$

where all quantities depend on (x, y). After fitting the approximator λ_θ using the standardized data, predictions for the original quantity λ can be obtained via

$$\lambda_c(x, y) \approx \exp(\sigma_d(x, y)\lambda_\theta(x, y) + \log(\sqrt{1+\alpha^2}) + \mu_d(x, y)). \tag{D.6}$$

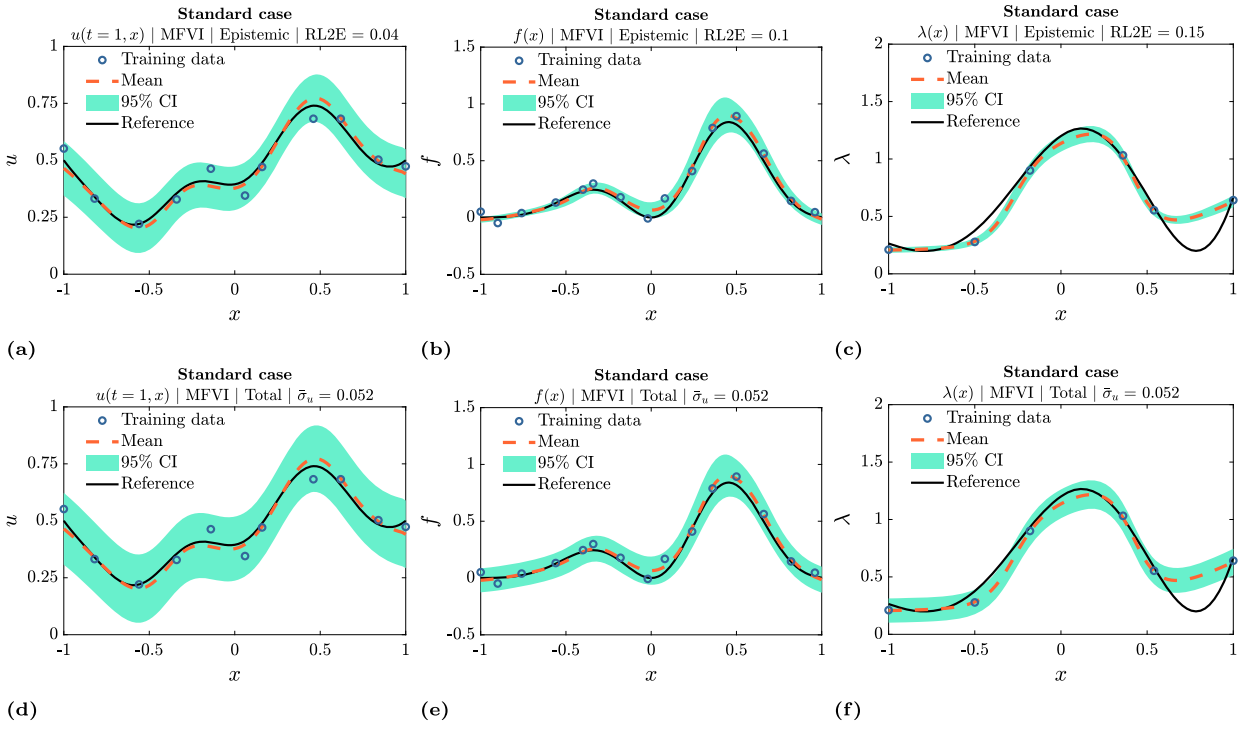


Fig. D.8. Mixed PDE problem of Eq. (21) — *Standard case*: training data, reference functions, as well as mean and uncertainty (95% CI) predictions of MFVI for u, f, and λ. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *learned* amount of aleatoric uncertainty, $\bar{\sigma}_u = 0.052$.

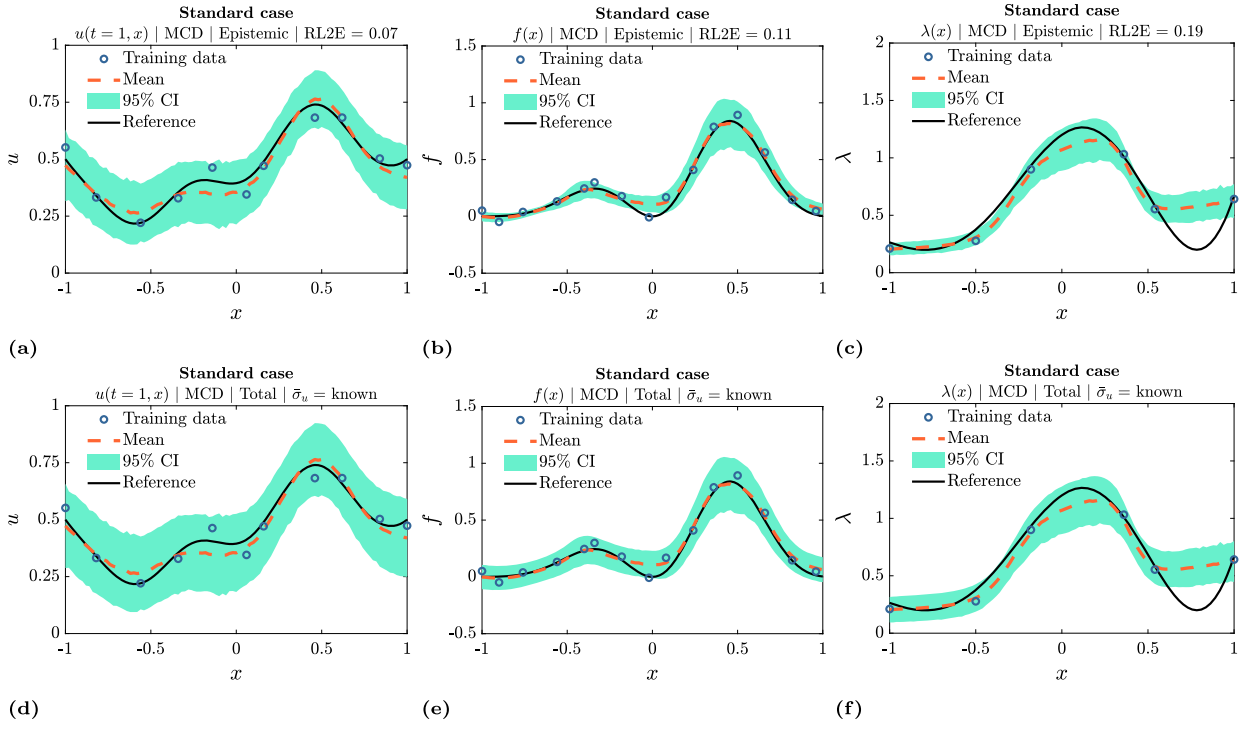


Fig. D.9. Mixed PDE problem of Eq. (21) — *Standard case*: training data, reference functions, as well as mean and uncertainty (95% CI) predictions of MCD for u, f, and λ. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *known* amount of aleatoric uncertainty, $\sigma_u = 0.05$.

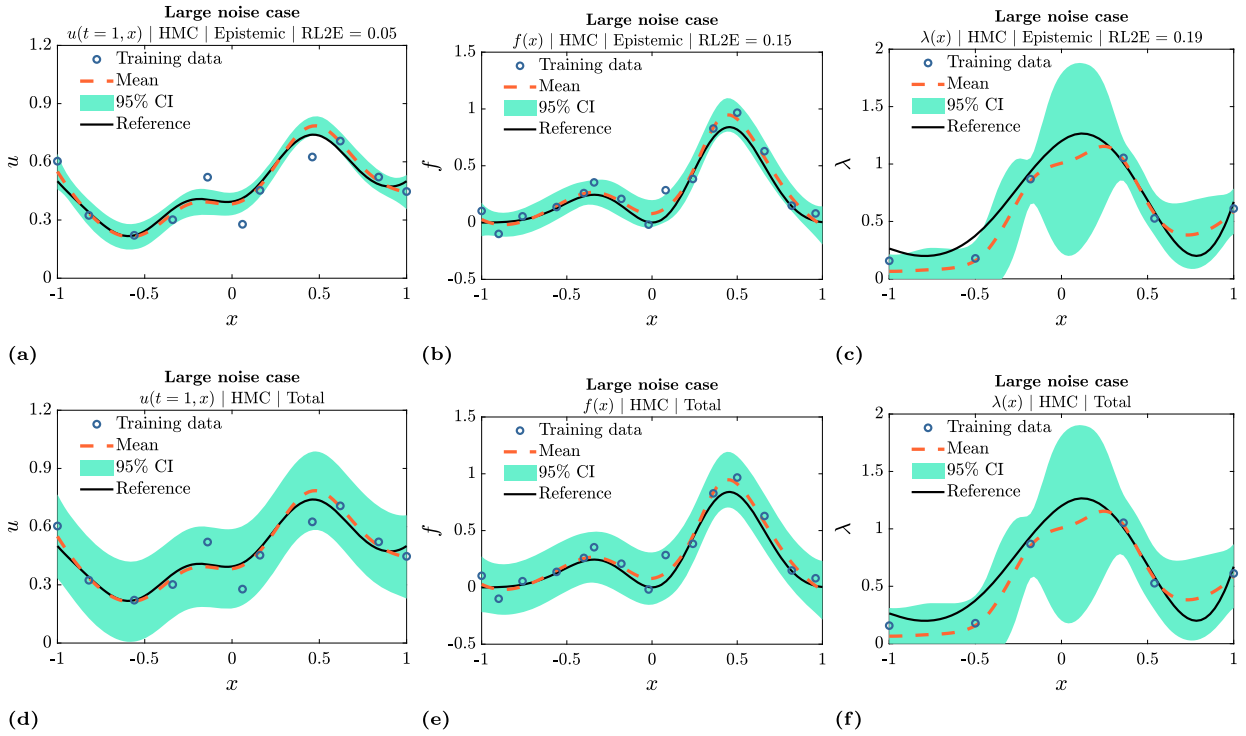


Fig. D.10. Mixed PDE problem of Eq. (21) — *Large noise case*: training data, reference functions, as well as mean and uncertainty (95% CI) predictions of HMC for u, f, and λ. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *known* amount of aleatoric uncertainty, $\sigma_u = 0.1$.

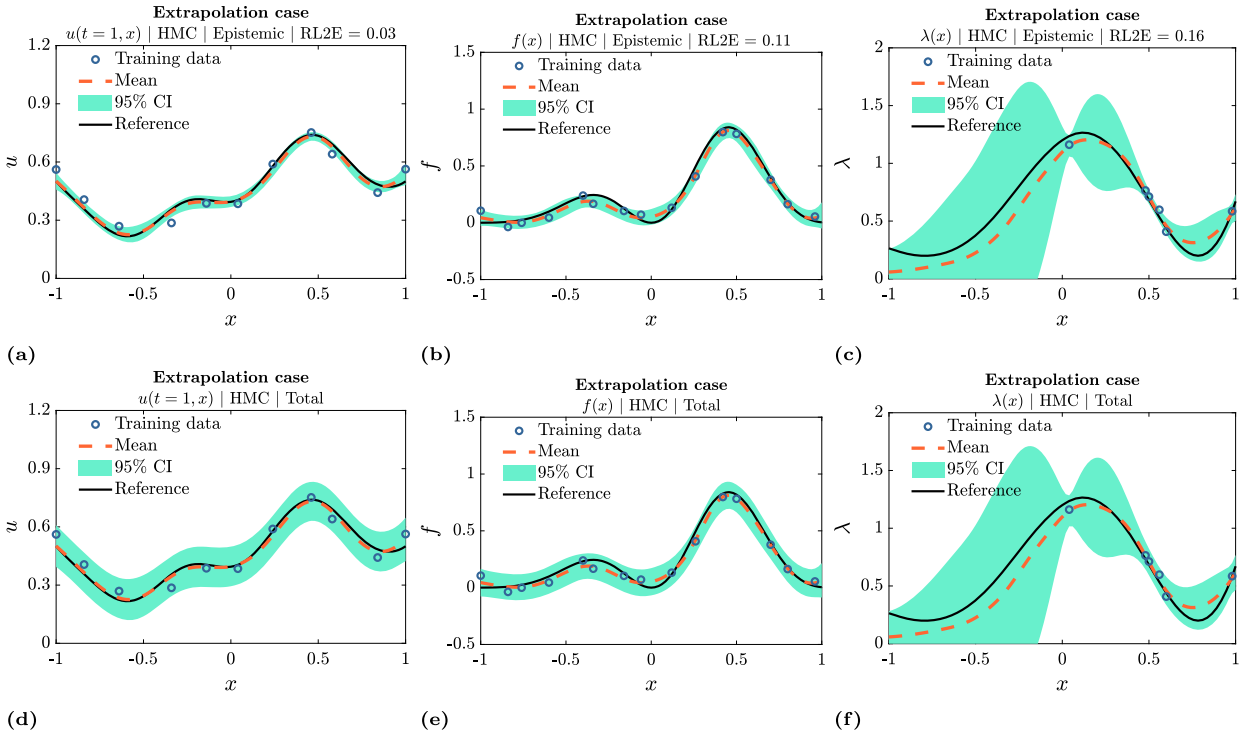


Fig. D.11. Mixed PDE problem of Eq. (21) — *Extrapolation case*: training data, reference functions, as well as mean and uncertainty (95% CI) predictions of HMC for u, f, and λ. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *known* amount of aleatoric uncertainty, $\sigma_u = 0.05$.

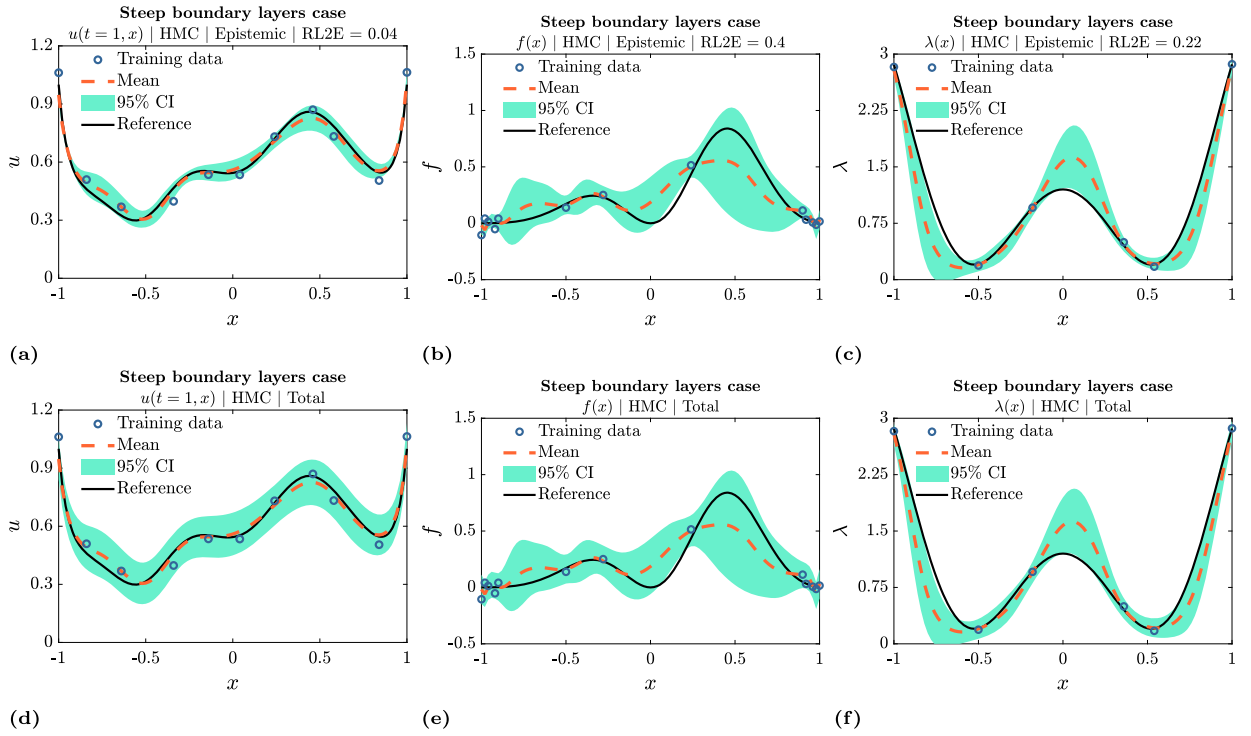


Fig. D.12. Mixed PDE problem of Eq. (21) — *Steep boundary layers case*: training data, reference functions, as well as mean and uncertainty (95% CI) predictions of HMC for u, f, and λ. **Top row:** epistemic uncertainty. **Bottom row:** total uncertainty including the *known* amount of aleatoric uncertainty, $\sigma_u = 0.05$.

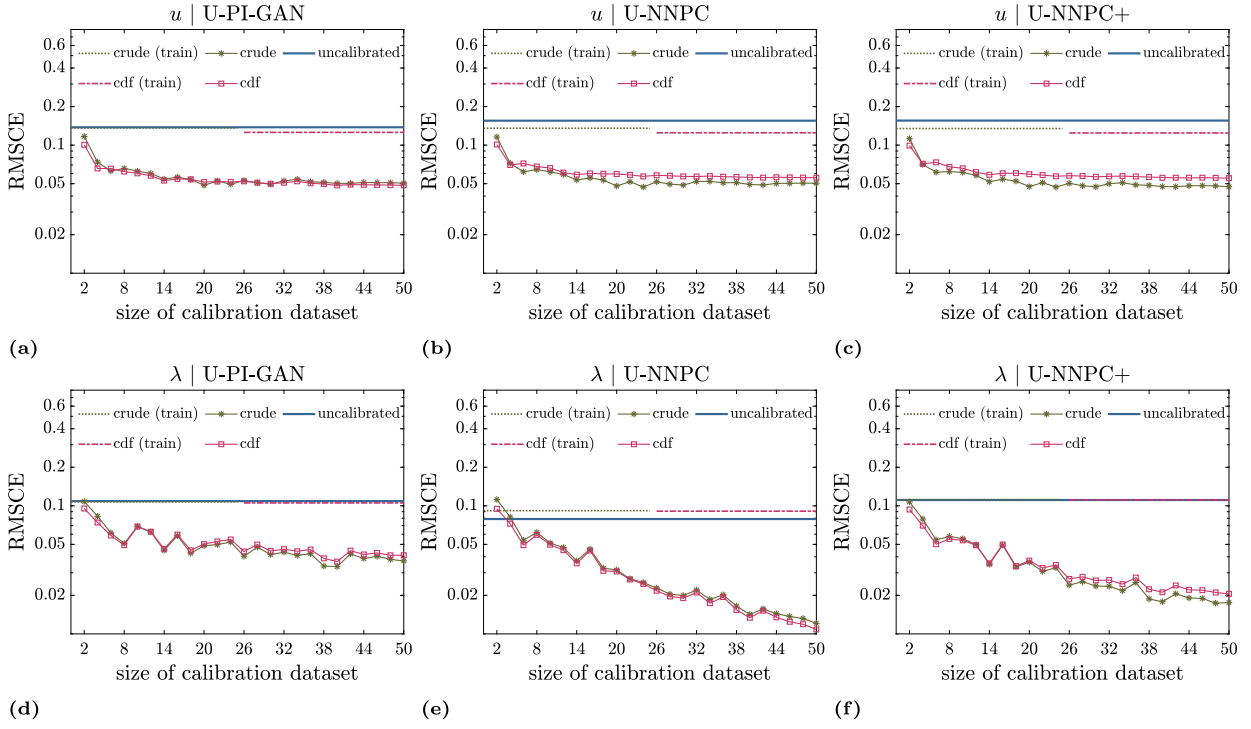


Fig. D.13. Mixed stochastic problem of Eq. (26): even a few left-out noisy realizations (2-10) of u and λ can be used for calibrating the total uncertainty predictions of all considered UQ methods. Effect of calibration approach (Section 5.2) and calibration dataset (number of left-out realizations) on RMSCE for U-PIGAN, U-NNPC, and U-NNPC+. Calibration is performed with total uncertainty, which in this case corresponds to epistemic and stochasticity uncertainties (see also Eq. (16)).

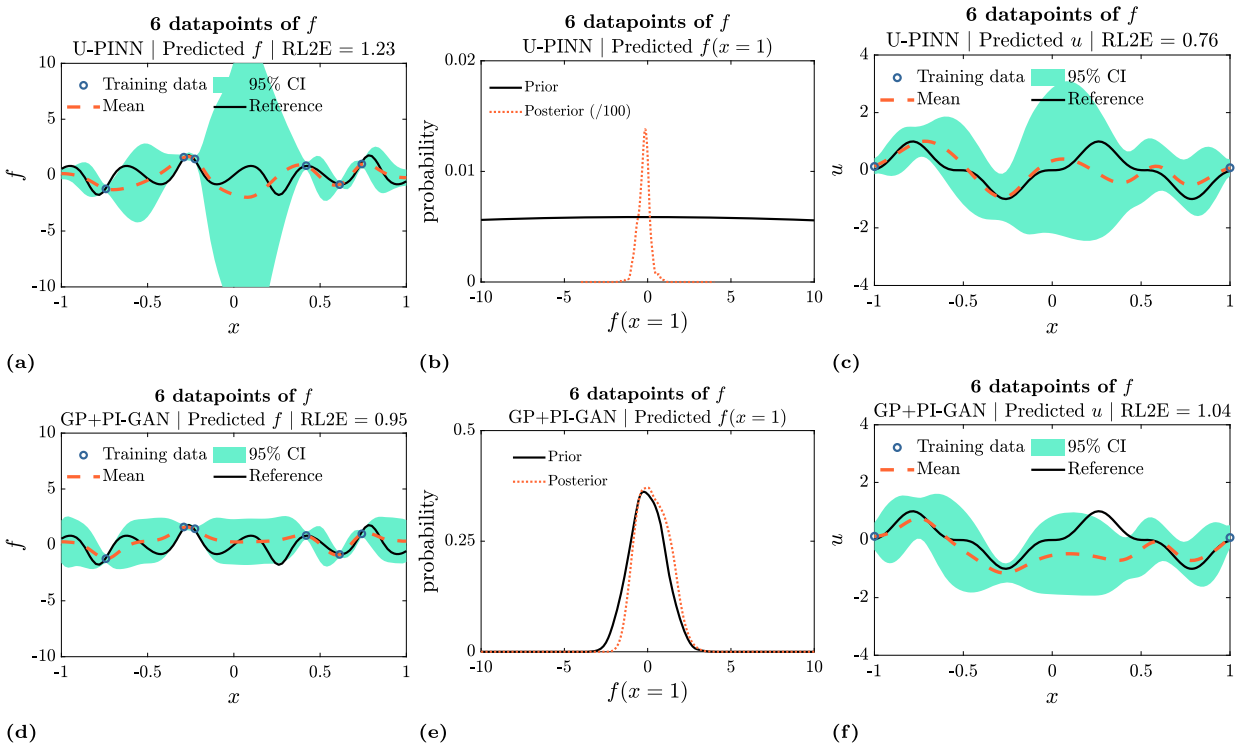


Fig. D.14. Forward PDE problem of Eq. (28) — $N_f = 6$ *datapoints of* f: we compare the U-PINN, which fits f and u simultaneously, with the herein proposed GP+PI-GAN, which first fits f and then u. In contrast to U-PINN, the uncertainty of u is underestimated by GP+PI-GAN, because the approximations of u and f are not coupled through the PDE of Eq. (28). Shown here are the training data, reference functions, as well as the mean and epistemic uncertainty (95% CI) predictions of U-PINN (top) and GP+PI-GAN (bottom) for u and f. The middle panels include the prior and posterior distributions of f evaluated at $x = 1$, i.e., at the right edge of the input domain where there is no available data for f.

Finally, note that the (x, y)-dependent standardizations we applied above limit the approximator λ_θ in the sense that it can only be evaluated on the original (x, y) grid used for training the DeepONet. To avoid this limitation, either the data is not standardized or a shared standardization is used for all points.

D.7. Additional results for the operator learning problem of Section 6.5

In this section, we provide representative results pertaining to OOD inference data in operator learning, as well as to a post-training calibration experiment.

In particular, we generate a sample for λ from the Gaussian process with $l = 0.2$ in Eq. (31). The corresponding reference solution for u is, subsequently, obtained by using the *Matlab PDE toolbox*. Similarly to the ID evaluation performed in Section 6.5, we assume that we have partial noisy measurements of λ and u, and aim to reconstruct the fields for λ and u. Specifically, we randomly select 20 and 10 data from the reference solutions of λ and u, respectively, and then contaminate them with the same Gaussian noise as used in Section 6.5.1. The results obtained by PA-BNN-FP and PA-GAN-FP are presented in Figs. D.15–D.16. It is shown that (1) the RL2E values for the predicted means of λ and u by PA-BNN-FP and PA-GAN-FP are much larger than those in Fig. 30 (ID evaluation), (2) the computational errors for λ and u are not covered within the 95% CI in PA-GAN-FP, and (3) the computational errors are mostly covered within the 95% CI in PA-BNN-FP, which may be attributed to the fact that the employed BNN here has a larger prior space than the GAN. More details on the detection of OOD data in operator learning are presented in Section 6.5.3.

Further, in Fig. D.17 we perform post-training calibration using 2-50 left-out noisy data from λ and u during inference. Clearly, because the training data of λ and u were much less than 50, the objective of this experiment is only to illustrate the trend of decreasing RMSCE with increasing calibration dataset size and compare the three calibration methods. It is observed that post-training calibration even with a few datapoints (2-10) can reduce the calibration error by half in some cases. An interesting future research direction is to perform a cost-effectiveness study for comparing post-training calibration with active learning, i.e., with techniques that re-adjust the mean and uncertainty predictions in light of new data.

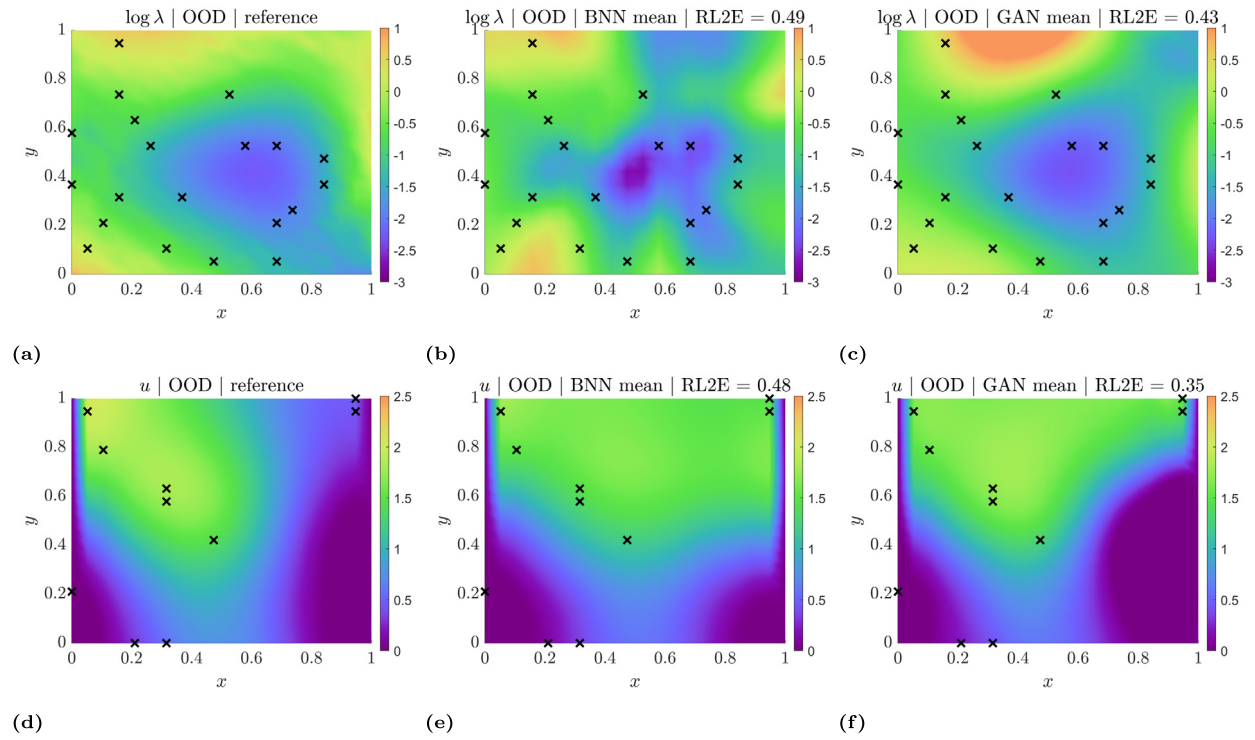


Fig. D.15. Operator learning problem of Eqs. (29)-(30) — *Limited and noisy inference data of λ and u (OOD)*: the performance of both PA-BNN-FP and PA-GAN-FP deteriorates significantly for OOD data with RL2E values close to 40-50%. Nevertheless, PA-BNN-FP is more calibrated than PA-GAN-FP, as indicated by the RMSCE values of Table 12. This shows that PA-GAN-FP that is pre-trained with historical data of λ produces confident predictions even for cases of unseen λ during pre-training. Shown here are the reference input $\tilde{\lambda} = \log\lambda$ and solution u, the corresponding inference data locations (x markers), as well as the mean predictions by PA-BNN-FP and PA-GAN-FP. Top row results correspond to input $\log\lambda$, whereas bottom row to solution u.

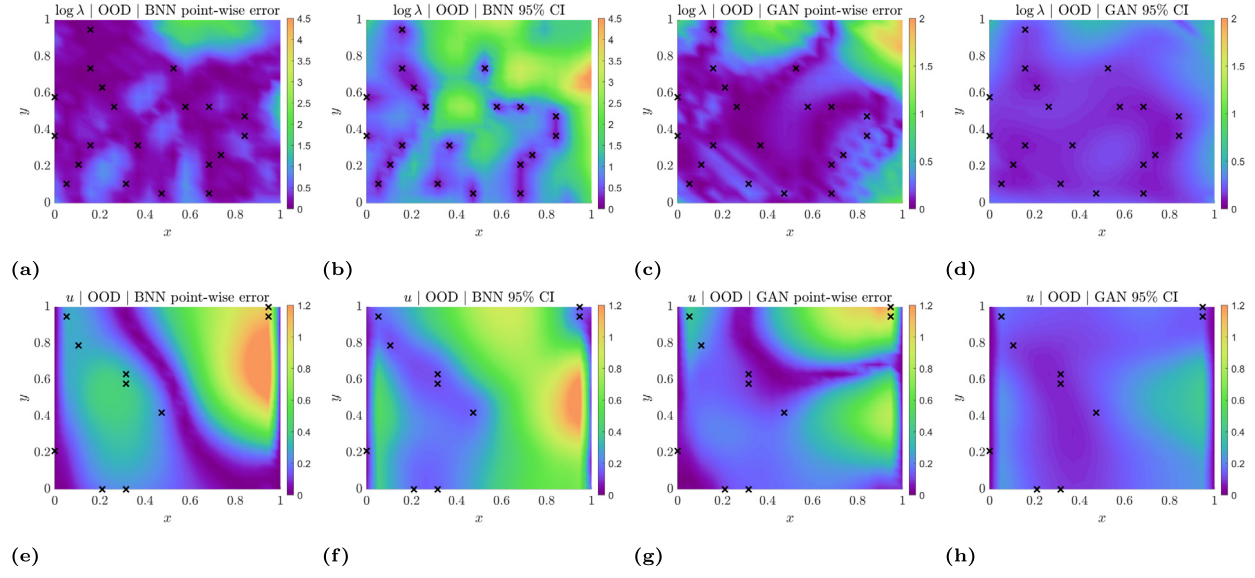


Fig. D.16. Operator learning problem of Eqs. (29)-(30) — *Limited and noisy inference data of λ and u (ID)*: the performance of both PA-BNN-FP and PA-GAN-FP deteriorates significantly for OOD data with RL2E values close to 40-50%. Nevertheless, PA-BNN-FP is more calibrated than PA-GAN-FP, as indicated by the RMSCE values of Table 12. In this figure, we notice that PA-GAN-FP is over-confident and produces small epistemic uncertainties. This shows that PA-GAN-FP that is pre-trained with historical data of λ produces confident predictions even for cases of unseen λ during pre-training. Shown here are the absolute point-wise error and epistemic uncertainty predictions corresponding to input $\tilde{\lambda} = \log\lambda$ (a-d) and solution u (e-f), as obtained by comparing PA-BNN-FP and PA-GAN-FP with reference solutions, as well as the corresponding inference data locations (x markers).

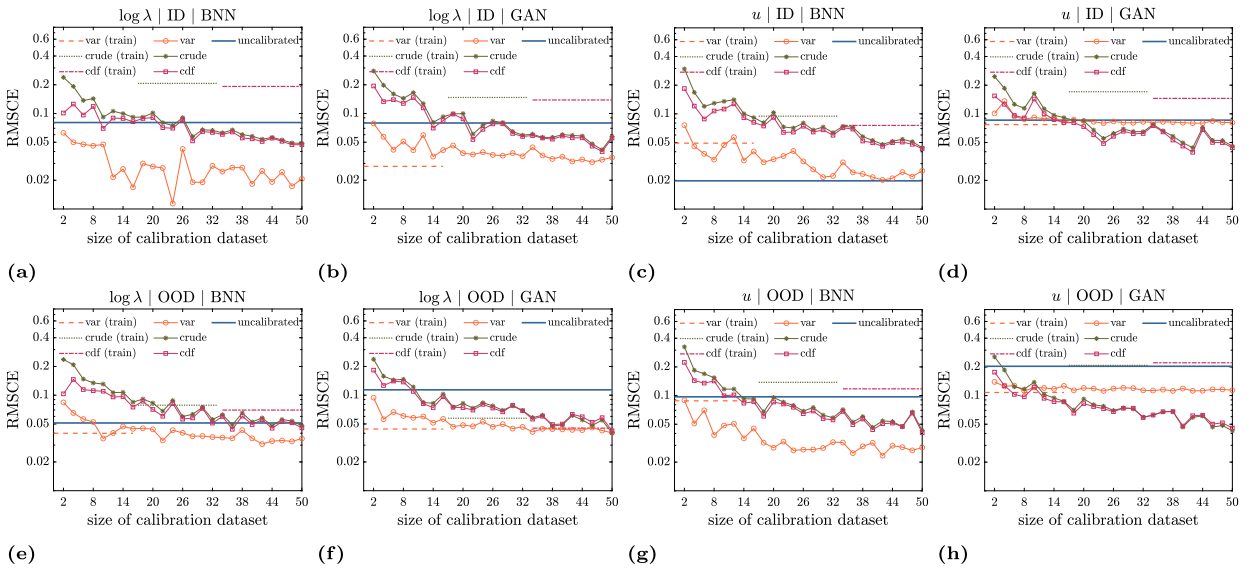


Fig. D.17. Operator learning problem of Eqs. (29)-(30) — *Limited and noisy inference data of λ and u (ID or OOD)*: performing post-training calibration even with a few left-out noisy datapoints (2-10) can reduce the calibration error by half in some cases. This is true especially for OOD data and for using the variance re-weighting technique. Shown here is the effect of calibration approach (Section 5.2) and calibration dataset on RMSCE for $\tilde{\lambda} = \log\lambda$ and u predictions using PA-BNN-FP and PA-GAN-FP. Note, however, that there are cases in which calibration can make the predictions worse (e.g., part c of the figure), because it calibrates "on average", as explained in Section 5.2.

Table D.3
Glossary of hyperparameters used in the comparative study. The hyperparameter ϵ in HMC without Gibbs sampling is the step size at the initial time, which changes at every step for the first $0.8T$ steps, and then is fixed for the remaining steps. For HMC with Gibbs sampling, ϵ is fixed in the whole sampling process. Furthermore, ϵ in the Adam optimizer represents the learning rate.

Glossary of hyperparameters used in the comparative study			
ϵ	learning rate or time step in leapfrog method	T	leapfrog steps
M	number of posterior samples	wd	weight decay constant
init	initialization method	train steps	number of training iterations
ϵ_{init}	maximum learning rate (beginning of cycle)	dr	dropout rate
T_{cycles}	number of cycles	ϵ_{final}	minimum learning rate (end of cycle)
h_1-h_4	conditional Gamma constants	T_{used}	number of last cycles used for sampling θ
T_c	number of discriminator iterations used per generator iteration	M'	stochastic realizations

D.8. Hyperparameters used in the comparative study

In this section, we provide all hyperparameters used in the comparative study. In Table D.3 we provide the glossary for the hyperparameters. Note that the Adam optimizer in *Tensorflow* is employed for optimization by default in the present study. In the Adam optimizer, we set the exponential decay rates for the first (i.e., β_1) and second (i.e., β_2) moment estimates in the training of GANs/PI-GANs as 0.5 and 0.9, respectively. They are set to the default values, i.e., $\beta_1 = 0.9$ and $\beta_2 = 0.999$, in the training of standard NNs. In Tables D.4–D.8, we present the hyperparameters used for obtaining the results of Section 6.1. Specifically, we note that: (1) all results from HMC without Gibbs sampling in this study are implemented using the open-source package *tensorflow probability* [165]. In addition, we employ HMC with adaptive step size to make the computations more robust. For cases using HMC with Gibbs sampling, i.e., Figs. 16 and 19, the step size is fixed throughout the whole HMC and Gibbs sampling process. We manually tune the step size to achieve the desired acceptance rate (around 0.6) in these two cases; and (2) for MFVI, we obtain the optimal training steps for early stopping based on the negative-log-likelihood of the validation data used in each case of Section 6.1. See more details regarding early stopping in Section B.2.2. The hyperparameters used for implementing the methods employed in Sections 6.2.2 and D.3.2-D.3.4 are provided in Tables D.9 and D.10. The hyperparameters used for implementing the methods employed in Section 6.3 are provided in Table D.11. The hyperparameters used for implementing the methods employed in Section 6.4 are provided in Table D.12. The hyperparameters used for implementing the various methods employed in Section 6.5 are provided in Tables D.13 and D.14.

Table D.4
Section 6.1.1: hyperparameters used for implementing each technique. Note that (1) init (mean) and init (std) for the MFVI represent the initializations for the mean and standard deviation of the posterior distribution, respectively, which are different, and (2) the batch size in the MFVI is 32. The specified MFVI initializations and batch size are used throughout the whole study and are not repeated in the rest of the hyperparameter tables for simplicity.

Method	Hyperparameter						
HMC	ϵ	T	burn-in steps	M			
	0.1	50	2×10^3	10^3			
LD	ϵ	burn-in steps	M				
	10^{-4}	5×10^3	10^3				
MFVI	ϵ	train steps	M	init (mean)	init (std)		
	10^{-3}	4,450	10^3	Xavier (normal)	$\log(1+\exp(-6))$		
MCD	ϵ	train steps	M	dr	init		
	10^{-3}	2×10^4	10^3	5×10^{-2}	Xavier (normal)		
LA	ϵ_{MAP}	train steps (MAP)	M	init			
	10^{-2}	3×10^4	10^3	Xavier			
DEns	ϵ	train steps	M	wd	init		
	10^{-4}	2×10^4	10	5×10^{-4}	Xavier (normal)		
SEns	ϵ_{init}	train steps	M	ϵ_{final}	T_{cycles}	T_{used}	init
	10^{-2}	2×10^4	20	10^{-4}	20	20	Xavier (normal)
SWAG	ϵ_{init}	train steps	M	ϵ_{final}	T_{cycles}	T_{used}	init
	10^{-2}	2×10^4	50	10^{-4}	10	5	Xavier (normal)

Table D.5
Dropout rates used in the epistemic uncertainty experiment of Fig. D.4.

(N, σ_u)	(32, 0.1)	(32, 0.3)	(32, 0.5)	(16, 0.1)	(16, 0.3)	(16, 0.5)
Dropout rate	0.05	0.2	0.3	0.1	0.2	0.3

Table D.6
Section 6.1.2: hyperparameters used for implementing each technique. ϵ_{MAP}: learning rate for the MAP training; ϵ_{Prior}: learning rate for the prior/noise scale optimization.

Method	Hyperparameter					
HMC	ϵ	T	burn-in steps	M	(h_1, h_2)	(h_3, h_4)
	2.5×10^{-3}	50	10^3	10^3	(2, 0.25)	(2, 10)
MFVI	ϵ	train steps	M			
	10^{-3}	22,170	10^3			
LA	ϵ_{MAP}	train steps (MAP)	init	ϵ_{Prior}	train steps (Prior)	M
	10^{-2}	3×10^4	Xavier (normal)	10^{-3}	2×10^3	10^4

Table D.7
Section 6.1.3: hyperparameters used for implementing each technique.

Method	Hyperparameter				
h-HMC	ϵ	T	burn-in steps	M	
	0.1	50	2×10^3	10^3	
h-MFVI	ϵ	train steps	M		
	10^{-3}	210	10^3		
h-DEns	ϵ	train steps	M	wd	init
	10^{-3}	2×10^4	10	10^{-2}	Xavier (normal)

Table D.8
Section D.2.2: hyperparameters used for implementing each technique.

Method	Hyperparameter			
h-HMC	ϵ	T	burn-in steps	M
	0.1	50	2×10^3	10^3
h-MFVI	ϵ	train steps	M	
	10^{-3}	2, 150	10^3	

Table D.9
Section 6.2.1: hyperparameters used for implementing each technique.

Method	Hyperparameter					
U-PINN	ϵ	T	burn-in steps	M	(h_1, h_2)	(h_3, h_4)
with HMC	10^{-2}	50	5×10^3	10^3	$(25, 4 \times 10^{-3})$	$(0.5, 40)$
U-PINN	ϵ	train steps	M			
with MFVI	10^{-3}	10^4	10^3			
U-PINN	ϵ	train steps	M	dr	init	
with MCD	10^{-3}	2×10^4	10^3	0.05	Xavier (normal)	
U-PINN	ϵ	train steps	M	wd	init	
with DEns	10^{-3}	5×10^3	10	0.1	Xavier (normal)	
PI-GAN-FP	ϵ	T	burn-in steps	M		
with HMC	0.1	50	2×10^3	10^3		

Table D.10
Sections 6.2.2 and D.3.2-D.3.4: hyperparameters used for implementing each technique.

Method	Hyperparameter			
U-PINN with h-HMC	ϵ	T	burn-in steps	M
in §6.2.2	10^{-3}	30	5×10^{-3}	10^3
U-PINN with HMC	ϵ	T	burn-in steps	M
in §D.3.2 (large noise)	10^-2	50	5×10^3	10^3
U-PINN with HMC	ϵ	T	burn-in steps	M
in §D.3.3 (extrapolation)	10^{-3}	50	5×10^3	10^3
U-PINN with HMC	ϵ	T	burn-in steps	M
in §D.3.4 (steep boundaries)	10^{-3}	50	5×10^3	10^3

Table D.11
Section 6.3: hyperparameters used for implementing each technique.

Method	Hyperparameter					
U-PI-GAN	ϵ	T_c	train steps	M	M'	
with DEns	10^{-3}	5	10^5	10^3		
U-NNPC	ϵ	train steps	M	M'	wd	init
with DEns	2×10^{-3}	10,001	3	500	10^{-3}	Xavier (normal)
U-NNPC+	ϵ	train steps	M	M'	wd	init
with DEns	2×10^{-3}	10,001	3	500	10^{-3}	Xavier (normal)

Table D.12
Section 6.4: hyperparameters used for implementing each technique.

Method	Hyperparameter			
U-PINN	ϵ	T	burn-in steps	M
with HMC	0.1	20	2×10^3	10^3
PI-GAN	ϵ	train steps	M	init
	10^{-4}	10^5	10^3	Xavier (normal)

Table D.13
Section 6.5.1: hyperparameters used for implementing each technique.

Method	Hyperparameter			
PA-BNN-FP	ϵ	T	burn-in steps	M
with HMC	0.1	10	3×10^3	10^3
PA-GAN-FP	ϵ	T	burn-in steps	M
with HMC	0.1	10	5×10^3	10^3

Table D.14
Section 6.5.2: hyperparameters used for implementing each technique.

Method	Hyperparameter			
U-DeepONet	ϵ	train steps	M	init
with DEns	10^{-3}	5×10^4	5	Xavier (normal)

Table D.15
Architecture and training steps of GANs/PI-GANs in Section 6. The width and depth correspond to the hidden layers. K is the dimension of θ, which is the same as the output dimension of each NN in G if we use two NNs for G. Note that (1) in Sections 6.1.2, 6.2.1, and 6.5.1, we employ two NNs for G, which take as inputs the Gaussian noise (the first architecture for G in this table) and coordinates (the second architecture for G in this table), respectively. This is to enhance the computational efficiency in posterior inference (see details in [136]). Further, (2) in Section 6.2.1, the architectures for u and λ are the same.

	Generator G		Discriminator D		K	Training steps
	width × depth	Activation	width × depth	Activation		
Section 6.1.2	$128 \times 2/128 \times 2$	tanh/tanh	128×3	Leaky ReLu	20	100,000
Section 6.2.1	$128 \times 2/128 \times 2$	tanh/tanh	256×3	Leaky ReLu	40	100,000
Section 6.4	128×2	tanh	128×3	Leaky ReLu	40	100,000
Section 6.3	128×2	tanh	128×3	Leaky ReLu	20	100,000
Section 6.5.1	$128 \times 2/128 \times 2$	tanh/tanh	256×3	Leaky ReLu	100	100,000

Table D.16
Architectures for NNPC and NNPC+ in Section 6.3. For each of u and λ, we use a NN for the mean and another one for the modes, e.g., 4×2 and 4×32, respectively; see [138] for more implementation details.

	NN for u		NN for λ		NN for f' (NNPC+ only)		Training steps
	width × depth	Activation	width × depth	Activation	width × depth	Activation	
Section 6.3	4×2 and 4×32	tanh	4×2 and 2×32	tanh	4×2 and 2×32	tanh	10,001

Table D.17
Architecture and training steps of DeepONets in Section 6.5.

	Branch Net		Trunk Net		Training steps
	width × depth	Activation	width × depth	Activation	
Section 6.5.1	128×2	tanh	128×3	tanh	50,000
Section 6.5.2	CNN	ReLu	128×3	tanh	30,000

D.9. Neural network architectures used in the comparative study

In the function approximation example of Section 6.1, we have compared more than ten different UQ methods and all the employed NNs use the same architecture except for GANs for functional priors (h-HMC+FP - details in the end of the section). That is, (1) 2 hidden layers with 50 neurons per layer, and (2) hyperbolic tangent function as the activation function. In addition, the prior distribution for each weight/bias of the BNNs is identical, which is a standard normal distribution.

In the mixed PDE problem of Section 6.2, we have compared six different UQ methods (including h-HMC). In each approach except for PI-GANs for functional priors (PI-GAN-FP - details in the end of the section), we employ two NNs as surrogates for the solution u to Eq. (21a)/(D.1) as well as the reaction rate λ. The NNs have the same architecture, i.e., (1) 2 hidden layers with 50 neurons per layer, and (2) hyperbolic tangent function as the activation function. Similarly to function approximation, the prior distribution for each weight/bias of the BNNs is identical, which is a standard normal distribution. The only difference in these two NNs lies in the input dimension, which for the former is 2 (because of space and time), whereas for the latter is 1 (space only).

In the forward PDE problem of Section 6.4, we use a BNN as the surrogate for the solution u to Eq. (28), which has the same architecture as well as prior distribution as the BNNs employed in Section 6.1. In addition, we also employ the same architecture/priors for the BNNs used in the operator learning problem of Section 6.5. Finally, we present the architectures for the GANs/PI-GANs, NNPC, and neural operators in Tables D.15, D.16, and D.17, respectively. Note that in Sections 6.1.2, 6.2.1, and 6.5.1, we employ two NNs for the generator G, which take as inputs the Gaussian noise (the first architecture for G in Table D.15) and coordinates (the second architecture for G in Table D.15), respectively. This is to enhance the computational efficiency in posterior inference (see details in [136]). In Sections 6.3 and 6.4, we use one NN for the generator G (see details in [111,136]), since we do not compute the posterior in these two cases. In addition, for the heteroscedastic noise case in Section 6.1.2 we have one more NN to approximate the varying noise scale, which takes x as input, and outputs the noise scale at x. For this NN, we use 2 hidden layers with 20 neurons per layer, and the activation function is ReLu.

References

[1] Z. Zou, X. Meng, A.F. Psaros, G.E. Karniadakis, NeuralUQ: a comprehensive library for uncertainty quantification in neural differential equations and operators, arXiv preprint arXiv:2208.11866, 2022.
[2] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, Nat. Rev. Phys. (2021).
[3] M. Alber, A. Buganza Tepole, W.R. Cannon, S. De, S. Dura-Bernal, K. Garikipati, G.E. Karniadakis, W.W. Lytton, P. Perdikaris, L. Petzold, E. Kuhl, Integrating machine learning and multiscale modeling—perspectives, challenges, and opportunities in the biological, biomedical, and behavioral sciences, npj Digit. Med. 2 (2019) 1–11.
[4] J. Willard, X. Jia, S. Xu, M. Steinbach, V. Kumar, Integrating scientific knowledge with machine learning for engineering and environmental systems, arXiv:2003.04919 [physics, stat], 2021.

[5] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, vol. 1, MIT Press, 2016.
[6] C.F. Higham, D.J. Higham, Deep learning: an introduction for applied mathematicians, SIAM Rev. 61 (2019) 860–891.
[7] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nat. Mach. Intell. 3 (2021) 218–229.
[8] S. Goswami, M. Yin, Y. Yu, G.E. Karniadakis, A physics-informed variational DeepONet for predicting the crack path in brittle materials, arXiv preprint arXiv:2108.06905, 2021.
[9] S. Wang, H. Wang, P. Perdikaris, Learning the solution operator of parametric partial differential equations with physics-informed DeepONets, arXiv:2103.10974 [cs, math, stat], 2021.
[10] S. Lanthaler, S. Mishra, G.E. Karniadakis, Error estimates for DeepOnets: a deep learning framework in infinite dimensions, arXiv preprint arXiv:2102.09618, 2021.
[11] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, arXiv:2010.08895 [cs, math], 2021.
[12] J. Darbon, T. Meng, On some neural network architectures that can represent viscosity solutions of certain high dimensional Hamilton–Jacobi partial differential equations, J. Comput. Phys. 425 (2021) 109907.
[13] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[14] I. Lagaris, A. Likas, D. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (1998) 987–1000.
[15] J. Sirignano, K. Spiliopoulos, DGM: a deep learning algorithm for solving partial differential equations, J. Comput. Phys. 375 (2018) 1339–1364.
[16] G. Pang, L. Lu, G.E. Karniadakis, fPINNs: fractional physics-informed neural networks, SIAM J. Sci. Comput. 41 (2019) A2603–A2626.
[17] A.D. Jagtap, G.E. Karniadakis, Extended physics-informed neural networks (XPINNs): a generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, Commun. Comput. Phys. 28 (2020) 2002–2041.
[18] X. Meng, Z. Li, D. Zhang, G.E. Karniadakis, PPINN: parareal physics-informed neural network for time-dependent PDEs, Comput. Methods Appl. Mech. Eng. 370 (2020) 113250.
[19] E. Kharazmi, Z. Zhang, G.E. Karniadakis, Hp-VPINNs: variational physics-informed neural networks with domain decomposition, Comput. Methods Appl. Mech. Eng. 374 (2021) 113547.
[20] Y. Shin, J. Darbon, G.E. Karniadakis, On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs, Commun. Comput. Phys. 28 (2020) 2042–2074, arXiv:2004.01806.
[21] S. Mishra, R. Molinaro, Estimates on the generalization error of physics-informed neural networks for approximating a class of inverse problems for PDEs, IMA J. Numer. Anal. (2021).
[22] X. Meng, G.E. Karniadakis, A composite neural network that learns from multi-fidelity data: application to function approximation and inverse PDE problems, J. Comput. Phys. 401 (2020) 109020.
[23] X. Meng, H. Babaee, G.E. Karniadakis, Multi-fidelity Bayesian neural networks: algorithms and applications, J. Comput. Phys. 438 (2021) 110361.
[24] L. Yang, X. Meng, G.E. Karniadakis, B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data, J. Comput. Phys. 425 (2021) 109913.
[25] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, SIAM Rev. 63 (2021) 208–228.
[26] O. Hennigh, S. Narasimhan, M.A. Nabian, A. Subramaniam, K. Tangsali, Z. Fang, M. Rietmann, W. Byeon, S. Choudhry, NVIDIA SimNet™: an AI-accelerated multi-physics simulation framework, in: International Conference on Computational Science, Springer, 2021, pp. 447–461.
[27] R.M. Neal, Probabilistic Inference Using Markov Chain Monte Carlo Methods, Department of Computer Science, University of Toronto, Toronto, Ontario, Canada, 1993.
[28] D.J. MacKay, Bayesian methods for neural networks: theory and applications, in: Neural Networks Summer School, University of Cambridge, 1995.
[29] R.M. Neal, Bayesian Learning for Neural Networks, Ph.D. thesis, University of Toronto, 1995.
[30] D. Barber, C.M. Bishop, Ensemble learning in Bayesian neural networks, Nato ASI Series F Comput. Syst. Sci. 168 (1998) 215–238.
[31] J. Lampinen, A. Vehtari, Bayesian approach for neural networks—review and case studies, Neural Netw. 14 (2001) 257–274.
[32] A. Gelman, J.B. Carlin, H.S. Stern, D.B. Dunson, A. Vehtari, D.B. Rubin, Bayesian Data Analysis, Chapman and Hall/CRC, 2015.
[33] A. Graves, Practical variational inference for neural networks, in: Advances in Neural Information Processing Systems, 2011, pp. 2348–2356.
[34] R.M. Neal, MCMC using Hamiltonian dynamics, arXiv:1206.1901 [physics, stat], 2012.
[35] M.D. Hoffman, D.M. Blei, C. Wang, J. Paisley, Stochastic variational inference, J. Mach. Learn. Res. 14 (2013) 1303–1347.
[36] Y.-A. Ma, T. Chen, E. Fox, A complete recipe for stochastic gradient MCMC, in: Advances in Neural Information Processing Systems, 2015, pp. 2917–2925.
[37] Y. Gal, Uncertainty in Deep Learning, University of Cambridge, vol. 1, 2016.
[38] A.G. Wilson, The case for Bayesian deep learning, arXiv:2001.10995 [cs, stat], 2020.
[39] A.G. Wilson, P. Izmailov, Bayesian deep learning and a probabilistic perspective of generalization, arXiv:2002.08791, 2020.
[40] D. Krueger, C.-W. Huang, R. Islam, R. Turner, A. Lacoste, A. Courville, Bayesian hypernetworks, arXiv preprint arXiv:1710.04759, 2017.
[41] D.M. Blei, A. Kucukelbir, J.D. McAuliffe, Variational inference: a review for statisticians, J. Am. Stat. Assoc. 112 (2017) 859–877.
[42] P. Izmailov, S. Vikram, M.D. Hoffman, A.G. Wilson, What are Bayesian neural network posteriors really like?, arXiv:2104.14421 [cs, stat], 2021.
[43] B. Lakshminarayanan, A. Pritzel, C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, in: Advances in Neural Information Processing Systems, 2017, pp. 6402–6413.
[44] G. Huang, Y. Li, G. Pleiss, Z. Liu, J.E. Hopcroft, K.Q. Weinberger, Snapshot ensembles: train 1, get M for free, arXiv:1704.00109 [cs], 2017.
[45] S. Mandt, M.D. Hoffman, D.M. Blei, Stochastic gradient descent as approximate Bayesian inference, J. Mach. Learn. Res. 18 (2017) 4873–4907.
[46] T. Garipov, P. Izmailov, D. Podoprikhin, D.P. Vetrov, A.G. Wilson, Loss surfaces, mode connectivity, and fast ensembling of dnns, in: Advances in Neural Information Processing Systems, 2018, pp. 8789–8798.
[47] M. Khan, D. Nielsen, V. Tangkaratt, W. Lin, Y. Gal, A. Srivastava, Fast and scalable Bayesian deep learning by weight-perturbation in Adam, in: International Conference on Machine Learning, PMLR, 2018, pp. 2611–2620.
[48] T. Pearce, A. Brintrup, M. Zaki, A. Neely, High-quality prediction intervals for deep learning: a distribution-free, ensembled approach, in: International Conference on Machine Learning, 2018, pp. 4075–4084.
[49] W.J. Maddox, P. Izmailov, T. Garipov, D.P. Vetrov, A.G. Wilson, A simple baseline for Bayesian uncertainty in deep learning, in: Advances in Neural Information Processing Systems, 2019, pp. 13153–13164.
[50] S. Fort, H. Hu, B. Lakshminarayanan, Deep ensembles: a loss landscape perspective, arXiv:1912.02757 [cs, stat], 2020.
[51] B. He, B. Lakshminarayanan, Y.W. Teh, Bayesian deep ensembles via the neural tangent kernel, arXiv preprint arXiv:2007.05864, 2020.
[52] G. Franzese, R. Candela, D. Milios, M. Filippone, P. Michiardi, Isotropic SGD: a practical approach to Bayesian posterior sampling, arXiv:2006.05087 [cs, stat], 2020.
[53] S. Kessler, A. Salas, V.W.C. Tan, S. Zohren, S. Roberts, Practical Bayesian learning of neural networks via adaptive optimisation methods, arXiv:1811.03679 [cs, stat], 2020.

[54] R. Rahaman, A.H. Thiery, Uncertainty quantification and deep ensembles, in: Advances in Neural Information Processing Systems, 2021.
[55] I. Osband, J. Aslanides, A. Cassirer, Randomized prior functions for deep reinforcement learning, Adv. Neural Inf. Process. Syst. 31 (2018).
[56] K. Ciosek, V. Fortuin, R. Tomioka, K. Hofmann, R. Turner, Conservative uncertainty estimation by fitting prior networks, in: International Conference on Learning Representations, 2019.
[57] A. Malinin, M. Gales, Predictive uncertainty estimation via prior networks, arXiv:1802.10501 [cs, stat], 2018.
[58] M. Sensoy, L. Kaplan, M. Kandemir, Evidential Deep Learning to Quantify Classification Uncertainty, Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc., 2018.
[59] A. Amini, W. Schwarting, A. Soleimany, D. Rus, Deep evidential regression, arXiv:1910.02600 [cs, stat], 2020.
[60] B. Charpentier, D. Zügner, S. Günnemann, Posterior network: uncertainty estimation without OOD samples via density-based pseudo-counts, arXiv preprint arXiv:2006.09239, 2020.
[61] A.-K. Kopetzki, B. Charpentier, D. Zügner, S. Giri, S. Günnemann, Evaluating robustness of predictive uncertainty estimation: are Dirichlet-based models reliable?, arXiv:2010.14986 [cs, stat].
[62] A. Malinin, S. Chervontsev, I. Provilkov, M. Gales, Regression prior networks, arXiv:2006.11590 [cs, stat], 2020.
[63] B. Charpentier, O. Borchert, D. Zügner, S. Geisler, S. Günnemann, Natural posterior network: deep Bayesian predictive uncertainty for exponential family distributions, arXiv:2105.04471 [cs, stat], 2021.
[64] N. Meinert, A. Lavin, Multivariate deep evidential regression, arXiv:2104.06135 [cs, stat], 2021.
[65] D. Ulmer, A survey on evidential deep learning for single-pass uncertainty estimation, arXiv preprint arXiv:2110.03051, 2021.
[66] A. Damianou, N. Lawrence, Deep Gaussian processes, in: Artificial Intelligence and Statistics, 2013, pp. 207–215.
[67] M. Raissi, G.E. Karniadakis, Deep multi-fidelity Gaussian processes, arXiv preprint arXiv:1604.07484, 2016.
[68] J. Lee, Y. Bahri, R. Novak, S.S. Schoenholz, J. Pennington, J. Sohl-Dickstein, Deep neural networks as Gaussian processes, in: International Conference on Learning Representations, 2018, p. 17.
[69] A.G.d.G. Matthews, M. Rowland, J. Hron, R.E. Turner, Z. Ghahramani, Gaussian process behaviour in wide deep neural networks, arXiv:1804.11271 [cs, stat], 2018.
[70] B. Adlam, J. Lee, L. Xiao, J. Pennington, J. Snoek, Exploring the uncertainty properties of neural networks' implicit priors in the infinite-width limit, arXiv preprint arXiv:2010.07355, 2020.
[71] K. Jakkala, Deep Gaussian processes: a survey, arXiv preprint arXiv:2106.12135, 2021.
[72] L. Aitchison, A. Yang, S.W. Ober, Deep kernel processes, in: Proceedings of the 38th International Conference on Machine Learning, PMLR, 2021, pp. 130–140.
[73] A.N. Angelopoulos, S. Bates, A gentle introduction to conformal prediction and distribution-free uncertainty quantification, arXiv:2107.07511 [cs, math, stat], 2021.
[74] N. Dewolf, B. De Baets, W. Waegeman, Valid prediction intervals for regression problems, 2021.
[75] N.G. Polson, V. Sokolov, Deep learning: a Bayesian perspective, Bayesian Anal. 12 (2017) 1275–1304.
[76] H.D. Kabir, A. Khosravi, M.A. Hosen, S. Nahavandi, Neural network-based uncertainty quantification: a survey of methodologies and applications, IEEE Access 6 (2018) 36218–36234.
[77] J. Caldeira, B. Nord, Deeply uncertain: comparing methods of uncertainty quantification in deep learning algorithms, Mach. Learn.: Sci. Technol. 2 (2020) 015002, arXiv:2004.10710.
[78] E. Goan, C. Fookes, Bayesian neural networks: an introduction and survey, in: Case Studies in Applied Bayesian Data Science, Springer, 2020, pp. 45–87.
[79] A. Loquercio, M. Segu, D. Scaramuzza, A general framework for uncertainty estimation in deep learning, IEEE Robot. Autom. Lett. 5 (2020) 3153–3160.
[80] N. Ståhl, G. Falkman, A. Karlsson, G. Mathiason, Evaluation of uncertainty quantification in deep learning, in: International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems, Springer, 2020, pp. 556–568.
[81] H. Wang, D.-Y. Yeung, A survey on Bayesian deep learning, arXiv:1604.01662 [cs, stat], 2020.
[82] J. Gawlikowski, C.R.N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, A survey of uncertainty in deep neural networks, arXiv preprint arXiv:2107.03342, 2021.
[83] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U.R. Acharya, V. Makarenkov, S. Nahavandi, A review of uncertainty quantification in deep learning: techniques, applications and challenges, arXiv:2011.06225 [cs], 2021.
[84] E. Hüllermeier, W. Waegeman, Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods, Mach. Learn. 110 (2021) 457–506.
[85] Z. Nado, N. Band, M. Collier, J. Djolonga, M.W. Dusenberry, S. Farquhar, A. Filos, M. Havasi, R. Jenatton, G. Jerfel, Uncertainty baselines: benchmarks for uncertainty & robustness in deep learning, arXiv preprint arXiv:2106.04015, 2021.
[86] X. Zhou, H. Liu, F. Pourpanah, T. Zengd, X. Wang, A survey on epistemic (model) uncertainty in supervised learning: recent advances and applications, arXiv preprint arXiv:2111.01968, 2021.
[87] L.V. Jospin, H. Laga, F. Boussaid, W. Buntine, M. Bennamoun, Hands-on Bayesian neural networks—a tutorial for deep learning users, IEEE Comput. Intell. Mag. 17 (2022) 29–48.
[88] A. Masegosa, Learning under model misspecification: applications to variational and ensemble methods, Adv. Neural Inf. Process. Syst. 33 (2020) 5479–5491.
[89] E. Pickering, G.E. Karniadakis, T.P. Sapsis, Discovering and forecasting extreme events via active learning in neural operators, arXiv:2204.02488 [nlin, physics:physics, stat], 2022.
[90] V. Fortuin, Priors in Bayesian deep learning: a review, Int. Stat. Rev. (2022).
[91] C. Nemeth, P. Fearnhead, Stochastic gradient Markov chain Monte Carlo, J. Am. Stat. Assoc. 116 (2021) 433–450.
[92] J. Yao, W. Pan, S. Ghosh, F. Doshi-Velez, Quality of uncertainty quantification for Bayesian neural network inference, arXiv:1906.09686 [cs, stat], 2019.
[93] I. Osband, Z. Wen, S.M. Asghari, V. Dwaracherla, X. Lu, B. Van Roy, Evaluating high-order predictive distributions in deep learning, arXiv preprint arXiv:2202.13509, 2022.
[94] Z. Wen, I. Osband, C. Qin, X. Lu, M. Ibrahimi, V. Dwaracherla, M. Asghari, B. Van Roy, From predictions to decisions: the importance of joint predictive distributions, arXiv:2107.09224, 2022.
[95] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, J. Snoek, Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift, in: Advances in Neural Information Processing Systems, 2019, pp. 13991–14002.
[96] O. Wiles, S. Gowal, F. Stimberg, S. Alvise-Rebuffi, I. Ktena, K. Dvijotham, T. Cemgil, A fine-grained analysis on distribution shift, arXiv:2110.11328, 2021.
[97] Y.-W. Luo, C.-X. Ren, Generalized label shift correction via minimum uncertainty principle: theory and algorithm, arXiv:2202.13043 [cs], 2022.
[98] R.K. Tripathy, I. Bilionis Deep UQ, Learning deep neural network surrogate models for high dimensional uncertainty quantification, J. Comput. Phys. 375 (2018) 565–588.
[99] C. Grigo, P.-S. Koutsourelakis, A physics-aware, probabilistic machine learning framework for coarse-graining high-dimensional systems in the small data regime, J. Comput. Phys. 397 (2019) 108842.
[100] Y. Yang, P. Perdikaris, Conditional deep surrogate models for stochastic, high-dimensional, and multi-fidelity systems, Comput. Mech. 64 (2019) 417–434.

[101] A. Olivier, M.D. Shields, L. Graham-Brady, Bayesian neural networks for uncertainty quantification in data-driven materials modeling, Comput. Methods Appl. Mech. Eng. 386 (2021) 114079.
[102] D.V. Patel, A.A. Oberai, GAN-based priors for quantifying uncertainty in supervised learning, SIAM/ASA J. Uncertain. Quantificat. 9 (2021) 1314–1343.
[103] K. Xu, E. Darve, Solving inverse problems in stochastic models using deep neural networks and adversarial training, Comput. Methods Appl. Mech. Eng. 384 (2021) 113976.
[104] J. O'Leary, J.A. Paulson, A. Mesbah, Stochastic physics-informed neural networks (SPINN): a moment-matching framework for learning hidden physics within stochastic differential equations, arXiv:2109.01621 [cs], 2021.
[105] K. Yeo, D.E.C. Grullon, F.-K. Sun, D.S. Boning, J.R. Kalagnanam, Variational inference formulation for a model-free simulation of a dynamical system with unknown parameters by a recurrent neural network, SIAM J. Sci. Comput. 43 (2021) A1305–A1335.
[106] L. Yang, S. Treichler, T. Kurth, K. Fischer, D. Barajas-Solano, J. Romero, V. Churavy, A. Tartakovsky, M. Houston, M. Prabhat, G.E. Karniadakis Highly-scalable, Physics-informed GANs for learning solutions of stochastic PDEs, in: 2019 IEEE/ACM Third Workshop on Deep Learning on Supercomputers (DLS), 2019, pp. 1–11.
[107] S. Atkinson, Bayesian hidden physics models: uncertainty quantification for discovery of nonlinear partial differential operators from data, arXiv: 2006.04228 [cs, stat], 2020.
[108] N. Geneva, N. Zabaras, Multi-fidelity generative deep learning turbulent flows, Found. Data Sci. 2 (2020) 391–428, arXiv:2006.04731.
[109] D.Z. Huang, K. Xu, C. Farhat, E. Darve, Learning constitutive relations from indirect observations using deep neural networks, J. Comput. Phys. 416 (2020) 109491.
[110] S. Karumuri, R. Tripathy, I. Bilionis, J. Panchal, Simulator-free solution of high-dimensional stochastic elliptic partial differential equations using deep neural networks, J. Comput. Phys. 404 (2020) 109120.
[111] L. Yang, D. Zhang, G.E. Karniadakis, Physics-informed generative adversarial networks for stochastic differential equations, SIAM J. Sci. Comput. 42 (2020) A292–A317.
[112] Y. Yang, M. Aziz Bhouri, P. Perdikaris, Bayesian differential programming for robust systems identification under uncertainty, Proc. R. Soc. A, Math. Phys. Eng. Sci. 476 (2020) 20200290.
[113] C. Bajaj, L. McLennan, T. Andeen, A. Roy, Robust learning of physics informed neural networks, arXiv:2110.13330 [cs, stat], 2021.
[114] A. Daw, M. Maruf, A. Karpatne PID-GAN, A GAN framework based on a physics-informed discriminator for uncertainty quantification with physics, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021, pp. 237–247, arXiv:2106.02993.
[115] J.N. Fuhg, I. Kalogeris, A. Fau, N. Bouklas, Interval and fuzzy physics-informed neural networks for uncertain fields, arXiv:2106.13727 [cs, math], 2021.
[116] Y. Gao, M.K. Ng, Wasserstein generative adversarial uncertainty quantification in physics-informed neural networks, arXiv:2108.13054 [cs, math], 2021.
[117] G. Lin, C. Moya, Z. Zhang, Accelerated replica exchange stochastic gradient Langevin diffusion enhanced Bayesian DeepONet for solving noisy parametric PDEs, arXiv preprint arXiv:2111.02484, 2021.
[118] J.P. Molnar, S.J. Grauer, Flow field tomography with uncertainty quantification using a Bayesian physics-informed neural network, arXiv:2108.09247 [physics], 2021.
[119] N. Wang, H. Chang, D. Zhang, Efficient uncertainty quantification for dynamic subsurface flow with surrogate by theory-guided neural network, Comput. Methods Appl. Mech. Eng. 373 (2021) 113492.
[120] P. Tsilifis, S. Ghosh, Inverse design under uncertainty using conditional normalizing flows, in: AIAA SCITECH 2022 Forum, AIAA SciTech Forum, American Institute of Aeronautics and Astronautics, 2021.
[121] L. Guo, H. Wu, T. Zhou, Normalizing field flows: solving forward and inverse stochastic differential equations using physics-informed flow models, arXiv:2108.12956 [cs, math], 2021.
[122] E.J. Hall, S. Taverniers, M.A. Katsoulakis, D.M. Tartakovsky GINNs, Graph-informed neural networks for multiscale physics, J. Comput. Phys. 433 (2021) 110192, arXiv:2006.14807.
[123] R.G. Ghanem, P.D. Spanos, Stochastic Finite Elements: A Spectral Approach, Dover Publications, 2003.
[124] D. Xiu, Numerical Methods for Stochastic Computations, Princeton University Press, 2010.
[125] D. Xiu, Stochastic collocation methods: a survey, in: Handbook of Uncertainty Quantification, 2015, pp. 1–18.
[126] S. Marelli, B. Sudret, UQLab: A Framework for Uncertainty Quantification in Matlab, American Society of Civil Engineers, 2014.
[127] A. Olivier, D.G. Giovanis, B. Aakash, M. Chauhan, L. Vandanapu, M.D. Shields, UQpy: a general purpose python package and development environment for uncertainty quantification, J. Comput. Sci. 47 (2020) 101204.
[128] B. Adams, W. Bohnhoff, K. Dalbey, M. Ebeida, J. Eddy, M. Eldred, R. Hooper, P. Hough, K. Hu, J. Jakeman Dakota, A Multilevel Parallel Object-Oriented Framework for Design Optimization, Parameter Estimation, Uncertainty Quantification, and Sensitivity Analysis: Version 6.13 User's Manual, Technical Report, Sandia National Lab. (SNL-NM), Albuquerque, NM (United States), 2020.
[129] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. 43 (2021) A3055–A3081.
[130] L. McClenny, U. Braga-Neto, Self-adaptive physics-informed neural networks using a soft attention mechanism, arXiv preprint arXiv:2009.04544, 2020.
[131] A.F. Psaros, K. Kawaguchi, G.E. Karniadakis, Meta-learning PINN loss functions, arXiv preprint arXiv:2107.05544, 2021.
[132] B. Stellato, B.P. Van Parys, P.J. Goulart, Multivariate Chebyshev inequality with estimated mean and variance, Am. Stat. 71 (2017) 123–127.
[133] R. Tanno, D.E. Worrall, E. Kaden, A. Ghosh, F. Grussu, A. Bizzi, S.N. Sotiropoulos, A. Criminisi, D.C. Alexander, Uncertainty modelling in deep learning for safer neuroimage enhancement: demonstration in diffusion MRI, NeuroImage 225 (2021) 117366.
[134] W. Wright, Neural network regression with input uncertainty, in: Neural Networks for Signal Processing VIII, in: Proceedings of the 1998 IEEE Signal Processing Society Workshop (Cat. No. 98TH8378), 1998, pp. 284–293.
[135] Y. Yang, G. Kissas, P. Perdikaris, Scalable uncertainty quantification for deep operator networks using randomized priors, arXiv:2203.03048 [cs, stat], 2022.
[136] X. Meng, L. Yang, Z. Mao, J.d.A. Ferrandis, G.E. Karniadakis, Learning functional priors and posteriors from data and physics, arXiv preprint arXiv: 2106.05863, 2021.
[137] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, in: Advances in Neural Information Processing Systems, 2014, pp. 2672–2680.
[138] D. Zhang, L. Lu, L. Guo, G.E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, J. Comput. Phys. 397 (2019) 108850.
[139] D. Zhang, L. Guo, G.E. Karniadakis, Learning in modal space: solving time-dependent stochastic PDEs using physics-informed neural networks, SIAM J. Sci. Comput. 42 (2020) A639–A665.
[140] A.G. Wilson, P. Izmailov, M.D. Hoffman, Y. Gal, Y. Li, M.F. Pradier, S. Vikram, A. Foong, S. Lotfi, S. Farquhar, Evaluating approximate inference in Bayesian deep learning, in: NeurIPS 2021 Competitions and Demonstrations Track, PMLR, 2022, pp. 113–124.
[141] I. Osband, Z. Wen, S.M. Asghari, V. Dwaracherla, B. Hao, M. Ibrahimi, D. Lawson, X. Lu, B. O'Donoghue, B. Van Roy, The neural testbed: evaluating joint predictions, arXiv:2110.04629, 2022.
[142] A. Ashukha, A. Lyzhov, D. Molchanov, D. Vetrov, Pitfalls of in-domain uncertainty estimation and ensembling in deep learning, arXiv:2002.06470 [cs, stat], 2021.

[143] Y. Chung, I. Char, H. Guo, J. Schneider, W. Neiswanger, Uncertainty toolbox: an open-source library for assessing, visualizing, and improving uncertainty quantification, arXiv:2109.10254 [cs, stat], 2021.
[144] F. Schmähling, J. Martin, C. Elster, A framework for benchmarking uncertainty in deep regression, Appl. Intell. (2022) 1–14.
[145] S. Rudy, T. Sapsis, Output-weighted and relative entropy loss functions for deep learning precursors of extreme events, arXiv preprint arXiv:2112.00825, 2021.
[146] E. Pickering, T.P. Sapsis, Structure and distribution metric for quantifying the quality of uncertainty: assessing Gaussian processes, deep neural nets, and deep neural operators for regression, arXiv:2203.04515 [cs, stat], 2022.
[147] T. Gneiting, F. Balabdaoui, A.E. Raftery, Probabilistic forecasts, calibration and sharpness, J. R. Stat. Soc., Ser. B, Stat. Methodol. 69 (2007) 243–268.
[148] V. Kuleshov, N. Fenner, S. Ermon, Accurate uncertainties for deep learning using calibrated regression, arXiv preprint arXiv:1807.00263, 2018.
[149] D. Levi, L. Gispan, N. Giladi, E. Fetaya, Evaluating and calibrating uncertainty prediction in regression tasks, arXiv preprint arXiv:1905.11659, 2019.
[150] H. Song, T. Diethe, M. Kull, P. Flach, Distribution calibration for regression, arXiv preprint arXiv:1905.06023, 2019.
[151] P. Cui, W. Hu, J. Zhu, Calibrated reliable regression using maximum mean discrepancy, arXiv preprint arXiv:2006.10255, 2020.
[152] E. Zelikman, C. Healy, S. Zhou, A. Avati, CRUDE: calibrating regression uncertainty distributions empirically, arXiv:2005.12496 [cs, stat], 2020.
[153] S. Zhao, T. Ma, S. Ermon, Individual calibration with randomized forecasting, arXiv preprint arXiv:2006.10288, 2020.
[154] D.J. MacKay, Gaussian processes-a replacement for supervised neural networks?, 1997.
[155] E.T. Nalisnick, On Priors for Bayesian Neural Networks, Ph.D. thesis, UC Irvine, 2018.
[156] G. Pang, G.E. Karniadakis, Physics-informed learning machines for partial differential equations: Gaussian processes versus neural networks, in: Emerging Frontiers in Nonlinear Science, Springer, 2020, pp. 323–343.
[157] S.-K. Au, Y. Wang, Engineering Risk Assessment with Subset Simulation, John Wiley & Sons, 2014.
[158] J. Quinonero-Candela, M. Sugiyama, A. Schwaighofer, N.D. Lawrence, Dataset Shift in Machine Learning, MIT Press, 2008.
[159] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, B.B. Gupta, X. Chen, X. Wang, A survey of deep active learning, ACM Comput. Surv. (CSUR) 54 (2021) 1–40.
[160] S. Liang, Y. Li, R. Srikant, Enhancing the reliability of out-of-distribution image detection in neural networks, arXiv:1706.02690 [cs, stat], 2020.
[161] J. Martin, C. Elster, Inspecting adversarial examples using the Fisher information, Neurocomputing 382 (2020) 80–86.
[162] J. Martin, C. Elster, Detecting unusual input to neural networks, Appl. Intell. 51 (2021) 2198–2209.
[163] R. Wang, Z. Chen, H. Dong, Q. Xuan, You can't fool all the models: detect adversarial samples via pruning models, IEEE Access 9 (2021) 163780–163790.
[164] D. Hendrycks, K. Gimpel, A baseline for detecting misclassified and out-of-distribution examples in neural networks, arXiv:1610.02136 [cs], 2018.
[165] J. Lao, C. Suter, I. Langmore, C. Chimisov, A. Saxena, P. Sountsov, D. Moore, R.A. Saurous, M.D. Hoffman, J.V. Dillon, Tfp.mcmc: modern Markov chain Monte Carlo tools built for modern hardware, arXiv:2002.01184 [cs, stat], 2020.
[166] E. Daxberger, A. Kristiadi, A. Immer, R. Eschenhagen, M. Bauer, P. Hennig, Laplace Redux–Effortless Bayesian deep learning, arXiv preprint arXiv:2106.14806, 2021.
[167] A.A. Alemi, I. Fischer, J.V. Dillon, K. Murphy, Deep variational information bottleneck, arXiv preprint arXiv:1612.00410, 2016.
[168] A.A. Alemi, I. Fischer, J.V. Dillon, Uncertainty in the variational information bottleneck, arXiv preprint arXiv:1807.00906, 2018.
[169] Z. Goldfeld, Y. Polyanskiy, The information bottleneck problem and its applications in machine learning, IEEE J. Sel. Areas Inf. Theory 1 (2020) 19–38.
[170] R. Zhao, Q. Ji, An empirical evaluation of Bayesian inference methods for Bayesian neural networks, in: NIPS Bayesian Deep Learning (BDL) Workshop, 2018.
[171] S. Ruder, An overview of gradient descent optimization algorithms, arXiv:1609.04747 [cs], 2017.
[172] S. Ghosh, J. Yao, F. Doshi-Velez, Model selection in Bayesian neural networks via horseshoe priors, J. Mach. Learn. Res. 20 (2019) 1–46.
[173] V. Fortuin, A. Garriga-Alonso, F. Wenzel, G. Rätsch, R. Turner, M. van der Wilk, L. Aitchison, Bayesian neural network priors revisited, arXiv:2102.06571 [cs, stat], 2021.
[174] P. Germain, F. Bach, A. Lacoste, S. Lacoste-Julien, PAC-Bayesian theory meets Bayesian inference, arXiv:1605.08636 [cs, stat], 2017.
[175] F. Wenzel, K. Roth, B.S. Veeling, J. Swiatkowski, L. Tran, S. Mandt, J. Snoek, T. Salimans, R. Jenatton, S. Nowozin, How good is the Bayes posterior in deep neural networks really?, arXiv preprint arXiv:2002.02405, 2020.
[176] P. Grünwald, T. van Ommen, Inconsistency of Bayesian inference for misspecified linear models, and a proposal for repairing it, Bayesian Anal. 12 (2017) 1069–1103.
[177] B. Leimkuhler, C. Matthews, T. Vlaar, Partitioned integrators for thermodynamic parameterization of neural networks, arXiv:1908.11843 [cs, stat], 2020.
[178] R. Zhang, C. Li, J. Zhang, C. Chen, A.G. Wilson, Cyclical stochastic gradient MCMC for Bayesian deep learning, arXiv preprint arXiv:1902.03932, 2019.
[179] S. Kapoor, W.J. Maddox, P. Izmailov, A.G. Wilson, On uncertainty, tempering, and data augmentation in Bayesian classification, arXiv preprint arXiv:2203.16481, 2022.
[180] C.E. Rasmussen, Gaussian processes in machine learning, in: Summer School on Machine Learning, Springer, 2003, pp. 63–71.
[181] D.M. Blei Build, Compute, critique, repeat: data analysis with latent variable models, Annu. Rev. Stat. Appl. 1 (2014) 203–232.
[182] M. Betancourt, A conceptual introduction to Hamiltonian Monte Carlo, arXiv preprint arXiv:1701.02434, 2017.
[183] T. Xifara, C. Sherlock, S. Livingstone, S. Byrne, M. Girolami, Langevin diffusions and the Metropolis-adjusted Langevin algorithm, Stat. Probab. Lett. 91 (2014) 14–19.
[184] M. Welling, Y.W. Teh, Bayesian learning via stochastic gradient Langevin dynamics, in: Proceedings of the 28th International Conference on Machine Learning (ICML-11), 2011, pp. 681–688.
[185] T. Chen, E. Fox, C. Guestrin, Stochastic gradient Hamiltonian Monte Carlo, in: International Conference on Machine Learning, 2014, pp. 1683–1691.
[186] Y. Chen, J. Chen, J. Dong, J. Peng, Z. Wang, Accelerating nonconvex learning via replica exchange Langevin diffusion, arXiv:2007.01990 [cs, math, stat], 2020.
[187] W. Deng, Q. Feng, L. Gao, F. Liang, G. Lin, Non-convex learning via replica exchange stochastic gradient MCMC, in: Proceedings of the 37th International Conference on Machine Learning, PMLR, 2020, pp. 2474–2483.
[188] W. Deng, Q. Feng, G. Karagiannis, G. Lin, F. Liang, Accelerating convergence of replica exchange stochastic gradient MCMC via variance reduction, arXiv:2010.01084 [cs, math, stat], 2021.
[189] S. Ahn, A. Korattikara, M. Welling, Bayesian posterior sampling via stochastic gradient Fisher scoring, arXiv:1206.6380 [cs, stat], 2012.
[190] C. Li, C. Chen, D. Carlson, L. Carin, Preconditioned stochastic gradient Langevin dynamics for deep neural networks, arXiv:1512.07666 [stat], 2015.
[191] S. Kim, Q. Song, F. Liang, Stochastic gradient Langevin dynamics algorithms with adaptive drifts, arXiv:2009.09535 [cs, stat], 2020.
[192] B. Delyon, M. Lavielle, E. Moulines, Convergence of a stochastic approximation version of the EM algorithm, Ann. Stat. 27 (1999) 94–128.
[193] A.E. Gelfand, S.E. Hills, A. Racine-Poon, A.F.M. Smith, Illustration of Bayesian inference in normal data models using Gibbs sampling, J. Am. Stat. Assoc. 85 (1990) 972–985.
[194] D.P. Kingma, M. Welling, An introduction to variational autoencoders, Found. Trends Mach. Learn. 12 (2019) 307–392, arXiv:1906.02691.
[195] A. Foong, D. Burt, Y. Li, R. Turner, On the expressiveness of approximate inference in Bayesian neural networks, Adv. Neural Inf. Process. Syst. 33 (2020) 15897–15908.
[196] C. Blundell, J. Cornebise, K. Kavukcuoglu, D. Wierstra, Weight uncertainty in neural network, in: International Conference on Machine Learning, PMLR, 2015, pp. 1613–1622.

[197] D.P. Kingma, T. Salimans, M. Welling, Variational dropout and the local reparameterization trick, in: Advances in Neural Information Processing Systems, 2015, pp. 2575–2583.
[198] C. Louizos, M. Welling, Multiplicative normalizing flows for variational Bayesian neural networks, in: International Conference on Machine Learning, PMLR, 2017, pp. 2218–2227.
[199] L. Mescheder, S. Nowozin, A. Geiger, Adversarial variational Bayes: unifying variational autoencoders and generative adversarial networks, arXiv: 1701.04722 [cs], 2018.
[200] J. Zhao, X. Liu, S. He, S. Sun, Probabilistic inference of Bayesian neural networks with generalized expectation propagation, Neurocomputing 412 (2020) 392–398.
[201] J. Bai, Q. Song, G. Cheng, Efficient variational inference for sparse deep learning with theoretical guarantee, arXiv preprint arXiv:2011.07439, 2020.
[202] R. Zhang, Y. Li, C. De Sa, S. Devlin, C. Zhang, Meta-learning divergences for variational inference, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2021, pp. 4024–4032.
[203] A. Unlu, L. Aitchison, Variational Laplace for Bayesian neural networks, arXiv preprint arXiv:2103.00222, 2021.
[204] D.P. Kingma, M. Welling, Auto-encoding variational Bayes, arXiv:1312.6114 [cs, stat], 2014.
[205] K. Sohn, H. Lee, X. Yan, Learning Structured Output Representation Using Deep Conditional Generative Models, Advances in Neural Information Processing Systems, vol. 28, Curran Associates, Inc., 2015.
[206] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, J. Mach. Learn. Res. 15 (2014) 1929–1958.
[207] S. Boluki, R. Ardywibowo, S.Z. Dadaneh, M. Zhou, X. Qian, Learnable Bernoulli dropout for Bayesian deep learning, in: Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, PMLR, 2020, pp. 3905–3916.
[208] J.S. Denker, Y. LeCun, Transforming neural-net output levels to probability distributions, in: Advances in Neural Information Processing Systems, 1991, pp. 853–859.
[209] D.J. MacKay, Probable networks and plausible predictions—a review of practical Bayesian methods for supervised neural networks, Netw. Comput. Neural Syst. 6 (1995) 469–505.
[210] A. Immer, M. Bauer, V. Fortuin, G. Rätsch, M.E. Khan, Scalable marginal likelihood estimation for model selection in deep learning, arXiv preprint arXiv:2104.04975, 2021.
[211] J. Martens, New insights and perspectives on the natural gradient method, arXiv preprint arXiv:1412.1193, 2014.
[212] H. Ritter, A. Botev, D. Barber, A scalable Laplace approximation for neural networks, in: 6th International Conference on Learning Representations, ICLR 2018-Conference Track Proceedings, in: International Conference on Representation Learning, vol. 6, 2018.
[213] H. Ritter, A. Botev, D. Barber, Online structured Laplace approximations for overcoming catastrophic forgetting, arXiv preprint arXiv:1805.07810, 2018.
[214] J. Lee, M. Humt, J. Feng, R. Triebel, Estimating model uncertainty of neural networks in sparse information form, in: International Conference on Machine Learning, PMLR, 2020, pp. 5702–5713.
[215] A. Immer, M. Korzepa, M. Bauer, Improving predictions of Bayesian neural nets via local linearization, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2021, pp. 703–711.
[216] E. Daxberger, E. Nalisnick, J.U. Allingham, J. Antoran, J.M. Hernandez-Lobato, Bayesian deep learning via subnetwork inference, in: Proceedings of the 38th International Conference on Machine Learning, PMLR, 2021, pp. 2510–2521.
[217] A.G.d.G. Matthews, J. Hron, R. Turner, Z. Ghahramani, Sample-then-optimize posterior sampling for Bayesian linear models, in: Advances in Neural Information Processing Systems, 2018.
[218] T. Pearce, F. Leibfried, A. Brintrup, Uncertainty in neural networks: approximately Bayesian ensembling, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2020, pp. 234–244.
[219] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, A.G. Wilson, Averaging weights leads to wider optima and better generalization, arXiv:1803.05407 [cs, stat], 2019.
[220] Y. Burda, H. Edwards, A. Storkey, O. Klimov, Exploration by random network distillation, arXiv preprint arXiv:1810.12894, 2018.
[221] T. Hospedales, A. Antoniou, P. Micaelli, A. Storkey, Meta-learning in neural networks: a survey, arXiv:2004.05439 [cs, stat], 2020.
[222] D. Flam-Shepherd, J. Requeima, D. Duvenaud, Mapping Gaussian process priors to Bayesian neural networks, in: NIPS Bayesian Deep Learning Workshop, 2017.
[223] D. Hafner, D. Tran, T. Lillicrap, A. Irpan, J. Davidson, Noise contrastive priors for functional uncertainty, arXiv preprint arXiv:1807.09289, 2018.
[224] W. Yang, L. Lorch, M.A. Graule, S. Srinivasan, A. Suresh, J. Yao, M.F. Pradier, F. Doshi-Velez, Output-constrained Bayesian neural networks, arXiv preprint arXiv:1905.06287, 2019.
[225] T. Pearce, R. Tsuchida, M. Zaki, A. Brintrup, A. Neely, Expressive priors in Bayesian neural networks: kernel combinations and periodic functions, in: Proceedings of the 35th Uncertainty in Artificial Intelligence Conference, PMLR, 2020, pp. 134–144.
[226] E. Nalisnick, J. Gordon, J.M. Hernandez-Lobato, Predictive complexity priors, in: Proceedings of the 24th International Conference on Artificial Intelligence and Statistics, PMLR, 2021, pp. 694–702.
[227] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, A.C. Courville, Improved training of Wasserstein GANs, in: Advances in Neural Information Processing Systems, 2017, pp. 5767–5777.